

Texts in Computer Science

Ronald T. Kneusel

Numbers and Computers

Third Edition



Springer

Texts in Computer Science

Series Editors

Orit Hazzan , Faculty of Education in Technology and Science, Technion—Israel
Institute of Technology, Haifa, Israel

Frank Maurer, Department of Computer Science, University of Calgary, Calgary,
Canada

Titles in this series now included in the Thomson Reuters Book Citation Index!

'Texts in Computer Science' (TCS) delivers high-quality instructional content for undergraduates and graduates in all areas of computing and information science, including core theoretical/foundational as well as advanced applied topics. TCS books should be reasonably self-contained and aim to provide students with modern and clear accounts of topics ranging across the computing curriculum. As a result, the books are ideal for semester courses or for individual self-study in cases where people need to expand their knowledge. All texts are authored by established experts in their fields, reviewed internally and by the series editors, and provide numerous examples, problems, and other pedagogical tools; many contain fully worked solutions.

The TCS series is comprised of high-quality, self-contained books that have broad and comprehensive coverage and are generally in hardback format and sometimes contain color. For undergraduate textbooks that are likely to be more brief and modular in their approach, Springer offers the flexibly designed *Undergraduate Topics in Computer Science* series, to which we refer potential authors.

Ronald T. Kneusel

Numbers and Computers

Third Edition

Ronald T. Kneusel
Thornton, CO, USA

ISSN 1868-0941

ISSN 1868-095X (electronic)

Texts in Computer Science

ISBN 978-3-031-67481-5

ISBN 978-3-031-67482-2 (eBook)

<https://doi.org/10.1007/978-3-031-67482-2>

1st & 2nd editions: © Springer International Publishing AG 2015, 2017

3rd edition: © The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2025

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

*To my parents, Janet and Tom, for fostering
my love of science.*

Preface

To the 3rd Edition

This book is about numbers and how they are represented and manipulated by computers. In this third edition, we refine the coverage of the first two editions by adding three new chapters introducing posits, floating-point and integer number formats for artificial intelligence, and, for fun, take current state-of-the-art large language models at their word to implement what they believe are interesting computer number formats. Additionally, the existing text has been revised and edited and software updated as necessary to move from Python 2.7 to Python 3.

The book's software is available on GitHub:

<https://github.com/rkneusel9/NumbersAndComputers>

Comments or questions? Contact me: rkneuselbooks@gmail.com.

Ronald T. Kneusel

Thornton, Colorado

August 2024

To the 1st and 2nd Editions

This is a book about numbers and how those numbers are represented in and operated on by computers.

Of course, numbers are fundamental to how computers operate because, in the end, everything a computer works with is a number. It is crucial that people who develop code understand this area because the numerical operations allowed by computers, and the limitations of those operations, especially in the area of floating point math, affect virtually everything people try to do with computers. This book aims to help by exploring, in sufficient, but not overwhelming, detail, just what it is that computers do with numbers.

Who Should Read This Book

This book is for anyone who develops software including software engineers, scientists, computer science students, engineering students, and anyone who programs for fun.

If you are a software engineer you should already be familiar with many of the topics in this book, especially if you have been in the field for any length of time. Still, I urge you to press on, for perhaps you will find a gem or two which are new to you.

If are old enough, you will remember the days of Fortran and mainframes. If so, like the software engineers above, you are probably also familiar with the basics of how computers represent and operate on numbers, but, also like the software engineers above, you will likely find a gem or two of your own. Scientists in particular should be aware of the limitations and pitfalls associated with using floating point numbers since few things in science are restricted to integers.

Students need this book because it is essential to know what the computer is doing under the hood. After all, if you are going to make a career of computers, why would you not want to know how the machine works?

How To Use This Book

This book consists of two main parts. The first deals with standard representations of integers and floating point numbers while the second details several other number representations which are nice to know about and handy from time to time. Either part is a good place to start, though it is probably best if the parts themselves are read from start to end. Later, after the book has been read, you can use it as a reference.

There are exercises at the end of each chapter. Most of these are of the straightforward pencil and paper kind, just to test your understanding, while others are small programming projects meant to increase your familiarity with the material. Exercises that are (subjectively) more difficult will be marked with either one or two stars (* or **) to indicate the level of difficulty.

Example code is in C and/or Python version 2.7 though earlier 2.x versions should work just as well. Intimate knowledge of these programming languages is not necessary in order to understand the concepts being discussed. If something is not easy to see in the code, it will be described in the text. Why C? Because C is a low-level language, close to the numbers we will be working with and because C is the grandfather of most common programming languages in current use including Python. In general, code will be offset from text and in a `monospace` font. For readers not familiar with C and/or Python there are a plethora of tutorials on the web and reference books by the bookcase. Two examples, geared towards people less familiar with programming, are *Beginning C* by Ivor Horton and *Python Programming Fundamentals* by Kent Lee. Both of these texts are available from Springer in print or ebook format.

At the end of each chapter are references for material presented in the chapter. Much can be learned by looking at these references. Almost by instinct, we tend to ignore sections like this as we are now programmed to ignore advertisements on web pages. In this former case, resist temptation; in the latter case, keep calm and carry on.

Acknowledgments

This book was not written in a vacuum. Here I want to acknowledge those who helped make it a reality. First, the reviewers, who gave of their time and talent to give me extremely valuable comments and friendly criticism: Robert Kneusel, M.S., Jim Pendleton, Ed Scott, Ph.D., and Michael Galloy, Ph.D. Gentlemen, thank you. Second, thank you to Springer, especially my editor, Courtney Clark, for moving ahead with this book. Lastly, and most importantly, thank you to my wife, Maria, and our children: David, Peter, Paul, Monica, Joseph, and Francis. Without your patience and encouragement, none of this would have been written.

Broomfield, CO, USA
December 2014

Ronald T. Kneusel

Contents

1	Number Systems	1
1.1	Representing Numbers	1
1.2	The Big Three (and One Old Guy)	6
1.3	Converting Between Number Bases	8
1.4	Summary	14
	References	15
2	Integers	17
2.1	Bits, Nibbles, Bytes, and Words	17
2.2	Unsigned Integers	19
2.2.1	Representation	19
2.2.2	Storage in Memory—Endianness	20
2.3	Operations on Unsigned Integers	23
2.3.1	Bitwise Logical Operations	23
2.3.2	Testing, Setting, Clearing, and Toggling Bits	28
2.3.3	Shifts and Rotates	31
2.3.4	Comparisons	35
2.3.5	Arithmetic	40
2.3.6	Square Roots	52
2.4	What About Negative Integers?	54
2.4.1	Sign-Magnitude	54
2.4.2	One's Complement	55
2.4.3	Two's Complement	55
2.5	Operations on Signed Integers	56
2.5.1	Comparison	56
2.5.2	Arithmetic	58
2.6	Binary-Coded Decimal	66
2.6.1	Introduction	66
2.6.2	Arithmetic with BCD	68
2.6.3	Conversion Routines	69
2.6.4	Other BCD Encodings	72
2.7	Summary	75
	References	78

3	Floating Point	79
3.1	Floating-Point Numbers	79
3.2	An Exceedingly Brief History of Floating-Point Numbers	82
3.3	Comparing Floating-Point Representations	84
3.4	IEEE 754 Floating-Point Representations	87
3.5	Rounding Floating-Point Numbers (IEEE 754)	93
3.6	Comparing Floating-Point Numbers (IEEE 754)	97
3.7	Basic Arithmetic (IEEE 754)	100
3.8	Handling Exceptions (IEEE 754)	102
3.9	Floating-Point Hardware (IEEE 754)	106
3.10	Binary-Coded Decimal Floating-Point Numbers	108
3.11	Summary	111
	References	112
4	Pitfalls of Floating-Point Numbers (And How To Avoid Them)	115
4.1	What Pitfalls?	115
4.2	Some Experiments	117
4.3	Avoiding the Pitfalls	128
4.4	Summary	132
	References	133
5	Big Integers and Rational Arithmetic	135
5.1	What is a Big Integer?	135
5.2	Representing Big Integers	136
5.3	Arithmetic with Big Integers	141
5.4	Alternative Multiplication and Division Routines	153
5.5	Implementations	162
5.6	Rational Arithmetic with Big Integers	166
5.7	When to Use Big Integers and Rational Arithmetic	172
5.8	Summary	175
	References	176
6	Fixed-Point Numbers	179
6.1	Representation (Q Notation)	179
6.2	Arithmetic with Fixed-Point Numbers	184
6.3	Trigonometric and Other Functions	189
6.4	When to Use Fixed-Point Numbers	199
6.5	Summary	200
	References	201
7	Decimal Floating Point	203
7.1	What is Decimal Floating Point?	203
7.2	The IEEE 754 Decimal Floating-Point Format	204
7.3	Decimal Floating Point in Software	213
7.4	Thoughts on Decimal Floating Point	220
7.5	Summary	220
	References	221

8	Interval Arithmetic	223
8.1	Defining Intervals	223
8.2	Basic Operations	225
8.3	Functions and Intervals	239
8.4	Implementations	244
8.5	Thoughts on Interval Arithmetic	248
8.6	Summary	249
	References	250
9	Arbitrary-Precision Floating Point	251
9.1	What is Arbitrary-Precision Floating Point?	251
9.2	Representing Arbitrary-Precision Floating-Point Numbers	251
9.3	Basic Arithmetic with Arbitrary-Precision Floating-Point Numbers	256
9.4	Comparison and Other Methods	258
9.5	Trigonometric and Transcendental Functions	259
9.6	Arbitrary-Precision Floating-Point Libraries	264
9.7	Thoughts on Arbitrary-Precision Floating Point	275
9.8	Summary	275
	References	276
10	Posits	277
10.1	Why Posits?	277
10.2	Understanding Posits	278
10.3	Standard Posits	281
10.4	Posit Range and Spacing	283
10.5	Experiments	285
10.5.1	Comparing Posit and IEEE Float Ranges	286
10.5.2	Comparing Posit and IEEE Float Precision	287
10.5.3	Comparing Posit and IEEE Compressibility	291
10.6	Posits and Neural Networks	294
10.6.1	Training the Models	294
10.6.2	Testing the Models	297
10.7	Discussion	301
	References	302
11	AI Number Formats	303
11.1	New Number Formats?	303
11.2	Model Training and Inference	304
11.3	Reduced-Precision Floating-Point Formats	306
11.3.1	Half-Precision (FP16) Formats	306
11.3.2	FP8 and FP4 Formats	308
11.4	Scaled Integer Formats	313
11.5	An Experiment	315
11.5.1	Quantization and Model Accuracy	316
11.5.2	Exploring the Code	319

11.6	Discussion	320
	References	321
12	AI-Designed Number Formats	323
12.1	How Large Language Models Work	323
12.2	Experimenting with Prompts	325
12.3	FlexInt: A Hybrid Integer Number Format	328
	12.3.1 Implementation	330
	12.3.2 Examples	334
12.4	MPIF: Mixed Precision Integer Float	337
	12.4.1 Implementation	339
	12.4.2 Examples	343
12.5	MetaInt	344
	12.5.1 Implementation	347
	12.5.2 Examples	350
12.6	Discussion	352
	References	354
13	Other Number Systems	355
13.1	Introduction	355
13.2	Logarithmic Number System	356
13.3	Double-Base Number System	368
13.4	Residue Number System	383
13.5	Redundant Signed-Digit Number System	390
13.6	Summary	397
	References	398
Index		401



Number Systems

1

Abstract

Computers use number bases other than the traditional base 10. In this chapter, we look at number bases focusing on those most frequently used in association with computers. We look at how to construct numbers in these bases and move numbers between different bases.

1.1 Representing Numbers

The ancient Romans used letters to represent their numbers. These are the “Roman numerals” which are often taught to children,

I	1
II	2
III	3
IV	4 (1 before 5)
V	5
X	10
L	50
C	100
D	500
M	1000

By grouping these numbers we can build larger numbers (integers),

$$\text{MCMXCVIII} = 1998$$

$$\begin{array}{ll}
 | = 1 & \cap = 10 \\
 || = 2 & \cap\cap = 20 \\
 ||| = 3 & \cap\cap\cap = 30 \\
 |||| = 4 & \text{⌚} = 100 \\
 \\
 ||||\cap\cap\text{⌚}\text{⌚} = 224
 \end{array}$$

Fig. 1.1 Egyptian numbers. The ancient Egyptians wrote numbers by summing units, tens, and hundreds. There were also larger valued symbols that are not shown. Numbers were written by grouping symbols until the sum equaled the desired number. The order in which the symbols were written, largest first or smallest first, did not matter, but common symbols were grouped. In this example, the smallest units are written first when writing from left to right to represent 224

The Romans built their numbers from earlier Egyptian numbers as seen in Fig. 1.1. Of course, we do not use either of these number systems for serious computation today for the simple reason that they are hard to work with.

For example, the Egyptians multiplied by successive doubling and adding. So, to multiply 17×18 to get 306 the Egyptians would make a table with two columns. The first column consists of the powers of two: $2^0, 2^1, 2^2, 2^3, 2^4 = 1, 2, 4, 8, 16$, etc. and the second column is the first number to be multiplied and its doublings. In this case, 17 is the first number in the problem, so the second column would be 17, 34, 68, 136, 272, etc., where the next number in the sequence is two times the previous number. Therefore, the table would look like this (ignore the starred rows for the moment),

1	17
2	34 *
4	68
8	136
16	272 *

Next, the Egyptians would mark the rows of the table that make the sum of the entries in the first column equal to the second number to multiply, in this case 18. These are marked in the table already, and we see that $2 + 16 = 18$. Lastly, the final answer is found by summing the entries in the second column for the marked rows, $34 + 272 = 306$.

This technique works for any pair of integers. If one were to ask the opposite question, what number times 17 gives 306, one would construct the table as before and then mark rows until the sum of the numbers in the second column is 306. The answer, then, is to sum the corresponding numbers in the first column to get 18. Hence, division was accomplished by the same trick. In the end, since we have algebra at our disposal, we can see that what the Egyptians were really doing was applying the distributive property of multiplication over addition,

$$17(18) = 17(2 + 16) = 34 + 272 = 306$$

But, still, this is a lot of effort. Fortunately, there is another way to represent numbers: place notation (or, more formally, place-value notation). In place notation, each digit represents the number of times that power of the base is present in the number with the powers increasing by one for each digit position we move to the left. Negative powers of the base are used to represent fractions less than one. The digit values we typically use range from 0 to 9 with 10 as the base. Here, we write 10 to mean $1 \times 10^1 + 0 \times 10^0$ thereby expressing the base of our numbers in the place notation using that base. Naturally, this fact is true regardless of the base, so 10_B is the base B for all $B > 1$. Computers use base 10 numbers but not very often though we will see a few examples later in the book.

In general, if the base of a number system is B , then numbers in that base are written with digits that run from 0 to $B - 1$. These digits, in turn, count how many instances of the base raised to some integer power are present in the number. Lots of words, but an equation may clarify,

$$abcd_B = a \times B^3 + b \times B^2 + c \times B^1 + d \times B^0$$

for digits a, b, c , and d . Notice how the exponent of the base is counting down. This counting continues after zero to negative exponents, which are just fractions,

$$B^{-n} = \frac{1}{B^n}$$

So, we can introduce a “decimal point” to represent fractions of the base. Formally, this is known as the *radix point* and is the character that separates the integer part of a number in a particular base from the fractional part. In this book, we will use the “.” (period) character and indicate the base of the number with a subscript after the number,

$$abcd.efg_B = a \times B^3 + b \times B^2 + c \times B^1 + d \times B^0 + e \times B^{-1} + f \times B^{-2} + g \times B^{-3}$$

for additional digits e, f, g and base B . If no explicit base is given, assume the base is 10.

Before moving on to modern number systems let's take a quick look at two ancient number systems that used place notation. The first is the Babylonian number system, which used base 60 (*sexagesimal*) but constructed its digits by grouping sets of

Fig. 1.3 YBC 7289 showing the calculation of the diagonal of a square with a side of length $\frac{1}{2}$. Note that this calculation uses a particularly good value for $\sqrt{2}$. Image copyright Bill Casselman and is used with permission, see <http://www.math.ubc.ca/~cass/Euclid/ybc/ybc.html>. The original tablet is in the Yale Babylonian Collection



The fact that the ancient Babylonians had a place notation is all the more impressive given that they wrote and performed their calculations on clay tablets that were allowed to dry in the sun. This was good as it preserved thousands of tablets for us to investigate today. Figure 1.3 is a particularly important mathematical tablet. This tablet shows a square with its diagonals marked. The side of the square is marked as 30, which is interpreted to be $\frac{1}{2}$. There are two numbers written by one of the diagonals. The first is 1, 24, 51, 10, and the one below is 42, 25, 35. The length of the diagonal of a square is $\sqrt{2}$ times the length of the side. If we multiply 30 by 1, 24, 51, 10 we do get 42, 25, 35. This means that the Babylonians used 1, 24, 51, 10 as an approximation to $\sqrt{2}$. How good was this approximation? Squaring 1, 24, 51, 10 gives 1, 59, 59, 59, 38, 1, 40, which is, in decimal, about 1.99999830, an excellent approximation indeed.

The Mayan people developed their place notation in base 20 using combinations of dots (one) and bars (five) to form digits. The numbers were written vertically from the bottom to the top of the page. Figure 1.4 shows on the left an addition problem written with this notation. Unlike the Babylonians, the Mayans did use a symbol for zero, shaped like a shell, and this is seen in Fig. 1.4 on the right.

It is interesting to see how the Maya, like the Babylonians before them, did not create unique symbols for all possible base 20 digits but instead used combinations of likely base symbols. This might imply that place notation evolved from an early tally system, which is a reasonable thing to imagine. Addition in the Mayan system worked by grouping digits, replacing five dots with a bar, and carrying to the next higher digit if the total in the lower digit was 20 or more.

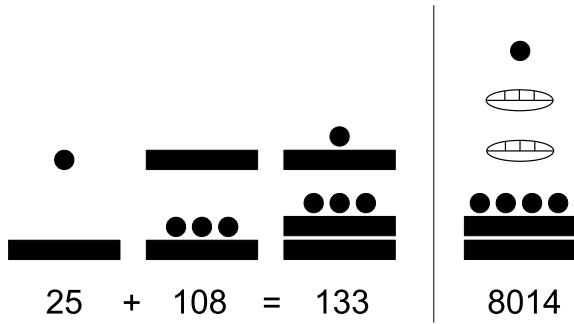


Fig. 1.4 Mayan numbers in base 20 written from bottom to top. Bars represent five, and dots represent ones, with groups of bars and dots used to show the numbers 1 through 19. The zero symbol, shown on the right, is shaped like a shell and holds empty places in the same way zero is used in base 10 so that the number on the right is $1 \times 20^3 + 0 \times 20^2 + 0 \times 20^1 + 14 \times 20^0 = 8014$

For more background on the fascinating history of numbers and number systems, including Mayan numbers, see [1,3]. For a detailed look at Babylonian numbers and mathematics, see [4,5]. Lastly, for a look at Egyptian numbers and mathematics see [2].

1.2 The Big Three (and One Old Guy)

In this book, we are primarily concerned with three number bases: base 2 (binary), base 10 (decimal), and base 16 (hexadecimal). These are the “big three” number bases for computers. In the past, base 8 (octal) was also used frequently, so we will mention it here. This is the “one old guy”. In this section, we look at how to represent numbers in these bases and how to move a number between these bases. Operations, such as arithmetic, will be covered in subsequent chapters from the point of view of how the computer works with numbers of a particular type.

Decimal Numbers. Relatively little needs to be said about decimal numbers as we have been using them all our lives to the point where working with them is second nature. Of course, the reason for this frequent appearance of ten is that we have ten fingers and ten toes. Decimal numbers, however, are not a good choice for computers. Digital logic circuits have two states, “on” or “off”. Since computers are built of these circuits, it makes sense to represent the “on” state (logical `True`) as a 1 and the “off” state (logical `False`) as a 0. If your digits are 0 and 1, then the natural number base to pick is 2, or binary numbers.

Binary Numbers. In the end, computers work with nothing more than binary numbers, which is quite interesting to think about: all the activities of the computer rely on numbers represented as ones and zeros. Following the notation above, we can see that a binary number such as 1011_2 is really

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11$$

Thinking back to the example of ancient Egyptian multiplication above we can now see that the left-hand column was nothing more than the powers of two which are the place values of a binary number. Since each power of two appears only once in a given number (integer) it is always possible to find a sum of powers of two that equals any number. The 1 values in a binary integer are only tick marks indicating which power of two is present. We know that each power of two appears at most once in a given integer because it is possible to represent any integer in binary, and the only digits allowed in a binary number are 0 and 1.

Hexadecimal Numbers. If you work with computers long enough, you will eventually encounter hexadecimal numbers. These are base 16 numbers since “hex” refers to six and “dec” refers to ten. A question immediately arises: we need 16 symbols for hexadecimal digits, but which 16? Zero through nine can be reused, but what should we use for 10 through 15? The flippant answer is whatever symbols we want. We could even invent 16 entirely new symbols, but typically, we follow the convention the ancient Greeks used (see [1] Chap. 4) and take the first six letters of the Latin alphabet as the missing symbols,

<i>A</i>	10
<i>B</i>	11
<i>C</i>	12
<i>D</i>	13
<i>E</i>	14
<i>F</i>	15

In this case, counting runs as 1, 2, 3, 4, 5, 6, 7, 8, 9, *A*, *B*, *C*, *D*, *E*, *F*, 10. Unsurprisingly, when writing hexadecimal numbers, one often sees lowercase letters, *a–f*, instead of *A–F*. Like most English speakers, computer scientists tend to shorten words whenever possible. So, you will often hear or see hexadecimal numbers referred to as “hex” numbers. No reference to magic is intended unless it is the magic by which computers work at all.

If we break down a hex number,

$$FDED_{16} = 15 \times 16^3 + 13 \times 16^2 + 14 \times 16^1 + 13 \times 16^0 = 65005$$

we see that it is a compact notation compared to binary. This should not be surprising. Instead of indicating which powers of two are in a number, as in binary, we are allowed 16 possible multipliers of each power.

Octal Numbers. While much less common than in the past, octal (base 8) numbers still occasionally appear. As these are base 8 numbers, we already know we only need the digits 0–7 to write numbers in this base. In Sect. 1.3, we will see why, if computers work in binary, people still use other number bases like octal or hexadecimal.

Table 1.1 The first 16 integers in decimal, binary, octal, and hexadecimal

Decimal	Binary	Octal	Hexadecimal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

As we expect, octal numbers can also be written as a sum of powers:

$$1234_8 = 1 \times 8^3 + 2 \times 8^2 + 3 \times 8^1 + 4 \times 8^0 = 668$$

and we will not be disappointed. In modern programming languages, octal numbers are typically reserved for character codes and will be rarely seen outside of that context.

1.3 Converting Between Number Bases

Sometimes, we need to convert a number between bases. In this section, we look at converting between the bases we introduced above. Table 1.1 will be handy when converting numbers between bases. It shows the first 16 integers in binary, octal, hexadecimal, and decimal.

Hexadecimal and Binary. Hexadecimal numbers are base 16, while binary numbers are base 2. We see that $2^4 = 16$ so the conversion between hexadecimal and binary, either direction, will involve groups of four binary digits. To convert a binary number to hexadecimal, simply separate the digits into groups of four, from right to left, and write each group as a hex digit referring to Table 1.1 if necessary,

$$\begin{aligned}
 101110010011110_2 &= && 101 & 1100 & 1001 & 1110 \\
 &= && 5 & C & 9 & E \\
 &= 5C9E_{16}
 \end{aligned}$$

Turning a hexadecimal number into binary is simply the reverse process,

$$\begin{aligned}
 FDDA_{16} &= && F & D & D & A \\
 &= 1111 & 1101 & 1101 & 1010 \\
 &= 1111110111011010_2
 \end{aligned}$$

This conversion process is simple enough, but why does it work? The key lies in the fact, already noted, that $2^4 = 16$. Recall that in a place notation system, each digit position to the left is one power of the base larger, just as the hundreds place in decimal is a factor of ten more significant than the tens place. If we look at base 2 and instead of moving one digit to the left, which is two times larger than the original digit value, we move four digits, we are now $2 \times 2 \times 2 \times 2$ larger or 16. But, in base 16, each digit position is 16 times larger than the next place to the right. Therefore, we have an equivalence between moving one digit to the left in a hexadecimal number and four digits to the left in a binary number. And this is why we can group the binary digits in sets of four to arrive at the equivalent hexadecimal digit.

Octal and Binary. If $2^4 = 16$ implies we group binary digits in sets of four, then $2^3 = 8$ implies we should group binary digits in sets of three to convert between binary and octal, and this is indeed the case,

$$\begin{aligned}
 101110010011110_2 &= && 101 & 110 & 010 & 011 & 110 \\
 &= && 5 & 6 & 2 & 3 & 6 \\
 &= 56236_8
 \end{aligned}$$

Likewise, we can reverse the process to get,

$$\begin{aligned}
 52431_8 &= && 5 & 2 & 4 & 3 & 1 \\
 &= 101 & 010 & 110 & 011 & 001 \\
 &= 101010110011001_2
 \end{aligned}$$

Lastly, should the need arise, conversion between hexadecimal and octal, or vice versa, is easily accomplished by first moving to binary and then to the other base from binary.

Decimal and Binary. The conversion technique above falls apart when moving between decimal and binary. We already know why: $2^x = 10$ has no integer solution, so we cannot merely group digits. Instead, we must process the conversions in a more cumbersome way.

Decimal to Binary. Perhaps the simplest way to convert a decimal number to binary by hand is to repeatedly divide the number by two, keeping track of the quotient and

remainder. The quotient gives the next number to divide by two and the remainder, always either 1 or 0, is one binary digit of the result. If the numbers are not too big, dividing by two is easy without paper and pencil.

For example, to convert 123 to binary,

$$\begin{aligned} 123 \div 2 &= 61 \text{ r } 1 \\ 61 \div 2 &= 30 \text{ r } 1 \\ 30 \div 2 &= 15 \text{ r } 0 \\ 15 \div 2 &= 7 \text{ r } 1 \\ 7 \div 2 &= 3 \text{ r } 1 \\ 3 \div 2 &= 1 \text{ r } 1 \\ 1 \div 2 &= 0 \text{ r } 1 \end{aligned}$$

The binary representation is read from bottom to top: $123 = 1111011_2$.

The conversion works, but it is good to know *why* it works, so let's take a closer look. If we write a seven-digit binary number, $b_6b_5b_4b_3b_2b_1b_0$, with b_k the k -th digit, in expanded form we get,

$$b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + b_3 \times 2^3 + b_4 \times 2^4 + b_5 \times 2^5 + b_6 \times 2^6$$

which, since $2^0 = 1$ and every other term is a multiple of two, can be written as

$$b_0 + 2(b_1 \times 2^0 + b_2 \times 2^1 + b_3 \times 2^2 + b_4 \times 2^3 + b_5 \times 2^4 + b_6 \times 2^5)$$

If we divide the above by 2 we get the quotient,

$$b_1 \times 2^0 + b_2 \times 2^1 + b_3 \times 2^2 + b_4 \times 2^3 + b_5 \times 2^4 + b_6 \times 2^5$$

and a remainder of b_0 . Therefore, it is clear that performing an integer division of a number by two leads to a new quotient and a remainder of either zero or one since b_0 is a binary digit. Further, this remainder is in fact the rightmost digit in the binary representation of the number. If we look at the quotient above it has the same form as the initial binary representation of the number but with every binary digit now shifted to the next lowest power of two. Therefore, dividing this quotient by two we will repeat the calculation to give b_1 as the remainder. This is the next digit in the binary representation. Finally, if we repeat this until the quotient is zero, we will generate all the digits in the binary representation in reverse order. Reversing the digits returns the desired binary representation.

Another method for converting a decimal number to binary involves making a table of the powers of two and asking repeated questions about sums from that table. This method works because the conversion process tallies the powers of two that add up to the decimal number.

For example, to convert 79 to binary, make a table like this,

64	32	16	8	4	2	1
----	----	----	---	---	---	---

with the powers of two written highest to lowest from left to right. Next, ask the question, “Is $64 > 79$?” If the answer is yes, put a 0 under the 64. If the answer is no, put a 1 under the 64 because there is a 64 in 79,

64	32	16	8	4	2	1
1						

Now add a new line to the table and ask, “Is $64 + 32 > 79$?” Since the answer is yes, we know that there is no 32 in the binary representation of 79 so we put a 0 below the 32,

64	32	16	8	4	2	1
1						
	0					

We continue the process, now asking “Is $64 + 16 > 79$?” In this case, the answer is again yes so we put a 0,

64	32	16	8	4	2	1
1						
	0					
		0				

We continue for each remaining power of two asking whether that power of two, plus all the previous powers of two that have a 1 under them, is greater or less than 79,

64	32	16	8	4	2	1
1						
	0					
		0				
			1			
				1		
					1	
						1

To complete the conversion, we collapse the table vertically and write the binary answer, $79 = 1001111_2$.

Binary to Decimal. If a binary number is small, the fastest way to convert it to a decimal is to remember Table 1.1. Barring that, there is a straightforward iterative way to convert binary to decimal. Suppose we have a k digit binary number, say $b_4b_3b_2b_1b_0$. We can use the following recurrence relation to find the decimal equivalent:

$$d_0 = 0$$

$$d_i = b_{k-i} + 2d_{i-1}, i = 1 \dots k$$

For example, let's convert 11001_2 to decimal using the recurrence relation. In this case, we get

i	d_i
0	0
1	$1 + 2(0) = 1$
2	$1 + 2(1) = 3$
3	$0 + 2(3) = 6$
4	$0 + 2(6) = 12$
5	$1 + 2(12) = 25$

This technique is easily implemented in code. Why does it work? Consider the five-digit binary number we started this section with, $b_4b_3b_2b_1b_0$. If we write the algebraic expression representing the recurrence relation after five iterations, we get

$$d_5 = b_0 + 2(b_1 + 2(b_2 + 2(b_3 + 2(b_4 + 2(0))))))$$

Which, if we work it out step by step, becomes

$$\begin{aligned}
 & b_0 + 2(b_1 + 2(b_2 + 2(b_3 + 2(b_4 + 2(0)))))) \\
 &= b_0 + 2(b_1 + 2(b_2 + 2(b_3 + 2b_4))) \\
 &= b_0 + 2(b_1 + 2(b_2 + 2b_3 + 2^2b_4)) \\
 &= b_0 + 2(b_1 + 2b_2 + 2^2b_3 + 2^3b_4) \\
 &= b_0 + 2b_1 + 2^2b_2 + 2^3b_3 + 2^4b_4 \\
 &= b_0 \times 2^0 + b_1 \times 2^1 + b_2 \times 2^2 + b_3 \times 2^3 + b_4 \times 2^4
 \end{aligned}$$

This is precisely the expanded form of a binary number with digits $b_4b_3b_2b_1b_0$.

Others to Decimal. The recurrence relation for converting binary numbers to decimal works for any number base. For example,

$$d_0 = 0$$

$$d_i = b_{k-i} + Bd_{i-1}, i = 1 \dots k$$

for a base B number with digits b_i .

With this knowledge, converting hexadecimal or octal numbers to decimal is straightforward. Simply use $B = 16$ for hexadecimal and $B = 8$ for octal. In the case of hexadecimal, a base greater than 10, the additional digits, $A - F$, take on decimal values 10 – 15.

For example, let's convert $56AD_{16}$ to decimal. Since this is a four-digit number $k = 4$ and the recurrence relation becomes

$$d_0 = 0$$

$$d_i = b_{k-i} + 16d_{i-1}, i = 1 \dots k$$

and calculation proceeds as

i	d_i
0	0
1	$5 + 16(0) = 5$
2	$6 + 16(5) = 86$
3	$10 + 16(86) = 1386$ (since $A_{16} = 10$)
4	$13 + 16(1386) = 22189$ (since $D_{16} = 13$)

so $56AD_{16} = 22189$.

Finally, a similar calculation works for octal numbers. Convert 7502_8 to decimal. Again, $k = 4$ and the recurrence relation is now,

$$d_0 = 0$$

$$d_i = b_{k-i} + 8d_{i-1}, i = 1 \dots k$$

giving,

i	d_i
0	0
1	$7 + 8(0) = 7$
2	$5 + 8(7) = 61$
3	$0 + 8(61) = 488$
4	$2 + 8(488) = 3906$

with a final result of $7502_8 = 3906$.

Frequent use of hexadecimal and octal numbers will quickly teach you to remember the first few powers of each base. Embedded system's programmers learn this early on for at least the first four digits. For hexadecimal, the powers of 16 are,

$$\begin{array}{l|l} 16^0 & 1 \\ 16^1 & 16 \\ 16^2 & 256 \\ 16^3 & 4096 \end{array}$$

and for octal,

$$\begin{array}{l|l} 8^0 & 1 \\ 8^1 & 8 \\ 8^2 & 64 \\ 8^3 & 512 \end{array}$$

With these, small hexadecimal and octal integers may be converted to decimal quickly,

$$1234_{16} = 1 \times 4096 + 2 \times 256 + 3 \times 16 + 4 \times 1 = 4660$$

$$1234_8 = 1 \times 512 + 2 \times 64 + 3 \times 8 + 4 \times 1 = 668$$

With a bit of practice such conversion can be performed mentally and quickly.

1.4 Summary

In this chapter, we briefly reviewed numbers, how they were represented historically, how place notation works, and how to manipulate numbers in the bases most frequently used by computers. It is helpful to become comfortable working with the conversion between number bases and it is beneficial to memorize the binary representation of the first 16 integers. The exercises will help build this confidence and the skill will be used frequently in the remainder of this book.

Exercises

1.1 Using Egyptian notation and method calculate 22×48 and $713 \div 31$.

1.2 Using Babylonian notation calculate $405 + 768$. (*Hint*: carry on sixty, not ten)

1.3 Using Mayan notation calculate $419 + 2105$. (*Hint*: carry on twenty, not ten)

1.4 Convert the following hexadecimal numbers to binary: $A9C1_{16}$, $20ED_{16}$, $FD60_{16}$.

1.5 Convert the following octal numbers to binary: 773_8 , 203_8 , 656_8 .

1.6 Using the table method, convert 6502 to binary.

1.7 Using the division method, convert 8008 to binary.

1.8 Write $e = 2.71828182845904 \dots$ to at least four decimal places using Babylonian notation. (*Hint*: You will need to use three negative powers of sixty) **

1.9 Write a program to convert a number in any base < 37 to its decimal equivalent using the recurrence relation. Use a string to hold the number and use 0 – 9 and capital letters A – Z for the digits. *

References

1. Boyer CB (1985) A history of mathematics. Princeton University Press, Princeton
2. Gillings R (1982) Mathematics in the time of the pharaohs. Dover Publications, New York
3. Menninger K (1992) Number words and number symbols. Dover Publications, New York
4. Neugebauer O (1945) Mathematical cuneiform texts. American oriental series, vol 29. American Oriental Society, New Haven
5. Neugebauer O (1952) The exact sciences in antiquity. Princeton University Press, Princeton



Abstract

Integers are perhaps the most important class of numbers. This is certainly true in the case of computers. In this chapter, we dive into the integers and how computers represent and operate on them. Without these operations, digital computers would not function. We begin with some preliminary notation and terminology. Next, we take a detailed look at the unsigned integers. We follow this with an examination of negative integers and their operations. Lastly, we finish with a look at binary-coded decimals.

2.1 Bits, Nibbles, Bytes, and Words

Our tour of integers will be more accessible if we get some terminology out of the way first. This will make discussing how numbers are represented inside a computer much easier. We'll start small and work our way up.

Bits. As we have seen, computers work with binary numbers. A single binary digit is known affectionately as a *bit*. This bit is either zero or one, off or on, false or true. The word *bit* is short for *binary digit* and appears in Claude Shannon's classic 1948 paper *A Mathematical Theory of Communication* [2]. Shannon attributed *bit* to John Tukey, who used it in a Bell Labs memo dated January 9, 1947. Throughout this book, we will play fast and loose with bit terminology and freely alternate between “on” and “true” when a bit is set to one and “off” or “false” when a bit is set to zero. The choice of words will depend on the context.

Since a bit is so simple, just a zero or a one, it is the fundamental unit of information in the digital world. A status bit set to one may indicate that the rocket is ready to launch, while a status bit set to zero indicates a fault. Indeed, some microcontroller

devices, like the 8051, make excellent use of their limited resources and support single bit data. We will make extensive use of bits in the remainder of this chapter. Zero or one, how hard can it be? Still, as valuable as a bit is, it is just, well, a *bit*. You need to group bits to build more expressive numbers.

Nibbles. A *nibble* (sometimes *nybble*) is a 4-bit number. As a 4-bit number, a nibble can represent the first 16 integers with $0000_2 = 0$ and $1111_2 = 15$. And, as we now know, the numbers 0 through 15 are exactly the digits of a hexadecimal number. So, a nibble is a single hexadecimal digit. This makes sense since we can represent $2^4 = 16$ possible values with 4 bits. A nibble is also the amount of information necessary to represent a single binary-coded decimal digit, but we are getting ahead of ourselves and will talk about binary-coded decimals later in this chapter. The origin of the word *nibble* is related to the origin of the word *byte*, so we will expand on it below.

Modern computers do not use nibbles as a unit of data for processing. However, the very first microprocessors, like the TMS 1000 by Texas Instruments and the 4004 by Intel, both introduced in 1971, were 4-bit devices and operated on 4-bit data. This makes the nibble a bit like octal numbers: appearing in the shadows but not a real player in modern computing.

Bytes. For a modern computer, the smallest unit of data is the *byte*, an 8-bit binary number. This means that a single byte can represent the integers from $0 \dots 255$. We'll ignore the concept of negative numbers for the moment.

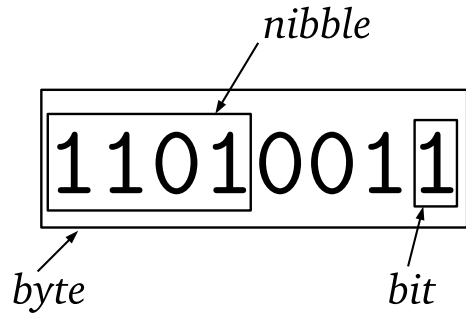
The term *byte* was first used by Werner Buchholz in July 1956 during the design of the IBM Stretch computer and was an intentional misspelling of “bite” to avoid confusion with “bit” [3]. Since *byte* came from “bite”, it is natural to call half a byte, a 4-bit number, a “nibble” since a nibble is a small bite. Hence *nibble* was born.

Buchholz intended a byte to represent a character. We still use bytes to represent characters in ASCII but have since moved to larger numbers for characters in Unicode. In this book, however, we will implicitly assume that characters are bytes to keep things simple. So, a text that contains 1000 characters will take 1000 bytes of memory to store. The universal use of bytes leads to the frequent appearance of the numbers $2^8 = 256$ and $2^8 - 1 = 255$. For example, colors on computers are often represented as triplets of numbers to indicate the amount of red, green, and blue that make up the color. Typically, these triplets are given as a set of three bytes so that 000000_{16} is black and $00FF00_{16}$ is bright green. This is just one example: bytes are everywhere. The relationship between bits, nibbles, and bytes is visualized in Fig. 2.1. Here, we see that a byte can be thought of as two nibbles, each a hexadecimal digit, or as 8 bits, each a binary digit.

Words. A byte is two nibbles, so is a *word* two bytes? Well, sometimes, yes. And sometimes it is four bytes, or eight, or some other value. Unfortunately, the term *word* is not generic like bit and byte; instead, it is tied to a particular computer architecture and describes its primary data size.

Historically, the word size of a computer could be anywhere from 4 bits for early microprocessors (Intel 4004) to 60 bits for early mainframes (CDC 6600), but modern computers have settled on powers of two for their word sizes, typically 32 or 64 bits

Fig. 2.1 A byte which consists of 2 nibbles and 8 bits. The number 211 is two nibbles, D_{16} and 3_{16} , giving $D3_{16}$, or 8 bits 11010011_2



with some second-generation microprocessors using 16-bit words (Intel 8086, WDC 65816). For our purposes, we can assume a 32-bit word size when describing data formats. On occasion, we will need to be more careful and explicitly declare the word size we are working with.

With these preliminaries understood, it is time to begin working with actual data representations. We start naturally enough with unsigned integers.

2.2 Unsigned Integers

Unsigned integers are our first foray into the depths of the computer as far as numbers are concerned. These are basic numbers, all positive integers, that frequently represent characters, counters, constants, or anything that can be mapped to the set $\{0, 1, 2, \dots\}$. Of course, computers have finite memory, so there is a limit to the size of an unsigned integer, as the next section illustrates.

2.2.1 Representation

Integers are stored in memory in binary using one or more bytes depending on their range. The example in Fig. 2.1 is a one-byte unsigned integer. If working in a typed language like C, we must declare a variable storing one byte. Typically, this means an *unsigned char* data type,

```
unsigned char myByte;
```

which can store a positive integer from $00000000_2 = 0$ to $11111111_2 = 255$. Therefore, this is the *range* of the `unsigned char` data type. Unsigned integers larger than 255 require more than one byte.

Table 2.1 shows standard C types for unsigned integers and the allowed range for that type. These are fixed in the sense that numbers must fit into this many bits at all times. If not, an underflow or overflow will occur.

Table 2.1 Unsigned integer declarations in C: the declaration, minimum value, maximum value, and number of data bits

Declaration	Minimum	Maximum	Number of bits
Unsigned char	0	255	8
Unsigned short	0	65,535	16
Unsigned int	0	4,294,967,295	32
Unsigned long	0	4,294,967,295	32
Unsigned long long	0	18,446,744,073,709,551,615	64

Some programming languages treat integers as abstract objects, separate from how they are stored in memory. For example, the unsigned integer operations discussed later in the chapter work nicely in Python, but the notion of range is a little nebulous; Python will move between internal representations as necessary.

If a number like $11111111_2 = 255$ is the most significant unsigned number that fits in a single byte, how many bytes are needed to store 256? Moving to the following number implies we will need 9 bits to store 256, implying that we will need two bytes. However, there is a subtlety here that needs to be addressed. If the number is to be stored in the computer’s memory, say starting at address 1200, how should the individual bits be written to memory location 1200 and the one following (1201)? The question has more than one answer.

2.2.2 Storage in Memory—Endianness

Addresses. To talk about how we store unsigned integers in memory, we must first discuss how memory is addressed. In this book, we have a working assumption of a 32-bit Intel-based computer running the Linux operating system. Additionally, we assume `gcc` to be our C compiler. In this case, we have *byte-addressable* memory, meaning we can refer to memory addresses using bytes even though the word size is four bytes. For example, this small C program,

```

1 | #include <stdio.h>
2 |
3 | int main() {
4 |     unsigned char *p;
5 |     unsigned short n = 256;
6 |
7 |     p = (unsigned char *)&n;
8 |     printf("address of first byte = %p\n", &p[0]);
9 |     printf("address of second byte = %p\n", &p[1]);
10 |
11 |     return 0;
12 | }
```


when compiled and run, produces output similar to

```
address of first byte = 0xbfdafe9e
address of second byte = 0xbfdafe9f
```

where the specific addresses, given in hexadecimal and starting with the `0x` prefix used by C, will vary from system to system and run to run. The key point is that the difference between the addresses is *one* byte. This is what is meant by a byte-addressable memory. If you don't know the particulars of the C language, don't worry. The program defines a two-byte number in line 5 (n) and a pointer to a single-byte number in line 4 (p). The pointer stores a memory address, which we set to the beginning of the memory used by n in line 7. We then ask the computer to print the numeric address of the memory location (line 8) and the next byte (line 9) using the `&` operator and indexing for the first (`p[0]`) and second (`p[1]`) bytes. Now, let's consider storing unsigned integers.

Bit Order. We use the 8 bits of the byte at a particular memory address to store the number $11011101_2 = 221$ in memory. The question then becomes, which bits of 221 map to which bits of the memory? If we number the bits from 0 for the lowest order bit, which is the rightmost bit when writing the number in binary, to 7 for the highest order bit, which is the leftmost bit when writing in binary, we get a one-to-one mapping,

$$\begin{array}{r} 7 \ 6 \ 5 \ 4 \ 3 \ 2 \ 1 \ 0 \\ \hline 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \end{array}$$

The highest order bit of the number is put into the highest order bit of the memory address. This seems sensible enough but one could imagine doing the reverse as well,

$$\begin{array}{r} 7 \ 6 \ 5 \ 4 \ 3 \ 2 \ 1 \ 0 \\ \hline 1 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \end{array}$$

So, which is correct? The answer depends on how bits are read from the memory location. In modern computer systems, the bits are read low to high, meaning the low-order bit is read first (bit 0) followed by the next higher bit (bit 1) so that the computer will use the first ordering of bits mapping 76543210 to 11011101.

Byte Order. Within a byte, bits are stored low to high. What about the bytes of a multibyte integer? We know that the number $100000000_2 = 256$ requires two bytes of memory, one storing the low set of 8 bits 00000000_2 and another storing the high set of 8 bits 00000001_2 where leading zeros indicate that the 1 is in the first bit position. But, which order in memory should we use

Memory address:	1200	1201
Low first:	00000000	00000001
High first:	00000001	00000000

low first or high first? The answer is either. If we put the low-order bytes of a multibyte number in memory first, we are using *little-endian* storage. If we put the high-order bytes first, we are using *big-endian* or *network order*. The choice is the *endianness* of the number, typically a function of the microprocessor, which has a preferred ordering. The terms little-endian and big-endian are derived from the book *Gulliver's Travels* by satirist Jonathan Swift [4]. Published in 1726, the book tells the story of Gulliver and his travels throughout a fictional world. When Gulliver comes to the land of Lilliput, he encounters two religious sects who are at odds over whether to crack open their soft-boiled eggs little-end first or big-end first. Intel-based computers use little-endian when storing multibyte integers. Motorola-based computers use big-endian. Additionally, big-endian is called network order because the Internet Protocol standard [5] requires that numeric values in packet headers be stored in big-endian order.

Let's look at some examples of little- and big-endian numbers. We will use a 32-bit number presented as four hexadecimal digits. For example, if the 32-bit number is

$$10101110111100000101011010010110_2 = \text{AE}_{16} \text{F0}_{16} 56_{16} 96_{16}$$

we will write it as AEF05696 dropping the 16 base indicator for the time being. Putting this number into memory starting with byte address zero and using big-ending ordering gives

Address:	0	1	2	3
Value:	AE	F0	56	96

where the first byte read, pulling bytes beginning with address zero, is the high-order byte. If we want to use little-endian, we would instead put the lowest order byte first, filling memory like so:

Address:	0	1	2	3
Value:	96	56	F0	AE

Reading from address zero will now give us the low-order byte first. Note that within a particular byte, we always use a bit ordering that maps the high-order bit (bit 7) to the high-order bit of the number. Reversing the bits within a byte when using little-endian is an error.

When working with data on a single computer within a single program, we typically do not need to pay attention to endianness. However, when transmitting data over the network or using data files written by other machines, it is a good idea to consider the endianness of the data. For example, when reading sensor values from a CCD camera, which are typically 16-bit numbers requiring two bytes to store, it is essential to know the endianness; otherwise, the image generated from the raw data will look strange because the high-order bits that contain the majority of the information about the image will be in the wrong place. From the examples above,

we see that correcting for endianness differences is straightforward; simply flip the order of the bytes in memory to convert from little-endian to big-endian and vice versa.

2.3 Operations on Unsigned Integers

Binary and unary operations on unsigned integers are the heart of what computers do. This section reviews common bit operations, mentioning essential things to consider when working with unsigned integers of fixed bit width. Examples will use byte values to keep it simple, but in some cases, multibyte numbers will be shown. Specific operations are shown in C and Python. Though Python syntax is often the same as C, the results may differ because the Python interpreter will change the internal representation as necessary.

2.3.1 Bitwise Logical Operations

Bitwise operators are binary, meaning they operate on two numbers (called operands) to produce a new binary number. In the previous sentence, the first instance of “binary” refers to the number of operands or arguments to the operation, while the second instance of “binary” refers to the base of the operands themselves.

Digital logic circuits, which are fascinating to study but well beyond the purview of this book, are built from logic gates that implement the logical operations we are discussing here in hardware. The basic set of logic operators are given names from Boolean logic: AND, OR, XOR, and the unary NOT. Negated versions of AND and OR, called NAND and NOR, are also in use, but these are easily constructed by taking the output of AND and OR and passing it through a NOT so we will ignore them here.

Logical operations are most easily understood via *truth tables*, which illustrate the function of the operator by enumerating the possible set of inputs and giving the corresponding output. They are called truth tables because of the correspondence between the 1 and 0 of a bit and the logical concept of `true` and `false` from Boolean algebra [6].

A truth table shows, for each row, the inputs to the operator followed by the output generated by those inputs. For example, the truth table for the AND operator is

AND		
0	0	0
0	1	0
1	0	0
1	1	1

which says that if the two inputs to the AND are 0 and 0 then the output bit is also 0. Likewise, if the inputs are 1 and 1 the output is 1. In fact, for the AND operation, the only way to get an output of 1 is for both input bits to be 1. We might think of AND as meaning “only both”.

The two other most commonly used binary logic operators are OR and XOR. The latter is called exclusive-OR and is sometimes indicated with EOR instead of XOR. Their truth tables are

OR			XOR		
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

where we see that OR means “at least one” and XOR means “one or the other but not both”.

The last of the standard bitwise logical operators is NOT with its Spartan truth table,

NOT	
0	1
1	0

The operation is simply turn 1 to 0 and 0 to 1.

Looking again at the truth tables for AND, OR, and XOR, we see that there are four rows in each table matching the four possible sets of inputs. We might interpret the 4 bits in the output column as a single 4-bit number. A 4-bit number has 16 possible values from 0000_2 to 1111_2 , implying there are 16 possible truth tables for a binary bitwise operator. Three of them are AND, OR, and XOR. Another three are the tables made by negating the output of these operators. That leaves ten other possible truth tables. What do these correspond to, and are they useful in computer programming? The complete set of possible binary truth tables is given in Table 2.2 along with an interpretation of each. While all of these operators have a name and use in logic, many are clearly of limited utility in computer programming. For example, always outputting 0 or 1 regardless of the input (FALSE and TRUE in the table) is of no practical use to a programmer.

Let’s consider a few logical operator examples beginning with AND,

$$\begin{array}{r} 10111101 = 189 \\ \text{AND } 11011110 = 222 \\ \hline 10011100 = 156 \end{array}$$

The binary representation of the number is in the first column, and the decimal is in the second.

We see that for each bit in the two operands, 189 and 222, the output bit is on only when the corresponding bits in the operand are on.

Table 2.2 The 16 possible binary bitwise logical operators. If a common name exists for the operation, it is given in the first column. The top two rows labeled p and q refer to the operands or inputs to the operator. A 0 is considered false while 1 is true. A description of the operator is given in the last column. In the description, the appearance of p or q in an if-statement implies that the operand is true

p :	0	0	1	1	
q :	0	1	0	1	
FALSE	0	0	0	0	Always false
AND	0	0	0	1	p and q
	0	0	1	0	if p then not q else false
	0	0	1	1	p
	0	1	0	0	If not p and q
	0	1	0	1	q
XOR	0	1	1	0	p or q but not (p and q)
OR	0	1	1	1	p , q or (p and q)
NOR	1	0	0	0	Not OR
XNOR	1	0	0	1	Not XOR
	1	0	1	0	Not q
	1	0	1	1	If q then p else true
	1	1	0	0	Not p
	1	1	0	1	If p then q else true
NAND	1	1	1	0	Not AND
TRUE	1	1	1	1	Always true

Next, consider the OR and XOR operators. For OR, we have

$$\begin{array}{r} 10111101 = 189 \\ \text{OR } 11011110 = 222 \\ \hline 11111111 = 255 \end{array}$$

This implies that at least one of the operands has a bit set at every bit position. The XOR operator will give us

$$\begin{array}{r} 10111101 = 189 \\ \text{XOR } 11011110 = 222 \\ \hline 01100011 = 99 \end{array}$$

One useful property of the XOR operator is that it undoes itself if applied a second time. Above, we calculated $189 \text{ XOR } 222$ to get 99. If we now perform $99 \text{ XOR } 222$ we get

$$\begin{array}{r}
 01100011 = 99 \\
 \text{XOR } 11011110 = 222 \\
 \hline
 10111101 = 189
 \end{array}$$

giving us back what we started with. As an aside, this is useful as a simple way to encrypt data. If Alice wishes to encrypt a stream of n bytes (M), to send it (relatively) securely to Bob, all she needs to do is generate a stream of n random bytes (S) and apply XOR, byte by byte, to the original stream M . This produces a new stream, M' . Alice can transmit M' to Bob any way she wishes. If adversary Carol intercepts the message, she cannot easily decode it without the same random stream S . Of course, Bob must somehow have S as well to recover the original message M . If S is truly random, used only once, and then discarded, this method becomes *one-time pad* encryption. The critical points are that S is truly random and only used once. Of course, this does not cover the possibility that Carol may get her hands on S as well, but if she cannot, there is (relatively) little chance she will be able to recover M from M' .

Another handy use for XOR allows us to swap two unsigned integers without using a third variable. For example, in C, instead of

```
unsigned char a,b,t;
...
t = a;
a = b;
b = t;
```

we can use

```
unsigned char a,b;
...
a ^= b; // a = a ^ b;
b ^= a;
a ^= b;
```

where $\wedge$ is the C operator for XOR and $a \wedge b$ is shorthand for $a = a \wedge b$.

Why does this work? If we write an example in binary, we see

a = 01011101		
b = 11011011		
a ^ b		01011101 ^ 11011011 → a=10000110
b ^ a		10000110 ^ 11011011 → b=01011101
a ^ b		10000110 ^ 01011101 → a=11011011

As mentioned, if we perform $a \text{ XOR } b \rightarrow c$ and then $c \text{ XOR } b$, we get a back. If we do $c \text{ XOR } a$ instead, we get b back. The trick makes use of this fact by first calculating $a \text{ XOR } b$ and then using that result, stored temporarily in a , to get a back but this time storing it in b and likewise for getting b back and storing it in a . Thereby swapping the two values in memory. This trick works for unsigned integers of any size. Another common use of XOR is in parity calculations. The parity of an integer is the number of 1 bits in its binary representation. If the number is odd, the parity is 1; otherwise, it is 0. This fact can be used as a simple checksum when transmitting a series of bytes. A checksum is an indicator of the integrity of the data. If the checksum calculated on the receiving end does not match, there has been a transmission error. If the last byte of data is the running XOR of all previous bytes, the parity of this byte can be used to determine if a bit was changed during transmission. To understand how to use XOR for parity calculations, we must wait until we discuss bit shifting later in the chapter.

The effect of NOT is straightforward,

$$\begin{array}{r} \text{NOT } 10111101 = 189 \\ \hline 01000010 = 66 \end{array}$$

NOT merely flips the bits from 0 to 1 and 1 to 0.

Figure 2.2 shows a C program implementing the AND, OR, XOR, and NOT logical operators. Note that the example uses the unsigned char data type, an 8-bit unsigned integer. The logical operators work with any size integer type, e.g., unsigned short or unsigned int, etc. Figure 2.3 presents the corresponding Python code.

Fig. 2.2 Bitwise logical operators in C

```

1 | #include <stdio.h>
2 |
3 | void pp(unsigned char z) {
4 |     printf("%3d (%02x)\n", z, z);
5 | }
6 |
7 | int main() {
8 |     unsigned char z;
9 |
10 |    z = 189 & 222; pp(z); // '&' is AND
11 |    z = 189 | 222; pp(z); // '|' is OR
12 |    z = 189 ^ 222; pp(z); // '^' is XOR
13 |    z = z ^ 222; pp(z); // returns 189
14 |    z = ~z; pp(z); // '~' is NOT
15 |
16 |    return 0;
17 | }
```

Fig. 2.3 Bitwise logical operators in Python

```

1 def pp(z):
2     print("{0:08b}".format(z & 0xff))
3
4 def main():
5     z = 189 & 222;   pp(z) # '&' is AND
6     z = 189 | 222;   pp(z) # '|' is OR
7     z = 189 ^ 222;   pp(z) # '^' is XOR
8     z = z ^ 222;     pp(z) # returns 189
9     z = ~z;          pp(z) # '~' is NOT
10
11 main()

```

Standard C does not support direct output of numbers in binary, so the example in Fig. 2.2 outputs in decimal and hexadecimal instead. Recall that in Chap. 1, we learned how to easily change hexadecimal numbers into binary by replacing each hexadecimal digit with 4 bits representing that digit. The Python code uses standard string formatting language features to output in binary directly. As Python is itself written in C, and the language was designed to show that heritage, the bitwise logical operator syntax is the same.

There is one subtlety with the Python code you may have noticed. The function `pp()` uses “`z & 0xff`” in the `format()` method call where you might have expected only “`z`”. If we do not AND the output value with FF_{16} , we will output a full range integer, which, as we will learn later in the chapter, Python will interpret as a negative number. The AND keeps only the lowest 8 bits, giving us the output we expect from the C code in Fig. 2.2.

2.3.2 Testing, Setting, Clearing, and Toggling Bits

A common operation on unsigned integers is the setting, clearing, and testing of particular bits. Setting a bit means to turn that bit on (1), while clearing is to turn the bit off (0). Testing a bit returns its current state, 0 or 1, without changing its value. In the embedded computing world, this is often done to set an output line high or low or to read the state of an input line. For example, a microcontroller uses specific pins on the device as digital inputs or outputs. Typically, this requires setting bits in a control register or reading bits in a register to know whether a voltage is present.

The bitwise logical operators introduced above are the key to working with bits. As Fig. 2.3 already alluded to, the AND operator can mask bits. Masking sets certain bits to zero. Let’s work through some examples.

First, consider a situation similar to the Python code of Fig. 2.3 that keeps the lower nibble of a byte while clearing the upper nibble. To do this, we AND the byte with 0F_{16} ,

$$\begin{array}{r}
 10111101 = \text{BD} \\
 \text{AND } 00001111 = 0\text{F} \\
 \hline
 00001101 = 0\text{D}
 \end{array}$$

where we are now displaying the binary and its hexadecimal equivalent. The output byte will have all zeros in the upper nibble because AND is only true when both operands are true. Since the second operand has zeros in the upper nibble, there is no situation where the output can be one. Likewise, the lower nibble is unaffected by the AND operation. If the two operands are one, the output bit is one, and if the second operand has a zero bit, the output is zero. In both cases, the output matches the bit of the second operand. A brief review of the AND truth table (above) will clarify why masking works.

AND can also test bits. Suppose we wish to know if bit 3 of BD_{16} is set. All we need to do is create a mask that has a one in bit position 3 and zeros in all other bit positions (recall that bit positions count from zero, right to left),

$$\begin{array}{r} 10111101 = BD \\ \text{AND } 00001000 = 08 \\ \hline 00001000 = 08 \end{array}$$

If the result of the AND is not zero, we know that the bit in position 3 of BD_{16} is set. If the bit was clear, the output of the entire operation would be exactly zero. C and Python treat nonzero as true, implying the following C code outputs “bit 3 on”:

```
if (0xBD & 0x08) {
    printf("bit 3 on\n");
}
```

Similarly, in Python,

```
if (0xBD & 0x08):
    print("bit 3 on")
```

We use AND to test bits and mask bits. We use OR to set bits. For example, to set bit 3 and leave all other bits as they are, do something like this,

$$\begin{array}{r} 10110101 = B5 \\ \text{OR } 00001000 = 08 \\ \hline 10111101 = BD \end{array}$$

which works because OR returns true when either operand or both have a one in a particular bit position. Just as AND with an operand of all ones returns the other operand, so will OR with all zeros. To set bits, then, we need OR, a bit mask, and assignment, as this C code demonstrates

```
#include <stdio.h>

int main() {
    unsigned char z = 0xB5;

    z = z | 0x08; // set bit 3
    printf("%x\n", z);
    z |= 0x40;    // set bit 6
    printf("%x\n", z);

    return 0;
}
```

The code outputs BD_{16} and FD_{16} after setting bits 3 and 6. Since we updated z each time, the set operations are cumulative.

We've seen how to test and set bits. Now, let's look at clearing bits without affecting other bits. We know that if we AND with a mask of all ones, we get back our first operand. So, if we AND with a mask of all ones with a zero in the bit position we want to clear, we will turn that bit off and leave all other bits as they were. For example, to turn off bit 3 do something like this

$$\begin{array}{r} 10111101 = BD \\ \text{AND } 11110111 = F7 \\ \hline 10110101 = B5 \end{array}$$

where the mask value for AND is $F7_{16}$. But, this begs the question of how we get the magic $F7_{16}$ in the first place? Here is where NOT comes into play. We know that to set bit 3, we need a mask with only bit 3 on and all other bits off. This mask is easy to calculate because the bit positions are simply powers of two. Therefore, the third position is $2^3 = 8$ implying that the mask is $8 = 00001000_2$. Applying NOT to this mask inverts all the bits, giving us the mask we need: 11110111_2 . This is readily accomplished in C,

```
unsigned char z = 0xBD;
z = z & (~0x08);
printf("%x\n", z);
```

The snippet outputs $B5_{16} = 10110101_2$ with bit 3 clear. The same syntax also works in Python.

Our next example toggles bits. When a bit is toggled, its value changes from off to on or on to off. We can use NOT to quickly toggle all the bits since this is what NOT does, but how do we toggle only 1 bit, leaving the rest unchanged? The answer lies in using XOR. The XOR operator returns true when the operands are different. If both are zero or one, the output is zero. This is what we need. We XOR the value using a mask with a one in the bit position we wish to toggle. For example, to toggle bit 1,

$$\begin{array}{r}
 10111101 = \text{BD} \\
 \text{XOR } 00000010 = 02 \\
 \hline
 10111111 = \text{BF}
 \end{array}$$

If bit 1 was already on we would have $1 \text{ XOR } 1 = 0$ which would turn it off. We toggle multiple bits by setting the positions we want to toggle in the mask before applying XOR. Consider,

```
unsigned char z = 0xBD;
z = z ^ 0x03;
printf("%x\n", z);
```

which will output $\text{BE}_{16} = 10111110_2$ as expected because $0x03$ toggled bits 0 and 1 from 01_2 to 10_2 .

2.3.3 Shifts and Rotates

So far, we have examined operations that manipulate bits more or less independently of other bits. Now, we look at sliding bits within the same value from one position to another. These manipulations are accomplished via the shift and rotate operators. A *shift* is as straightforward as it sounds; just move bits from lower positions in the value to higher if shifting to the left or from higher positions to lower if shifting to the right. When shifting, bits simply “fall off” the left or right edge of the integer as needed. Falling off implies that we impose a specific number of bits on the integer. We will stick with 8-bit unsigned integers for our examples, though these operations work equally well on integers of any size. Let’s consider what happens when we shift a value to the left 1 bit position using binary notation,

$$10101111 \leftarrow 1 = 01011110$$

The $\leftarrow$ symbol means a shift to the left, and the 1 is the number of bit positions to shift.

In the example, the leading 1 drops off the left end, and a zero moves in from the right to fill the empty position. All other bits move up one position. Now, consider what happens when only a single bit is set, and we shift one position to the left,

$$00000010 \leftarrow 1 = 00000100$$

we see that we started with a value of $2^1 = 2$ and we ended with a value of $2^2 = 4$. Therefore, a single position shift to the left will move each bit to the next highest bit position, the same as multiplying by two. As this happens to all bits, the net effect is

to multiply the number by two. Of course, if bits that were set fall off the left end, we will lose precision, but the remaining bits will be multiplied by two. For example,

$$00101110 \leftarrow 1 = 01011100$$

takes $00101110_2 = 46$ to $01011100_2 = 92$ which is 46×2 . Shifting to the left by more than 1-bit position is equivalent to repeated shifts by one position, so shifting by two positions will multiply the number by $2 \times 2 = 4$,

$$00101110 \leftarrow 2 = 10111000$$

giving $10111000_2 = 184$ as expected.

It is natural to think that if a left shift multiplies by two, a right shift will divide by two, and this is indeed the case,

$$00101110 \rightarrow 1 = 00010111$$

gives $00010111_2 = 23$ which is $46 \div 2$. Just as bits falling off the left end of the number will result in a loss of precision, so will bits falling off the right end with one significant difference. If a bit falls off the right end of the number, it is lost from the ones position. If the ones position is set, the number is odd and not evenly divisible by two. If the bit is lost, the result is still the number divided by two, but the division is *integer division*, which ignores any remainder. Information is lost since a right shift followed by a left shift results in a number that is one less than what we started with if the original number was odd. We can see this by shifting two positions to the right,

$$00101110 \rightarrow 2 = 00001011$$

to get $00001011_2 = 11$ which is $46 \div 4$ ignoring the remainder of 2.

Both C and Python support shifts using the $<<$ and $>>$ operators for left and right shifts, respectively. The $<<$ operator is frequently used with an argument of 1 to build bit masks,

$$\begin{aligned} 1 << 3 &= 00001000_2 \\ 1 << 5 &= 00100000_2 \end{aligned}$$

Before we leave shifts, let's return to the parity calculation mentioned above. Recall that the parity of a number is determined by the number of 1 bits in its binary representation. If odd, the parity is 1; otherwise, it is 0. The critical observation here is that XOR preserves the parity of its arguments. For example, if, in binary, $a = 1101$ and $b = 0111$, the parity of the two is zero since there are six 1 bits. If we apply XOR, we get $a \text{ XOR } b = 1010$, which also has an even number of 1 bits and therefore has the same parity. This suggests the trick. If we XOR a number with itself but first shift the number half its bit width, the resulting bits of the lower half of the output of XOR will have the same parity as the original number. We can see

this if we look at $a = 01011101$ and XOR it with itself after shifting down by 4 bits, which is half the width of the number,

$$\begin{array}{r} 01011101 \\ \text{XOR } 00000101 \\ \hline \text{xxxx1000} \end{array}$$

where we are ignoring the upper 4 bits. The original number has five 1 bits; therefore, the parity is 1. If we look at the lower 4 bits after the XOR we see it also has a parity of 1. If we repeat the process but use the new value, this time shifting by half its effective width, which is 4 bits, we end up with a number with the same parity as we started with and the same parity as the original. If we repeat this all the way, the final result will be a 1-bit number that is the parity of the original number. Therefore, for 01011101,

$$\begin{array}{rll} \text{original} & 01011101 & \\ \text{right shift 4} & 00000101 & \\ \hline \text{XOR} & 01011000 & \\ \text{right shift 2} & 00010110 & \\ \hline \text{XOR} & 01001110 & \\ \text{right shift 1} & 00100111 & \\ \hline \text{XOR} & 01101001 & \\ & \text{parity bit} & \text{xxxxxxxx1} \end{array}$$

For an 8-bit number, we can define a parity function in C as

```
unsigned char parity(unsigned char x) {
    x = (x >> 4) ^ x;
    x = (x >> 2) ^ x;
    x = (x >> 1) ^ x;
    return x & 1;
}
```

where the `return x & 1;` line returns only the lowest order bit since all other bits are meaningless at this point. The extension of this function to wider integers is straightforward.

Shifting moves bits left or right and drops them at the ends. This is not the only option. Instead of dropping the bits, we might want to move the bits that would have fallen off to the opposite end. This is a *rotation*, which, like a shift, can be to the left or right. Unfortunately, while many microprocessors contain rotate instructions as primitive machine language operations, neither C nor Python support rotation directly. However, rotations can be simulated in code, and an intelligent compiler will even turn the simulation into a single machine instruction (e.g., `gcc`).

Again, to keep things simple, we will work with 8-bit numbers. For an 8-bit number, say $AB_{16} = 10101011_2$, a rotation one bit to the right looks like this,

$$10101011 \Rightarrow 1 = 11010101$$

We introduce the $\Rightarrow$ symbol to mean rotation to the right instead of simple shifting ($\rightarrow$). Rotation to the left 1-bit position gives

$$10101011 \Leftarrow 1 = 01010111$$

The bit that would have fallen off at the left end has now moved around to the right.

To simulate rotation operators in C and Python, we use a combination of $<<$ and $>>$ with an OR operator. In this case, we must pay attention to the number of bits in the data type. For example, in C, we can define rotations to the right by any number of bits with

```
unsigned char rotr(unsigned char x, int n) {
    return (x >> n) | (x << 8 - n);
}
```

with a similar definition for rotations to the left,

```
unsigned char rotl(unsigned char x, int n) {
    return (x << n) | (x >> 8 - n);
}
```

One thing to note is the 8 in the second line of these functions. This number represents the number of bits used to store the data value. Since we have declared the argument x to be of type `unsigned char` we know it uses 8 bits. To modify the functions for 32-bit integers, replace 8 with 32 or, in general, use `sizeof(x) * 8` to convert the byte size to bits.

The Python versions of the C rotation functions are similar

```
def rotr(x,s):
    return ((x >> s) | (x << 8 - s)) & 0xff
```

and

```
def rotl(x,s):
    return ((x << s) | (x >> 8 - s)) & 0xff
```

where the only change is that the return value is AND'ed with FF_{16} to keep the lowest 8 bits of the result. To make the functions support 16-bit integers, replace 8 with 16, as in the C versions, but also make the mask keep the lowest 16 bits by replacing 0xff with 0xffff .

The rotation functions are helpful, but why do they work? Let's contemplate the operations individually and then combine them to produce the final result. We start with the original input, $\text{AB}_{16} = 10101011_2$, and show, row by row, the first shift to the left, the second shift to the right, and finally the OR to combine them. The values between the vertical lines are those that fit within the 8-bit range of the number; the other bits are lost in the shift operations,

$$\begin{array}{r|l}
 10101011 & \text{AB}_{16} \\
 01010101 & 1 \quad \text{AB}_{16} >> 1 \\
 1010101 & 10000000 \quad \text{AB}_{16} << 8 - 1 \\
 11010101 & \text{OR}
 \end{array}$$

Notice that the bit lost when shifting to the right one position has been added back at the front by the left shift and OR operation. Since shifts always introduce a zero bit, the OR will always set the output bit to the proper value because $1 \text{ OR } 0 = 1$ and $0 \text{ OR } 0 = 0$. This works for any number of bits to rotate,

$$\begin{array}{r|l}
 10101011 & \text{AB}_{16} \\
 00101010 & 11 \quad \text{AB}_{16} >> 2 \\
 1010101 & 11000000 \quad \text{AB}_{16} << 8 - 2 \\
 11101010 & \text{OR}
 \end{array}$$

2.3.4 Comparisons

Magnitude comparison operators take two unsigned integers and return a truth value about whether the operator's implied relationship holds for the operands. Here we look at the basic three, equality ($A = B$), greater than ($A > B$), and less than ($A < B$). There is a second set easily created from the first with NOT and AND operators: not equal ($A \neq B$), greater than or equal ($A \geq B$), and less than or equal ($A \leq B$).

At its core, a computer uses digital logic circuits to compare bits. Figure 2.4 illustrates the layout of a 1-bit comparator. Comparators for larger numbers of bits are repeated instances of this basic pattern. As this is not a book on digital logic, we will go no further down this avenue but will talk about comparing unsigned integers from a more abstract point of view.

Most microprocessors have primitive instructions for comparing integers, as this operation is fundamental to computing. Besides direct comparison, many instructions affect processor flags based on privileged numbers like zero. For example, the 8-bit 6502 microprocessor, which has a single accumulator, A, for arithmetic, performs comparisons with the CMP instruction but also sets processor status flags whenever a value is loaded from memory using the LDA instruction. The 6502 uses branch

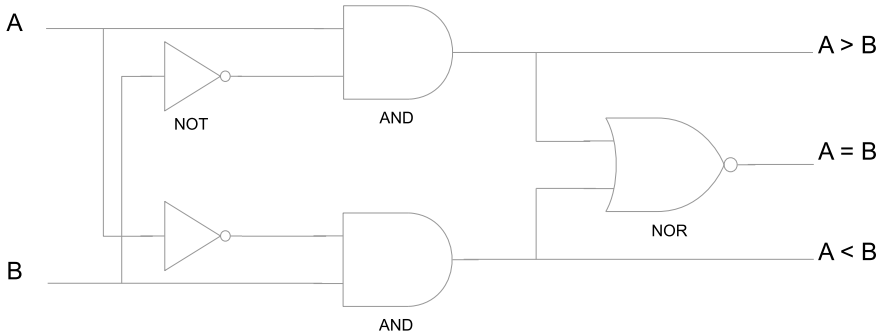


Fig.2.4 A 1-bit digital comparator. The three output values represent the three possible relationships between the input values A and B. The output is 1 for the true relationship and 0 for those not met. Cascades of this basic form can be used to create multi-bit comparators

instructions like BEQ and BNE to branch on equal or not equal, respectively. This also applies to the implied comparison with the special value 0, which happens when the LDA instruction loads the accumulator.

Armed with this brief review of an old 8-bit microprocessor, we can see that the following set of instructions would indeed perform an 8-bit comparison between 22_{16} already in the accumulator via the first LDA instruction and $1A_{16}$ stored in memory location 2035_{16} and branch if they are not equal. Additionally, we also perform an implicit comparison of the value of memory location $20FE_{16}$ with zero and branch if it is

```
LDA #$22      ; A = $22
CMP $2035     ; compare to location $2035
BNE noteql    ; branch to "noteql" if not equal

LDA #$20FE    ; A = contents of $20FE
BEQ iszero    ; branch to "iszero" if zero
```

where we use the classic notation of $\$22 = 22_{16}$.

Why introduce this example? Partly to show that comparison is fundamental to computers and performed as efficiently as possible in hardware and partly to set the stage for our less efficient alternatives to digital logic. The comparisons we are implementing in code are pure hardware, even in the simplest of microprocessors.

Since for any two integers A and B precisely one of the following is true: $A = B$, $A < B$, or $A > B$, it follows that if we know how to test for any two the last condition comes along for free: it is true when neither of the other two applies. In our case, consider the situation where we know how to test for equality ($A = B$) and greater

than $(A > B)$. We do this with two predicate functions that return 1 if the relationship holds for the arguments and 0 otherwise. Let's call these predicates `isZero(A)` and `isGreater(A, B)` and consider how we might implement them directly in C for 8-bit values using the `unsigned char` data type.

Why choose to define `isZero(A)` instead of the perhaps more obvious `isEqual(A, B)`? Given our experience with the XOR operator, we know that

$$a \text{ XOR } a \rightarrow 0$$

implying that

$$\text{isEqual}(A, B) = \text{isZero}(A \text{ XOR } B)$$

but how do we implement `isZero(A)`? One approach in code would be to shift the bits and test the lowest order one. If we find a test that is not zero, then the number is not zero. The test is via an OR, which is only zero when both operands are zero. We use AND to make the bit comparison and then shift the result down so that the compared bit is in the lowest position. Then, the OR of all these tests will be 1 if any bits are set and 0 if not. This is the exact opposite of what we want, so we add a NOT to reverse the sense of the logic and a final AND with 1 to mask out all other bits and return the state of the lowest bit only. Here, then, is the full predicate function `isZero(x)`,

```
unsigned char isZero(unsigned char x) {
    unsigned char ans;
    return ~(((x & (1<<7)) >> 7) |    // test bit 7
             ((x & (1<<6)) >> 6) |
             ((x & (1<<5)) >> 5) |
             ((x & (1<<4)) >> 4) |
             ((x & (1<<3)) >> 3) |
             ((x & (1<<2)) >> 2) |
             ((x & (1<<1)) >> 1) |
             (x & 1)) & 1;           // test bit 1
}
```

This function has no actual comparison operators, only logical bitwise operators. With this, we can immediately implement `isEqual(x, y)`,

```

unsigned char isEqual(unsigned char x,
                      unsigned char y) {
    return isZero(x ^ y);
}

```

We now need `isGreater(A,B)`, which is implemented with bit operators, shifts, and a call to `isZero(x)`. The C code is given first, followed by an explanation of why it works,

```

1 unsigned char isGreater(unsigned char a,
2                          unsigned char b) {
3     unsigned char x,y;
4
5     x = ~a & b;
6     y = a & ~b;
7
8     x = x | (x >> 1);
9     x = x | (x >> 2);
10    x = x | (x >> 4);
11
12    return ~isZero(~x & y) & 1;
12}

```

To tell if `a` is greater than `b`, we need to know the first place where their respective bits do not agree. Once we know this bit position, we know that `a` is greater than `b` if at that bit position `a` has a 1 while `b` has a 0. So, we need to find the locations where the bits differ. To make the example concrete, we let `a` be $00011101_2 = 29$ and `b` be $00011010_2 = 26$. If we look at line 5, we see

$$x = \sim a \ \& \ b;$$

which sets `x` to the AND of the NOT of `a` and `b`. This leaves `x` with a 1 in all the positions where the bit in `a` is less than the same bit in `b`. For our example, this sets `x` to 00000010_2 , which tells us that the only bit position in `a` that is less than the corresponding bit position in `b` is bit 1. Likewise, line 6 asks: where are the bit positions where `a` is greater than `b`? In this case, we set `y` to 00000101_2 to indicate that in bit 0 and bit 2, the value of `a` is greater than `b`. In order to see if `a` is greater than `b` we need to find the highest bit position where the two differ and see if that bit is set. We can do this if we take the value in `x`, which tells us where `a` bits are less than `b` bits, and build a mask which is 1 for all bit positions at or below the highest bit position where `a` is first less than `b`. We do this with lines 8 through 10. This operation, which ORs the value with shifted versions of itself, duplicates the highest 1 bit among all the lower bits. In this case,

```

x                → 00000010
x = x | (x >> 1) → 00000011
x = x | (x >> 2) → 00000011
x = x | (x >> 4) → 00000011

```

where the last two steps add nothing new since $x \gg 2$ and $x \gg 4$ both result in zero, which will set no new bits. We now have a mask in x that tells us all the bit positions below the first place where the bit in a is less than the bit in b . If we NOT this mask, $\neg 00000011 \rightarrow 11111100$, we can use the new mask to tell us all the bit positions where a was not less than b . Lastly, with this mask and the value in y , which tells us where the bits in a were greater than the bits in b , we can perform one final AND, $\sim x \& y$, which will result in zero if $a \leq b$ since no bits will be set in y in the region where the bits of a were greater than those of b , or a nonzero value since at least one bit will be set in y in that region. Line 12 asks if this result is zero by passing the output to `isZero`. It then applies NOT to change the output of `isZero` since the result is zero when $a \leq b$ and not zero when $a > b$. The final AND with 1 gives us only the final bit since the NOT will change all the bits of the result of `isZero`.

We are nearly finished with our comparison operators. We have equality (`isEqual`) and greater than (`isGreater`). With these we see that `isLess` is

```

unsigned char isLess(unsigned char x,
                    unsigned char y) {
    return (!isEqual(x,y)) && (!isGreater(x,y));
}

```

which is reasonable since for any two unsigned integers A and B , if $A \not\leq B$ then $A < B$ must be true. Testing for not equal is even simpler,

```

unsigned char isNotEqual(unsigned char x,
                        unsigned char y) {
    return !isEqual(x,y);
}

```

since the only way for A to *not* not equal B is if the two are indeed equal. Less than or equal and greater than or equal follow directly from the functions already defined,

```

unsigned char isLessOrEqual(unsigned char x,
                          unsigned char y) {
    return isEqual(x,y) || isLess(x,y);
}

```

and

```
unsigned char isGreaterOrEqual(unsigned char x,
                               unsigned char y) {
    return isEqual(x,y) || isGreater(x,y);
}
```

which completes our implementation of comparison operators using only bitwise operations.

2.3.5 Arithmetic

Just as comparison is a fundamental microprocessor operation, so is arithmetic. In this section, we look at arithmetic with unsigned binary integers, first from the point of view of doing it “by hand” and then from the point of view of a simple 8-bit microprocessor. These two approaches illustrate the mechanism behind the operations. We will not, however, attempt to implement these operations in C as we did for the comparison operators, though we will use C to demonstrate overflow and underflow conditions.

The addition facts in binary are

$$\begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 0 \text{ carry } 1 \end{array}$$

From this, we see that one fact produces a carry, resulting in a two-digit number. As in decimal arithmetic, the carry is applied to the next digit to the left. Therefore, to add two unsigned 8-bit binary numbers we move right to left, bit by bit, adding and moving any carry to the next digit to the left,

$$\begin{array}{rcl} 11111 & \leftarrow & \text{carry} \\ 01101110 & \leftarrow & \text{first operand} \\ + 00110101 & \leftarrow & \text{second operand} \\ \hline 10100011 & \leftarrow & \text{answer} \end{array}$$

The carry from the second to the leftmost bit does not cause difficulty since the highest bits of each number are zero. However, what would happen if there was a carry from the leftmost bit? In mathematics, nothing special would happen; there would simply be a new leftmost bit, but with computers this is not the case. Recall that we are working with 8-bit unsigned numbers, which means all numbers fit in 8 bits in memory. If we use 8 bits for numbers, we have no place in which to put any final carry bit. This results in an *overflow* condition. The computer discards this new 9th bit and retains the lowest 8 bits. Consider,

$$\begin{array}{r}
 11111 \\
 11101110 \quad EE_{16} \\
 + 00110101 \quad 35_{16} \\
 \hline
 1\ 00100011 \quad 123_{16}
 \end{array}$$

which is stored as 23_{16} discarding the carry on the leftmost bit. The following C code demonstrates:

```
#include <stdio.h>

int main() {
    unsigned char x, y, z;

    x = 0xEE;
    y = 0x35;
    z = x + y;
    printf("%x\n", z);
}
```

Let's look at how a simple 8-bit microprocessor implements an addition, in particular, an unsigned 16-bit addition requiring two addition operations. Working again with the 6502 processor mentioned earlier, we see that an 8-bit addition between a value in memory locations 23_{16} and 25_{16} involves a load into the accumulator (LDA), a clearing of the carry flag which catches any overflow bit (CLC), and an addition with memory (ADC). Specifically, we assume memory location 23_{16} contains EE_{16} and memory location 25_{16} contains 35_{16} . We then load the accumulator, clear the carry, and add

LDA \$23	$A \leftarrow EE$
CLC	$C \leftarrow 0$
ADC \$25	$A \leftarrow A + 35 + C$
	$A \leftarrow 23, C \leftarrow 1$

with the accumulator set to 23_{16} , the lowest 8 bits of the sum of EE_{16} and 35_{16} , and the carry flag set to 1 to indicate an overflow occurred. Setting the carry flag is the key to implementing multibyte addition. In C, we declare the variables as data type `unsigned short`, i.e., 16 bits, and add as before. For example, in C, we have

```
#include <stdio.h>

int main() {
    unsigned short x, y, z;

    x = 0xEE;
    y = 0x35;
    z = x + y;
    printf("%x\n", z);
}
```

The code gives us a 16-bit answer of 123_{16} . In memory, using little-endian representation for multibyte numbers, we have

memory location \$24 : 23
\$25 : 01

since we store the lowest byte first.

In the simpler world of the 8-bit microprocessor, we store the lowest part of the sum, the low byte, and add the high bytes without clearing the carry. Assuming memory is set to

memory location \$23 : EE
\$24 : 00
\$25 : 35
\$26 : 00

we clear the carry flag, add the two low bytes, store the partial sum, add the high bytes with any carry, and store the final part of the sum like this

LDA \$23	$A \leftarrow EE$
CLC	$C \leftarrow 0$
ADC \$25	$A \leftarrow A + 35 + C, A = 23$
STA \$27	$\$27 \leftarrow 23, C \leftarrow 1$
LDA \$24	$A \leftarrow 0$
ADC \$26	$A \leftarrow A + 0 + C, A = 1$
STA \$28	$\$28 \leftarrow 1$

Notice the introduction of a new instruction, *STA*, which stores the accumulator in memory. When this sequence of instructions is complete, we have the final answer in memory locations \$27 and \$28 as 23_{16} and 1_{16} , respectively, as expected of a little-endian number.

The addition above is equivalent to this single 16-bit addition,

$$\begin{array}{r}
 1\ 11111 \\
 00000000\ 11101110\ EE_{16} \\
 +\ 00000000\ 00110101\ 35_{16} \\
 \hline
 00000001\ 00100011\ 123_{16}
 \end{array}$$

where we have separated the upper 8 bits of the high byte from the lower 8 bits of the low byte.

The addition of unsigned binary numbers is straightforward. We add, left to right, bit by bit, using a carry bit when necessary. If the result is too large, we overflow and retain the lowest n bits where n is the width of the data type in bits. We now move to the subtraction of unsigned binary numbers.

The subtraction facts in binary are

$$\begin{array}{l}
 0 - 0 = 0 \\
 0 - 1 = 1, \text{ underflow} \\
 1 - 0 = 1 \\
 1 - 1 = 0
 \end{array}$$

where the *underflow* condition will require a borrow from the next higher bit position. Like overflow, underflow is a situation where we cannot correctly represent the number using the number of bits we have to work with. In this case, the underflow happens when we attempt to subtract a larger number from a smaller one, and we cannot represent the resulting negative number because we are assuming unsigned numbers. We'll address the issue of negative numbers later in the chapter.

To continue the example we used for addition, we now evaluate $EE_{16} - 35_{16}$ to get $B9_{16}$. In binary, using the subtraction facts, we have

$$\begin{array}{r}
 01 \\
 01\ 01 \\
 11101110\ EE_{16} \\
 -\ 00110101\ 35_{16} \\
 \hline
 10111001\ B9_{16}
 \end{array}$$

with each borrow written above the bit and the next bit set to one less than it was. If a second borrow is necessary for a bit position, we write it above a second time. Let's look at the subtraction again, bit by bit, right to left,

<i>bit 0</i>	$10 - 1 = 1, \text{ borrow}$
<i>bit 1</i>	$0 - 0 = 0$
<i>bit 2</i>	$1 - 1 = 0$
<i>bit 3</i>	$1 - 0 = 1$
<i>bit 4</i>	$10 - 1 = 1, \text{ borrow}$
<i>bit 5</i>	$10 - 1 = 1, \text{ borrow}$
<i>bit 6</i>	$0 - 0 = 0$
<i>bit 7</i>	$1 - 0 = 1$

Reading from bottom to top gives $10111001_2 = B9_{16}$, as expected.

What happens if we need to borrow across more than one bit position? For example, in base 10, a problem like

$$\begin{array}{r} 7003 \\ - 972 \\ \hline 6031 \end{array}$$

involves borrowing across two digits in order to subtract the 7 of 972 which we can be written as

$$\begin{array}{r} 69^103 \\ - 972 \\ \hline 6031 \end{array}$$

where we change the 700 into 69^10 to subtract 9 7 giving the partial result 603. We subtracted one from the next two digits and added it as a ten to the digit we were working with. The same thing happens in binary. Consider this subtraction problem,

$$\begin{array}{r} 01 \\ 1010\cancel{1}0^101 \text{ A9}_{16} \\ - 101001 \text{ 10 A6}_{16} \\ \hline 00000011 \text{ 03}_{16} \end{array}$$

where we attempt to subtract 1 from 0 in the second bit position (bit 1). We must borrow from bit 2, but since this is also zero, we instead borrow from bit 3 and change 100 into 01^10 to complete the subtraction.

As we are working with 8-bit unsigned integers, one will eventually be tempted to ask what happens if we try to subtract 1 from 0 since we cannot represent -1 . We arrive at the result we would get if we had an extra bit at the leftmost position and borrowed from it. Consider,

$$\begin{array}{r} 0 \ 1111111 \\ \pm \ 0000000^10 \\ - \quad 0000000 \ 1 \\ \hline 11111111 \end{array}$$

Subtracting one from the smallest number we can represent, namely, zero, results in the largest number we can represent, which, for an 8-bit unsigned integer, is every bit set, $2^8 - 1 = 255$. Another way to think about it is that the numbers form a loop with 00000000 and 11111111 set next to each other. If we move one position below zero, we wrap around to the top and get 255. Likewise, if we move up one position from 255, we wrap around to the bottom and get zero. Moving below zero is an underflow condition, while moving above 255 is an overflow.

Before we contemplate the multiplication and division of unsigned integers, let's look at one more subtraction example, one that would result in a negative number. We'll use our running example but swap the operands,

$$\begin{array}{r} 1\ 00110101\ 35_{16} \\ -\quad 11101110\ EE_{16} \\ \hline 01000111\ 47_{16} \end{array}$$

where we have indicated the implied 1 bit from which we can borrow. This implied 1 bit is in the bit position for $2^8 = 256$, which suggests another way to think about negative answers when subtracting unsigned numbers. For our example, $35_{16} - EE_{16} = -185$, but if we add 256, which is essentially what we are doing when thinking there is an implied extra high bit, we get $-185 + 256 = 71 = 47_{16}$ which is the answer we found previously.

We have been working with 8-bit wide unsigned integers. If we are using 16-bit integers, the implied bit 16 (recall, we count bits from zero) is $2^{16} = 65536$, meaning we add 65536 to any negative value to get the result we expect from unsigned subtraction.

The following C code demonstrates all that we have been discussing:

```
#include <stdio.h>

int main() {
    unsigned char x=0xEE, y=0x35, z;

    z = x - y;
    printf("%X\n", z);

    z = y - x;
    printf("%X\n", z);

    z = 0 - 1;
    printf("%X\n", z);
}
```

The output is

$$\begin{aligned} B9_{16} &= 10111001_2 \\ 47_{16} &= 01001110_2 \\ FF_{16} &= 11111111_2 \end{aligned}$$

which is what we saw in the examples above.

Let's examine a useful trick involving AND. If 1 bit in a number is set, this implies that the number is a power of two since every position in a binary number is, by

definition, a power of two. If we know which bit we want to test, using a mask and checking that bit is straightforward. But what if we wanted to know if the number in question was any power of two? We can use AND here as well. If the number we want to test is $00100000_2 = 32$, we notice that it is a power of two, and only one bit is on. Now, subtract one from this number to get $00011111_2 = 31$. What has happened is that the single bit that was on is now off, and some of the off bits are now on. Finally, what happens if we AND these two values together? We get

$$\begin{array}{r} 00100000 = 32 \\ \text{AND } 00011111 = 31 \\ \hline 00000000 = 00 \end{array}$$

which is precisely zero. From this, we see that we will only get zero when one of two conditions is met: either the number is itself zero, or it is a power of two with only 1 bit set. This is nicely captured in a simple C function,

```
unsigned char is_power_of_two(unsigned char n) {
    return (n == 0) ? 0
           : (n & (n-1)) == 0;
}
```

which returns 1 if the argument is a power of two and 0 otherwise. The function checks if the argument is zero and, if so, returns 0. If not, then check whether $n \& (n - 1)$ is exactly 0. If it is, the expression is true, and the function returns 1 to indicate a power of two. Otherwise, it returns 0. While written for an unsigned char data type, the function will work for any unsigned integer type.

We've looked in detail at addition and subtraction. Now, we turn our attention to multiplication and division. Modern microprocessors perform multiplication and division as operations in hardware. This is a very good thing but makes it difficult for us in a sense, so we will, as before, look at more primitive approaches that might be used in small 8-bit microcontrollers that lack hardware instructions for multiplication and division. We will illustrate the algorithms in C to keep things simple, even though this would never be done in practice.

Integer multiplication is merely repeated addition, so one approach to finding $n \times m$, where n and m are unsigned integers, would be to add n to itself m times or vice versa. Naturally, it would make sense to run the loop as few times as possible, implying a loop over the smaller of n or m adding the other number. In C, we have

```
unsigned short mult1(unsigned char n, unsigned char m) {
    unsigned char i;
    unsigned short ans = 0;

    if (n < m) {
        for(i=0; i < n; i++)
```

```

        ans += m;
    } else {
        for(i=0; i < m; i++)
            ans += n;
    }

    return ans;
}

```

which leaves the product of 8-bit numbers n and m in the now possibly 16-bit value p . Why is the product possibly 16 bits? Because the largest product of two 8-bit numbers requires 16 bits to store. This is because $255 \times 255 = 65025$ which is greater than $2^8 - 1 = 255$ meaning it needs more than 8 bits to store but is less than $2^{16} - 1 = 65535$ which is the maximum for a 16-bit unsigned integer.

Is this a good way to multiply numbers, however? Probably not. The loop needs to be repeated for the smaller of n or m which may be up to 255 times. Given that we must do a 16-bit addition inside the loop, we understand that the multiplication may turn into thousands of individual machine instructions. Surely we can do better than this? To answer this question, let's look a little more closely at multiplication in binary as we might do it by hand

$$\begin{array}{r}
 \begin{array}{r}
 00010100 \quad 14_{16} = 20 \\
 \times 00001110 \quad 0E_{16} = 14 \\
 \hline
 00000000 \\
 00010100 \\
 00010100 \\
 + 00010100 \\
 \hline
 00100011000 \quad 118_{16} = 280
 \end{array}
 \end{array}$$

We observe that if the binary digit in the multiplier is 0, we simply copy down all zeros, and if it is a 1, we copy the multiplicand lining it up beneath the multiplier bit as we would do in decimal. Then, again as in decimal multiplication, we add all the partial products to arrive at the final answer. For simplicity we did not write the leading zeros which would be present if showing all 16 bits of the result.

The method suggests a possible improvement to `mult1`. Rather than repeatedly adding the multiplier or multiplicand, we instead copy the algorithm above by shifting the multiplicand into position and adding it to the partial product if the multiplier bit is 1, otherwise ignore those that are 0. This leads to a second multiplication function in C

```

unsigned short mult2(unsigned char n, unsigned char m) {
    unsigned char i;
    unsigned short ans = 0;

    for(i=0; i < 8; i++) {

```

```
        if (m & 1) {
            ans += n << i;
        }
        m >>= 1;
    }

    return ans;
}
```

which, when compared to `mult1` and run ten million times, proves to be about 1.6x faster. Let's look at what `mult2` is doing.

We are multiplying two 8-bit numbers, so we must consider each bit in the multiplier, `m`. This is the source of the `for` loop. The `if` statement ANDs the multiplier with 1 to extract the lowest bit. If this bit is set, we want to add the multiplicand, `n`, to the partial product stored in `ans`. Note, though, that before we add the multiplicand, we must shift it to the proper bit position. Note also that this works because the result of the operation is a 16-bit value, which will not lose any of the bits of `n` when we do the shift. Regardless of whether we add anything to the partial product, we need to shift the multiplier down 1 bit so that in the next pass through the loop, the `if` will examine at the next highest bit of the original `m`. Lastly, we see no side effects to this function because C passes all arguments by value, meaning `n` and `m` are copies local to the function.

The speed improvement between `mult1` and `mult2` becomes much more dramatic when we move from multiplying 8-bit numbers to multiplying 16-bit numbers. To do this, we take the source for `mult1` and `mult2` and replace all instances of `unsigned char` by `unsigned short` and all instances of `unsigned short` by `unsigned int`. Lastly, we change the loop limit in `mult2` from 8 to 16. Doing this makes `mult2` nearly 3500x faster than `mult1` for the same arguments (assuming both to be near the limit of 65535).

What about division? We'll discuss two division-related operations: integer division (`/`), which returns the quotient, and modulo (`%`), which returns the remainder. For example, we need an algorithm that produces these answers

$$\begin{aligned} 123 / 4 &= 30 \\ 123 \% 4 &= 3 \end{aligned}$$

since $123/4 = 30$ with a remainder of 3.

We might implement division via repeated subtraction. If we count the number of times we can subtract the divisor from the dividend before we get a partial result that is less than the divisor, we will have the quotient and the remainder. We might code this in C as

```

unsigned char div1(unsigned char n,
                  unsigned char m,
                  unsigned char *r) {
    unsigned char q=0;

    *r = n;

    while (*r > m) {
        q++;
        *r -= m;
    }

    return q;
}

```

and test it with

```

int main() {
    unsigned char n=123, m=7;
    unsigned char q,r;

    q = div1(n, m, &r);
    printf("quotient=%d, remainder=%d\n", q,r);
}

```

which prints `quotient = 30, remainder = 3`, as expected.

The function requires three arguments since we want to return the quotient as the function value and the remainder as a side effect. This is why we pass the remainder as a third argument using a pointer. Inside of `div1`, we set the remainder (`r`) to the dividend and continually subtract the divisor (`m`) until arriving at a result less than the divisor. While doing this, we keep a count of the number of times we subtract in `q`, which we return as the quotient.

Like the `mult1` example above, `div1` is an inefficient way to implement division. What happens if the dividend is large and the divisor is small? We must loop many times in that case. The problem is even worse if we use integers larger than 8 bits. What to do, then?

Let's look at binary division by hand. Unlike decimal long division, binary division is rather simple. Either the divisor is less than or equal to the dividend, in which case the quotient bit is 1, or the quotient bit is 0; there are no trial multiplications. Therefore, dividing $123 = 01111011_2$ by $4 = 100_2$ proceeds as so

Fig. 2.5 Shift, test, and restore unsigned integer division

```

1| unsigned char div2(unsigned char n,
2|                     unsigned char m,
3|                     unsigned char *r) {
4|     unsigned char i;
5|
6|     *r = 0;
7|
8|     for(i=0; i<8; i++) {
9|         *r = (*r << 1) + ((n & 0x80) != 0);
10|        n <<= 1;
11|
12|        if ((*r-m) >= 0) {
13|            n |= 1;
14|            *r -= m;
15|        }
16|    }
17|
18|    return n;
19| }
```

$$\begin{array}{r}
 00011110 \\
 100 \overline{) 01111011} \\
 \underline{0} \\
 01 \\
 \underline{0} \\
 011 \\
 \underline{0} \\
 0111 \\
 \underline{100} \\
 0111 \\
 \underline{100} \\
 0110 \\
 \underline{100} \\
 0101 \\
 \underline{100} \\
 11 \\
 \underline{0} \\
 11
 \end{array}$$

with $00011110_2 = 30$ and a remainder of $11_2 = 3$, as expected.

Modern microprocessors implement division in hardware. However, we can consider unsigned division as it might be implemented in a more primitive microprocessor or microcontroller. This leads to Fig. 2.5, which requires some explanation.

The key to understanding Fig. 2.5 is to observe that binary division by hand is really a matter of testing whether or not we can subtract the divisor from the partial dividend. If so, we set a one in that bit of the quotient. Otherwise, we set a zero. The algorithm of Fig. 2.5 is configured for 8-bit division using 8-bit dividends. Therefore, we must examine all 8 bits of the dividend, starting with the highest bit.

We take advantage of the fact that C passes arguments by value to use *n* as dividend and quotient. After examining all 8 bits, the value in *n* becomes the quotient. As we look at each bit of the dividend, we shift it to the left while shifting in the new bit of the quotient from the right.

We store the remainder in *r* and pass it out of the function via a pointer. To start the division process, we set *r* to zero and the quotient to the dividend in *n*. Since *n* already has the dividend, there is no explicit operation to do this, we get it for free. For example, if we call `div2(123, 4, &r)`, the state of affairs in binary after the execution of line 6 in Fig. 2.5 is

<i>i</i>	<i>r</i>	<i>n</i>	<i>m</i>
<i>undefined</i>	00000000	01111011	100

where the dividend is in *n*, and the remainder is zero. We next hit the loop, beginning in line 8. This loop executes eight times, once for each bit of the dividend. Lines 9 and 10 perform a double left shift. This is the equivalent of treating *r* and *n* as a single 16-bit variable with *r* the high-order bits. First we shift *r* one bit to the left (`*r << 1`) and then comes the rather cryptic expression,

$$((n \& 0x80) != 0)$$

which tests whether the highest bit in *n* is set. Recall our discussion of AND above. If the high bit is set, we add it into *r* because we are about to left shift *n* 1 bit, and if the bit we are shifting out of *n* is set, we need to move it to *r* to complete the virtual 16-bit shift of *r* and *n*. Next, we shift *n* in line 10.

Line 12 checks whether or not we can subtract the divisor in *m* from the partial dividend, which is being built, bit by bit, in *r*. If we can, we set the first bit of *n*, our quotient, in line 13 and then update the partial dividend by subtracting the divisor in line 14. Recall that we are examining the dividend 1 bit at a time by moving it into *r*. We are simultaneously storing the quotient in *n* 1 bit at a time by putting it on the right side. Since we have already shifted to the left, the first bit in *n* is always zero; we only update it if the subtraction succeeds. After this first pass through the loop, we have

<i>i</i>	<i>r</i>	<i>n</i>	<i>m</i>
0	00000000	11110110	100

with the first bit of the quotient, a zero, in the first bit of *n*. Continuing through the loop generates this sequence of values

i	r	n	m
1	00000001	11101100	100
2	00000011	11011000	100
3	00000011	10110001	100
4	00000011	01100011	100
5	00000010	11000111	100
6	00000001	10001111	100
7	00000011	00011110	100

We end with a quotient of 30 in `n`, which is the return value of the function, and a remainder of 3 in `r`. Notice how `n` changes as we move through the loop. Each binary digit is shifted into `r` from the right as the new quotient bits are assigned from the left until all bits are examined. Unlike the `div1` example, this algorithm operates in constant time. There are a fixed number of operations needed regardless of the input values.

2.3.6 Square Roots

Here, we briefly consider a simple integer square root algorithm that uses an interesting mathematical fact. The algorithm works by counting the number of times an ever-increasing odd number can be subtracted before reaching or going below zero. The algorithm itself is easy to implement in C,

```
unsigned char sqr(unsigned char n) {
    unsigned char c=0, p=1;

    while (n >= p) {
        n -= p;
        p += 2;
        c++;
    }
    return c;
}
```

Notice that we again use the fact that C passes arguments by value, allowing modification of `n` inside the function. Our count, the square root of `n`, is initialized to zero in `c`. We start our odd number in `p` at 1 and then move to 3, 5, 7, and so on. The `while` loop is checking to see if our `n` value is still larger or the same as `p` and if so, we subtract `p` and count one more subtraction in `c`. When `n` is less than `p`, we are done counting and return `c` as the square root. The algorithm underestimates the square root when `n` is not a perfect square.

If we call `sqr` with 64 as the argument, we get the following sequence of values in the `while` loop:

<code>n</code>	<code>p</code>	<code>c</code>
63	3	1
60	5	2
55	7	3
48	9	4
39	11	5
28	13	6
15	15	7
0	17	8

where the final value of `n` is zero since 64 is a perfect square and `c` is 8, which is the square root of 64.

The algorithm works, but why? Because the sum of the sequence of odd numbers is always a perfect square. For example,

$$\begin{aligned}
 1 &= 1 \\
 1 + 3 &= 4 \\
 1 + 3 + 5 &= 9 \\
 1 + 3 + 5 + 7 &= 16 \\
 1 + 3 + 5 + 7 + 9 &= 25
 \end{aligned}$$

or more compactly

$$\sum_{i=1}^n 2i - 1 = n^2$$

where n^2 is the argument to `sqr` and n is the square root.

Another way to see this, courtesy of Eric Spellman, is to consider the difference between n^2 and $(n+1)^2$. The latter is $n^2 + 2n + 1$, which implies that the difference between a squared integer and the next integer squared is $2n + 1$, which is always an odd number.

For example, we can apply this same observation to computing the unsigned cube root of an integer. In this case, we have n^3 and $(n+1)^3 = n^3 + 3n^2 + 3n + 1$ implying that the difference between cubes is $3n^2 + 3n + 1$. With this in mind a simple modification of the square root code now returns the cube root,

```

unsigned int cbroot(unsigned int n) {
    unsigned int c=0, p=1;

    while (n >= p) {
        n -= p;
        c++;
    }
}

```

```
        p = 3*c*c+3*c+1;
    }

    return c;
}
```

where instead of adding 2 to p every iteration, we compute $p = 3c^2 + 3c + 1$.

2.4 What About Negative Integers?

In the previous section, we examined unsigned integers and the operations computers typically perform with them. Without a doubt, unsigned integers are the mainstay of computers, but often, it is necessary to represent quantities that are less than zero. What do we do about that? This section examines three options for representing signed integers: sign-magnitude, one's complement, and two's complement.

2.4.1 Sign-Magnitude

The most natural way to track the sign of an integer is to reserve one bit of its representation for the sign. For example, we might reserve the highest order bit for the sign and use the remaining bits for the magnitude, i.e., store the number in *sign-magnitude* form. Consider these examples,

01111111	=	127
...		
00000010	=	2
00000001	=	1
00000000	=	0
10000001	=	-1
10000010	=	-2
...		
11111111	=	-127

Sign-magnitude seems a perfectly reasonable way to store a signed integer but for the following:

- We lose range in terms of magnitude. An unsigned 8-bit number can take values from 0 to 255 while a sign-magnitude number is restricted to -127 to $+127$. As we will see, keeping track of the sign always results in a loss of range.

- There are two ways to represent zero: $+0 = 00000000$ and $-0 = 10000000$. This seems unnecessary and wasteful of a bit pattern.
- Arithmetic becomes more tedious because we must always be mindful of the sign of the number, thereby complicating hardware implementations.

For these reasons, modern computer systems have abandoned the sign-magnitude representation for integers. Let's try to do better.

2.4.2 One's Complement

Our first candidate for a suitable replacement to the sign-magnitude form is *one's complement*. One's complement notation represents negative numbers by taking the positive form and calculating the one's complement. The one's complement is simple to do, merely negate (logical NOT) every bit in the positive form of the number. For example,

01111111	=	127
...		
00000010	=	2
00000001	=	1
00000000	=	0
11111110	=	-1
11111101	=	-2
...		
10000000	=	-127

One's complement means we need only examine the highest order bit to determine the sign of a number. Also, one's complement arithmetic can be implemented without too much difficulty, though there remain two ways to represent zero: 00000000 and 11111111 . We can do better still.

2.4.3 Two's Complement

The one's complement of a positive number is the bit pattern formed by changing all the zeros to ones and all the ones to zeros. The *two's complement* of a positive number is the bit pattern we found by taking the one's complement and adding one. Consider these examples,

```

01111111 = 127
...
00000010 = 2
00000001 = 1
00000000 = 0
11111111 = -1
11111110 = -2
...
10000000 = -128

```

Two's complement has only one way to represent zero,

```

00000000 → 11111111 → 00000000
positive  one's complement two's complement

```

since adding one to 11111111 maps back around to 00000000 with the overflow bit ignored. Moreover, we increase the range by one to represent numbers from -128 to $+127$ instead of -127 to 127 as with one's complement or sign-magnitude. Modern computers have adopted two's complement notation. Therefore, so shall we for the remainder of the chapter unless otherwise indicated.

2.5 Operations on Signed Integers

We would like to perform operations on signed integers. Bit-level operations like AND, OR, and NOT work the same for signed integers as for unsigned integers. To these operators, the bits are merely bits; the “fact” of a negative integer is a convention forced on certain bit patterns. Therefore, we only need to consider how to compare negative integers; perform arithmetic on negative integers; and, as a special operation, deal with the sign of a two's complement integer when changing the number of bits used to represent the number. Let us first start by comparing two signed integers.

2.5.1 Comparison

Comparison of two signed integers, A and B , implies determining which relational operator, $<$, $>$, or $=$, should be placed between them. When we compared unsigned integers we examined the bits from highest to lowest. That remains the case with signed integers, but we must first consider the signs of the two numbers. If the signs differ, we quickly know the relationship between the numbers without considering the bits. If the signs match, either positive or negative, we must examine the magnitude of the bits to see where they might differ.

A C function like that of Fig. 2.6 will do the trick. Note that we are working with variables of type `signed char`, i.e., signed 8-bit integers.

Fig. 2.6 Comparison of two signed integers *a* and *b*. The function returns 0 if *a* = *b*, 1 if *a* > *b* and -1 if *a* < *b*

```

1|signed char bset(signed char v,
2|                signed char n) {
3|    return (v & (1 << n)) != 0;
4|}
5|
6|signed char scomp(signed char a,
7|                signed char b) {
8|    unsigned char i;
9|
10|    if ((bset(a,7) == 0) && (bset(b,7) == 1))
11|        return 1;
12|    if ((bset(a,7) == 1) && (bset(b,7) == 0))
13|        return -1;
14|
15|    for(i=0; i<7; i++) {
16|        if ((bset(a,6-i) == 0) && (bset(b,6-i) == 1))
17|            return -1;
18|        if ((bset(a,6-i) == 1) && (bset(b,6-i) == 0))
19|            return 1;
20|    }
21|
22|    return 0;
23|}

```

The helper function (*bset*, lines 1–4) returns 1 if the *n*-th bit of *v* is set, otherwise it returns 0. The helper uses the shift and AND mask trick we learned earlier to test a bit position value. The main function, *scomp*, looks first at the signs of the arguments (lines 10–13) and returns the relationship if they differ. If the sign bit of *a* is zero, *a* is positive. If the sign bit of *b* is one, then *b* is negative and *a* must be greater than *b*, so return 1 to indicate *a* > *b*. If the signs are reversed, *a* is less than *b*, so return -1.

If the signs of *a* and *b* match, either positive or negative, we must examine the remaining bits from highest to lowest to detect any differences. This is the loop of lines 15 through 20 in Fig. 2.6. If we find a bit position where *a* has a zero and *b* has a one, we know *a* < *b* must be true, so return -1. Likewise, if we find that *a* is one and *b* is zero at some bit position, we know that *a* > *b* and return 1. Finally, if we make it through all the bits detecting no differences, return 0 because the integers are equal. A comparison function like *scomp* makes it straightforward to create predicate functions for equal, less than, and greater than. For example,

```

unsigned char isEqual(signed char a,
                    signed char b) {
    return scomp(a,b) == 0;
}

unsigned char isLessThan(signed char a,
                       signed char b) {
    return scomp(a,b) == -1;
}

unsigned char isGreaterThan(signed char a,

```

```

                                signed char b) {
    return scomp(a,b) == 1;
}

```

define such functions.

2.5.2 Arithmetic

What about basic arithmetic operations on signed integers? The focus is on operations involving negative numbers, as operations involving positive numbers follow the unsigned integer techniques described earlier in the chapter.

Addition and Subtraction. Addition in one's complement notation is nearly identical to unsigned addition with one extra operation should there be a final carry. To see this, consider adding two negative one's complement integers,

$$\begin{array}{r}
 11000000 \quad -63 \\
 + 11000010 \quad -61 \\
 \hline
 1\ 10000010 \quad -125 \\
 10000011 \quad -124
 \end{array}
 \quad (\text{one's complement})$$

The carry at the end, shown by the extra 1 on the left, is added to the result to arrive at the correct answer of -124 . This addition of any carry, the *end-around carry*, is the extra twist necessary when adding one's complement numbers.

The same addition in two's complement produces a carry which we ignore,

$$\begin{array}{r}
 11000001 \quad -63 \\
 + 11000011 \quad -61 \\
 \hline
 1\ 10000100 \quad -124
 \end{array}
 \quad (\text{two's complement})$$

since we see that 10000100 is -124 by making it positive,

$$\begin{array}{r}
 01111011 \\
 + 00000001 \\
 \hline
 01111100 \quad 124
 \end{array}$$

Adding a positive and a negative works similarly for one's and two's complement numbers. For example, in one's complement, we have

$$\begin{array}{r}
 + 01111100 \quad 124 \\
 11000010 \quad -61 \\
 \hline
 1\ 00111110 \quad 62 \\
 00111111 \quad 63
 \end{array}
 \quad (\text{one's complement})$$

where we have again used the end-around carry to give us the correct answer. The two's complement version is similar

$$\begin{array}{r}
 + \quad 01111100 \quad 124 \\
 \quad 11000011 \quad -61 \quad (\text{two's complement}) \\
 \hline
 1 \quad 00111111 \quad 63
 \end{array}$$

where we again ignore the carry and keep only the lower 8 bits. If we are working with 16-bit or 32-bit numbers, we keep that many bits in the answer.

Computers implement subtraction by negation and addition. This allows for only one set of hardware circuits to be used for both operations. Therefore, subtraction becomes particularly simple. To calculate $124 - 61 = 63$, we calculate $124 + (-61) = 63$, which is exactly the example calculated above. For calculation by hand, it is helpful to think of subtraction as an operation separate from addition. However, with the appropriate notation for negative numbers, subtraction becomes an “illusion” and is nothing more than addition with a negative number.

While addition and subtraction are straightforward, especially with two’s complement notation, we must consider one question: what happens if the operation result does not fit the number of bits we are working with? Let’s consider only two’s complement numbers. We already observed in the examples above that we could ignore the carry to the 9th bit because the lower 8 bits were correct. Is this always the case?

A signed 8-bit two’s complement number ranges from -128 to 127 . We cannot represent the answer correctly if we attempt an operation outside this range. Detecting this situation requires following two simple rules:

1. *If the sum of two positive numbers is negative, overflow has happened.*
2. *If the sum of two negative numbers is positive, overflow has happened.*

We need not worry about the sum of a positive and negative number because both the positive and negative numbers are already in the allowed range, and it is impossible, because of the difference in sign, for the sum to be outside this range. This is why we ignored the last carry bit when adding -61 to 124 .

Let’s look at cases that prove the rules. If we try to calculate $124 + 124 = 248$ we know we will have trouble because 248 is greater than 127 which is the largest positive 8-bit two’s complement number. Consider,

$$\begin{array}{r}
 + \quad 01111100 \quad 124 \\
 \quad 01111100 \quad 124 \quad (\text{two's complement}) \\
 \hline
 11111000 \quad -8
 \end{array}$$

which is clearly a wrong answer. According to our rule for the addition of two positive numbers, we know overflow has happened because the sign bit, bit 7, is one, indicating a negative answer. Similarly, two large negative numbers added prove our second rule,

$$\begin{array}{r}
 + \quad 10000100 \quad -124 \\
 \quad 10000100 \quad -124 \quad (\text{two's complement}) \\
 \hline
 00001000 \quad 8
 \end{array}$$

where we have ignored the carry to the 9th bit. We see that the result is positive since bit 7 is zero, thereby indicating an overflow.

Multiplication. We now consider the multiplication of signed integers. One approach to signed multiplication uses multiplication rules to track the result's sign. If we track the sign, we can make any negative number positive, perform unsigned integer multiplication as described above in Sect. 2.3.5, and negate the result if necessary to make it negative. This approach will work for both one's and two's complement numbers. As we recall from school, when multiplying two numbers there are four possible scenarios related to the signs,

1. *positive* $\times$ *positive* = *positive*
2. *positive* $\times$ *negative* = *negative*
3. *negative* $\times$ *positive* = *negative*
4. *negative* $\times$ *negative* = *positive*

With this in mind, it is straightforward to extend the `mult2` example from Sect. 2.3.5 to check the signs of the inputs, negate the negative numbers to make them positive before multiplying, and negate again if the answer should be negative. In C, this gives us Fig. 2.7.

The main loop in lines 18 through 21 has not changed, but before we run it we check the signs of the inputs to see if we need to negate any negative numbers to make them positive. The variable `s` holds the flag to tell us that the answer needs to be negative.

```

1 | signed short signed_mult2(signed char n, signed char m) {
2 |     unsigned char i, s=0;
3 |     signed short ans=0;
4 |
5 |     if ((n > 0) && (m < 0)) {
6 |         s = 1;
7 |         m = -m;
8 |     }
9 |     if ((n < 0) && (m > 0)) {
10 |         s = 1;
11 |         n = -n;
12 |     }
13 |     if ((n < 0) && (m < 0)) {
14 |         n = -n;
15 |         m = -m;
16 |     }
17 |
18 |     for(i=0; i < 8; i++) {
19 |         if (m & 1) ans += n << i;
20 |         m >>= 1;
21 |     }
22 |
23 |     if (s) ans = -ans;
24 |     return ans;
25 | }
```

Fig. 2.7 Unsigned integer multiplication modified for signed numbers


```

1 | signed short multb8(signed char m, signed char r) {
2 |     signed int A, S, P;
3 |     unsigned char i;
4 |
5 |     A = m << 9;
6 |     S = (-m) << 9;
7 |     P = (r & 0xff) << 1;
8 |
9 |     for(i=0; i < 8; i++) {
10 |         switch (P & 3) {
11 |             case 1: // 01
12 |                 P += A;
13 |                 break;
14 |             case 2: // 10
15 |                 P += S;
16 |                 break;
17 |             default: // 11 or 00
18 |                 break;
19 |         }
20 |         P >>= 1;
21 |     }
22 |
23 |     return P>>1;
24 | }

```

Fig.2.8 The Booth algorithm for multiplying two's complement 8-bit integers

In this example, we have taken advantage of the fact that the C compiler will properly negate a value as in line 7 regardless of the underlying notation used for negative numbers. We know, however, that in practice, this will typically be two's complement.

Can we multiply numbers directly in two's complement? Yes, there are several existing algorithms to choose from. Let's consider one of the more popular of them, the Booth algorithm [1] developed by Andrew Booth in 1950. A C implementation of this algorithm for multiplying two signed 8-bit integers is given in Fig. 2.8. Let's walk through it.

Booth's essential insight was that when we multiply two binary numbers a string of ones can be replaced by a positive and negative sum in the same way that $16 \times 6 = 16 \times (8 - 2)$, but since we are working in binary, we can always write any string of ones as the next higher bit minus one. So, we have

$$00011100 = 00100000 + 000000 - 10$$

where we have written a -1 for a specific bit position to indicate subtraction. If we scan across the multiplicand and see that at bit position i and $i - 1$ there is a 0 and 1, respectively, we can add the multiplier. Similarly, when we see a 1 and 0, we can subtract the multiplier. At other times, we neither add nor subtract the multiplier. After each pair of bits, we shift to the right.

In Fig. 2.8, we initialize the algorithm by setting A to the multiplier, m , shifted nine places to the left, and similarly set S to the negative of the multiplier (two's

complement form), also shifted nine positions to the left. The product, `P`, is initialized to the multiplicand in `r` but shifted one position to the left. This is done in lines 5 through 7. Why all the shifting? We are multiplying two 8-bit signed numbers so the result may have as many as 16 bits, hence using `signed int` for `A`, `S`, and `P`. This is the origin of 8 of the 9-bit positions. The extra bit position for `A` and `S`, and the single extra bit for `P` (line 7), is so that we can look at the last bit position and the one that would come after which we always set to zero. This means that the last bit position, bit 0, and the next, bit -1, could be 1 and 0 to signal the end of a string of ones. We mask the multiplicand, `r`, with `0xFF` to ensure that the sign is not extended when we move from the `signed char` to `signed int` data type. The next section details sign extension.

The loop in lines 9 through 21 examines the first 2 bits of `P`, which are the two we are currently considering, and decides what to do based on their values. Our options are

bit <i>i</i>	bit <i>i</i> - 1	operation
0	0	<i>do nothing</i>
0	1	add multiplier to product
1	0	subtract multiplier from product
1	1	<i>do nothing</i>

which is reflected in the `switch` statement of line 10. The phrase `P&3` masks off the first 2 bits of `P`, which is what we want to examine. After the operation, we shift the product (`P`) to the right to examine the next pair of bits. When the loop finishes, we shift `P` once more to the right to remove the extra bit we added at the beginning in line 7. This completes the algorithm, and we have the product in `P`, which we return.

The algorithm substitutes a starting add and ending subtraction for what might be a long string of additions for each 1 bit in a string of 1 bits. Also, when not adding or subtracting, we merely shift bits, thereby making the algorithm particularly efficient.

Sign Extension and Signed Division. As with multiplication above, we can modify the unsigned integer algorithm for division in Fig. 2.5 to work with signed integers by determining the proper sign of the output, then making all arguments positive and dividing, negating the answer if necessary. However, before we do that, let's take a quick look at sign extension.

Sign Extension. What happens if we take an 8-bit number and make it a 16-bit number? If the number is positive, we simply set the upper 8 bits of the new 16-bit number to zero and the lower 8 bits to our original number like so,

$$00010110 \rightarrow 0000000000010110$$

which is pretty straightforward. Now, if we have a negative 8-bit signed integer in two's complement notation, we know that the leading bit will be a 1. If we simply add the leading zeros, we get

$$11011101 \rightarrow 000000011011101$$

which is no longer a negative number because the leading bit is now a 0. To avoid this problem and preserve the value, we extend the sign when we form the 16-bit number by making all the new higher bits 1 instead of 0,

$$11011101 \rightarrow 1111111111011101$$

which we know is the same value numerically and we can check it by looking at the magnitude of the number. Recall we convert between positive and negative two's complement by flipping the bits and adding one. So, the 8-bit version becomes

$$11011101 \rightarrow 00100010 + 1 \rightarrow 00100011 = 35_{10}$$

and the 16-bit version becomes

$$1111111111011101 \rightarrow 0000000000100010 + 1 \rightarrow 0000000000100011 = 35_{10}$$

which means that both bit patterns represent -35 as desired. We intentionally frustrated sign extension in Fig. 2.8 by masking r with $0xFF$ before assigning it to P , a 32-bit integer.

Signed Division. Figure 2.5 implements unsigned division. If we track the signs properly we can modify it to work with signed integers. Division returns two results. The first is the quotient and the second is any remainder. The sign we should use for the quotient is straightforward enough,

<i>Dividend</i>	<i>Divisor</i>	<i>Quotient</i>
+	+	+
+	−	−
−	+	−
−	−	+

Ambiguity appears when we think about the sign to apply to the remainder. It turns out that different programming languages have adopted different conventions. For example, C chooses to make the remainder have the same sign as the dividend while Python gives the remainder the sign of the divisor. Unfortunately, the situation is more complicated still. When dividing negative numbers, we often return an approximate quotient (unless the remainder is zero), and that approximate quotient has to be rounded in a particular direction. All division algorithms satisfy $d = nq + r$, which means the quotient, q , times the divisor, n , plus the remainder, r , equals the dividend, d . However, we have choices regarding how to set the signs and values of q and r . There are three options:

1. *Round towards zero.* In this case, we select q as the integer closest to zero that, when multiplied by n , is less than or equal to d . In this case, if d is negative, r will also be negative. For example, $-33/5 = -6$ with a remainder of -3 so that $-33 = 5(-6) + (-3)$. This is the option used by C.

2. *Round towards negative infinity.* Here, we round the quotient down and use a remainder with the same sign as the divisor. In this case, $-33/5 = -7$ with a remainder of 2 giving $-33 = 5(-7) + 2$. This is the option used by Python.
3. *Euclidean definition.* This definition makes the remainder always positive. If $n > 0$ then $q = \text{floor}(d/n)$ and if $n < 0$ then $q = \text{ceil}(d/n)$. In either case, r is always positive, $0 \leq r < |n|$.

Let’s make these definitions concrete. The table below shows the quotient and remainder for several examples in C and Python. These give us an intuitive feel for how these operations work.

<i>Dividend</i>	<i>Divisor</i>	<i>Quotient</i>	<i>Remainder</i>	
33	7	4	5	C
		4	5	Python
−33	7	−4	−5	C
		−5	2	Python
33	−7	−4	5	C
		−5	−2	Python
−33	−7	4	−5	C
		4	−5	Python

We see that differences only arise when the signs of the dividend and divisor are opposite. Here, the C choice of rounding towards zero and the Python choice of rounding towards negative infinity come into play. The C choice seems more consistent initially because it always results in quotients and remainders with the same magnitude; only the signs change. From a mathematical point of view, this approach is less desirable because certain expected operations do not give valid results. For example, to test whether or not an integer is even, it is common to check if the remainder is zero or one when divided by two. If the remainder is zero, the number is even; if one, it is odd. This works in Python for negative integers since $-43 \% 2 = 1$ as expected, but in C, this fails since we get $-43 \% 2 = -1$ because of the convention to give the remainder the sign of the dividend.

We can now update the `div2` algorithm in Fig. 2.5 to handle signed integers. We show the updated algorithm, now called `signed_div2`, in Fig. 2.9. Let’s look at what has changed.

The division algorithm in lines 20 through 30 is the same as in Fig. 2.5 since we still perform unsigned division. Lines 6 through 18 check whether any of the arguments, dividend (`n`) or divisor (`m`) or both, are negative. We use two auxiliary variables, `sign` and `rsign`, to track how we deal with the sign of the answer. If the dividend is negative but the divisor is positive, the quotient should be negative, so we set `sign` to 1. If the dividend is positive but the divisor is negative, we must also make the quotient negative. If both the dividend and divisor are negative, the quotient is positive. In all cases we make the dividend and divisor positive after we know how to account for the sign of the quotient. For the remainder, we use the variable `rsign` to decide when to make it negative. The division algorithm itself

Fig. 2.9 Shift, test, and restore unsigned integer division updated to handle signed integers

```

1 | signed char signed_div2(signed char n,
2 |                         signed char m,
3 |                         signed char *r) {
4 |     unsigned char i, sign=0, rsign=0;
5 |
6 |     if ((n < 0) && (m > 0)) {
7 |         sign = rsign = 1;
8 |         n = -n;
9 |     }
10 |    if ((n > 0) && (m < 0)) {
11 |        sign = 1;
12 |        m = -m;
13 |    }
14 |    if ((n < 0) && (m < 0)) {
15 |        rsign = 1;
16 |        n = -n;
17 |        m = -m;
18 |    }
19 |
20 |    *r = 0;
21 |
22 |    for(i=0; i<8; i++) {
23 |        *r = (*r << 1) + ((n & 0x80) != 0);
24 |        n <<= 1;
25 |
26 |        if ((*r-m) >= 0) {
27 |            n |= 1;
28 |            *r -= m;
29 |        }
30 |    }
31 |
32 |    if (sign) n = -n;
33 |    if (rsign) *r = -*r;
34 |
35 |    return n;
36 | }
```

will make the remainder, in `*r`, positive but for our answer to be sound we must sometimes make `*r` negative. When to do this? A sound answer will always satisfy

$$n = m \times q + r$$

so if the dividend in `n` is negative, we also require the remainder to be negative. In this case, we follow the C convention.

If we run `signed_div2` on `n = 123` and `m = 4` with all combinations of signs, we get the following output:

<i>n</i>	<i>m</i>	<i>q</i>	<i>r</i>	$m \times q + r$
123	4	30	3	$4(30) + 3$
-123	4	-30	-3	$4(-30) - 3$
123	-4	-30	3	$-4(-30) + 3$
-123	-4	30	-3	$-4(30) - 3$

where the column on the right shows that our choice of sign for the remainder is correct.

In this section, we reviewed implementations of signed arithmetic on integers. In some cases, we could build on existing algorithms for unsigned arithmetic, while in others, we worked directly in two's complement format.

2.6 Binary-Coded Decimal

Binary-Coded Decimal (BCD) numbers make use of specific bit patterns corresponding to the digits 0 . . . 9 to store one or two decimal digits in each byte. Storing numbers in this format allows for decimal operations in place of binary and, indeed, some early microprocessors, such as the Western Digital 6502, had BCD modes. In this section, we describe how to encode numbers in BCD and how to do simple arithmetic with those numbers.

2.6.1 Introduction

As we saw earlier in this chapter, working with numbers expressed in binary can be cumbersome. Binary is the natural base for a computer to use given its construction, but humans, with ten fingers and ten toes, generally prefer to work in decimal. One possibility is to encode decimal digits in binary and let the computer work with data in this format. Binary-coded decimal does just this. For our purposes, we will work with what is generally called *packed BCD*, where we use each nibble of a byte to represent exactly one decimal digit. Historically, other sizes were used, one digit per byte (unpacked), for example, or even other values, typically with early computers. We will examine one of these early formats below (zoned decimal). In addition, while any set of distinct bit patterns can be used to represent decimal digits, we will use the obvious choice, $0 = 0000$, $1 = 0001$, . . . , $9 = 1001$, so there is a direct conversion between a decimal digit and its BCD bit pattern. The remaining six bit patterns, $1010 \dots 1111$, are not used or allowed in properly formatted BCD numbers. In the previous sections, we ignored the difficulties of converting human-entered decimal data to binary and vice versa. BCD simplifies this process. BCD is no longer frequently used, but it reappears when we consider some of the more specialized ways computers represent numbers, particularly Decimal Floating-Point numbers, Chap. 7.

If we consider only positive numbers, a single byte can represent decimal numbers from zero through 99 as so,

<i>BCD</i>	<i>Decimal</i>
0 0	0
0 1	1
0 2	2
1 0	10
1 1	11
...	
9 8	98
9 9	99

where the two digits in the *BCD* column on the left represent the two 4-bit nibbles of the byte. This works but does not allow for negative numbers. Historically, several variations were used for storing a sign with a BCD number. The form we will use is directly analogous to signed binary integers, but instead of two's complement, we will use *ten's complement* with a leading nibble to represent the sign. If the leading nibble is 0000, the number is positive; if 1001, the number is negative, and the value is in *ten's complement format*. We use the leading sign nibble because, unlike two's complement where the leading bit is always 1 if the number is negative, ten's complement has no such quick test.

In two's complement, we negate a number by flipping all the digits, making $0 \rightarrow 1$ and $1 \rightarrow 0$ and then add one. To find the ten's complement of a decimal number, we first find the nine's complement, which involves subtracting the number from 99, for a single byte, and then add one. For example, the BCD number 51 can be thought of as -49 because,

$$99 - 51 = 48, \quad 48 + 1 = 49$$

A multidigit BCD number is represented with multiple bytes. Endian issues arise again in this case, and we chose to use big-endian so that it is easier to see the decimal numbers in the binary bit patterns. With this convention, including the sign nibble, the decimal number 123 is represented in BCD as

$$\begin{array}{r} + \quad | \quad 0 \quad 1 \quad | \quad 2 \quad 3 \\ 0000 | 0000 \ 0001 | 0010 \ 0011 \end{array}$$

where we need the leading 0 digit to fill out the byte representing the leading two digits. Similarly, using the negative sign nibble, we can represent -732 as

$$\begin{array}{r} - \quad | \quad 0 \quad 2 \quad | \quad 6 \quad 8 \\ 1001 | 0000 \ 0010 | 0110 \ 1000 \end{array}$$

because $999 - 732 = 267$, $267 + 1 = 268$ is the ten's complement of 732.

2.6.2 Arithmetic with BCD

Let’s explore arithmetic using BCD numbers. We only consider addition and subtraction as multiplication and division are rarely performed in BCD. For performance reasons, it is faster to convert from BCD to binary, perform the multiplication or division in binary, and then convert the back to BCD.

The addition of two BCD numbers is straightforward at first glance. We simply add nibble by nibble,

123	0000	0001 0010 0011
+ 732	+ 0000	0111 0011 0010
855	0000	1000 0101 0101

For each nibble, we add in binary as if the numbers were unsigned integers. For this example, everything works nicely. But consider this case,

123	0000	0001 0010 0011
+ 738	+ 0000	0111 0011 1000
861	0000	1000 0101 1011

Here, we encounter a small problem. The sum of the first digits gives us 1011, which is 11, not an allowed BCD bit pattern.

We know the cause of the problem: $3 + 8 = 11$, meaning we have produced a carry. A straightforward approach to dealing with any carries is to take the resulting bit pattern, 1011, and add six, 0110, which will give us the correct bit pattern for the decimal digit and a carry of 1 for the next digit. This correction gives

$$1011 + 0110 = 1\ 0001$$

with the carry separated from the remaining digit, the BCD representation of 1. Applying this correction to the addition leads to the expected result

123	0000	0001 0010 0011
+ 738	+ 0000	0111 0011 1000
	0000	1000 0101 1011
861	0000	1000 0110 0001

We added the carry into the next digit to change the 5 to a 6.

Why add six? There are 16 possible 4-bit nibbles, but we are only using the first 10 of them for decimal digits. If we exceed 10 for any single digit, adding six will wrap the bit pattern around to the digit we would get if we, in decimal form, subtracted ten. Adding six also leaves the carry set for the next digit. If adding the carry to the next higher digit results in 10 or greater the “add six” trick can be repeated as many times as necessary until the BCD number is in the proper form.

We have chosen to represent negative BCD numbers using ten’s complement. This format works the same way as two’s complement, so we add to subtract. For example, consider,

123	0000		0001 0010 0011
- 732	+ 1001		0010 0110 1000
	1001		0011 1000 1011
-609	1001		0011 1001 0001

where the overflow in the first digit is addressed by adding six and then adding the carry to the next higher digit. The sign is negative, and the BCD number reads as 391. Since the number is negative, we expect that 391 is the ten’s complement form of -609 , which is the actual answer. To see that it is, we negate it and add one,

$999 - 391 = 608, \quad 608 + 1 = 609$

proving that we have, in fact, reached the correct result.

2.6.3 Conversion Routines

Binary to BCD and BCD to binary conversion is straightforward. First, let’s consider binary to BCD. This routine could be used during the multiplication or division of BCD numbers or might be used on its own to prepare for the output of a binary value as a decimal number. Once the value is in BCD, conversion to ASCII for output is straightforward; simply examine each nibble, add it to 48, which is the ASCII code for “0”. Then, output the resulting byte as an ASCII character.

Our binary-to-BCD conversion routine converts an 8-bit unsigned binary number to a three-digit BCD number. It goes by the amusing name of the “double dabble” algorithm. Our implementation is in Fig. 2.10. The routine itself is referred to briefly in [7].

What is this code doing? We have an unsigned 8-bit binary number passed in as `b`. We will return the three-digit BCD number in `p`. Since a single byte can hold any value from zero to 255, we need exactly 12 bits to store the equivalent BCD number. Using an unsigned `short` as a return value gives us 16 bits, the top four of which we leave as zero. Inside the algorithm, we use `p` as an unsigned `int` giving us 32 bits to work with. The algorithm is going to perform eight shifts to the left so that it can examine each of the 8 bits in `b`. For example, if `b` is $123 = 01111011_2$, we initialize `p` with `b` so that at the start `p` looks like

<code>p</code>											
0000	0000	0000	0000	0000	0000	0000	0111	1011			
			<i>100s</i>			<i>10s</i>		<i>1s</i>			

where the bit positions of the hundreds, tens, and ones digits for our BCD representation are indicated.

The loop in lines 7 through 24 runs eight times and shifts `p` 1 bit position to the left each time. However, before the shift, we examine each BCD digit location to see

```

1 | unsigned short bin2bcd8(unsigned char b) {
2 |     unsigned int p=0;
3 |     unsigned char i,t;
4 |
5 |     p = (unsigned int)b;
6 |
7 |     for(i=0; i<8; i++) {
8 |         t = (p & 0xf00) >> 8;
9 |         if (t >= 5) {
10 |             t += 3;
11 |             p = ((p>>12)<<12) | (t<<8) | (p&0xff);
12 |         }
13 |         t = (p & 0xf000) >> 12;
14 |         if (t >= 5) {
15 |             t += 3;
16 |             p = ((p>>16)<<16) | (t<<12) | (p&0xffff);
17 |         }
18 |         t = (p & 0xf0000) >> 16;
19 |         if (t >= 5) {
20 |             t += 3;
21 |             p = ((p>>20)<<20) | (t<<16) | (p^0xffff);
22 |         }
23 |         p <<= 1;
24 |     }
25 |
26 |     return (unsigned short) (p>>8);
27 | }

```

Fig.2.10 Unsigned 8-bit binary to three-digit BCD conversion

if the value there is five or greater. If it is, we add three before shifting to the left. Line 8 pulls out the 4 bits representing the ones value in the BCD representation. Specifically, it first masks `p` with `0xf00`, which leaves only the 4 bits in the ones place nonzero. It then shifts to the right eight bits and assigns this value to `t`. This sets `t` to the ones value. Line 9 asks if this value is greater than or equal to five. If so, line 10 increments `t` by three. Line 11 then updates the proper position in `p` with the new value of `t`. Let's look at this line more closely. Recall that OR combines bits, and here, three pieces are combined to replace the proper 4 bits of `p` with the value of `t`. First, `((p >> 12) << 12)` loses the lower 12 bits of `p`, which includes the 4 bits of the BCD ones digit along with the 8 bits originally set to `b`. It then shifts back up so that the net result is a value that has the lowest 12 bits set to zero and the remaining bits set to whatever they were previously. This is OR-ed with `t` shifted up eight bits so that the lower 4 bits of `t`, which are the only ones to have a nonzero value, are in the proper position to be the BCD ones digit. Lastly, `p&0xff` keeps the lowest 8 bits of `p` and OR's them in as well. When each of these operations is complete `p` is the same as it was previously, except that the four bits in the BCD ones position have been increased by three. We will see below why three was added and why five was the cutoff value. Lines 13 through 22 perform the same check for the BCD tens and BCD hundreds digits. The only change is the mask and shift values to isolate

and work with the correct 4 bits of p . Lastly, line 23 shifts all of p one position to the left to move the next bit of b (in the lowest 8 bits of p) into position. After all bits of b have been examined the final BCD result is in bits 9 through 20 so we shift down eight positions to get the final BCD value.

The operation of the entire algorithm is shown below for $b = 123$. For convenience we ignore the top three nibbles of p , a 32-bit unsigned integer. This leaves us with

<i>100s</i>	<i>10s</i>	<i>1s</i>	<i>b</i>	<i>comment</i>
0000	0000	0000	0111 1011	initial
0000	0000	0000	1111 0110	shift 1
0000	0000	0001	1110 1100	shift 2
0000	0000	0011	1101 1000	shift 3
0000	0000	0111	1011 0000	shift 4
0000	0000	1010	1011 0000	add 3, ones
0000	0001	0101	0110 0000	shift 5
0000	0001	1000	0110 0000	add 3, ones
0000	0011	0000	1100 0000	shift 6
0000	0110	0001	1000 0000	shift 7
0000	1001	0001	1000 0000	add 3, tens
0001	0010	0011	0000 0000	shift 8

which after the last left shift has 123 as a BCD number in the indicated columns. The `return` statement shifts this value to the right 8 bits to return the final three-digit BCD number.

The algorithm works, but why? To understand the algorithm, recall that to convert a BCD digit that is not proper, say 1011, to a proper digit is to add six, 0110. This will produce a carry bit to the next BCD digit position while preserving the proper value in the existing digit. So, one way to think of converting binary to BCD is to look at each digit position and subtract ten before doubling if it is greater than ten. This is to say, calculate

$$2x + 6$$

for cases where the digit, x , is ten or greater before looking at the next bit of our binary number. This is equivalent to considering if the value is greater than five and if so adding three and then doubling because

$$2x + 6 = 2(x + 3)$$

which has the added advantage of not requiring an extra bit since $2x + 6$ may be greater than 15 while $x + 3$ never will be for x a valid BCD digit. This is the approach of the double-dabble algorithm shown in Fig. 2.10.

Conversion from a three-digit BCD number to binary is especially straightforward. We simply examine each nibble, multiply it by the proper power of 10, and accumulate the result,

```

unsigned char bcd2bin8(unsigned short b) {
    unsigned char n;

    n = (b & 0x0f);
    b >>= 4;
    n += 10*(b & 0x0f);
    b >>= 4;
    n += 100*(b & 0x0f);

    return n;
}

```

The phrase `b&0x0f` masks off the lowest nibble of `b`. This is the ones digit, so we simply set the output value in `n` to it. We then shift `b` to the right by 4 bits to make the lowest nibble the tens digit. We add this to the running total after first multiplying it by ten. We then shift one last time to make the hundreds digit the first nibble, multiply it by 100, and add to `n` to get the final value of the three-digit BCD number.

2.6.4 Other BCD Encodings

So far in this section we have used packed BCD encoding. Here, we explore two others: zoned decimal and densely packed decimal (DPD). We will only briefly discuss densely packed decimals as we discuss it in detail in Sect. 7.2 as it is a critical part of the IEEE decimal floating-point format.

Densely Packed Decimal. The DPD format encodes three-decimal digits using 10 bits. It is a refined version of Chen-Ho encoding [8] and is used by the IEEE 754-2008 Decimal Floating-Point Format [9] to encode the significant in base ten. There is a 2-bit savings in storage over packed BCD which requires 12 bits to encode three-decimal digits.

Packing three BCD numbers into a DPD declet is straightforward. We label the bits of each BCD digit as *abcd*, *efgh*, and *ijklm*. Using these labels for the bits, we can take any set of three BCD digits and encode them into a DPD declet using Table 2.3. The ten bits of the declet are labeled *pqr stu v wxy*, which can be read from the table by finding the row matching the *aei* bit values from the input BCD numbers.

For example, to create a declet encoding the number 359, we first encode the digits in packed BCD as 0011, 0101, and 1001, and then look for the row in Table 2.3 that starts with 001 (the first bits of each packed BCD digit). This directs us to line 2 so that we can map the remaining input bits to the output DPD bits like so,

$$\begin{aligned}
 pqr &\rightarrow bcd = 011 \\
 stu &\rightarrow fgh = 101 \\
 v &\rightarrow i = 1 \\
 wxy &\rightarrow 00m = 001
 \end{aligned}$$

Table 2.3 Packing three BCD numbers (*abcd efgh ijk*m) into a declet (*pqr stu v wxy*). Locate the row by matching the first bit of each BCD number to the value under the *aei* column, then the bits of the declet are read from the remaining columns of that row. After [11]

<i>aei</i>	<i>pqr</i>	<i>stu</i>	<i>v</i>	<i>wxy</i>
000	bcd	fgh	0	jkm
001	bcd	fgh	1	00m
010	bcd	jkh	1	01m
011	bcd	10h	1	11m
100	jdk	fgh	1	10m
101	fgd	01h	1	11m
110	jdk	00h	1	11m
111	00d	11h	1	11m

Table 2.4 Unpacking a declet (*pqr stu v wxy*) by translating it into three groups of binary digits representing three decimal digits (*abcd efgh ijk*m). After [11]

<i>vxwst</i>	<i>abcd</i>	<i>efgh</i>	<i>ijk</i> m
0-----	0pqr	0stu	0wxy
100---	0pqr	0stu	100y
101---	0pqr	100u	0sty
110---	100r	0stu	0pqy
11100	100r	0pqu	100y
11110	0pqr	100u	100y
11111	100r	100u	100y

implying that the final output declet is 011 101 1 001.

A declet can be unpacked using Table 2.4 by forming the value *vxwst* and locating the matching row. For example, to undo the example above where 359 → 011 101 1 001 (*pqr stu v wxy*), we form *vxwst* = 10010 which matches line 2 of Table 2.4 so that the bits of the unencoded declet become

$$\begin{aligned}abcd &\rightarrow 0pqr = 0011 = 3 \\efgh &\rightarrow 0stu = 0101 = 5 \\ijkm &\rightarrow 100y = 1001 = 9\end{aligned}$$

as we expect.

Zoned Decimal. Zoned decimal is a storage format supported by early IBM mainframes, beginning with the IBM 360, if not earlier. The format is still used, in fact, and is supported by specific programming languages, for example, RPG IV [10].

Zoned decimal is a BCD format used primarily during input and output. While packed BCD stores two decimal digits per byte, zoned decimal stores only one in the lower nibble. The upper nibble, the “zone”, stores a value that makes the entire byte

a valid EBCDIC or ASCII character code for the number itself. In this way, output is made simple. The only twist is storing the sign of the decimal number. In this case, the upper nibble of the lowest order digit is altered to indicate that the entire number is negative. We will implement zoned decimal routines that store up to eight digits in an unsigned 64-bit integer. If we do this we see that

8 → F0F0F0F0F0F0F0F8
 -8 → F0F0F0F0F0F0F0D8
 8675309 → F0F8F6F7F5F3F0F9
 -8675309 → F0F8F6F7F5F3F0D9

where we are using the EBCDIC representation, which uses F for the high nibble and D if the number is negative. The conversion is similar in ASCII, but 3 is used for the high nibble and 7 if the number is negative.

For example, the single decimal digit, 8, maps to F0F0F0F0F0F0F0F8 where the number is stored in the 64-bit unsigned integer with leading zeros, seven of them, as F0, the EBCDIC code for “0”, followed by F8 for the digit, “8”. For -8, the conversion is the same, but the lowest order byte is now D8 to indicate that the entire number is negative. Notice that an unsigned 64-bit integer can store eight bytes and that each zoned decimal digit is a byte. Therefore, the range of representable integers is [−99999999, +99999999], which fits in a standard C signed integer. The code for converting a signed integer to zoned decimal is

```

1 | #include <inttypes.h>
2 | #define PNIB 0xF0
3 | #define NNIB 0xD0
4 |
5 | uint64_t bin2zoned(int b) {
6 |     uint8_t i,s = (b<0) ? 1:0;
7 |     uint64_t d,z=0,p=10000000;
8 |
9 |     b = abs(b);
10 |    for(i=7; i>0; i--) {
11 |        d = b / p;
12 |        b -= d * p;
13 |        p /= 10;
14 |        z += (PNIB+d) << (uint64_t)(i*8);
15 |    }
16 |    z += (s) ? (NNIB+b) : (PNIB+b);
17 |    return z;
18 | }
```

where we include the `inttypes` standard C library to access the `uint64_t` data type. Lines 2 and 3 define the upper nibbles for the EBCDIC representation. Replace these with `0x30` and `0x70`, respectively, for ASCII. The `bin2zoned` function (line 5) accepts a standard signed integer value (`b`) and returns the eight-digit zoned decimal representation. Line 6 sets the sign of the input (`s`) and then works with the positive version (line 9). The loop starting in line 10 sets the highest seven digits of the

output. The last digit is a special case because the upper nibble changes if the input is negative. The output value is in `z`.

Line 11 sets `d` to the integer part of the input divided by the power of ten the current zoned digit represents (`p`, initialized in line 7). The loop counts down, so we process the input from the most significant to the least significant digit. Line 12 subtracts the digit value from the input. Line 13 drops `p` to the next lower power of ten. Finally, line 14 adds the digit value to the positive upper nibble value, `PNIB+d`, and shifts it into the proper byte position, `i*8`.

The lowest order digit is handled in line 16. At this point the value in `b` is the digit we need so we check the sign in `s` to decide which upper nibble to add before adding the final digit value to `z`. Line 17 returns the zoned decimal representation.

Converting a zoned decimal to binary is similarly straightforward,

```

1 | int zoned2bin(uint64_t z) {
2 |     int32_t b=0,p=1;
3 |     int8_t i,s=1;
4 |
5 |     for (i=0; i<8; i++) {
6 |         b += (z & 0xF) * p;
7 |         p *= 10;
8 |         if ((z & 0xF0) == NNIB) s=-1;
9 |         z >>= 8;
10 |    }
11 |    return s*b;
12 |}
```

We loop over each digit in the input, lowest to highest, starting in line 5. The output is stored in `b` and we mask off the lowest nibble of the input, `z`, and multiply it by the current power of ten represented by that digit, `p` (line 6). Line 7 moves to the next higher power of ten. Line 8 checks if the current zoned digit's high nibble is negative. If so, it sets the sign in `s`. Line 9 shifts the input down so that the highest zoned digit is in the first byte. The binary value is multiplied by the sign and returned (line 11). Conversion from zoned decimal to packed BCD is left as an exercise for the reader (see Problem 2.9).

2.7 Summary

In this lengthy but essential chapter we covered a lot of important topics. We learned vital terminology regarding bits, bytes, and words. We explored unsigned integers at length, learning how to perform such low-level functions as manipulation of individual bits to the basic logic operations of AND, OR, NOT, and XOR. We completed our tour of low-level manipulation by studying shifts and rotates and how they affect the value of an unsigned integer. We then examined how to compare the magnitude of two unsigned integers.

Unsigned integer arithmetic was our next topic and with it we learned how to implement the basic operations of addition, subtraction, multiplication, and division with remainder. To round out the unsigned integers we reviewed one approach for calculating the square root of a number.

Signed integers were our next target. We built upon what we learned with unsigned integers and delved into the main ways signed data is stored: sign-magnitude, one's complement, and two's complement. We learned how to compare signed integers and how to either extend existing unsigned arithmetic routines to handle signed arguments or how to implement algorithms that operate on two's complement numbers directly.

To round out our examination, we explored binary-coded decimal numbers, including how to add and subtract them in ten's complement and how to convert between binary-coded decimal and pure binary and vice versa.

It would not be too much to say that integers are the heart of computers. We now know how to work with them at any level. There are more specialized representations of integers, and we will examine them in other chapters, but for the vast majority of computer work involving data that does not need fractions, including characters, fixed-width integers are our main tools and understanding them in some detail is well worth the effort.

Exercises

2.1 Interpret the following bit patterns as little-endian and then as big-endian unsigned short integers. Convert your answer to decimal.

- 1101011100101110
- 1101010101100101
- 0010101011010101
- 1010101011010111

2.2 Given $A = 6B_{16}$ and $B = D6_{16}$, write the result of each expression in binary and hexadecimal.

- $(A \text{ AND } B) \text{ OR } (\text{NOT } A)$
- $(A \text{ XOR } B) \text{ OR } (A \text{ AND } (\text{NOT } B))$
- $((\text{NOT } A) \text{ OR } (\text{NOT } B)) \text{ XOR } (A \text{ AND } B)$

2.3 Using C syntax write an expression that achieves each of the following. Assume that the unsigned char $A = 0x8A$.

- Set bits 3 and 6 of A.
- Keep only the low nibble of A.

- Set unsigned char `B` to 100 if bit 3 of `A` is not set.
- Clear bit 1 of `A`.
- Toggle bits 4 and 5 of `A`.

2.4 Using C syntax write an expression for each of the following.

- Swap the upper and lower nibbles of unsigned char `v = 0xC4`.
- Multiply `v` by 5 using at least one shift operation.

2.5 Express each of the following numbers in one's and two's complement notation. Write your answer in binary and hexadecimal. Assume 8-bit signed integers.

- -14
- -127
- -1

2.6 Write a function to reverse the bits of an unsigned 8-bit integer. *

2.7 Write a function to count the number of 1 bits in an unsigned 32-bit integer. *

2.8 Modify the cube root algorithm in Sect. 2.3.6 to work correctly with *signed* integers.

2.9 Modify the `zoned2bin` function of Sect. 2.6.4 to return a *packed BCD* representation of the input. Call this new function `zoned2bcd`. Note, the input is an unsigned 64-bit integer representing eight digits, so the packed BCD representation will require 32 bits and will fit in an unsigned 32-bit integer.

2.10 The *Hamming distance* between two integers is the number of places where their corresponding bits differ. For example, the Hamming distance between 1011 and 0010 is 2 because the numbers differ in bits 0 and 3. Write a function to calculate the Hamming distance between two unsigned short integers. *

2.11 A *Gray code* is a sequence of bit patterns in which two adjacent patterns differ by only 1 bit. Gray codes were developed to aid in error correction of mechanical switches but are used more generally in digital systems. For example, for 4 bits, the first six values in the Gray code are: 0000, 0001, 0011, 0010, 0110, 0111. The rule to convert a binary number n into the corresponding Gray code g is to move bit by bit, from lowest to highest bit, setting the output bit to the original bit or inverted if the next higher bit is 1. So, to change $n = 0011$ to $g = 0010$ we output a 0 for bit 0 of g since bit 1 of n is set and output the existing bit value of n for all other positions since the next higher bit is not set.

Using this rule, write a C function that calculates the Gray code g for any unsigned short n input. (*Hint: There are many ways to write a function that works but in the end it can be accomplished in one line of code. Think about how to check if the next higher bit of the input is set by shifting and how to invert the output bit if it is.*) **

References

1. Booth AD (1951) A signed multiplication technique. Q J Mech Appl Math IV(Part 2)
2. Shannon C (2001) A mathematical theory of communication. ACM SIGMOBILE Mob Comput Commun Rev 5(1):3–55
3. As mentioned in *Anecdotes*. Ann Hist Comput 6(2):152–156 (1984)
4. Swift J, Gulliver's Travels (1726) Reprinted Dover Publications; Unabridged edition (1996)
5. The Internet Protocol. RFC 791 (1981)
6. Boole G (1854) An investigation of the laws of thought. Reprinted with corrections, Dover Publications, New York (1958). (reissued by Cambridge University Press, 2009)
7. Gao S, Al-Khalili D, Chabini N (2012) An improved BCD adder using 6-LUT FPGAs. In: IEEE 10th international new circuits and systems conference (NEWCAS 2012), pp 13–16
8. Chen TC (1971) Decimal number compression (Internal memo to Irving T. Ho). San Jose Research Laboratory: IBM, pp 1–4
9. IEEE Standards Association (2008) Standard for floating-point arithmetic. IEEE 754-2008
10. IBM Rational Development Studio for i, ILE RPG language reference 7.1 (2010)
11. Duale A et al (2007) Decimal floating-point in z9: an implementation and testing perspective. IBM J Res Develop 51(1.2):217–227



Abstract

We live in a world of floating-point numbers, and we make frequent use of floating-point numbers when working with computers. In this chapter, we dive deeply into how floating-point numbers are represented in a computer. We briefly review the distinction between real numbers and floating-point numbers. Then comes a brief historical look at the development of floating-point numbers. After this, we compare two popular floating-point representations and then focus exclusively on the IEEE 754 standard. For IEEE 754, we consider representation, rounding, arithmetic, exception handling, and hardware implementations. We conclude the chapter with some comments about binary-coded decimal floating-point numbers.

3.1 Floating-Point Numbers

This chapter is about floating-point numbers, a highly complex area with active research even today. As such, we can only scratch the surface of representation and computation with floating-point numbers in a single chapter. Therefore, after some preliminaries, we will focus on the current Institute of Electrical and Electronics Engineers (IEEE) 754-2008 standard, which we will refer to simply as “IEEE 754”. For a more in-depth treatment of floating-point numbers and arithmetic, please see Muller et al. [1].

It would be nice to have an idea of what, exactly, a “floating-point” number is and how it differs from a “real” number. Let’s start with the latter.

From school, we know that a real number is any number found on the number line. We have an intuitive feel for what a real number is, and we know many of them because we also know that all integers ($\mathbb{Z}$) and rationals ($\mathbb{Q}$) are also real numbers. But, of course, there are still many more real numbers besides these. For example,

π is transcendental and, therefore, a real number, but it is neither an integer nor a rational (capable of being written as p/q with $p, q \in \mathbb{Z}$).

How, then, to define the real numbers, $\mathbb{R}$? Formally, there are several ways to define $\mathbb{R}$. We will consider the synthetic approach that specifies $\mathbb{R}$ by defining several axioms that hold only for $\mathbb{R}$. The set of numbers that satisfy these axioms is, by definition, $\mathbb{R}$. Specifically, $\mathbb{R}$ may be defined by considering a set $(\mathbb{R})$, two special elements of $\mathbb{R}$, 0 and 1, two operations called addition (+) and multiplication ($\times$), and finally a relation on $\mathbb{R}$ called $\leq$ which orders the elements of $\mathbb{R}$. With this and the following four axioms, we arrive at a specification of the real numbers. The axioms are

1. $(\mathbb{R}, +, \times)$ forms a field,
2. $(\mathbb{R}, \leq)$ forms a totally ordered set,
3. $+$ and $\times$ maintain $\leq$,
4. the order $\leq$ is complete in a formal sense,

where a complete description of their meaning and implications is beyond the scope of this text. The interested reader is directed to [2] for a presentation on fields and rings. For the present, in this sense, $+$ is simply regular addition and $\times$ is regular multiplication. The order $\leq$ also acts as expected.

A *floating-point number* approximates a real number with something that is representable on a computer. It is “close enough” (we’ll see below what that means) to the actual real number we wish to represent to be useful and lead to meaningful results.

To be specific, in this chapter, a floating-point number is a number that is represented in exponential form as

$$\pm d_0.d_1d_2d_3 \dots d_{p-1} \times \beta^e, \quad 0 \leq d_i < \beta$$

where $d_0.d_1d_2d_3 \dots d_{p-1}$ is the p digit *significand* (or *mantissa*), β is the base and e is the integer exponent. We follow the notation given in [3] for the above. While the base β can be any integer, we restrict ourselves to $\beta = 2$ for this chapter, implying our numbers are binary and the only allowed digits are 0 and 1.

If there are p digits in the significand, then this part of the floating-point number can have β^p unique values. This defines the precision with which values can be represented for a particular exponent. To complete the format, we must also specify the range for the exponent from e_{min} to e_{max} . Before moving on to a brief history of floating-point numbers in computers, let’s take a quick look at working with floating-point numbers in base 2.

The significand, $d_0.d_1d_2d_3 \dots d_{p-1}$ is a number with the value of

$$d_0 + d_1\beta^{-1} + d_2\beta^{-2} + d_3\beta^{-3} + \dots + d_{p-1}\beta^{-(p-1)}$$

which is then scaled by the exponent β^e , so that the final value of the number is

$$(d_0 + d_1\beta^{-1} + d_2\beta^{-2} + d_3\beta^{-3} + \dots + d_{p-1}\beta^{-(p-1)})\beta^e$$

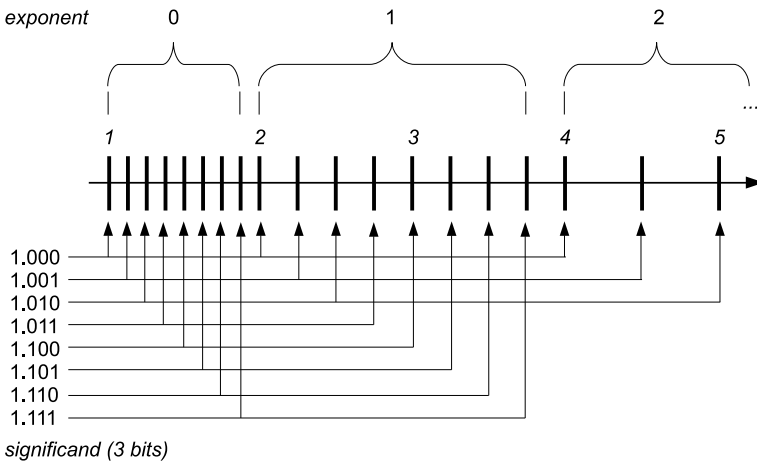


Fig. 3.1 The distribution of floating-point numbers. For this figure, the number of significand bits is three with an implied leading bit of one: $1.m_0m_1m_2 \times 2^e$. The location of representable numbers for exponents of $e = 0, 1, 2$ is shown. For each increase in the exponent, the number of representable numbers is halved. So, between $[1, 2)$ there are eight values while between $[2, 3)$ there are only four and between $[4, 5)$ there are only two

where we have ignored a possible minus sign to make the entire number negative.

The above means that the distribution of floating-point numbers on the real number line is not uniform, as shown in Fig. 3.1.

Floating-point numbers are not uniformly distributed because the digits of the significand become multipliers on the exponent value β^e . So, for a fixed exponent, the smallest interval between two floating-point numbers is $\beta^{-(p-1)}\beta^e$, which is of fixed size as long as the exponent does not change. Once the exponent changes from e to $e + 1$ the minimum interval becomes

$$\beta^{-(p-1)}\beta^e \rightarrow \beta^{-(p-1)}\beta^{e+1} = \left(\beta^{-(p-1)}\beta^e\right)\beta$$

which is β times larger than the previous smallest interval.

Above, we said that there are β^p possible significands. However, not all of them are useful if we want to make floating-point numbers unique. For example, in decimal 1.0×10^2 is the same as 0.1×10^3 and the same holds true for any base β . Therefore, to make floating-point numbers uniquely representable, we add a condition requiring the significand's first digit to be nonzero. For binary floating point, this means that the first digit of the significand must be a 1. Floating-point numbers with this convention are said to be *normalized*. The requirement buys us one extra digit of precision when the base is 2 because the only nonzero digit is 1, which means all normal binary floating-point numbers have a known leading digit for the significand. The known digit need not be stored in memory, leaving room for a $p + 1$ -th digit in the significand.

To make all this concrete, let's examine some binary floating-point numbers ($\beta = 2$). We will calculate the actual decimal values for the floating-point numbers shown in Fig. 3.1. They are

<i>Significand</i>	<i>Exponent</i>	<i>Expanded</i>	<i>Decimal value</i>
1.000	0	$(1) \times 1$	1.0000
1.001	0	$(1 + \frac{1}{8}) \times 1$	1.1250
1.010	0	$(1 + \frac{1}{4}) \times 1$	1.2500
1.011	0	$(1 + \frac{1}{4} + \frac{1}{8}) \times 1$	1.3750
1.100	0	$(1 + \frac{1}{2}) \times 1$	1.5000
1.101	0	$(1 + \frac{1}{2} + \frac{1}{8}) \times 1$	1.6250
1.110	0	$(1 + \frac{1}{2} + \frac{1}{4}) \times 1$	1.7500
1.111	0	$(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8}) \times 1$	1.8750
1.000	1	$(1) \times 2$	2.0000
1.001	1	$(1 + \frac{1}{8}) \times 2$	2.2500
1.010	1	$(1 + \frac{1}{4}) \times 2$	2.5000
1.011	1	$(1 + \frac{1}{4} + \frac{1}{8}) \times 2$	2.7500
1.100	1	$(1 + \frac{1}{2}) \times 2$	3.0000
1.101	1	$(1 + \frac{1}{2} + \frac{1}{8}) \times 2$	3.2500
1.110	1	$(1 + \frac{1}{2} + \frac{1}{4}) \times 2$	3.5000
1.111	1	$(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8}) \times 2$	3.7500
1.000	2	$(1) \times 4$	4.0000
1.001	2	$(1 + \frac{1}{8}) \times 4$	4.5000
1.010	2	$(1 + \frac{1}{4}) \times 4$	5.0000

where we see that the 3 bits of the significand have indeed set the precision of each number for a fixed exponent but that the larger the exponent, the greater the delta between successive floating-point numbers.

3.2 An Exceedingly Brief History of Floating-Point Numbers

Perhaps the first description of floating-point numbers, meaning a method for storing and manipulating real numbers with a machine, is that of Leonardo Torres y Quevedo in his paper "Essays on Automatics" (1914). This paper is included in Randell's book "The Origins of Digital Computers" [4]. Torres y Quevedo implemented an electromechanical calculating machine, the "electromechanical arithmometer", inspired by the work of Charles Babbage, which was capable of accepting typed statements such as 365×256 via a typewriter and would respond, by typing on the same line, with $= 93440$ and then advance to the next line. This machine was demonstrated at a conference in Paris in 1920.

As for floating point, Torres describes a way to store floating-point numbers consistently in his “Essays” paper. First, he states that numbers will be stored in exponential format as $n \times 10^m$. He then offers three rules by which consistent manipulation of floating-point numbers by a machine could be implemented. They are, in his own words (as translated into English),

1. *n will always be the same number of digits (six, for example).*
2. *The first digit of n will be of the order of tenths, the second of hundredths, etc.*
3. *One will write each quantity in the form: n; m.*

On the whole, this is quite remarkable for 1914. The format Torres proposed shows that he understood the need for a fixed-sized significand as is presently used for floating-point data. He also fixed the location of the decimal point in the significand so that each representation was unique. Finally, he even showed concern for how to format such numbers by specifying a syntax that could be entered through a typewriter, as it was with his “electromechanical arithmometer” some years later. We use essentially the same syntax for most programming languages today; simply replace the “;” with an “E” or “e” so that where he would have written *0.314159;1*, we now write *0.314159E1*. It is unfortunate that Torres y Quevedo’s work is not more widely known outside his native Spain.

Konrad Zuse is typically credited with being the first to implement practical floating-point numbers for his *Z1* and *Z3* computers [5]. The *Z1* was developed in 1938, with the *Z3* completed in 1941. These computers used binary floating-point numbers with 1 bit for the sign, 14 bits for the significand, and a signed exponent of 7 bits. Zuse also recognized the value of an implicit “1” bit in the significand and stored the numbers in normalized format. This format is in many ways identical to the modern floating-point formats we will investigate in this chapter and is a testament to Zuse’s often unrecognized brilliance.

According to [5], Zuse called any number with an exponent of -64 “zero” and any number with an exponent of 63 “infinity”. This means that the smallest number possible on the *Z3* was $2^{-63} = 1.08 \times 10^{-19}$ while the largest was $1.999 \times 2^{62} = 9.2 \times 10^{18}$. Numbers were entered in decimal and converted to binary format. Similarly, output was converted from binary to decimal. However, this conversion did not cover the entire range of the machine, and the user was limited to a much smaller floating-point range for input and output.

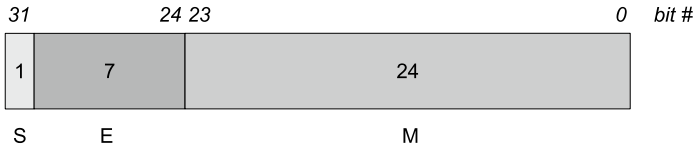
Computers in the 1950s through the 1970s used various floating-point formats. For example, the Burroughs B6700 used a base 8 ($\beta = 8$) format, while the IBM System/360 used a base 16 ($\beta = 16$) format. Many other computers copied the IBM format, including the Data General Nova and Eclipse computers.

In 1985, IEEE released a standard for floating-point arithmetic. This standard has since dominated the industry and has been implemented in hardware by most microprocessors. This standard is the main focus of this chapter, and we begin by comparing its floating-point representation to the older IBM format.

3.3 Comparing Floating-Point Representations

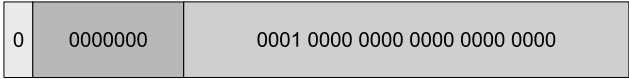
In this section, we examine the 32-bit IBM S/360 format for floating-point numbers and compare it to the 32-bit IEEE 754 format. We then focus exclusively on the IEEE format for the remainder of the section and chapter. The comparison will illustrate why $\beta = 2$ is superior to $\beta = 16$.

A single-precision floating-point number in the IBM S/360 format was stored in memory as



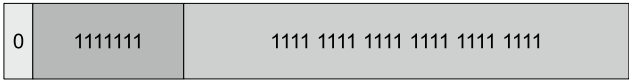
with 1 bit for the sign (S), 7 bits for the exponent (E), and 24 bits for the significand (M). The exponent was stored in excess-64 format, meaning an exponent of 0 represented -64, and an exponent of 127 represented 63. The significand was in binary but manipulated in groups of 4 bits since the base was 16 and $2^4 = 16$. Because of this, one can think of the significand as representing six hexadecimal digits for the first six negative powers of 16.

To normalize a floating-point number, at least one of the first 4 high bits of the significand had to be nonzero. Because of this, the smallest positive floating-point number was



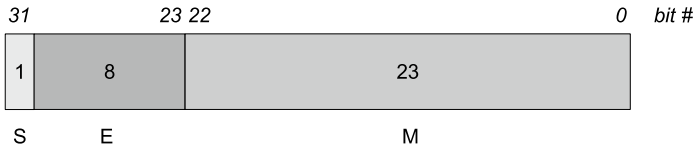
which is, in base 16, $0.1_{16} \times 16^{0-64} = 1 \times 16^{-1} \times 16^{-64} = 5.397605 \times 10^{-79}$.

Similarly, the largest positive number was



equal to $0.\text{FFFFFF}_{16} \times 16^{127-64} = 15 \times 16^{-1} + 15 \times 16^{-2} + 15 \times 16^{-3} + 15 \times 16^{-4} + 15 \times 16^{-5} + 15 \times 16^{-6} \times 16^{63} = (1 - 16^{-6}) \times 16^{63} = 7.237005 \times 10^{75}$ since the significand of $0.\text{FFFFFF}_{16}$ is only one 16^{-6} away from 1, hence $0.\text{FFFFFF}_{16} = 1 - 16^{-6}$. The number zero was represented by all bits set to zero.

The IEEE 754 standard defines several different floating-point sizes. For the moment, we consider only the *binary32* single-precision format. On modern x86-based hardware, this is the format used when one declares a variable to be of type `float` in C, for example. This format is stored in memory as



with 1 bit for the sign (S), 8 bits for the exponent (E), and 23 bits for the significand (M). The exponent is stored in excess-127 format with two special exponent values of 0 and FF_{16} reserved. The base is 2 ($\beta = 2$), and the significand has an implied leading 1 bit always, if the number is normalized. Therefore, in this format, a number is stored as

$$\pm 1.b_{22}b_{21}b_{20} \dots b_0 \times 2^{E-127}$$

where b_n is the value of that bit of the significand. Since the base is 2, the bits are interpreted as individual bits instead of groups of 4 bits, like in the IBM format.

Because of the implied 1 bit, the smallest allowed *binary32* number is

0	00000001	000 0000 0000 0000 0000 0000
---	----------	------------------------------

which is $1 \times 2^{1-127} = 1.1754944 \times 10^{-38}$. Similarly, the largest positive number is

0	11111110	111 1111 1111 1111 1111 1111
---	----------	------------------------------

which, using the reasoning above for the significand of the largest IBM S/360 format number, is $(2 - 2^{-23}) \times 2^{254-127} = 3.4028235 \times 10^{38}$. The mantissa is $(2 - 2^{-23})$ instead of $(1 - 2^{-23})$ because of the implied leading 1 bit of the *binary32* significand.

Immediately, we see that the IBM format has a much larger range than the IEEE format, approximately $[10^{-79}, 10^{75}]$ versus $[10^{-38}, 10^{38}]$. This is encouraging in terms of expressing large numbers using fewer bits of memory, but the significand has a hidden problem. Namely, for the IBM format, a normalized number is one in which the first hexadecimal digit of the six hexadecimal digit significand is nonzero, meaning any of the possible bit patterns for 1 through 15 are allowed. If the bit pattern is 0001_2 , then there are three fewer significant bits in the significand than if the leading digit is 1000_2 up to 1111_2 . This represents a potential loss of precision of up to one decimal digit and is due entirely to how the number is represented in base 16.

This effect is known as *wobbling precision* and is more severe the larger the base, as is the case with the IBM format. The following example illustrates that wobbling precision can lead to highly inaccurate results (borrowed from Cody and Waite [7]).

The approximate value of $\pi/2$ in hexadecimal is $1.922_{16} \times 16^0$. We get this from Python using the `float.hex()` class method:

```
float.hex(pi/2.0) = 0x1.921fb54442d18p+0
                  = 1.921fb54442d1816 × 20
```

We must be careful because Python expresses floats in hexadecimal with a base of 16, not 2. The conversion to binary is straightforward; however, simply replace each hexadecimal digit with 4 binary digits equal to the hexadecimal digit. The exponent is zero in this case, so the actual value is merely the significand. Similarly, the approximate value of $2/\pi$ in hexadecimal is $\text{a.2f9}_{16} \times 16^{-1}$ since

$$\begin{aligned}
 \text{float.hex}(2.0/\pi) &= 0x1.45f306dc9c883p-1 \\
 &= 1.45f306dc9c883_{16} \times 2^{-1} \\
 &= 1.45f306dc9c883_{16}/2 \\
 &\approx a.2f9_{16} \times 16^{-1}
 \end{aligned}$$

with the critical point being the leading digit of the base 16 representation of these two numbers. This digit would become the leading digit of the IBM floating-point version of these numbers. For $\pi/2$, we see that it is 0001_2 , but for $2/\pi$ we get 1010_2 . The former has three leading zeros, meaning only 21 significant bits in the IBM representation, while the latter has no leading zeros, meaning all 24 significant bits matter. This difference of 3 bits is a loss of about one decimal digit of accuracy.

What happens in the IEEE base 2 format? In this case, there is no loss of precision, which we see immediately from the Python output. For $\pi/2$ we get $1.921fb54442d18_{16} \times 2^0$ while for $2/\pi$ we have $1.45f306dc9c883_{16} \times 2^{-1}$ both of which have a leading 1 digit as the IEEE format specifies. Therefore, all the significand bits are meaningful, and there is no wobbling precision effect.

Before we leave this section, we describe a small C function for converting 32-bit IBM S/360 floating-point numbers to IEEE double-precision numbers. Double precision is necessary, as we have seen that the single-precision IEEE format does not have the numeric range necessary to hold all 32-bit IBM floats. The IBM format is assumed to be stored in an unsigned 32-bit integer which allows all bit patterns. The function is

```

1 double ibm_to_ieee_64(uint32_t n) {
2     int s, e, m;
3     double ans;
4
5     s = (n & (1<<31)) != 0;
6     e = (n >> 24) & 0x7F;
7     m = n & 0xFFFFFFF;
8
9     ans = m * pow(16,-6) * pow(16,e-64);
10    return (s == 1) ? -ans : ans;
11 }
```

This function may be helpful if one encounters an old data file in IBM format. It could happen.

The sign of the IBM float is extracted in line 5, the exponent in line 6, and the significand in line 7 using masking and shifting operations. The resulting double-precision floating-point number is calculated in line 9. The significand of the IBM format is of the form $0.h_0h_1h_2h_3h_4h_5$ so the value in *m* must be multiplied by 16^{-6} before the final multiplication by 16^{e-64} where we subtract 64 because the exponent is in excess-64 format. Line 10 simply sets the sign of the output and returns it. This function assumes that it is running on a system which uses IEEE floating point by default.

3.4 IEEE 754 Floating-Point Representations

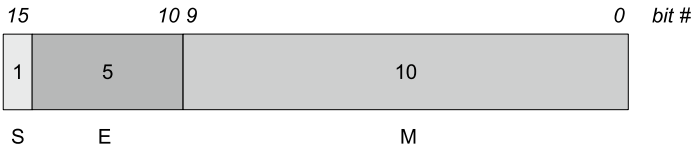
The IEEE 754 standard defines multiple floating-point formats for binary ($\beta = 2$) and decimal ($\beta = 10$) numbers. We are only concerned with the binary formats here. These are, with their ranges,

Name	Minimum	Maximum
<i>binary16</i>	$2^{-14} \approx 6.104 \times 10^{-5}$	$(2 - 2^{-10}) \times 2^{15} = 65504.0$
<i>binary32</i>	$2^{-126} \approx 1.175 \times 10^{-38}$	$(2 - 2^{-23}) \times 2^{127} \approx 3.403 \times 10^{38}$
<i>binary64</i>	$2^{-1022} \approx 2.225 \times 10^{-308}$	$(2 - 2^{-52}) \times 2^{1023} \approx 1.798 \times 10^{308}$

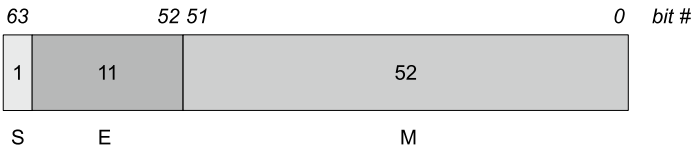
where *binary16* is half-precision, *binary32* is single precision (C `float`), *binary64* is double precision (C `double`), and *binary128* (not shown) is quadruple precision, sometimes called “quad precision”.

The *binary128* format uses 128 bits, and some C compilers implement it in software. More often, however, compilers for x86 architectures will use the 80-bit extended precision hardware type instead. Because of this, we will largely ignore *binary128* in this chapter. If you are familiar with Fortran, the *binary32* format is roughly equivalent to a `REAL`, while the *binary64* format is similar to `REAL*8` and the *binary128* format is like `REAL*16`.

The *binary32* format was illustrated above in Sect. 3.3. For completeness, we illustrate *binary16*



and *binary64*



which follow the same format as *binary32* but differ in the exponent and significand bits. For *binary16*, the exponent is 5 bits and is stored in excess-15 format. The exponent for *binary32* is 8 bits and uses excess-127 format. Lastly, *binary64* uses an exponent of 11 bits stored in excess-1023 format.

The observant reader will notice that the exponents in the table do not cover the full range of possible bit values. For example, the *binary32* format uses 8 bits for the exponent, which means any exponent from $0 - 127 = -127$ to $255 - 127 = 128$ should be possible but the actual exponent range is only -126 to 127 . This is because IEEE 754 reserves the smallest and largest exponent values for specific purposes. The smallest exponent value, the bit pattern of all zeros, designates either zero itself, or a *subnormal* number. We will explore subnormal numbers later in the chapter.

The largest exponent bit pattern, either 11111_2 , 1111111_2 , or 1111111111_2 for *binary16*, *binary32*, or *binary64*, respectively, is reserved for infinity or NaN (not-a-number). The selection between zero and a subnormal number or infinity and NaN depends upon the value of the significand (M). Specifically,

Exponent	M = 0	M ≠ 0
<i>largest</i>	$\pm\infty$	NaN
<i>smallest</i> (=0)	± 0	<i>subnormal</i>

where the sign bit determines positive or negative. Note carefully that this convention implies that zero is signed, but the standard requires $+0 = -0$, i.e., zero is interpreted as zero, regardless of the sign bit.

Most readers are likely familiar with the single- and double-precision floating-point numbers, *binary32* and *binary64*, respectively. These generally map to the well used C data types of `float` and `double`. Note, however, that the Python data type `float` is really a *binary64* number. The addition of *binary16* to the IEEE standard seems a bit odd as it does not map to any primitive data type typically used in modern programming languages, but, as we'll learn in Chap. 11, it has found a home in the world of artificial intelligence.

Additionally, half-precision floats, another common name for *binary16* numbers, are supported in recent versions of `gcc` for ARM targets. Newer Intel microprocessors also have instructions for working with *binary16* numbers. In both cases, the storage in memory uses half-precision, but once read from memory, the numbers are converted to 32-bit floats. The main use for these numbers is to increase the precision of values stored on graphics processors. Indeed, *OpenGL* supports *binary16* numbers. Naturally, however, this comes at a price, namely, a rapidly increasing loss of precision as the value stored moves further and further from zero. As illustrated in Fig. 3.1, the difference between floating-point values increases as one moves further away from zero. On the plus side, this means that if one needs to work with numbers in the range $[0, 1]$, then unless very high precision is required, *binary16* is a reasonable way to keep relatively high precision (about 6 or 7 decimals) while using half the memory of a 32-bit float and one-quarter the memory of a 64-bit float. This might be significant if there are large arrays of these values to work with, as the bandwidth to move that memory is much less than it might otherwise be. It also may help by allowing more data to fit in a processor cache, reducing the number of expensive and slow accesses from main memory. Finally, NVIDIA CUDA, starting with version 7.5, supports half-precision floating-point storage and arithmetic.

Infinity For any of the IEEE formats, if the exponent is maximum, i.e., all bits are set to 1, and the significand is zero, then the number is infinite (∞) and positive or negative depending upon the sign bit. Infinity is produced when an operation's result exceeds the range of numbers that can be expressed in the given floating-point format. For example,

$$\begin{aligned}\exp(1000000.0) &= \text{Inf} = +\infty \\ -\exp(1000000.0) &= -\text{Inf} = -\infty\end{aligned}$$

Infinity is also returned for certain other operations, such as

$$\begin{aligned}1.0/0.0 &= \text{Inf} = +\infty \\ \log(0.0) &= -\text{Inf} = -\infty\end{aligned}$$

which follow the usual rules of arithmetic. The $-\infty$ result for $\log(0.0)$ makes sense as a limit as $x \rightarrow 0$ from the right.

As mentioned in the IEEE standard, the following operations are valid and work as expected when using ∞ as an argument and x finite (and not zero),

$$\begin{aligned}\infty + x &= \infty, & x + \infty &= \infty \\ \infty - x &= \infty, & x - \infty &= -\infty \\ \infty \times x &= \infty, & x \times \infty &= \infty \\ \infty \div x &= \infty, & x \div \infty &= 0\end{aligned}$$

Additionally, the square root of infinity is defined to be infinity.

Not-A-Number While infinity is straightforward, Not-a-Number, or NaN, gets messy. A NaN is stored as a floating-point number with the largest possible exponent (all bits 1) and a significand that is *not* zero. There are two kinds of NaNs: quiet and signaling. We will discuss this distinction more fully below when discussing floating-point exceptions. For now, we are concerned with how the two kinds of NaNs are stored so that we can distinguish between them.

According to the standard, the first bit of the significand, which is d_1 in the representation $\pm d_0.d_1d_2d_3 \dots d_{p-1} \times \beta^e$ since d_0 is always implied and 1, determines the NaN type. If this bit is zero, the NaN is a signaling NaN; otherwise, it is a quiet NaN. This implies that a signaling NaN must have at least one other bit of the significand be nonzero; otherwise, the number is not a NaN but infinity. The remaining $p - 2$ digits of the significand are available for a “payload”, to use the standard’s term, which may contain diagnostic information about what caused the NaN in the first place.

The standard is intentionally fuzzy about what, if anything, should constitute the payload of a NaN. However, the standard is explicit that NaNs, when used as an argument in an operation, should preserve the payload of at least one of the NaNs, or the NaN, if it is the only NaN in the operation. Operations on NaNs produce NaNs as output. The vague nature of the NaN payload invites creative use. Indeed, some groups and companies have used NaNs for their own purposes. Since it is not specified beyond indicating a bit for quiet or signaling, users can use NaNs as they see fit. One possibility might be to use NaNs as symbols in a symbolic expression parser. Another would be to use NaNs as missing data values and the payload to indicate a source for the missing data or its class.

The following set of C functions can be used to check for NaN and to set and get a 22-bit payload from a NaN,

```

1 | typedef union {
2 |     float f;
3 |     unsigned int d;
4 | } fp_t;
5 |
6 | char nan_isnan(float nan) {
7 |     fp_t x;
8 |     x.f = nan;
9 |     return (((x.d >> 23) & 0xFF) == 0xFF) && ((x.d & 0x7FFFFFFF) != 0);
10| }
11|
12| float nan_set_payload(float nan, unsigned int payload) {
13|     fp_t x;
14|     x.f = nan;
15|     x.d |= (payload & 0x3FFFFFFF);
16|     return x.f;
17| }
18|
19| unsigned int nan_get_payload(float nan) {
20|     fp_t x;
21|     x.f = nan;
22|     return (x.d & 0x3FFFFFFF);
23| }

```

The function `nan_isnan` checks to see if a float is actually a NaN, `nan_set_payload` sets the free 22 bits of a NaN to a user-specified value, and `nan_get_payload` returns this value from a NaN. Most systems already have a `isnan` function; we include it here for completeness. The functions use a trick to work with the same piece of memory as both a float and an unsigned integer, both of which occupy 32 bits. This is the `fp_t` data type defined in lines 1 through 4. The function `nan_isnan` accepts a *binary32* number and puts it into `x` (line 8). We then interpret the memory of the float as an unsigned integer by accessing the `d` field. Line 9 is the magic in two parts, both of which come directly from the definition of a NaN. The first part, `((x.d >> 23) & 0xFF) == 0xFF`, examines the exponent field of the number. Recall that a *binary32* number has 23 bits in the significand. We shift 23 bits down to move the exponent field into bit position 0. We then mask this to keep the first 8 bits (the AND with `0xFF`). This removes any sign bit, which the standard ignores for NaNs. If this is the exponent of an actual NaN, then, by definition, the exponent is all 1 bits, which is `0xFF`, so we check if the value is equal to `0xFF`. However, an exponent of all 1 bits is not enough to define a NaN, as the number could still be infinity. We must consider the significand itself. This is the second part of line 9, `((x.d & 0x7FFFFFFF) != 0)`, which masks off the 23 bits of the significand by the AND with `0x7FFFFFFF` and then checks to see if this value is zero. If the number is a NaN, this value will not be zero. If it is a quiet NaN, the highest bit will be set at a minimum, and if it is a signal NaN, at least one of the other 22 bits will need to be set. If both of these parts are true, the number is a NaN, and the result of the final logical AND is returned.

The `nan_set_payload` function accepts a NaN and a payload, which must fit in 22 bits. A `fp_t` is used to hold the NaN. The payload is masked with `0x3FFFFFF` to keep it in range and then OR'ed into the bits of the NaN in line 15. The updated NaN is returned in line 16. To access the NaN value, use `nan_get_payload`.

To illustrate the idea of NaN propagation, consider the following code snippet, compiled on a 32-bit Linux system using `gcc`,

```

1 | const char *pp(unsigned int x) {
2 |     static char b[33];
3 |     unsigned int z;
4 |     b[0] = '\0';
5 |     for (z = (1<<31); z > 0; z >>= 1)
6 |         strcat(b, ((x & z) == z) ? "1" : "0");
7 |     return b;
8 | }
9 |
10 | int main() {
11 |     fp_t x;
12 |     float n = log(-1);
13 |     unsigned int d;
14 |
15 |     x.f = n;
16 |     printf("%d original      = %s\n", nan_isnan(n), pp(x.d));
17 |     n = nan_set_payload(n, 12345);
18 |
19 |     x.f = n;
20 |     printf("%d set payload = %s\n", nan_isnan(n), pp(x.d));
21 |     d = nan_get_payload(n);
22 |     printf("payload is %d (%s)\n", d, pp(d));
23 |
24 |     n = 123 * n;
25 |
26 |     d = nan_get_payload(n);
27 |     printf("payload is still %d (%s)\n", d, pp(d));
28 | }
```

The function `pp` prints the given 32-bit unsigned integer in binary as a string. In line 12, we define `n` and set it so `log(-1)` which is a NaN. In line 15, we put this NaN into `x` and print the result of asking whether `n` is a NaN (it is) and showing the bits of `n`,

```
1 original = 01111111110000000000000000000000
```

Line 17 sets the payload for the NaN to the unimaginative air-shield combination of 12345. Lines 19 through 22 show that the NaN is still a NaN but now has a payload,

```
1 set payload = 01111111110000000011000000111001
payload is 12345 (00000000000000000011000000111001)
```

where $11000000111001_2 = 12345$. So far, this isn't anything awe-inspiring. Line 24, however, multiplies `n` by 123 and assigns back to `n` to illustrate NaN propagation, which the IEEE standard strongly encourages. The output of line 27 is

```
payload is still 12345 (00000000000000000000000011000000111001)
```

which demonstrates that the payload was propagated.

Subnormal Numbers A normal number is one in which the leading bit of the significand is an implied 1 and the exponent, e , is $0 < e < m$, for m the maximum exponent value, which depends upon the precision of the number. For *binary32*, $m = 255$ since 8 bits are used to store the exponent. Therefore, the smallest normal *binary32* number is

$$0\ 00000001\ 000000000000000000000000_2 = 2^{-126} = 1.1754944 \times 10^{-38}$$

where we have separated the binary representation into the sign, exponent, and significand. If we insist on using an implied leading 1 bit for the significand, the next smallest number we can represent is immediately zero. However, if we are willing to suffer a loss of precision in the significant, we can drop the exponent to zero and change the implied leading digit of the significand from 1 to 0 to represent numbers as

$$\pm 0.d_1d_2 \dots d_{23} \times 2^{-126}$$

Numbers of this form are called *subnormal* or *denormal*. Subnormal numbers allow us to work with very small numbers around zero but at the small cost of losing precision in the significand. However, there is a considerable cost in terms of performance as modern floating-point hardware does not work with subnormal numbers, implying calculations must be performed in software.

The program in Fig. 3.2 illustrates the transition from normal to subnormal numbers for 32-bit floats. It initializes a float to 1.0 and then repeatedly divides it by 2 until the result is zero. The function `ppx` prints the bits of the floating-point number with a space between the sign, the exponent, and the significand. If we run this program, the tail end of the output is


```

x = (00000000100000000000000000000000)2.3509887e - 38
x = (00000000100000000000000000000000)1.1754944e - 38
x = (00000000010000000000000000000000)5.8774718e - 39
x = (00000000001000000000000000000000)2.9387359e - 39
x = (00000000000100000000000000000000)1.4693679e - 39
x = (00000000000010000000000000000000)7.3468397e - 40
x = (00000000000001000000000000000000)3.6734198e - 40
x = (00000000000000100000000000000000)1.8367099e - 40
x = (00000000000000010000000000000000)9.1835496e - 41
x = (00000000000000001000000000000000)4.5917748e - 41
x = (00000000000000000100000000000000)2.2958874e - 41
x = (00000000000000000010000000000000)1.1479437e - 41
x = (00000000000000000001000000000000)5.7397185e - 42
x = (00000000000000000000100000000000)2.8698593e - 42
x = (00000000000000000000010000000000)1.4349296e - 42
x = (00000000000000000000001000000000)7.1746481e - 43
x = (00000000000000000000000100000000)3.5873241e - 43
x = (00000000000000000000000010000000)1.793662e - 43
x = (00000000000000000000000001000000)8.9683102e - 44
x = (00000000000000000000000000100000)4.4841551e - 44
x = (00000000000000000000000000010000)2.2420775e - 44
x = (00000000000000000000000000000100)1.1210388e - 44
x = (00000000000000000000000000000010)5.6051939e - 45
x = (00000000000000000000000000000001)2.8025969e - 45
x = (00000000000000000000000000000000)1.4012985e - 45
x = (00000000000000000000000000000000)0

```

where the first two numbers are normalized, the exponent is nonzero. After these start the subnormalized numbers, the first of which is

$$0\ 00000000\ 1000000000000000000000_2 = 2^{-1} \times 2^{-126} = 5.8774718 \times 10^{-39}$$

and the smallest positive subnormal number is therefore $2^{-23} \times 2^{-126} = 1.4012985 \times 10^{-45}$.

The penalty in performance for using subnormal numbers can be found by timing how long it takes to run 10 million iterations of a simple multiplication of x by 1 where x is first the smallest normal number and then the largest subnormal number. If we do this, being careful to make sure the loop is actually performed and not optimized away by the compiler, we see that subnormal numbers are about 23 times slower than normal numbers on the machine used.

3.5 Rounding Floating-Point Numbers (IEEE 754)

Often, when a real number is expressed in floating-point format, it is necessary to round it so that it can be stored as a valid floating-point number. The IEEE standard defines four rounding modes, or rules, for binary floating point. These are available to

```
1 | #include <stdio.h>
2 | #include <string.h>
3 |
4 | const char *ppx(unsigned int x) {
5 |     static char b[36];
6 |     unsigned int z, i=0;
7 |     b[0] = '\0';
8 |     for (z = (1<<31); z > 0; z >>= 1) {
9 |         strcat(b, ((x & z) == z) ? "1" : "0");
10 |         if ((i == 0) || (i == 8))
11 |             strcat(b, " ");
12 |         i++;
13 |     }
14 |     return b;
15 | }
16 |
17 | typedef union {
18 |     float f;
19 |     unsigned int d;
20 | } fp_t;
21 |
22 | int main() {
23 |     fp_t x;
24 |     int i;
25 |     x.f = 1.0;
26 |     for(i=0; i < 151; i++) {
27 |         printf("x = (%s) %0.8g\n", ppx(x.d), x.f);
28 |         x.f /= 2.0;
29 |     }
30 | }
```

Fig. 3.2 A program to display the transition from normal to subnormal floating-point numbers

Round to Nearest	Round to the nearest floating-point number. Ties round to even.
Round Towards Zero	Round down, if positive, or up, if negative, always towards zero.
Round Towards +∞	Round up, always towards positive infinity.
Round Towards −∞	Round down, always towards negative infinity.

Fig. 3.3 The four IEEE 754 binary floating-point rounding modes

programmers in C on Linux if using the gcc compiler (among others). The rounding modes are listed in Fig. 3.3.

The default mode for gcc, and the one recommended by the IEEE standard, is to round towards the nearest value with ties rounding towards the even value. The even value is the one with a zero in the lowest order bit of the significand. For example, using decimal and rounding to an integer value “round to nearest” acts like so

$$\begin{aligned} +123.7 &\rightarrow +124 \\ +123.4 &\rightarrow +123 \\ +123.5 &\rightarrow +124 \\ -123.7 &\rightarrow -124 \\ -123.4 &\rightarrow -123 \\ -123.5 &\rightarrow -124 \end{aligned}$$

which is more or less the way we expect rounding to work. The stipulation of rounding ties to even helps prevent the accumulation of rounding errors as in binary this will happen about 50% of the time.

The other, directed, rounding modes work in this way,

Value	Round towards Zero	Round towards $+\infty$	Round towards $-\infty$
+123.7	+123	+124	+123
+123.4	+123	+124	+123
+123.5	+123	+124	+123
-123.7	-123	-123	-124
-123.4	-123	-123	-124
-123.5	-123	-123	-124

The directed rounding modes shift results towards $\pm\infty$ or zero, always. Round to nearest is used so often that many software libraries do not function properly if one of the directed modes is used [8].

It is possible to alter the rounding mode programmatically. Figure 3.4 is a Linux program that changes the rounding mode while displaying the floating-point representation of $\exp(1.1)$ and $-\exp(1.1)$.

This program produces the following output:

```
FE_TONEAREST:
  exp(1.1) = (0 10000000 10000000100010001000010) 3.00416613
  -exp(1.1) = (1 10000000 10000000100010001000010) -3.00416613

FE_UPWARD:
  exp(1.1) = (0 10000000 10000000100010001000010) 3.00416613
  -exp(1.1) = (1 10000000 10000000100010001000001) -3.00416589

FE_DOWNWARD:
  exp(1.1) = (0 10000000 10000000100010001000001) 3.00416589
  -exp(1.1) = (1 10000000 10000000100010001000010) -3.00416613

FE_TOWARDZERO:
  exp(1.1) = (0 10000000 10000000100010001000001) 3.00416589
  -exp(1.1) = (1 10000000 10000000100010001000001) -3.00416589
```

```

1 #include <stdio.h>
2 #include <fenv.h>
3 #include <math.h>
4 #include <string.h>
5
6 // compile: gcc rounding.c -o rounding -lm -frounding-math
7
8 int main() {
9     fp_t fp;
10
11     printf("\nFE_TONEAREST:\n");
12     fesetround(FE_TONEAREST);
13     fp.f = exp(1.1);
14     printf("    exp(1.1) = (%s) %0.8f\n", ppx(fp.d), fp.f);
15     fp.f = -exp(1.1);
16     printf("   -exp(1.1) = (%s) %0.8f\n\n", ppx(fp.d), fp.f);
17
18     printf("FE_UPWARD:\n");
19     fesetround(FE_UPWARD);
20     fp.f = exp(1.1);
21     printf("    exp(1.1) = (%s) %0.8f\n", ppx(fp.d), fp.f);
22     fp.f = -exp(1.1);
23     printf("   -exp(1.1) = (%s) %0.8f\n\n", ppx(fp.d), fp.f);
24
25     printf("FE_DOWNWARD:\n");
26     fesetround(FE_DOWNWARD);
27     fp.f = exp(1.1);
28     printf("    exp(1.1) = (%s) %0.8f\n", ppx(fp.d), fp.f);
29     fp.f = -exp(1.1);
30     printf("   -exp(1.1) = (%s) %0.8f\n\n", ppx(fp.d), fp.f);
31
32     printf("FE_TOWARDZERO:\n");
33     fesetround(FE_TOWARDZERO);
34     fp.f = exp(1.1);
35     printf("    exp(1.1) = (%s) %0.8f\n", ppx(fp.d), fp.f);
36     fp.f = -exp(1.1);
37     printf("   -exp(1.1) = (%s) %0.8f\n\n", ppx(fp.d), fp.f);
38 }

```

Fig. 3.4 A program to display the effect of different rounding modes. Note that `fp_t` and `ppx()` are defined in Fig. 3.2

which shows the floating-point number in decimal as well as the actual bits used to store the value. Note that Fig. 3.4 includes a comment line describing the actual command used to compile the program. The `-frounding-math` option is required; otherwise, `fesetround()` will not actually change the rounding mode.

Examining the output, we notice that `FE_TONEAREST`, the default rounding mode, produces results that match what we expect since negating the value is not expected to alter the result other than the sign bit. We see this clearly by looking at the bits of the results; they are all the same except for the sign bit.

The remaining directed modes act as expected. For `FE_UPWARD`, we see that the negative result is smaller than the positive, in magnitude, because it has been rounded towards positive infinity. Similarly, we see the magnitudes reversed for `FE_DOWNWARD` which is rounding towards negative infinity. Lastly, `FE_TOWARDZERO` rounds towards zero, giving the smaller magnitude result for both positive and negative cases. The floating-point bit patterns for `FE_TONEAREST` and `FE_TOWARDZERO` are the same in each case except for the sign bit, while the other two cases are similar, but the positive and negative bit patterns are flipped (for the significand).

If the default of “round to nearest” is so widely used, why are the other rounding modes in the IEEE standard? One example of why can be found in [9] where directed rounding significantly improved the accuracy of the calculations. We will use rounding modes in Chap. 8 when implementing interval arithmetic. However, specialized uses aside, it is unnecessary to modify the rounding mode from the default in most cases.

3.6 Comparing Floating-Point Numbers (IEEE 754)

The IEEE standard requires floating-point numbers to maintain an explicit ordering for comparisons. The simplest way to compare two floating-point numbers is to look at their bit patterns. For example, the following two numbers are as close as two 32-bit floating-point numbers can be without being equal, meaning no representable floating-point numbers exist between them. The numbers are

```
0.500000000 = 00111111000000000000000000000000 = 1056964608
0.500000060 = 00111111000000000000000000000001 = 1056964609
```

where the floating-point value is on the left, the actual bit patterns are in the center, and the equivalent 32-bit signed integer is on the right. Note that when viewed as an integer, the smaller number, 0.5, is also smaller. This is no accident. The floating-point format is designed to make floating-point comparison as straightforward as comparing two signed integers, where negative floating-point numbers are interpreted in two’s-complement format. The only exception is zero since it can be signed or unsigned. But it is easy to detect this single case.

The C code for a floating-point comparison function that checks for the special case of signed zero is

```

1 | typedef union {
2 |     float f;
3 |     int d;
4 | } sp_t;
5 |
6 | int fp_compare(float a, float b) {
7 |     sp_t x,y;
8 |
9 |     x.f=a;
10 |    y.f=b;
11 |
12 |    if ((x.d == (int)0x80000000) &&
13 |        (y.d == 0)) return 0;
14 |    if ((y.d == (int)0x80000000) &&
15 |        (x.d == 0)) return 0;
16 |
17 |    if (x.d == y.d) return 0;
18 |    if (x.d < y.d) return -1;
19 |    return 1;
20 |}

```

where we introduce `sp_t` instead of `fp_t` which we used previously. We now use a signed integer instead of an unsigned integer. The signed integer is necessary so that the integer comparisons will treat negative floating-point numbers properly.

The `fp_compare` function accepts two floats, a and b , as input and returns -1 if $a < b$, 0 if $a = b$, and +1 if $a > b$. In lines 9 and 10, the input floats are placed into two `sp_t` variables, x and y . This is so we can get the equivalent signed integer from the floating-point bit pattern. Lines 12 and 13 are the special check for negative zero compared to positive zero. In this case, we want a return value of 0 since the standard says these values are equal. The cryptic `(int)0x80000000` is the bit pattern for a negative zero number cast to a signed integer. If this is the first value and the second is positive zero, consider them the same. Lines 14 and 15 make the same comparison with the arguments reversed. Line 17 checks if the bit patterns are the same and returns zero if they are to declare the values equal. Line 18 checks to see if $a < b$ and returns -1 if they are. The only option left is $a > b$, so we return +1 in line 19, thereby completing the predicate function.

For completeness, let's define functions for the basic comparison operators. We can then check these against the intrinsic operators in C. The functions are

```
1 int fp_eq(float a, float b) {
2     return fp_compare(a,b) == 0;
3 }
4
5 int fp_lt(float a, float b) {
6     return fp_compare(a,b) == -1;
7 }
8
9 int fp_gt(float a, float b) {
10    return fp_compare(a,b) == 1;
11 }
```

And we can use them with the following code:

```
1 int main() {
2     float a,b;
3
4     a = 3.13; b = 3.13;
5     printf("3.13 == 3.13: %d %d\n", fp_eq(a,b), a==b);
6
7     a = 3.1; b = 3.13;
8     printf("3.1 < 3.13: %d %d\n", fp_lt(a,b), a<b);
9
10    a = 3.2; b = 3.13;
11    printf("3.2 > 3.13: %d %d\n", fp_gt(a,b), a>b);
12
13    a = -1; b = 1;
14    printf("-1 < 1: %d %d\n", fp_lt(a,b), a<b);
15 }
```

which shows that comparing the integer representations gives the results we would expect

```
3.13 = 3.13 : 11
3.10 < 3.13 : 11
3.20 > 3.13 : 11
-1.00 < 1.00 : 11
```

In this discussion, we have ignored NaNs and infinity. According to the standard, NaNs are unordered even when compared to themselves. Infinities of the same sign are equal, while opposite signs should compare as expected. This is not the case with the simple example above in `fp_compare`, which should be updated to compare infinities properly.

3.7 Basic Arithmetic (IEEE 754)

The standard mandates that conforming implementations supply the expected addition (+), subtraction (−), multiplication (×), and division (/) operators. It also requires the square root of a positive number and a fused multiply-add, which implements $(xy) + z$ with no exceptions thrown during the multiplication. This combined operation only involves one rounding step, not the two that would happen if the multiplication was done separately from the addition.

Addition and Subtraction. The addition of two floating-point numbers is straightforward. Let's consider the steps necessary to calculate $8.76 + 1.34$. First, we represent each number in binary as

$$\begin{aligned} 8.76 &\rightarrow 8.760000229 = 01000001000011000010100011110110 \\ 1.34 &\rightarrow 1.340000033 = 0011111101010111000010100011111 \end{aligned}$$

where we immediately see that neither 1.34 nor 8.76 can be exactly represented using a 32-bit float and that we have again shown the binary representation separating the sign, exponent, and significand. If we convert the bit pattern to actual binary, we get

$$\begin{aligned} 8.76 &\rightarrow 1.00011000010100011110110 \times 2^3 \\ 1.34 &\rightarrow 1.01010111000010100011111 \times 2^0 \end{aligned}$$

where we see that the exponents are not the same. To add the significands, we must first make the exponents the same. So, we rewrite the smaller number to have the same exponent as the larger one. This means we add the difference of the exponents to the exponent of the smaller number, $3 - 0 = 3 \rightarrow 3 + 0 = 3$, and shift the significand of the smaller number that many bit positions to the left to get

$$\begin{aligned} 8.76 &\rightarrow 1.00011000010100011110110 \ 000 \times 2^3 \\ 1.34 &\rightarrow 0.00101010111000010100011 \ 111 \times 2^3 \end{aligned}$$

with the bits that would have been shifted out of the 32-bit float separated from the rest of the significand by a space. Usually, the calculation is performed at a higher precision than the storage format, and rounding is applied. Three zeros fill in the top value for the extended precision bits.

With the exponents the same, the addition is straightforward: add the significand, bit by bit, as one would add 2.5×10^3 and 4.6×10^3 to get 7.1×10^3 ,

$$\begin{array}{r} 8.76 \rightarrow 1.00011000010100011110110 \ 000 \times 2^3 \\ + 1.34 \rightarrow 0.00101010111000010100011 \ 111 \times 2^3 \\ \hline 1.01000011001100110011001 \ 111 \times 2^3 \end{array}$$

If the answer does not have a single 1 to the left of the decimal point, we shift the significand to the right until it does to make our answer a normalized number. If we shift the significand n bits to the right, we must raise the exponent by adding n . The

answer is already normalized in the example above since the first and only bit to the left of the decimal point is 1.

There is one step remaining, which is to account for rounding. Assuming that the rounding mode is the default of “round to nearest”, we must consider the first 1 in the three lowest order bits of the answer, the 3 bits on the far right of the significand. To round to the nearest floating-point number, we add one to the rightmost bit of the significand, before the space, to make $001 \rightarrow 010$. In this case, then, the final answer is

$$10.100000381 \rightarrow 1.01000011001100110011010 \times 2^3$$

Subtraction works in the same way. Specifically, the steps to add or subtract two floating-point numbers x and y are

1. Raise the exponent of the number with the smaller exponent by the difference between the larger and smaller exponent. Also, shift the significand of the smaller number to the right by the same number of bits. This causes both numbers to have the same exponent. If the smaller number was y , the shifted number is y' .
2. Add or subtract the significands of x and y' as they now have the same exponent. Put the answer in z .
3. If there is not a single 1 to the left of the decimal point in z , shift the significand of z to the right until there is. Increase the exponent of z by the number of bits shifted.

Multiplication. To multiply two numbers in scientific notation, we multiply the significands and add the exponents

$$(a \times 10^x)(b \times 10^y) = ab \times 10^{x+y}$$

Similarly, for two floating-point numbers expressed in binary, we multiply the significands, add the exponents, and then normalize. Therefore, to multiply 3.141592 by 2.1, we have

$$\begin{array}{lll} 3.141592 & \rightarrow 3.141592026 & \rightarrow 01000000010010010000111111011000 \\ 2.1 & \rightarrow 2.099999905 & \rightarrow 01000000000001100110011001100110 \end{array}$$

where the first column is the real number we want to multiply, the second column is the actual floating-point number we get when using *binary32*, and the last column is the actual bit pattern in memory. Using the bit pattern, we can write these values in binary as

$$\begin{array}{ll} 3.141592 & \rightarrow 1.10010010000111111011000 \times 2^1 \\ 2.1 & \rightarrow 1.00001100110011001100110 \times 2^1 \end{array}$$

which means that our product will have an exponent of $1 + 1 = 2$. We can multiply the significands bit by bit, which is tedious on paper, so instead, we approximate the product using only 7 bits after the radix point,

$$\begin{array}{r}
 1.1001001 \\
 \times \quad 1.0000110 \\
 \hline
 0 \\
 1110010010 \\
 11100100100 \\
 \dots \\
 110010010000000 \\
 + 1.10100100110110
 \end{array}$$

where we have already moved the radix point to the proper position in the final sum. The $\dots$ represents a series of all zero places corresponding to the string of four zeros in the multiplier. The last line shows the last partial product. The full product of the significands is $1.10100110001110101101111$.

Since the significand product already has a leading value of one, we do not need to shift. For example, if the significand product started with $10.00101\dots$, we would shift the radix point one position to the left to make the significand $1.000101\dots$, and increase the exponent by one.

Finally, our now approximate product (because we dropped many significand bits) becomes

$$1.10100100110110 \times 2^2 \rightarrow 6.5756835938$$

while the full precision product is $3.141592 \times 2.1 = 6.5973429545$.

In summary, then, the steps to multiply two floating-point numbers x and y are

1. Add the exponents of x and y .
2. Multiply the significands of x and y .
3. Shift the product of the significands to get a single leading 1 bit to the left of the radix point. Adjust the exponent as well by adding one for every bit position the product of the significands is shifted to the right. This is equivalent to shifting the radix point to the left.
4. Reconcile the signs so that the final product has the expected sign.

This concludes our look at floating-point arithmetic. Next, we consider an essential part of the IEEE 754 standard we have neglected until now, exceptions.

3.8 Handling Exceptions (IEEE 754)

Not all floating-point operations produce meaningful results for all operands. An essential part of the IEEE 754 standard addresses what to do when exceptions to normal floating-point processing arise. The standard recommends a default set of

actions for these cases and then allows implementations to give users the power to trap and respond to exceptions. In this section, we learn what these exceptions are, the default responses the standard recommends for them, and how to trap exceptions to bring them under user control in C. It is important to remember that floating-point exception handling is distinct from the concept of exceptions in programming languages, though they are, of course, related and could be intermingled. Modern processors, including the x86 line, process floating-point exceptions in hardware and, as such, are programming language-independent.

Exceptions and Default Handling The standard defines five types of exceptions: *inexact*, *invalid*, *underflow*, *overflow*, and *divide-by-zero*:

Exception type	Description	Examples
<i>inexact</i>	When the result is not exact due to rounding	2.0/3.0
<i>invalid</i>	When the operands are not valid for the operation	0/0, $0 \times \infty$, $\sqrt{x}$, $x < 0$
<i>underflow</i>	When the result is too small to be accurate or when sub-normal	$(1.17549435 \times 10^{-38})/3.0$
<i>overflow</i>	When the result is too large to represent	$(3.40282347 \times 10^{38})^2$
<i>divide-by-zero</i>	When a finite number is divided by zero	1/0

The standard also defines default behavior for these exceptions. The goal of the default behavior is to allow the computation to proceed by substituting a particular value for the exception or to ignore the exception altogether. For example, the default behavior for an *inexact* exception is to ignore it and continue with the calculation. The default behavior for an *invalid* exception is generating a NaN and then propagating it through the remainder of the calculation. If *divide-by-zero* happens, the result is passed on as $\pm\infty$ depending upon the signs of the operands.

The *overflow* exception is raised when the result does not fit. The default behavior depends upon the rounding mode in effect at the time and the sign of the number. Specifically,

Round to Nearest	Return $\pm\infty$ depending upon the sign of the result
Round Towards Zero	Return the largest number for the format with the sign of the result
Round Towards $+\infty$	Return $+\infty$ if positive, largest negative number if negative
Round Towards $-\infty$	Return $-\infty$ if negative, largest positive number if positive

Lastly, *underflow* is raised when the number is too small. The implementer is allowed to determine what “too small” means. The default result returned will be a subnormal number or zero.

Trapping Exceptions For many programs, the default behavior is perfectly acceptable. However, for total control over floating-point operations, the standard allows for traps that run user code when an exception happens. For x86 architectures, traps are in hardware but can be used from programming languages such as C. When a trap is executed or used by a higher level library, the programmer can control what happens when exceptions occur. Let's explore how to do this using `gcc`.

The first thing to be aware of is that by default, the `gcc` compiler is not 100% IEEE 754 compliant. Therefore, to get the expected performance, we need to use `-frounding-math` and `-fsignaling-nans` when compiling. Also, we must include the `fenv.h` library. With these caveats, the set of functions we use depends upon how we want to control floating-point exceptions. We can throw a signal when they happen (`SIGFPE`), or we can test individual operations for individual exceptions. Let's look at using a signal first.

A signal is a Unix concept. It is a message to a program to indicate that a particular event has happened. There are many possible signals, but the one we want is `SIGFPE` to signal a floating-point error. We must enable floating-point exceptions, then arrange a signal handler like so

```

1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <signal.h>
4 #include <fenv.h>
5
6 void fpe_handler(int sig) {
7     if (sig != SIGFPE) return;
8     printf("Floating point exception!\n");
9     exit(1);
10 }
11
12 int main() {
13     float a=1, b=0, c;
14
15     feenableexcept(FE_INVALID | FE_DIVBYZERO |
16                  FE_OVERFLOW | FE_UNDERFLOW);
17     signal(SIGFPE, fpe_handler);
18
19     c = a/b;
20 }
```

Lines 1 through 4 include necessary libraries. Lines 6 through 10 define the signal handler. We check to see if the signal is really a floating-point exception (line 7) and then report it and exit. In `main`, we define some floats and then enable floating-point exception traps by calling `feenableexcept()` with the bit masks for the floating-point exceptions we want to trap. Line 17 ties the `SIGFPE` signal to our handler, `fpe_handler()`. Finally, line 19 tries to divide-by-zero, triggering the exception. This code should be compiled with,

`$ gcc fpe.c -o fpe -lm -frounding-math -fsignaling-nans`
assuming `fpe.c` to be the filename.

Another way to work with floating-point exceptions is to check explicitly if any were raised during a calculation. In this case, we need the `feclearexcept()` and `fetestexcept()` functions to clear any exceptions before the calculation and then to test for particular exceptions after. For example, this program

```
1 #include <stdio.h>
2 #include <signal.h>
3 #include <fenv.h>
4
5 int main() {
6     float c;
7
8     feclearexcept(FE_ALL_EXCEPT);
9     c = 0.0/0.0;
10    if (fetestexcept(FE_INVALID) != 0)
11        printf("FE_INVALID happened\n");
12
13    feclearexcept(FE_ALL_EXCEPT);
14    c = 2.0/3.0;
15    if (fetestexcept(FE_INEXACT) != 0)
16        printf("FE_INEXACT happened\n");
17
18    feclearexcept(FE_ALL_EXCEPT);
19    c = 1.17549435e-38 / 3.0;
20    if (fetestexcept(FE_UNDERFLOW) != 0)
21        printf("FE_UNDERFLOW happened\n");
22
23    feclearexcept(FE_ALL_EXCEPT);
24    c = 3.40282347e+38 * 3.40282347e+38;
25    if (fetestexcept(FE_OVERFLOW) != 0)
26        printf("FE_OVERFLOW happened\n");
27
28    feclearexcept(FE_ALL_EXCEPT);
29    c = 1.0 / 0.0;
30    if (fetestexcept(FE_DIVBYZERO) != 0)
31        printf("FE_DIVBYZERO happened\n");
32 }
```

will produce this output

```
FE_INVALID happened
FE_INEXACT happened
FE_UNDERFLOW happened
FE_OVERFLOW happened
FE_DIVBYZERO happened
```

when compiled with

```
$ gcc fpe_traps.c -o fpe_traps -lm -frounding-math
-fsignaling-nans
```

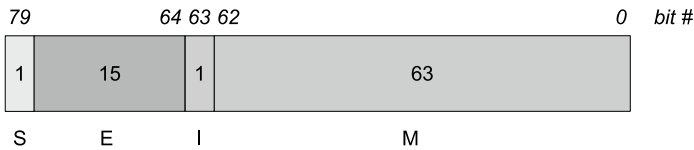
Note, just as `feenableexcept()` took an argument that was the bitwise OR of one or more exceptions, so too will `fetestexcept()`. For example, to test for *invalid* and *divide-by-zero* use

```
int i = fetestexcept(FE_INVALID | FE_DIVBYZERO)
and then check for either with i != 0 or one explicitly with (i & FE_INVALID)
!= 0.
```

3.9 Floating-Point Hardware (IEEE 754)

For this section, we assume a system using modern x86 processors. These processors implement an 80-bit extended precision version of floating point in hardware. This is why floating-point calculations are so fast when, in the past, floating point was generally done in software, especially for smaller systems.

An x86 extended precision float is stored in registers as



where it is essential to note that unlike the IEEE storage formats, there is no implied leading 1 bit for the significand. Instead, it is given explicitly as the first bit marked a “I” above. The sign (S) is 0 for positive and 1 for negative. The exponent (E) is stored as excess-16383, and the fractional part of the significand (M) is 63 bits long. Therefore, a floating-point number (n) is

$$n = (-1)^s i_0 . d_{62} d_{61} \dots d_0 \times 2^{e-16383}$$

where there is now an explicit integer part (i_0) and the binary significand ($d_{62} \dots d_0$). How many bits of the significand are used in a particular calculation is a function of the processor (32 bit vs 64 bit), the compiler, and various compiler settings. We ignore these important but intricate details and instead focus on the representation.

Just as an IEEE *binary32* or *binary64* float uses certain bit combinations to indicate NaN and infinity, so does the hardware version. The explicit integer part helps quickly determine the type of number being manipulated. Table 3.1 illustrates how to decide whether or not the float is normal, subnormal, zero, infinity, NaN, or invalid.

The top part of the table illustrates normal, subnormal, and zero. As is the case for IEEE, there are two possible representations for zero: signed and unsigned. The sign is ignored in calculations. An integer part of zero indicates a subnormal number. If the integer part is 1 and the exponent is any value except all zeros or all ones, the number is a normalized floating-point number. The exceptional values are in the

Table 3.1 How to interpret the bit patterns of an x86 extended precision floating-point number. Only the meaningful combinations of bits are shown; all others should be interpreted as *invalid*. When the CPU performs floating-point calculations, it uses this format, converting to and from IEEE 754 formats like *binary32* and *binary64* as needed. In (a), the exponent and integer parts are used to decide between zero, subnormal, and normal numbers. All normal numbers have an integer part of 1 matching the implied leading 1 bit of the IEEE formats. For (b), the exponent is the special case of all ones. The integer part and leading bit of the significand determine the number type

(a) Normal, Subnormal and Zero:

Exponent	<i>I</i>	<i>M</i>	Value
000000000000000	0	all 0	zero (sign ignored)
000000000000000	0	non-0	subnormal
any but all 1's	1	any	normal

(b) Infinity, Not-a-Number, and Invalid (exponent = 11111111111111):

<i>I</i>	<i>M</i> ₆₂	<i>M</i> ₆₁ . . . <i>M</i> ₀	Value
1	0	0	±∞ (uses sign)
1	0	non-0	signaling NaN
1	1	0	invalid result (eg, 0/0)
1	1	non-0	quiet NaN

bottom part of the table. All of these have the integer part set to 1 and the exponent set to all ones. These values are distinguished based on the value of the significand's leading fractional part and the significand's remaining bits.

If the leading bit, *M*₆₂, is zero, the number is infinity (all remaining significand bits zero) or a signaling NaN. Recall that a signaling NaN raises an exception. If *M*₆₂ is set, the number is either a quiet NaN (the remainder of the significand bits are not zero) or an invalid number (all zero bits).

The bit patterns covered in Table 3.1 are not exhaustive. There are other combinations which might be present. In the past, some of these were used, but now all are considered invalid and will not be generated by the floating-point hardware.

The large number of bits in the significand means that 32-bit and 64-bit floating-point operations can be performed with complete precision and proper rounding of results. Above, we discussed the basic arithmetic operations specified in the IEEE 754 standard. There is one we intentionally ignored at the time: the fused multiply-add operation, *xy* + *z*, which first calculates *xy* and then adds *z* *without* rounding after the multiplication. If this were not done, the expression would involve two roundings, one for the multiplication and another for the addition. This is unnecessary and, with the extended precision of the x86 floating-point format, will result in fewer rounding artifacts. The *xy* + *z* form was chosen because it appears frequently in expressions. An intelligent compiler will look for these expressions to use the fused multiply-add capability whenever possible.

3.10 Binary-Coded Decimal Floating-Point Numbers

In this section, we consider a seemingly obscure representation that is, in fact, one of the most widely used representations in the world: binary-coded decimal floating-point or *BCD float*. BCD floats represent numbers using packed BCD for the significand and ten as the exponent. The integer BCD format is described in detail in Chap. 2, Sect. 2.6. The floating-point version described here is found in a similar form in virtually all desktop calculators, making it one of the world's most widely used number formats. We'll review two different BCD float formats.

The popular line of TI calculators (Texas Instruments) uses the first format. This format uses 9 bytes to store a floating-point number. This seems excessive, but desktop calculators only operate on a few numbers at a time, so the price in processing time is well worth the precision gained. The format is described in more detail in [10]. As a C structure, the format is, ignoring guard digits used to minimize rounding errors during calculations,

```
struct FP {
    uint8_t sign;
    uint8_t exponent;
    uint8_t significand[7];
};
```

where the overall sign of the number is stored as 0x00 for positive and 0x80 for negative. Other values are used for complex numbers, but we'll ignore those here. The exponent is stored with a bias of 0x80, giving a range of $[-128, +127]$. I.e., the actual exponent is found by subtracting 0x80 from the value stored in the exponent field. The significand is stored as packed BCD in 7 bytes, giving 14 decimal digits of precision.

The second BCD float format is found in compilers for microcontrollers. For example, the Raisonance RC-51 C compiler for the 8051 microcontroller supports three sizes of BCD floats: 32 bit (6 decimal digits), 48 bit (10 decimal digits), and 56 bit (14 decimal digits). We will describe the 32-bit format in detail and develop conversion routines to and from IEEE doubles.

A BCD float-32 number is stored in 4 bytes as



where the significand is 3 bytes long (6 decimal digits), and 1 byte is used to store the signs and exponent. The first sign (“S” above) is the overall sign of the number, where $S = 1$ means the number is negative. Instead of being stored as a two's complement number, the exponent uses a sign bit (“N” above). However, in this case, the sense of the sign bit is reversed so that $N = 0$ means that the exponent is negative. The

magnitude of the exponent is stored in 6 bits, which means that the BCD float-32 format can store numbers with exponents in the range $[-63, +63]$. This same format is used for the other BCD floats; only the number of digits in the significand changes.

For example, the number -3.14159×10^6 is stored in BCD float-32 format as $0x314159C7 = 00110001010000010101100111000111_2$,

$$-3.14159 \times 10^6 \rightarrow \begin{array}{cccccccc} 0011 & 0001 & 0100 & 0001 & 0101 & 1001 & 11 & 000111 \\ & 3 & 1 & 4 & 1 & 5 & 9 & S N 7 \end{array}$$

where the significand is represented so there is no integer part: 0.314159 instead of 3.14159. Therefore, the number is stored as 0.314159×10^7 with $S = 1$ because it is negative and $N = 1$ because the exponent is positive.

Conversion from BCD float-32 to double precision (to allow for exponents > 38) is straightforward,

```

1 double bcdfloat32_to_double(uint32_t bcd) {
2     uint32_t sig = bcd >> 8;
3     uint8_t i, sexp = bcd & 0xFF;
4     double ans=0, exp;
5
6     for(i=6; i > 0; i--) {
7         ans += (sig & 0xF) * pow(10,-i);
8         sig >>= 4;
9     }
10
11    exp = (sexp & 0x40) ? (sexp & 0x3F) : -(sexp & 0x3F);
12    ans *= pow(10,exp);
13
14    return (sexp & 0x80) ? -ans : ans;
15 }
```

where `sig` stores the significand (all but lowest order byte, line 2). We store the sign and exponent byte in `sexp` (line 3). Lines 6 through 9 process the nibbles of the significand and add them into the output (`ans`) after scaling them by the appropriate power of ten. Line 7 looks at the lowest significand nibble, multiplies it by 10^{-i} , and adds it to the output. Line 8 shifts the next most-significant digit of the significand into place. At the end of the loop, `ans` contains the significand, so we only need to examine the exponent. Line 11 extracts the exponent by masking off the lowest order bits (AND with 0x3F) and also sets the exponent sign by looking to see if bit 6 is set (`sexp & 0x40`). Line 12 then scales the significand by the exponent. Line 14 checks the overall sign of the number and returns it, negating if necessary.

Conversion from a double to a BCD float-32 is similarly straightforward. For simplicity, we ignore range checks and implement the conversion as

```

1 | uint32_t double_to_bcdfloat32(double d) {
2 |     char s[20];
3 |     uint32_t ans=0, nsgn=1, sgn=0, sexp=0;
4 |     int32_t exp;
5 |
6 |     sprintf(s, "%+0.5e", d);
7 |     if (s[0] == '-') sgn=1;
8 |     if (s[9] == '-') nsgn=0;
9 |     exp = 10*(s[10]-'0') + (s[11]-'0');
10 |    exp = (s[9]=='-') ? -exp : exp;
11 |    exp++;
12 |    sexp = (abs(exp) & 0x3F) + (sgn*0x80) + (nsgn*0x40);
13 |
14 |    ans += (s[1]-'0') << 7*4;
15 |    ans += (s[3]-'0') << 6*4;
16 |    ans += (s[4]-'0') << 5*4;
17 |    ans += (s[5]-'0') << 4*4;
18 |    ans += (s[6]-'0') << 3*4;
19 |    ans += (s[7]-'0') << 2*4;
20 |    ans += sexp;
21 |
22 |    return ans;
23 |}

```

where we cheat a bit by using a call to `sprintf` to convert the double input to a formatted base-10 character string (line 6). Once we have the string representation, we only need to pull it apart to set the fields of the BCD float-32 number. Lines 7 and 8 set the bits for the signs. Lines 9 through 11 extract the exponent. We add one to the exponent (line 11) because the conversion expresses the significand as “3.14159” and not “0.314159”. Line 12 sets the bits of the lowest order byte of the output using the absolute value of the exponent and the signs. Lines 14 through 19 extract the digits of the significand from the string representation. The numeric value is the digit character minus the ASCII code for zero. These are then shifted to the proper nibble position and added to the output. Finally, the signs and exponent are added (line 20), and the resulting value returned.

So, why do calculators use BCD floating point? The best answer to that question is found in Chap. 7, which discusses the IEEE decimal floating-point format, a variation on packed BCD. Here, we will give an example showing that decimal calculations are generally more precise, if slower, than binary floating-point calculations.

Consider this expression,

$$(1001 \times 0.01 - 10) \times 100 \rightarrow 1$$

Performing the calculation in Python using IEEE floating-point returns 0.9999999999999787, not 1, as expected. However, an inexpensive Rockwell 9TR calculator from 1976 returns precisely 1 because of BCD floating point.

3.11 Summary

In this chapter, we examined floating-point numbers and how they are represented in a computer. We discussed what we mean by “floating point” and reviewed some history surrounding floating point and computers. We compared two popular floating-point representations. We then explored the main parts of the IEEE 754 standard covering representations, rounding modes, comparisons, basic arithmetic, and exceptions. Next, we discussed the relationship between the IEEE standard and common floating-point hardware. Lastly, we explored packed BCD decimal floating-point as typically implemented in desktop and pocket calculators.

Exercises

3.1 Convert the following 32-bit IEEE bit patterns to their corresponding floating-point values:

- a) 0 10000000 10010010000111111011011
- b) 0 10000000 01011011111100001010100
- c) 0 01111111 10011110001101110111101
- d) 0 01111110 01100010111001000011000
- e) 0 01111111 01101010000010011110011

The first number is the sign, the second group of 8 bits is the exponent, and the remaining bits are the significand. It will help to write a computer program to handle the powers of two in the significand.

3.2 Assume a simple floating-point representation that uses 4 bits in the significand, 3 bits for the exponent, and 1 for the sign. The exponent is stored in excess-3 format, and all exponent values are valid. In this case, we have the following representations:

$$\begin{aligned}
 4.25 &= 01010100 \\
 1.5 &= 00111000 \\
 -1.5 &= 10111000 \\
 -4.25 &= 11010100
 \end{aligned}$$

Work out the following sums and express the answer in bits. Use a floating-point intermediate with 8 bits for the significand, and then round your answer to the nearest expressible value. Find $4.25 + 1.5$ and $-4.25 + -1.5$. **

3.3 Modify `fp_compare` to handle infinities properly.

3.4 Totaling the elements of an array is a common operation. Adding each new array element to the accumulated sum of all the previous elements produces an overall round-off error proportional to the size of the array.

The error is negligible for a small array but might matter for a large array. One way to avoid the accumulated round-off error is to use *pairwise summation* [6]. Pairwise summation is a recursive divide-and-conquer algorithm that checks whether the input array length is below a cutoff value, say five elements. If so, the algorithm returns their sum via sequential addition. Otherwise, the algorithm takes the length of the input array, divides it by two using integer division, and calls itself recursively on each of the subarrays, adding the return values together to form the final output value. In pseudo-code,

```
function pairwise(x) {
    if (length(x) <= 5) {
        s = x[0] + x[1] + ...
    } else {
        n = length(x) / 2
        s = pairwise(x[0..n-1]) + pairwise(x[n..])
    }
    return s
}
```

Implement this function in C and Python. Demonstrate the effect of round-off by comparing the output of this function with the naive summation for input arrays of 10,000 elements or more. Use a pseudo-random number generator to generate [0, 1) values. **

3.5 Extend the BCD float-32 routines in Sect. 3.10 to process BCD float-48 and BCD float-56. For simplicity, store these in 64-bit unsigned integers and convert to and from C doubles.

References

1. Muller J, Brisebarre N et al (2010) Handbook of floating-point arithmetic. Birkhäuser, Boston
2. Saracino D (1980) Abstract algebra: a first course. Addison-Wesley
3. Goldberg D (1991) What every computer scientist should know about floating-point arithmetic. ACM Comput Surv (CSUR) 23(1):5–48
4. Randell B (1982) The origins of digital computers-selected papers. Springer
5. Rojas R (1997) Konrad Zuse's legacy: the architecture of the Z1 and Z3. Ann Hist Comput IEEE 19(2):5–16
6. Higham N (2002) Accuracy and stability of numerical algorithms. Siam
7. Cody W, Waite W (1980) Software manual for the elementary functions. Prentice-Hall
8. Monniaux D (2008) The pitfalls of verifying floating-point computations. ACM Trans Program Lang Syst (TOPLAS) 30(3):12

9. Rump SM (2013) Accurate solution of dense linear systems, part ii: algorithms using directed rounding. *J Comput Appl Math* 242:185–212
10. McLaughlin S (2016) Learn TI-83 plus assembly in 28 days. <http://tutorials.eeems.ca/ASMin28Days/lesson/day18.html>. (Accessed 09 Sep 2016)



Pitfalls of Floating-Point Numbers (And How To Avoid Them)

4

Abstract

Floating-point numbers can be hazardous. In this chapter, we take a look at some of the pitfalls of floating-point computation and develop rules of thumb through which we can (hopefully) avoid most of them. We first examine some of the catastrophic and even deadly consequences of using floating-point computation poorly. We then bring things down a bit and examine expressions and processes that can lead to a loss of precision. Next, we enumerate some rules of thumb that can help us design code that is less susceptible to these pitfalls, thereby leading to more robust software. Finally, we describe a software tool that can help refactor floating-point code to be more reliable.

4.1 What Pitfalls?

Programmers frequently use floating-point numbers, typically without carefully considering the possible consequences due to the inherent imprecision of floating-point numbers. This chapter examines several cases where floating-point numbers have led to disaster, often with the loss of human life. After these cautionary tales, we will run some experiments illustrating the effects of casual floating-point use. The chapter closes by offering some general (and hopefully helpful) advice on improving our chances of creating correctly functioning programs involving floating point and describing a software tool that can help improve the reliability of floating-point expressions.

Throughout this chapter, it will be good to keep this quote from Kahan as given by Krämer [1] in mind,

Significant discrepancies [between the computed and the true result] are very rare, too rare to worry about all the time, yet not rare enough to ignore. [2]

Ariane 5 Explosion. The European Space Agency has a successful track record for launching sophisticated spacecraft. However, the maiden flight of its Ariane 5 rocket on June 4, 1996 was a total loss, to the tune of 500 million USD, all because one of the software modules attempted to put a 64-bit floating-point value into a 16-bit signed integer.

The rocket self-destructed under 40 seconds after launch from Kourou, French Guiana. The software putting a 64-bit floating-point value into the signed 16-bit integer had caused an overflow (i.e., the value exceeded 32,767), which caused the rocket to believe it was off-course. Therefore, the rocket fired thrusters to “correct” its course, which induced enough stress to cause the rocket to self-destruct. The final report of the inquiry board summarizes the events [5].

Schleswig-Holstein Parliament Election. This story does not involve loss of life or property but is politically troubling. In the German state of Schleswig-Holstein, no party receiving less than 5% of the vote is seated in parliament. For the election of 1992, it appeared the Green party had precisely 5% of the vote, granting them a seat. At least, this was what the software that prints the vote tally indicated. However, closer examination showed that the Green party only had 4.97% of the vote, less than the required 5% and that the software had simply rounded to one decimal to make 4.97% $\rightarrow$ 5.0%. Because of this, the Green party votes were dropped, granting a one seat majority to the Social Democrats [6].

Vancouver Stock Exchange. This story does not involve loss of life or property, either, but it was embarrassing. In 1982, the Vancouver Stock Exchange created a new index set to an initial value of 1000.000 [7]. The “000” is meaningful; the index was calculated to three decimals. However, a programming error truncated the index to three decimals without rounding so, to use the example from the original Wall Street Journal article, if the actual index value was 540.32567, the software truncated the index to 540.325 dropping the final “67” digits. Of course, this is not a good idea as the proper three-decimal value is 540.326, found by rounding the “5” up to “6” based on the value of the fourth decimal. The error is small but cumulative, and as the index was updated some 2800 times per day, it didn’t take long before the index value was completely meaningless. Eventually, the error was detected. Recalculation by hand put the index value at over 1098 instead of the 574 reported by the faulty software.

4.2 Some Experiments

The examples above illustrate that floating point can sometimes lead to fatal and costly disasters. More practically, floating point, in combination with compilers, hardware, and even the form of the equations used, can lead to wildly inaccurate results, as this section’s experiments demonstrate.

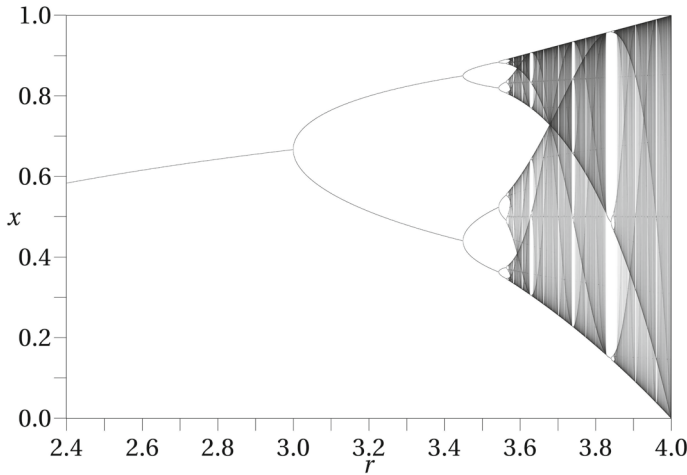


Fig. 4.1 The bifurcation plot of the logistic map. The plot shows several hundred x values from the iterated logistic map for each r value as a function of r illustrating the period-doubling route to chaotic behavior. For the experiment, $r = 3.8$ which is in the chaotic region. (“Logistic Map” by Jordan Pierce, Creative Commons CC0 1.0 Universal Public Domain Dedication License)

The Logistic Map. A favorite equation for illustrating the onset of chaotic behavior is the logistic map, $x_{i+1} = rx_i(1 - x_i)$ with r a constant between 1 and < 4 and x a value from $(0, 1)$. This map exhibits fascinating behavior as an initial x_0 is repeatedly iterated with r varying from 2 to just under 4.

Selecting an initial value of 0.1 (or any other), running the iteration forward a thousand times, and then plotting the next thousand iterations as a function of r produces the bifurcation plot in Fig. 4.1. The plot shows the period-doubling route to chaotic behavior as r increases. There is a deep correlation between the structure of this plot and the famous Mandelbrot set. Our concern is with the sequence of iterates produced when r is in the chaotic regime using different but algebraically equivalent forms of the logistic equation. This experiment is inspired by previous work by Colonna [8].

We can rewrite the logistic equation in multiple algebraically equivalent ways,

$$x_{i+1} = rx_i(1 - x_i) = rx_i - rx_ix_i = x_i(r - rx_i) = rx_i - rx_i^2$$

Our test program will compute each of the four forms. The program itself is straightforward,

```

1 #include <stdio.h>
2 #include <math.h>
3
4 int main() {
5     double x0,x1,x2,x3;
6     double r = 3.8;
7     int i;
8
9     x0 = x1 = x2 = x3 = 0.25;
10
11     for(i=0; i < 100000; i++) {
12         x0 = r*x0*(1.0 - x0);
13         x1 = r*x1 - r*x1*x1;
14         x2 = x2*(r - r*x2);
15         x3 = r*x3 - r*pow(x3,2);
16     }
17
18     printf("      x0          x1          x2          x3\n");
19
20     for(i=0; i < 8; i++) {
21         x0 = r*x0*(1.0 - x0);
22         x1 = r*x1 - r*x1*x1;
23         x2 = x2*(r - r*x2);
24         x3 = r*x3 - r*pow(x3,2);
25         printf("%0.8f %0.8f %0.8f %0.8f\n", x0,x1,x2,x3);
26     }
27 }

```

The program iterates each of the four versions of the logistic map 100,000 times before outputting the final eight iterates. Note that the program uses double-precision values and that the initial value is 0.25, which is not a repeating binary, meaning it is exactly represented in memory. The program produces this output,

x0	x1	x2	x3
0.76670670	0.65123778	0.80925715	0.38542453
0.67969664	0.86308311	0.58656806	0.90011535
0.82729465	0.44904848	0.92152269	0.34164928
0.54293721	0.94013498	0.27481077	0.85471519
0.94299431	0.21386855	0.75730128	0.47187310
0.20427297	0.63888941	0.69842500	0.94699374
0.61767300	0.87669699	0.80038457	0.19074708
0.89738165	0.41077765	0.60712262	0.58657800

when compiled with

```
$ gcc logistic.c -o logistic -lm
```

We already notice the effect of the different ways the compiler has decided to evaluate the logistic expressions even though they are identical algebraically. If we compile again with optimization,

```
$ gcc logistic.c -o logistic -lm -O2
```

we get

x0	x1	x2	x3
0.18402020	0.67978266	0.32056222	0.92875529
0.57059572	0.82717714	0.82764792	0.25144182
0.93106173	0.54322944	0.54205800	0.71523156
0.24390598	0.94289862	0.94327827	0.77396643
0.70078024	0.20459509	0.20331661	0.66478109
0.79681171	0.61839658	0.61552007	0.84681933
0.61523066	0.89673255	0.89928943	0.49292215
0.89954320	0.35189247	0.34415822	0.94980964

which is different still. Clearly, how an expression is coded matters significantly. Which value, if any, is correct? How can we tell? One way would be to use arbitrary precision floating point, the topic of Chap. 9.

Alternatively, we might use rational arithmetic and convert back to floating point at the end. Rational arithmetic is discussed in Chap. 5, but be warned that iteration quickly leads to fractions with very large numerators and denominators requiring a considerable amount of memory.

Taylor Series for e^x when $x < 0$. Let's consider another experiment. In this case, we want to compute e^x for any x using the Taylor series expansion,

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = 1 + \sum_{i=1}^{\infty} \frac{x^i}{i!}$$

which we translate into Python as

```
1 def fact(n):
2     if (n == 1):
3         return 1.0
4     return n * fact(n-1)
5
6 def exponential(x, tol=1e-10):
7     ans = 0.0
8     term= 1.0
9     p = 1.0
10
11     while (abs(term) > tol):
12         ans += term
13         term = x**p / fact(p)
14         p += 1
15
16     return ans
```

We accumulate terms of the series until one of the terms is smaller than a given threshold. If we run this program for positive values of x and compare them with

the output of a gold standard `exp(x)` function (recall that Python floats are double precision), we get

20	4.8516519540979016e+08	4.8516519540979028e+08
21	1.3188157344832141e+09	1.3188157344832146e+09
22	3.5849128461315928e+09	3.5849128461315918e+09
23	9.7448034462489052e+09	9.7448034462489033e+09
24	2.6489122129843472e+10	2.6489122129843472e+10
25	7.2004899337385880e+10	7.2004899337385880e+10

where the first column is the argument, the second is the output of `exponential()`, and the third is the output of our gold standard function. Our Taylor series approximation for e^x is working quite well.

Now, let's run again, but this time, make $x < 0$. In this case, we find

-20	4.9926392547479328e-09	2.0611536224385579e-09
-21	2.7819487708032662e-08	7.5825604279119066e-10
-22	-2.4369901527907362e-08	2.7894680928689246e-10
-23	3.1518333727167153e-09	1.0261879631701890e-10
-24	3.7814382919759864e-07	3.7751345442790977e-11
-25	1.1662837734562158e-06	1.3887943864964021e-11

Not so good. What happened, and how can we fix it? What happened is that the Taylor series for e^x , $x < 0$ is made up of terms that are alternately even and odd powers of x . This means that the sign of the term changes from term to term. In other words, the series sum is made up of values that are wildly different from each other. Consider the approximation for e^{-20} ,

$$\begin{aligned}
 e^{-20} &= 1 - 20 + \frac{20^2}{2} - \frac{20^3}{6} + \frac{20^4}{24} - \frac{20^5}{120} + \frac{20^6}{720} - \dots \\
 &= 1 - 20 + 200 - 1333.3 + 6666.7 - 26666.7 + 88888.9 - \dots
 \end{aligned}$$

where the effect of each term changing sign is that the sum to term n is always the opposite sign as term $n + 1$. Adding two very different numbers leads to a rapid loss of precision, making the approximation functionally useless in practice even though it is mathematically valid.

In this case, the fix is to use only positive x and recall that $e^{-x} = 1/e^x$. If we rerun our Python program with only positive x and report `1.0/exponential(x)` we greatly improve our final results

-20	2.0611536224385583e-09	2.0611536224385579e-09
-21	7.5825604279119097e-10	7.5825604279119066e-10
-22	2.7894680928689241e-10	2.7894680928689246e-10
-23	1.0261879631701888e-10	1.0261879631701890e-10
-24	3.7751345442790977e-11	3.7751345442790977e-11
-25	1.3887943864964019e-11	1.3887943864964021e-11

thereby proving that forethought is always best when expecting meaningful results from floating-point operations.

Summing an Array of Numbers. Summing the elements of an array is a frequent operation with floating-point numbers. For example, calculating the mean of a collection of data points typically involves storing the data in an array and then processing the elements of the array to calculate the mean.

The straightforward way to sum an array is to add the elements one after the other. As we will see, this simple approach suffers from round-off error. We can do better and compensate for the error by using Kahan summation [3], a form of compensated summation [9]. For example, the following code generates a vector of random values and sums them using naive summation and Kahan summation:

```
1 #include <stdio.h>
2 #include <time.h>
3 #include <stdlib.h>
4
5 #define N 10001
6 double A[N];
7
8 int main() {
9     double sum, c, y, t;
10    int i;
11
12    srand(time(NULL));
13    for(i=0; i<N; i++)
14        A[i] = 1001.0*((double)rand()/(double)RAND_MAX);
15
16    sum = 0.0;
17    for(i=0; i<N; i++)
18        sum += A[i];
19    printf("simple sum = %0.16f\n", sum);
20
21    sum = c = 0.0;
22    for(i=0; i<N; i++) {
23        y = A[i] - c;
24        t = sum + y;
25        c = (t - sum) - y;
26        sum = t;
27    }
28    printf("Kahan sum = %0.16f\n", sum);
29    return 0;
30 }
```

where lines 12 through 14 fill the array `A` with random numbers in the range `[0, 1001]`. The simple sum is in lines 16 through 18. It is self-explanatory. We print the result in line 19. Kahan summation is in lines 21 through 27. These lines need some explanation. The main goal of Kahan summation is to preserve some notion of the error (`c`) and compensate for it. Line 23 adjusts the to-be-added value (`A[i]`) by the

current value of c . This subtracts the estimated error from the previous value, which may be negative. Line 24 stores the new sum, adding in the current value (t). Line 25 would, in the absence of error, be exactly zero. The difference $t - \text{sum}$ should be exactly $A[i] - c$ as is y , so the value of the expression should be zero. However, round-off error will (sometimes) make this value nonzero. This is the estimated error introduced by adding $A[i]$, and we compensate for it when the next term is added (line 23). Line 26 updates the actual returned sum.

How well does Kahan summation work? We can calculate it. If we sum an array of 10001 random values, each in the range $[0, 1001]$, using simple summation and Kahan summation, we can compare the result with a high-precision sum of the same array using 100 decimal digits. Taking the 100 decimal digit answer as the gold standard, we can calculate the deviation for the simple and Kahan sums. Repeating this process, say 300 times, leads to an average deviation. Doing precisely this gives

$$\begin{aligned}\text{simple sum } |\bar{\Delta}| &= 0.0000000103093065730 \\ \text{Kahan sum } |\bar{\Delta}| &= 0.0000000002381457745\end{aligned}$$

Rerunning the test will return slightly different results but will consistently show that Kahan summation is approximately two orders of magnitude more precise than simple summation. The high-precision floating-point library used as the gold standard is developed in Chap. 9.

Comparing Floating-Point Numbers. Our final example involves one of the most common activities with floating-point numbers: testing for equality.

To illustrate the problem, consider the expression $0.1 + 0.2 == 0.3$ where $==$ implies a test for equality. This expression fails when using 32-bit or 64-bit IEEE floating point. However, this expression does not fail for IEEE floating point: $0.1 \times 10^1 + 0.2 \times 10^1 == 0.3 \times 10^1$. So, for what values of x in 10^x will the test pass? Testing empirically using only positive exponents to avoid cluttering the final plot produces Fig. 4.2 where we plot 1 if the test passes for that particular exponent x and 0 otherwise. The results for negative exponents are similar. Examining the figure, we see islands where the expression works as expected and many places where it fails. Comfortingly, there is a large set of exponents covering a widespread range of values encountered in practice where the expression is evaluated correctly, with the notable exception of $x = 0$.

Why is the expression not always appropriately evaluated? Round-off error is to blame. We know that 0.1 is infinitely repeating in binary, so the requirement to fit it in a finite number of bits will inevitably result in an error that can make a mathematically true expression appear false on the computer.

Another area where comparison round-off error appears is inside a loop. It is not uncommon to want to iterate over a range of floating-point values,

```
for(d=start; d < end; d += inc)
    ...
```

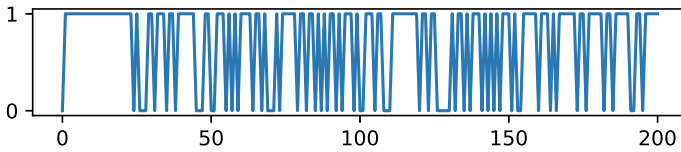


Fig. 4.2 Comparing $0.1 \times 10^x + 0.2 \times 10^x = 0.3 \times 10^x$, $x \in [0, 200]$ using double-precision floating-point numbers. If the comparison passes plot 1, otherwise plot 0. There are islands where the comparison works as expected, but also many places where it fails

where `start` is a starting value, `end` is an ending value, and `inc` is an increment. For example, we can set `inc` using `inc = (end - start) / n` in order to calculate `n` values in the loop. This seems straightforward, but round-off error and the test `d < end` will not let us easily predict whether the loop will execute `n` or `n + 1` times, which is bad if we hope in the loop body to fill an array of exactly `n` elements.

We can determine the frequency of this error with a simple program that counts the number of times the body of a loop defined in this way actually executes for random `start` and `end` values holding `n` fixed,

```

1 int main() {
2     double start, end, inc, d, s=0.0;
3     int i=0, j=0, n, m;
4
5     srand(time(NULL));
6     n = 100;
7     m = 10000000;
8
9     for(j=0; j<m; j++) {
10        start = 100001.0*((double)rand()/((double)RAND_MAX));
11        end = start + 1001.0*((double)rand()/((double)RAND_MAX));
12        inc = (end - start) / n;
13        i = 0;
14
15        for(d=start; d < end; d += inc)
16            i++;
17        s += i-n;
18    }
19
20    printf("fraction with extra loop = %0.9f\n", s/(double)m);
21    return 0;
22 }
```

We set `start` to a random value in the range `[0, 100001]` and `end` to `start` plus a value in the range `[0, 1001]`. We fix `n` at 100, expecting the loop in lines 15 and 16 to execute exactly `n` times. We count the actual number of loop iterations in `i` and accumulate a running total in `s` (line 17). When we've run `m` tests, we report the fraction of loops that executed `n + 1` times (line 20). Running this program many times always reports a fraction very close to 49.5%, meaning we perform 101 iterations nearly half the time.

We can correct the problem by rewriting the loop,

```
for(j=0; j < n; j++) {  
    d = start + j*inc;  
    ...  
}
```

We are now assured of executing exactly n times, and we set `d` at the beginning of each iteration so we can use it in the loop's body.

What about comparing for equality in general? Can we be accurate when using floating-point values? The answer is “yes” with some attention paid to what we mean by equality. We learned in Chap. 3 that the bit representation of IEEE floating-point values are ordered, and when two values are the same, their bit values are the same, with one notable exception. Therefore, a rigorous test for floating-point equality is to look at the bits of the representations. If they are identical, the numbers must be the same. The exception is zero, which may be signed or unsigned according to IEEE 754. The numbers are the same in this case, but the bit values are different. We should also be careful in the case of NaNs. Recall that a NaN has a payload intended to convey application-specific information about the source of the NaN. So, while we might want to say that two NaNs are equal, they might have different payloads. Infinity, positive or negative, is not an issue because, by definition, only 1 bit pattern represents each.

With this in mind, we can define a rigorous equality test like this,

```
1 uint8_t equal_strict(float a, float b) {  
2     uint32_t A = *(uint32_t *)&a;  
3     uint32_t B = *(uint32_t *)&b;  
4  
5     if ((a == 0.0) && (b == 0.0)) return 1;  
6     if (isnan(a) && isnan(b)) return 1;  
7     return (A == B);  
8 }
```

where we take the inputs and re-interpret them as unsigned integers (lines 2 and 3). This will let us compare the bits directly. Line 5 checks for zero since the IEEE standard demands $0.0 == -0.0$. Line 6 looks at NaNs, regardless of the payload. Lastly, line 7 directly compares the bits now that we know the inputs are not NaNs or zero. This is the strictest possible test for floating-point equality and is probably not what we want in most cases because it demands *exact* bit-for-bit equality.

A commonly used alternative technique is to use a tolerance value. If we know the range of values we are working with, for example, the result of actual physical measurements, we can usually set a tolerance below which we are willing to call the values the same. In this case, we can use an expression like


```
if (fabs(a-b) < t) ...
```

to check if the difference between two floating-point values, here doubles because we are using `fabs`, is less than a tolerance value, `t`. This is a perfectly reasonable thing to do in many cases. However, this is not a general solution because, in the general case, the magnitudes of `a` and `b` may be out of concordance with the given tolerance. To be more accurate and still allow a tolerance over which we are willing to call two floating-point values equal, we can return to the integer representation and measure the difference between the two values in integer space, and as long as that is within a certain tolerance, we can call the numbers “equal” (with the tests above for zero and NaN cases).

We introduce a new term, “ulps”, which means “units in the last place”. An ulp is the smallest difference between two consecutive floating-point numbers. In integer space, this is the difference between a floating-point number with integer representation n and a floating-point number with representation $n + 1$. We can use this to test for equality by passing the maximum ulps difference we are willing to tolerate and still call the numbers equal. This difference may be quite significant if we are in a situation where we are willing to call two very different numbers equal.

Our ulps equality routine is a simple modification of the strict equality routine above,

```
1 uint8_t equal_ulps(float a, float b, uint32_t ulps) {
2     uint32_t A = *(uint32_t *)&a;
3     uint32_t B = *(uint32_t *)&b;
4
5     if ((a == 0.0) && (b == 0.0)) return 1;
6     if (isnan(a) && isnan(b)) return 1;
7     return abs(A-B) < ulps;
8 }
```

where the only change, besides passing in a tolerance value (`ulps`) is line 7, which checks if the difference between the two inputs, as integers, is less than the max tolerance. Note that `equal_ulps` will not be correct for numbers close to zero. This is because the high bit of an IEEE float is the sign, and even if the numbers are both close together, if one is negative and the other positive, then the raw difference in ulps will be huge indeed. We leave correcting `equal_ulps` as an exercise for the reader, see Problem 4.1.

We can test our new function with this program,

```

1 int main() {
2     float a,b;
3     uint32_t A,B,i;
4     uint32_t S=0, UMAX=0, UMIN=80000000;
5     uint32_t p0=0,f0=0,p1=0,f1=0, M=50000000;
6
7     srand(time(NULL));
8
9     for(i=0; i < M; i++) {
10        a = (float)(1001.0*(double)rand()/(double)RAND_MAX);
11        b = 2.1*a;
12        b = b - 2.0*a;
13        b = 10.0*b;
14        A = *(uint32_t*)&a;
15        B = *(uint32_t*)&b;
16        S += abs(A-B);
17        if (abs(A-B) > UMAX) UMAX = abs(A-B);
18        if (abs(A-B) < UMIN) UMIN = abs(A-B);
19        if (equal_ulps(a,b,30)) p0++; else f0++;
20        if (a==b) p1++; else f1++;
21    }
22
23    printf("p0=%8d, f0=%8d, passed=%0.8f\n", p0, f0, (double)p0/(double)M);
24    printf("p1=%8d, f1=%8d, passed=%0.8f\n", p1, f1, (double)p1/(double)M);
25    printf("mean difference=%0.8f, [%u,%u]\n", (double)S/(double)M,UMIN,UMAX);
26
27    return 0;
28 }

```

This program performs many tests of the difference between a random floating-point value (a) and b which is set to $(2.1a - 2.0a) \times 10$ which, mathematically, should be exactly equal to a but which we know will not be because of round-off error. We split the formula among several lines to stymie the compiler lest it do any advanced optimizations. The variable S accumulates the difference between a and b in ulps. The largest and smallest difference encountered is stored in $UMAX$ and $UMIN$, respectively. Line 19 calls the new equality routine and increments the counters for passed and failed. Line 20 does the same for the straightforward equality test.

When the loop completes, the program outputs the totals for each case along with the fraction of tests that passed, the mean difference, minimum difference, and maximum difference in ulps. A typical run produces output like

```

p0=50000000, f0=          0, passed=1.00000000
p1= 2408450, f1=47591550, passed=0.04816900
mean difference=5.37257440, [0,20]

```

Notice that all the calls to `equal_ulps` have passed because we set the tolerance to 30 ulps; in this case, the maximum difference encountered was 20 ulps. Notice that only 4.8% of the direct comparisons were successful. These percentages persist when the range of a is changed across many orders of magnitude.

Note that the routines above work for double-precision floating point as well. Simply map `float` $\rightarrow$ `double` and `uint32_t` $\rightarrow$ `uint64_t`.

4.3 Avoiding the Pitfalls

The previous section illustrated through empirical experiments some of the pitfalls of floating-point computation. The topics addressed are not exhaustive by any means. However, we can now put together a short list of “rules of thumb” worth considering when writing floating-point code:

1. Do not use floating-point numbers if integers will suffice. As a corollary, do not use floating-point numbers if exact computation is required, rather, scale if a fixed number of decimals will always be used. For example, if the values are dollars, do the computations in cents and divide by 100 when done.
2. Do not represent floating-point numbers in text form in files or as strings unless absolutely necessary. If necessary, use scientific notation and be sure to have enough digits after the mantissa’s decimal point to represent the significand accurately. Use at least 8 digits for a 32-bit float and 16 for a 64-bit float. For example, in Python,

```
>>> from math import pi
>>> float("%.15g" % pi) == pi
False
>>> float("%.16g" % pi) == pi
True
```

The `0.15g` format specifier uses 15 digits after the decimal, which is insufficient; 16 are required for the text representation of π , when converted to a floating-point number, to match the original. Floating-point numbers are often found in XML and JSON files and frequently without enough precision.

3. The spacing between floating-point numbers increases as the number increases. If working with large numbers, rescale, if possible, to map to values closer to one.
4. Comparing two floating-point values for equality can be problematic if they were computed using a different code sequence. If possible, replace expressions like `a == b` with `abs(a-b) < e` where `e` is a tolerance value, or use the equality functions defined above. Even the tolerance value is dangerous in that the choice of `e` depends upon the scale of the values being worked with.
5. Subtracting two nearly equal numbers results in a large loss of precision as the result comprises only the lowest order bits of the significand. Generally, this should be avoided. Naturally, the rule above does just this, so it is best to compare integer representations directly when possible and absolute precision is needed.
6. For real numbers, we know that $(ab)c = a(bc)$; however, this is sometimes not the case for floating-point numbers due to rounding. Use care when assuming associativity.

The rules above are ad hoc examples of a large research area known as *numerical methods*. An essential part of numerical methods involves the analysis of the

equations being implemented by the computer and reworking those equations to minimize floating-point errors to improve the program’s stability. The stability of numerical algorithms is a vast field, as might be expected. For a scholarly introduction, see Higham [9], especially its early chapters. See Acton [10] for a more popular treatment.

Using A Tool—Herbie. An emerging area of research in numerical methods is automatically rewriting floating-point expressions to improve their reliability. Here, we will take a look at one such tool, Herbie. The description of how Herbie works is beyond the scope of this book; see [11] for more information. We will describe how to install and test Herbie and how to use it to simplify expressions. There is also a web demo of Herbie that uses a more familiar syntax; see <http://herbie.uwplse.org/demo/> (retrieved 4-Mar-2024).

Herbie is written in Racket. Racket is a programming language derived from Scheme, which itself is derived from Lisp. In a previous life, Racket was called MzScheme, though Racket goes well beyond MzScheme. To use Herbie, we need to install Racket. We can install Racket by downloading it here: <http://download.racket-lang.org/>. After we install Racket, we can install Herbie from <https://herbie.uwplse.org/doc/latest/installing.html>.

```
git clone https://github.com/uwplse/herbie

where can test the installation with
```

```
racket src/reports/run.rkt bench/hamming/
```

If 24 or more tests pass, we can assume that Herbie is working properly (patience, it takes a while). Compile Herbie with

```
raco make src/reports/run.rkt
```

to improve performance. We are now ready to use Herbie to improve some floating-point expressions.

Herbie requires us to enter the expressions we wish to optimize in an external file using Lisp syntax. For example, consider the following translations from standard infix to prefix format:

<i>infix</i>	<i>prefix</i>
$x + y$	<code>(+ x y)</code>
$x - y$	<code>(- x y)</code>
$2x^2 - 3x + 4$	<code>(+ (- (* 2 (sqr x)) (* 3 x)) 4)</code>
$100(10.01x - 10)$	<code>(* 100 (- (* 10.01 x) 10))</code>
$\sqrt{x^2 + y^2}$	<code>(sqrt (+ (sqr x) (sqr y)))</code>
$\cos^2 x + \sin^2 x$	<code>(+ (sqr (cos x)) (sqr (sin x)))</code>

Herbie understands $+$, $-$, $*$, $/$, abs , sqrt , sqr , exp , log , expt , sin , cos , tan , cot , asin , acos , atan , sinh , cosh , and tanh where the functions have their usual meaning. Use $(\text{expt } x \ y)$ for x^y . Herbie also knows PI for π and E for e .

The expressions need to be entered into the file are as follows:

```
(FPCore (x)
  (* 100 (- (* 10.01 x) 10)) )
```

where the expression is, using Lisp syntax, the second line. The expression is wrapped with

```
(FPCore (x) ... )
```

where (x) is a list of the variables in the expression and $\dots$ is the expression.

So, to use Herbie to process the expression $x(\sqrt{x-1} - \sqrt{x})$ we enter

```
(FPCore (x) (* x (- (sqrt (- x 1)) (sqrt x))) )
```

into a file named `expr.fpcore` (the `fpcore` extension is suggested) and evaluate it with

```
racket src/herbie.rkt expr.fpcore
```

to produce Herbie's rewritten expression

```
(FPCore (x) (/ (- x) (+ (sqrt x) (sqrt (- x 1)))))
```

indicating that, according to Herbie, we should change

$$x(\sqrt{x-1} - \sqrt{x}) \rightarrow \frac{-x}{\sqrt{x} + \sqrt{x-1}}$$

to maximize our accuracy. Let's test Herbie's claims using a simple Python program

```

1 | import random
2 | from APFP import *
3 | from math import sqrt
4 |
5 | def f(x):
6 |     return x*(sqrt(x-1) - sqrt(x))
7 |
8 | def g(x):
9 |     return (-x) / (sqrt(x) + sqrt(x-1))
10 |
11 | def main():
12 |     APFP.dprec = 100
13 |     M = 10000
14 |     SA = APFP(0)
15 |     SB = APFP(0)
16 |
17 |     for i in range(M):
18 |         x = 1001.0*random.random() + 1.0
19 |         a = f(x)
20 |         b = g(x)
21 |         X0 = (APFP(x) - APFP(1)).sqrt()
22 |         X1 = APFP(x).sqrt()
23 |         B = APFP(x)*(X0 - X1)
24 |         DA = abs(B - APFP("%0.18f" % a))
25 |         DB = abs(B - APFP("%0.18f" % b))
26 |         SA += DA
27 |         SB += DB
28 |
29 |     print("Running %d tests:" % M)
30 |     print("      mean deviation using f() = %s" % (SA / APFP(M)))
31 |     print("      mean deviation using g() = %s" % (SB / APFP(M)))

```

where the `APFP` library is the fixed-point arbitrary precision floating-point library developed in Chap. 9. `APFP` can perform calculations with an arbitrary number of digits after the decimal point. Here, we use 100 digits after the decimal point. There is no practical limit to the size of the integer portion. The functions `f()` and `g()` are the two forms of the expression we are testing, the original and the version Herbie suggests. The main loop (line 17) draws a random value, gets the output of the two forms of the expression, and then calculates the original expression using arbitrary precision (lines 21 through 23). We determine the deviation from the arbitrary precision result for each expression (lines 24 and 25) and accumulate the deviation to calculate the mean (lines 30 and 32).

Running this program produces output similar to (truncating the 100 digits in the actual display)

```

Running 10000 tests:
mean deviation using f() = 0.00000000000005792391185219435
mean deviation using g() = 0.0000000000000006209725959118

```

where $\mathfrak{f}()$ is $x(\sqrt{x-1} - \sqrt{x})$, the original expression, and $\mathfrak{g}()$ is $\frac{-x}{\sqrt{x} + \sqrt{x-1}}$, the expression Herbie suggested we use instead. Indeed, Herbie is correct; the rewritten expression produces, on average, 1/1000-th the deviation from the arbitrary precision result when compared to the original expression.

This simple test shows that tools like Herbie can improve program floating-point accuracy.

4.4 Summary

This chapter examined real-world instances where floating-point errors caused severe damage and loss of life or property. We then used experiments to understand some issues in applying floating-point numbers. These helped us formulate some rules of thumb for using floating point. Lastly, we examined a powerful software tool for automatically rewriting floating-point expressions to improve their accuracy and demonstrated the efficacy of the rewritten expressions.

Exercises

4.1 Correct the deficiency in `equal_ulps` above by treating the input floating-point numbers as sign-magnitude integers. I.e., bit 31 of the integer representation is the sign and needs to be explicitly examined in some fashion.

4.2 The function `equal_ulps` uses a tolerance value to indicate equality. One could also use a tolerance value to help in deciding whether a floating-point number, `a`, is greater than or less than another number, `b`. Write a new function, `compare_ulps(a,b,u)`, which returns -1 if `a < b`, 0 if `a = b`, and +1 if `a > b`. The third argument, `u`, is a tolerance in ulps. If `a` and `b` are within this tolerance, they are considered equal. If not, and `a` is larger than `b`, return +1 otherwise, return -1. (Hint: Recall that bit 31 of an IEEE float is the sign bit. You will need to take this into account in your function.)

4.3 Experiment using Herbie to rewrite other expressions. Test them using the Python code in Sect. 4.3. In particular, consider expressions using $\sqrt{f(x)}$, $e^{f(x)}$, $\log f(x)$, and trig functions like $\sin f(x)$, etc. Is Herbie most effective with any specific transcendental function?

References

1. Krämer W (1998) A priori worst case error bounds for floating-point computations. *IEEE Trans Comput* 47(7):750–756
2. Kahan WM (1989) The regrettable failure of automated error analysis. In: Mini-course, conference computers and mathematics. Massachusetts Institute of Technology
3. Kahan WM (1965) Further remarks on reducing truncation errors. *Commun ACM* 8(1):40
4. B-247094, report to the house of representatives. Washington, D.C.: GAO, information management and technology division, 4 Feb 1992. www.fas.org/spp/starwars/gao/im92026.htm (Accessed 15 Oct 2014)
5. Lions JL (1996) (chair), ARIANE 5 flight 501 failure report by the inquiry board, Paris, 19 July 1996
6. Weber-Wulff D (1992) The risks digest 13(37)
7. Quinn K (1983) Ever had problems rounding off figures? *Stock Exchange Has*, Wall Street J 37
8. Colonna JF (2014) The subjectivity of computers. <http://www.lactamme.polytechnique.fr/descripteurs/subject.01..html>. (Accessed 15 Oct 2014)
9. Higham N (2002) Accuracy and stability of numerical algorithms. Siam
10. Acton F (2013) Real computing made real: preventing errors in scientific and engineering calculations. Courier Dover Publications
11. Panchekha P et al (2015) Automatically improving accuracy for floating point expressions. *ACM SIGPLAN Notices* 50(6):1–11



Big Integers and Rational Arithmetic

5

Abstract

Big integers differ from standard integers in that they are of arbitrary size; the number of digits is limited only by the memory available. This chapter examines how big integers are represented in memory and how to perform arithmetic with them. We also discuss some implementations that might be of use when using programming languages that do not support big integers natively. Next, we examine rational arithmetic with big integers. Finally, we conclude with some advice on when using big integers and rational numbers might be advantageous.

5.1 What is a Big Integer?

Big integers, also known as bignums, bigints, multiple precision, arbitrary precision, or infinite precision integers, are integers that are not limited to a fixed number of bits. The standard representation for integers in computers involves a fixed number of bits, typically 8, 16, 32, or 64. These numbers, therefore, have a fixed range. A big integer, however, uses as much memory as necessary to represent the digits and, therefore, is only limited in range by the memory available. Several programming languages, including Python, include big integers as part of their specification.

This chapter will examine how big integers are represented in memory and describe algorithms for dealing with big integer input and output when the base is convenient. We then contemplate arithmetic and explore common big integer algorithms. Next, we look at more sophisticated big integer algorithms suitable when huge numbers are involved. Following that, we tackle rational arithmetic, a natural place to use big integers and big integer libraries we expect to be long-lived. We conclude with comments on when to resort to big integers or rational arithmetic.

5.2 Representing Big Integers

We know that place notation represents numbers using a given base, B , and the coefficients of the powers of that base, which sum to the number we want to represent. Specifically, we can represent a number in any base with

$$abcd.efg_B = a \times B^3 + b \times B^2 + c \times B^1 + d \times B^0 + e \times B^{-1} + f \times B^{-2} + g \times B^{-3}$$

where we typically use $B = 10$ as the base. When we represent a big integer in memory, we would like to use a base that allows us to accomplish two things simultaneously,

1. Use a base that makes converting text input and output straightforward.
2. Use a base in which products of the “digits” of our base fit in a standard word.

If we have a 32-bit system and look at using signed 32-bit integers to represent digits, we need a base where any two digits, when multiplied, give us a value that fits in a 32-bit integer. We will consider the representation of signs for big integers later. This will satisfy the second requirement above. What about the first? If one looks at the standard number bases used in computer systems, they are all powers of two. Octal is base $2^3 = 8$ while hexadecimal is base $2^4 = 16$. In these cases, we saw that changing a binary number into an octal or hexadecimal number was particularly simple and vice versa. We can use this observation when choosing a base for our big integers. Since we would like to make things convenient for users, we want to be able to input and output base ten numbers, which suggests using a power of ten for our big integer base, but which power? Since we assume a 32-bit computer in this book, we want a base such that the base squared fits in a signed 32-bit integer. If we choose $B = 10,000$, we see that $B^2 = 100,000,000$, which does fit nicely in a signed 32-bit integer. However, if we go to the next power of ten, $B = 100,000$, we see that $B^2 = 10,000,000,000$, which is too large to fit in 32 bits. Therefore, let's work with $B = 10,000$.

Why choose such a large base? Why not simply use $B = 10$? The answer is that the larger the base, the smaller the number of unique digits required to represent a large number. In binary, we need 8 bits to represent a number, which is only three digits long in base ten. So, the larger the base, the less memory we need to store our numbers. As we are, by definition, working with big numbers, we would like to be as economical as possible. If we were willing to work harder to handle input and output, we could, naturally, use an even larger base. The most significant value that fits in an unsigned 32-bit integer is 4,294,967,295, and we see that $65,535^2 = 4,294,836,225$ will fit, meaning that is the largest base we could use without moving to 64-bit integers. However, as it is not a power of ten, we must work harder to implement base-10 input and output. An additional feature of our choice of base is that it fits in a *signed* 32-bit integer, allowing us to make our internal representation quite compact, as we will see.

We have selected to represent our big integers in memory as base 10,000 numbers with 10,000 digits. We will use standard binary to represent the digit, with each word in memory representing a single digit. When we want to output a big integer, we only need to output each signed 32-bit integer as a group of four decimal digits because our base is $10^4 = 10,000$. Similarly, when reading a number from a user, we will take the input as a string of ASCII characters and group it in sets of four to form the digits of our base 10,000 number in memory.

The previous paragraph contains a subtle piece of information. It implies that our big integers will be stored in memory in big-endian format. This means that if we use an array of signed integers to hold the digits, the first integer in that array will be the most significant power of the base, not the smallest as it would be in little-endian format. This choice again simplifies our input and output operations.

Therefore, we will use arrays of signed integers to represent the digits. Additionally, the very first element of the array will contain the number of digits in the number, and the sign of this element will be the sign of the integer. Pictorially,

0	1	2	3
-3	123	4567	8910
sign/length	d_2	d_1	d_0

which illustrates how the number “-12,345,678,910” will be stored in memory using an array of four signed 32-bit integers. The first element of the array contains “-3”, which says there are three digits in this number and that it is negative. The remaining three elements of the array are the digit values in base 10,000. We will use this simple format going forward.

Input and Output. Figure 5.1 presents a simple C routine to convert a string into a base 10,000 number. Similarly, Fig. 5.2 shows a routine to output a base 10,000 number as a decimal.

Figure 5.1 accepts a string in `s` and returns a pointer to a newly allocated big integer. Note that this memory must be released by a call to `free`. Let’s look in detail at what this particular function is doing.

Line 10 checks to see if the input is negative. If it is, we set the sign to 1 (default is 0) and skip past the sign character by incrementing our character index `j`. Line 15 calculates the number of digits in the big integer. The input string is in base 10 while the output integer is in base 10,000, so we divide the input length, accounting for the sign character, by four to see how many digits we need. We take the ceiling of this to avoid truncating the length. Line 16 allocates new memory on the heap for the number, using `n+1` to add one element for the length and sign. For clarity, we skip any check on the return value of `malloc` to see if the allocation succeeded. In line 17, the sign and length are set.

Lines 19 through 36 work through the big integer, digit by digit, using sets of four characters from `s` to set the value. The first digit, the highest order digit, is a special case handled by lines 21 through 29 because the highest order digit is the only digit

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <math.h>
4 | #include <string.h>
5 |
6 | signed int *bigint_input(char *s) {
7 |     int i, j=0, n, sign=0;
8 |     signed int *b;
9 |
10 |     if (s[0] == '-') {
11 |         sign = 1;
12 |         j++;
13 |     }
14 |
15 |     n = (int)ceil((strlen(s)-sign) / 4.0);
16 |     b = (signed int *)malloc((n+1)*sizeof(signed int));
17 |     b[0] = (sign) ? -n : n;
18 |
19 |     for(i=1; i <= n; i++) {
20 |         if ((i==1) && ((strlen(s)-sign)%4) != 0) {
21 |             b[i] = 0;
22 |             switch ((strlen(s)-sign)%4) {
23 |                 case 3:
24 |                     b[i] += (s[j++]-'0') * 100;
25 |                 case 2:
26 |                     b[i] += (s[j++]-'0') * 10;
27 |                 case 1:
28 |                     b[i] += (s[j++]-'0');
29 |             }
30 |         } else {
31 |             b[i] = (s[j++]-'0') * 1000;
32 |             b[i] += (s[j++]-'0') * 100;
33 |             b[i] += (s[j++]-'0') * 10;
34 |             b[i] += (s[j++]-'0');
35 |         }
36 |     }
37 |
38 |     return b;
39 | }

```

Fig. 5.1 Big integer input in C. This routine takes a string representing a big integer in decimal and converts it to a base 10,000 number. The required standard C libraries are also listed

that might have fewer than four digits in the element. The digit is set to zero; then the `switch` statement determines how many excess digits there are in the input string.

Notice there are no `break` statements in the `switch`. This will cause the first matched condition to fall through to all the lower ones, which is what we want. Lines 31 through 34 handle all other digits of the output big integer. In this case, since we know there are four characters matched to this digit, we do not need the `switch` statement, and the logic is easier to follow. We successively add each character multiplied by the proper power of ten and convert it to a digit value by subtracting the ASCII value for zero. When the loop over output digits is complete, the number is complete, and a pointer to the memory is returned in line 38.

Figure 5.2 takes a pointer to a big integer in `b` and a pointer to an output string buffer in `s`. The big integer is then put into `s` as an ASCII string. This conversion

```

1 #include <stdio.h>
2
3 void bigint_output(signed int *b, char *s) {
4     int i, j=0;
5     char t[5];
6
7     if (b[0] < 0)
8         s[j++] = '-';
9
10    for(i=1; i <= abs(b[0]); i++) {
11        sprintf(t, "%04d", b[i]);
12        if (!((t[0] == '0') && (i == 1))) s[j++] = t[0];
13        if (!((t[1] == '0') && (i == 1))) s[j++] = t[1];
14        if (!((t[2] == '0') && (i == 1))) s[j++] = t[2];
15        s[j++] = t[3];
16    }
17
18    s[j] = '\0';
19 }

```

Fig. 5.2 Big integer output in C. This routine takes a base 10,000 big integer and outputs it as a decimal string

is simpler than going the other way, as in Fig. 5.1. If the first element of the big integer is negative, the number is negative, so we output the sign in line 8, bumping the output character index, *j*. Lines 10 through 16 pass over the big integer, digit by digit, and convert each digit to a set of up to four ASCII characters. To save space, we use a temporary string, *t*, and the C library function `sprintf`, which prints into a string. This changes the *i*-th big integer digit into a four-character string. This string may have leading zeros because of the `%04d` format statement. If *i* is one, not zero, because we skip the sign/length element, then we do not want leading zeros, which explains the origin of the somewhat cryptic code in lines 12 through 14. Each of these `if` statements asks if we are working with the highest order digit, when *i* is one, and if that digit is zero. It then negates this to execute when we are not working with the highest order digit and the digit is zero. In that case, we want to copy the proper character from the temporary string, *t*, to the output string, *s*. Since we always have at least one character in the digit we are working with, line 15 has no `if` statement; it is always executed.

When the loop in lines 10 through 16 completes we have filled in all of the output string, character by character, with the decimal representation of the input big integer. All that remains is adding a null at the end of the output to make a valid C-style string (line 18).

Comparisons. Let's take a quick look at comparisons using big integers.

Figure 5.3 shows a basic big integer comparison function. The function `bigint_compare` takes two big integer arguments, *a* and *b*, and returns -1 if *a* < *b*, 0 if *a* = *b*, and +1 if *a* > *b*.

```

1 int bigint_compare(signed int *a, signed int *b) {
2     signed int i;
3
4     if ((a[0] > 0) && (b[0] < 0)) return 1;
5     if ((a[0] < 0) && (b[0] > 0)) return -1;
6
7     if ((abs(a[0]) > abs(b[0])) && (a[0] > 0)) return 1;
8     if ((abs(b[0]) > abs(a[0])) && (a[0] > 0)) return -1;
9     if ((abs(a[0]) > abs(b[0])) && (a[0] < 0)) return -1;
10    if ((abs(b[0]) > abs(a[0])) && (a[0] < 0)) return 1;
11
12    if (a[0] > 0) {
13        for(i=1; i <= abs(a[0]); i++) {
14            if (a[i] > b[i]) return 1;
15            if (b[i] > a[i]) return -1;
16        }
17    } else {
18        for(i=1; i <= abs(a[0]); i++) {
19            if (a[i] > b[i]) return -1;
20            if (b[i] > a[i]) return 1;
21        }
22    }
23
24    return 0;
25 }

```

Fig. 5.3 Big integer comparison. The function compares two big integers, a and b , and returns -1 if $a < b$, 0 if $a = b$, and +1 if $a > b$. With this basic comparison function it is straightforward to define $<$, $>$, $=$, $<=$, $>=$, and $!=$ via small wrapper functions

Lines 4 and 5 quickly compare the signs of the numbers. If they differ, we already know the proper return value and do not need to look at the actual data values. Lines 7 through 10 do similar quick tests. We already know that the signs of a and b are the same, so we look to see which has a larger number of digits. In line 7, we look to see if a has more digits than b and that they are both positive. If so, we know that a is larger, so we return 1. If both are positive and b has more digits, the case in line 8, we return -1 because a is the smaller value, and we are determining the relationship between a and b (not b and a). Lines 9 and 10 make the same comparison but for both a and b negative. In this case, the return values are flipped since if a has more digits than b , it must be larger in magnitude and negative, making it smaller than b .

Line 12 looks at the sign of a . At this point, we know a and b have the same sign and number of digits. We must, therefore, look digit by digit, the largest power of 10,000 first, to see where they might differ to determine which of the two is greater. The loop in lines 13 through 16 moves through a and b and uses digit values to decide if one is larger than the other. The loop in lines 19 through 21 does the same, but in this case, we know that a and b are both negative and have the same number of digits, so the return values are reversed. Lastly, if we reach line 24, we know that the numbers have the same sign, number of digits, and digit values, implying they are equal.

The function in Fig. 5.3 makes it easy to define wrapper functions to implement the actual comparison operations. If we were using C++, we would define a big integer class and overload $<$, $>$, $=$, etc. to implement the comparisons. As we are

using plain C, we instead implement a function library. A possible definition for the equal function is

```
int bigint_equal(signed int *a, signed int *b) {  
    return bigint_compare(a,b) == 0;  
}
```

Defining the remaining operators is left as an exercise.

Now that we know how to store big integers, how to read them from strings, how to write them to strings, and how to compare them, we are ready to implement basic big integer arithmetic.

5.3 Arithmetic with Big Integers

Basic arithmetic with big integers generally follows the school book methods, though, as we will learn, there are some effective multiplication alternatives if the number of digits becomes huge.

Let's develop routines for addition, subtraction, multiplication, and division. We will not implement C versions of the alternative multiplication routines, but we will discuss the operation of the algorithms. Note that the routines given here are bare-bones only. We are omitting necessary checks for memory allocation, etc. to make the routines as concise and easy to follow as possible. We understand that a proper big integer package would have all these checks in place.

Addition and Subtraction. Before contemplating addition and subtraction, let's consider how we will handle negative numbers.

We are using a sign-magnitude representation because using a complement notation might mean manipulating all the digits of the number when we can often be more clever about handling operations with regard to the signs. In addition, we have the situations in Table 5.1 where we always work with the absolute value of the integers a and b . Similarly, for subtraction, we have Table 5.2.

It becomes apparent that for general addition and subtraction, we need the ability to compare the absolute value of two big integers, negate a big integer, and add and subtract two positive big integers. Therefore, to properly implement addition and subtraction, we need two helper functions: `bigint_ucompare` to compare the absolute value of two big integers, and `bigint_negate` to negate a big integer. These are given in Fig. 5.4.

These functions are straightforward. Indeed, `bigint_negate` is about as simple as can be given the efficient way we have selected to represent our big integers in memory. To change the sign, we only need to negate the first element of the array. In addition, we return a reference to the array to compose the negate call with other big integer operations. The function `bigint_ucompare` is a reduced version of `bigint_compare` as given in Fig. 5.3 where we ignore the sign. It returns -1

Table 5.1 Steps to add two big integers paying attention to their signs

Sign of a	Sign of b	Steps to find $a + b$
+	+	Add $ a $ and $ b $
+	−	$ a > b $, subtract $ b $ from $ a $ $ a < b $, subtract $ a $ from $ b $. Negate answer
−	+	$ a > b $, subtract $ b $ from $ a $. Negate answer $ a < b $, subtract $ a $ from $ b $
−	−	Add $ a $ and $ b $. Negate answer

Table 5.2 Steps to subtract two big integers paying attention to their signs

Sign of a	Sign of b	Steps to find $a - b$
+	+	$ a \geq b $, subtract $ b $ from $ a $ $ a < b $, subtract $ a $ from $ b $. Negate answer
+	−	Add $ a + b $
−	+	Add $ a + b $. Negate answer
−	−	$ a \leq b $, subtract $ a $ from $ b $ $ a > b $, subtract $ b $ from $ a $. Negate answer

```
1 signed int *bigint_negate(signed int *a) {
2     a[0] = -a[0];
3     return a;
4 }
5
6 int bigint_ucompare(signed int *a, signed int *b) {
7     signed int i;
8
9     if (abs(a[0]) < abs(b[0])) return -1;
10    if (abs(a[0]) > abs(b[0])) return 1;
11
12    for(i=1; i <= abs(a[0]); i++) {
13        if (a[i] > b[i]) return 1;
14        if (b[i] > a[i]) return -1;
15    }
16
17    return 0;
18 }
```

Fig. 5.4 Big integer helper functions to compare the absolute value of two big integers, `bigint_ucompare`, and to negate a big integer, `bigint_negate`


```

1 signed int *bigint_uadd(signed int *a, signed int *b) {
2     signed int *c, *x, *y, *t;
3     int i, m, n, yy, carry=0;
4
5     if (abs(a[0]) > abs(b[0])) {
6         n = abs(a[0]); m = abs(b[0]);
7         x = a;         y = b;
8     } else {
9         n = abs(b[0]); m = abs(a[0]);
10        x = b;         y = a;
11    }
12
13    c = (signed int *)malloc((n+1)*sizeof(signed int));
14    c[0] = n;
15
16    for(i=n; i>0; i--) {
17        yy = (i>(n-m)) ? y[i-(n-m)] : 0;
18        c[i] = x[i] + yy + carry;
19        if (c[i] > 9999) {
20            carry = 1;
21            c[i] -= 10000;
22        } else
23            carry = 0;
24    }
25
26    if (carry) {
27        t = (signed int *)malloc((n+2)*sizeof(signed int));
28        for(i=1; i<=n; i++) t[i+1] = c[i];
29        t[0] = n+1;
30        t[1] = 1;
31        free(c);
32        c = t;
33    }
34    return c;
35}

```

Fig. 5.5 Low-level big integer addition using the school method

if $|a| < |b|$, 0 if $|a| = |b|$, and +1 if $|a| > |b|$. With these in hand, we can implement addition and subtraction using low-level functions that ignore the sign of their arguments. After that, we will use a higher level version of addition and subtraction that looks at the signs of the arguments to decide how to proceed.

Our low-level addition routine will pay no attention to the sign of the arguments. It will assume they are positive. This routine implements addition using the simple method we learned in school. We add digits, from right to left, carrying on every 10,000. We will add a new first digit to the output, if necessary, based on any carry in the top digit position. This is directly analogous to addition in any other base. We call the routine `bigint_uadd` and it is given in Fig. 5.5.

Recall that we will use a high-level driver routine to decide when to call the low-level add and subtract routines. Because of this, `bigint_uadd` will ignore the sign of its input and treat all arguments as positive. Lines 5 through 11 set up the addition. We use extra pointers, `x` and `y`, and assign them the arguments `a` or `b` so that `x` will always point to the larger number (line 5). We also use `n` and `m` to hold the number of digits in these numbers. We do this to align the digits properly when we move from right to left across the number, adding as we go. Since `y` will always have as many or fewer digits as `x` we can use this fact to substitute zero for digits that do not exist in `y`. Line 13 allocates memory for the addition, while line 14 sets the length of the output. We know that when adding two big integers, we may overflow in the most significant digit. If that happens, we adjust for it later in the routine and initially assume that the answer will fit in `n` digits, which is the number of digits in the larger of `a` or `b`. Lines 16 through 24 perform the actual addition, digit by digit, from the least significant digit to the most significant. We use `yy` to hold the digit value of the lesser of `a` and `b` (in terms of number of digits). Line 17 either returns the actual digit of `y` or zero if we are looking at digits of `x` that have no match in `y`. Line 18 sets the output digit in `c`. We add the current digit of `x` and `y` and any carry from the previous digit addition. We set `carry` to zero in line 3 when it was declared.

After adding the digits, we must check if we caused a carry. We are using base 10,000, so we know that any two digits added together will still fit in a single `signed int`. We simply need to see if the result is larger than 9999. If so, we subtract the base and set the carry for the next digit. Otherwise, we ensure that the carry is clear (line 23).

The addition loop continues until all the digits of `x` have been examined. We have the answer in `c` at the end of this, but we need to do one more check. We need to check for overflow of the most significant digit of `c`. This is line 26. If the carry is set from the last digit addition, we have overflowed the `n` digits we allotted to `c` and need to add one more most significant digit. Lines 27 through 33 create a new output array in `t` with `n+1` digits. Line 28 copies the digits from `c` to `t` leaving room for the new, most significant digit. Lines 29 and 30 set the number of digits in `t` and set the most significant digit to 1 since that is the carry. Lines 31 and 32 release the old memory used by `c` and point `c` to `t`. Line 34 returns the pointer to the new big integer, which holds the sum of `a` and `b`. This addition routine can, temporarily, use double the memory because of the copy when `c` overflows. For our present purposes, this inefficiency is tolerable and will seldom happen in practice.

Similarly, we implement the low-level subtraction routine, `bigint_usub`, in Fig. 5.6. This routine, like `bigint_uadd`, ignores the sign of its inputs. It also assumes that `a` is always larger, in terms of magnitude, than `b`. This means we will never underflow and end up with a negative answer. Our high-level driver routine will arrange things so that this assumption is always true. Lines 5 through 8 get the number of digits of our arguments in `n` and `m` and allocate room for our answer in `c`, which will always need `n+1` digits. We set `c[0]` to the number of digits.

Lines 10 through 19 perform the subtraction from least significant digit to most significant, using the same trick we used in `bigint_uadd` to get the digit of the smaller number in `y`, which may be zero if we are looking at digits `b` does not have.

```

1 signed int *bigint_usub(signed int *a, signed int *b) {
2     signed int *c, *t;
3     int i, m, n, y, borrow=0, zero=0;
4
5     n = abs(a[0]);
6     m = abs(b[0]);
7     c = (signed int *)malloc((n+1)*sizeof(signed int));
8     c[0] = n;
9
10    for(i=n; i>0; i--) {
11        y = ((i-(n-m)) > 0) ? b[i-(n-m)] : 0;
12        c[i] = a[i] - y - borrow;
13        if (c[i] < 0) {
14            borrow = 1;
15            c[i] += 10000;
16        } else
17            borrow = 0;
18        zero += c[i];
19    }
20
21    if ((c[1] == 0) && (zero != 0)) {
22        t = (signed int *)malloc(n*sizeof(signed int));
23        for(i=2; i<=n; i++) t[i-1] = c[i];
24        t[0] = n-1;
25        free(c);
26        c = t;
27    }
28
29    if (zero == 0) {
30        free(c);
31        c = (signed int *)malloc(2*sizeof(signed int));
32        c[0] = 1;
33        c[1] = 0;
34    }
35
36    return c;
37 }

```

Fig. 5.6 Low-level big integer subtraction using the school method

Line 12 subtracts the current digit and puts it in `c`. Note that we subtract any borrow that happened in the previous digit subtraction. We set `borrow` to zero when it is defined in line 3. How do we know if we borrowed? If the digit answer is negative, we must borrow a base value (10,000) from the next higher digit. The check for this is in line 13. If true, `borrow` is set to 1 (we only ever need to borrow 10,000), and we add the borrow into the current digit value to bring it positive. If no borrow is needed, we set `borrow` to zero and continue to the next most significant digit. Line 18 appears to be curious. It adds the current digit value to a single variable we are calling `zero`. This is set to zero when defined in line 3. We use this variable to decide if the result of the subtraction is exactly zero. If possible, this allows us to simplify the return value, `c`, in lines 29 through 34. If we are subtracting two large

```

1 signed int *bigint_add(signed int *a, signed int *b) {
2
3     if ((a[0]>0) && (b[0]>0))
4         return bigint_uadd(a,b);
5
6     if ((a[0]<0) && (b[0]<0))
7         return bigint_negate(bigint_uadd(a,b));
8
9     if ((a[0]>0) && (b[0]<0)) {
10        if (bigint_ucompare(a,b) == -1)
11            return bigint_negate(bigint_usub(b,a));
12        else
13            return bigint_usub(a,b);
14    }
15
16    if ((a[0]<0) && (b[0]>0)) {
17        if (bigint_ucompare(a,b) == 1)
18            return bigint_negate(bigint_usub(a,b));
19        else
20            return bigint_usub(b,a);
21    }
22 }

```

Fig. 5.7 High-level big integer addition

integers and they happen to be exactly the same value, the result, without this check, would use as much memory as the arguments themselves with every digit set to zero. This check lets us save that memory and return a straightforward result that is only one digit long. The new result releases the memory the initial result uses and sets up a single-digit big integer that is exactly zero instead. We also check to see if the subtraction has left us with a leading zero. This is the purpose of lines 21 through 27. If the leading digit is zero, but the entire value is not zero, the leading digit is removed by allocating a new value in line 22 and copying the digits from *c* to this value, ignoring the initial zero. Once the loop in lines 10 through 19 ends, the answer is in *c*, and we return it in line 36.

Our final `bigint_add` and `bigint_sub` routines look at the signs of their arguments and proceed to call the low-level versions of add and subtract as appropriate. Figure 5.7 shows `bigint_add` while Fig. 5.8 shows `bigint_sub`.

Our high-level addition routine examines the signs and magnitudes of the arguments to determine how to get the proper result. The checks in the code are directly analogous to those of Tables 5.1 and 5.2. In addition, if the arguments are both positive, we simply add (line 3). If the arguments are both negative, we add and make the answer negative (line 6). If the signs are mixed, we need to consider the relationship between the magnitudes of the numbers. If *a* is positive and *b* is negative (line 9) we see two cases: $|a| < |b|$ which is handled in line 11 and $|a| \geq |b|$ which is handled in line 13. Similarly, for a negative and *b* positive (line 16) we consider cases when $|a| > |b|$ in line 17 and when $|a| \leq |b|$ in line 20.

```

1 | signed int *bigint_sub(signed int *a, signed int *b) {
2 |
3 |     if ((a[0]>0) && (b[0]>0)) {
4 |         if (bigint_ucompare(a,b) == -1)
5 |             return bigint_negate(bigint_usub(b,a));
6 |         else
7 |             return bigint_usub(a,b);
8 |     }
9 |
10 |    if ((a[0]<0) && (b[0]<0)) {
11 |        if (bigint_ucompare(a,b) == 1)
12 |            return bigint_negate(bigint_usub(a,b));
13 |        else
14 |            return bigint_usub(b,a);
15 |    }
16 |
17 |    if ((a[0]>0) && (b[0]<0))
18 |        return bigint_uadd(a,b);
19 |
20 |    if ((a[0]<0) && (b[0]>0))
21 |        return bigint_negate(bigint_uadd(a,b));
22 | }

```

Fig. 5.8 High-level big integer subtraction

For high-level subtraction in Fig. 5.8 we follow Table 5.2. If both a and b are positive, we look at $|a| < |b|$ in line 4 or otherwise in line 7. Similarly, if both are negative, we look at $|a| > |b|$ in line 11 and otherwise in line 14. The simpler cases here are if a is positive and b is negative, line 17, or a is negative and b is positive, line 20. The signs of the arguments will fit one of these cases for both addition and subtraction, so we know that all possibilities have been considered. Note that for subtraction, we always ensure that the first argument to `bigint_usub` is the same size or larger than the second.

Multiplication. Big integer multiplication can be implemented using the school method without much difficulty. We do that in Fig. 5.9, where we pay no attention to the signs of the arguments. Figure 5.10 is our high-level routine that manages the signs. Let's examine Fig. 5.10 first. The rules of multiplication say that when the signs of the multiplicand and multiplier are the same, either both positive or negative, then the answer is positive. This check is made in line 3. If the signs are the same, we simply call `bigint_umult` as that will always give a positive result. If the signs are different, we drop to line 7 and negate the result since the result should be negative.

Figure 5.9 is really three separate functions, all of which are used to multiply the numbers stored in a and b . Lines 1 through 7 define a simple helper function that duplicates an existing big integer. It allocates space for the new integer and copies all the bytes of a to b by calling the C library function, `memcpy`. It then returns the reference to this new big integer.

```

1 signed int *bigint_copy(signed int *a) {
2     signed int *b;
3
4     b = (signed int *)malloc((abs(a[0])+1)*sizeof(signed int));
5     memcpy((void *)b, (void *)a, (abs(a[0])+1)*sizeof(signed int));
6     return b;
7 }
8
9 signed int *bigint_umultd(signed int *a, signed int d, unsigned int s) {
10     signed int *p, carry=0, n;
11     int i;
12
13     n = abs(a[0]) + s + 2;
14     p = (signed int *)malloc(n*sizeof(signed int));
15     memset((void *)p, 0, n*sizeof(signed int));
16     p[0] = n-1;
17
18     for(i=abs(a[0]); i>=1; i--) {
19         p[i+1] = a[i] * d + carry;
20         if (p[i+1] > 9999) {
21             carry = p[i+1] / 10000;
22             p[i+1] = p[i+1] % 10000;
23         } else
24             carry = 0;
25     }
26
27     if (carry) p[1] = carry;
28     return p;
29 }
30
31 signed int *bigint_umult(signed int *a, signed int *b) {
32     int i, n;
33     signed int *m, *x, *y;
34     signed int *p=(signed int *)NULL, *q=(signed int *)NULL;
35
36     if (abs(a[0]) > abs(b[0])) {
37         x = a; y = b;
38     } else {
39         x = b; y = a;
40     }
41
42     m = (signed int *)malloc(2*sizeof(signed int));
43     m[0] = 1; m[1] = 0;
44
45     for(i=abs(y[0]); i>=1; i--) {
46         if (p != NULL) free(p);
47         if (q != NULL) free(q);
48         p = bigint_umultd(x, y[i], abs(y[0])-i);
49         q = bigint_uadd(m, p);
50         if (m != NULL) free(m);
51         m = bigint_copy(q);
52     }
53
54     if (p != NULL) free(p);
55     if (q != NULL) free(q);
56     return m;
57 }

```

Fig. 5.9 Low-level big integer multiplication using the school method

```

1 | signed int *bigint_mult(signed int *a, signed int *b) {
2 |
3 |     if ((a[0]>0) && (b[0]>0)) ||
4 |         ((a[0]<0) && (b[0]<0))
5 |         return bigint_umult(a,b);
6 |
7 |     return bigint_negate(bigint_umult(a,b));
8 | }

```

Fig. 5.10 High-level big integer multiplication

Lines 9 through 29 define `bigint_umultd` which multiplies a big integer in `a` by a single digit, `d`. When we multiply two numbers using the school method, we accumulate partial products, shifting them for the power of the base we are currently multiplying by. We then add these together to get the final result. For example,

$$\begin{array}{r}
 1776 \leftarrow \text{multiplicand} \\
 \times 1492 \leftarrow \text{multiplier} \\
 \hline
 3552 \leftarrow \text{1st partial product} \\
 159840 \\
 710400 \\
 + 1776000 \leftarrow \text{4th partial product} \\
 \hline
 2649792
 \end{array}$$

We can get the same result by starting with a product of zero and adding each digit times the multiplicand after shifting to the left as many places as the digit's location in the multiplier. This is exactly what we do when we write and then sum the partial products. This same approach works regardless of base. We do the same thing in `bigint_umultd`. The multiplicand is in `a`, the digit to multiply by is in `d`, and the number of places to shift the result to the left is in `s`. Line 13 of Fig. 5.9 sets `n` to the number of output digits. We need room for all the digits in `a`, which we get with `abs(a[0])`. To this, we add `s`; these are extra digits at the end of the number we will initialize to zero. This takes the place of the zeros added to the end of the partial products in the example above. We need to add one more array element to hold the sign and length of the number, and to this, we add still one more digit to handle any overflow (carry in the last digit multiplied). This is the origin of the `+ 2` in line 13. Line 14 allocates the product and points `p` to it. Line 15 erases the entire number and sets it to zero, while line 16 sets the length of the number. This initializes the partial product.

Lines 18 through 25 do the multiplication by the digit, `d`. We look at all the digits of the multiplicand `a` from least significant to most significant. This defines the loop in line 18. Line 19 does the multiplication of the digit of `a` by `d`. There are two things to note in this line. First, we add `carry`, just as we did for addition, but in this case, the carry may be more than one. Second, we assign this result to `p[i+1]` and not `p[i]`. This is done to leave a leading zero in `p` to handle any final overflow. Recall that array elements further from `p[0]` are less significant. Lines 20 and 24 handle

any carry. We have a carry if the resulting multiplication is greater than 9999. Unlike addition, the carry is not simply one but is the value of `p[i+1]` divided by the base. The actual value that should be in `p[i+1]` is the remainder when dividing by the base. If the carry has not happened, line 24 clears `carry`.

When the loop exits, it is possible that `carry` is not zero. Line 27 checks for this and sets the leading zero already allocated to the carry. Line 28 returns the partial product. A full implementation would do something about the leading zero in `p`, but we ignore it here.

The function that does the actual unsigned multiplication is `bigint_umult`. The result of the multiplication will be in `m`, which we initialize to zero in lines 42 and 43 of Fig. 5.9. Before this, lines 36 through 40 set pointers `x` and `y` so that `x` always points to the input with the largest number of digits, which may be the same as the other input. We do this because we need fewer passes through the loop in line 45 if we make the multiplier the value with the least number of digits.

The loop in lines 45 through 52 accumulates the partial products of the digits of `y` (the shorter of `a` and `b`) with `x`. This code is inefficient because it requires copying and freeing memory more than is necessary, but doing so makes the algorithm easier to follow. We use `p` and `q` to point to temporary results. In this case, `p` points to the partial product returned by the call to `bigint_umultd`. This call multiplies `x` by the current digit of `y` and shifts the result `abs(y[0]) - i` places to the left. As we move from right to left over the digits of `y`, we need to shift each partial product from the current digit position to the left, and this is exactly `abs(y[0]) - i`. When this call finishes, the partial product is in `p` and must be added to our accumulated product. We reuse `bigint_uadd` to do this and store the new running product value in `q`. We really want this result in `m`, so we release memory used by `m` and call `bigint_copy` to copy `q` to `m`. This is necessary so that the next pass through the loop can update `q`. This is a consequence of using C, not a language with automatic garbage collection. At the end of the loop, we have the final product in `m`. To avoid leaking memory, we must free `p` and `q` at the end of the loop. Finally, we return the product in line 56.

This implementation is pedagogical as there are more efficient ways to implement the multiplication routine. The school approach is well known to be of order n^2 , denoted $O(n^2)$. Below, we will discuss alternate big integer multiplication routines that scale better than this. If a routine runs in $O(n^2)$ time, it means that as n , here the number of the digits, increases, the running time increases as the square of the number of digits (with an unstated multiplier that is generally ignored). An $O(n^2)$ routine may be perfectly acceptable for small inputs (relatively few digits) but quickly becomes unusable when the number of digits becomes large.

Division. The division of big integers is rather complex to implement quickly. The school algorithm for division is typically implemented, but for this book, even that is too long. So, to provide a minimalist but complete package for big integer arithmetic, we will instead implement the most straightforward approach possible. However, in the next section, we will discuss more comprehensive algorithms for division and multiplication.


```

1 signed int *bigint_udiv(signed int *a, signed int *b) {
2     signed int *q, *t, *c, *x=NULL, *y=NULL;
3
4     q = (signed int *)malloc(2*sizeof(signed int));
5     q[0] = 1;
6     q[1] = 0;
7
8     c = (signed int *)malloc(2*sizeof(signed int));
9     c[0] = 1;
10    c[1] = 1;
11
12    t = bigint_copy(a);
13
14    while (1) {
15        if (x != NULL) free(x);
16        if (y != NULL) free(y);
17        x = bigint_usub(t,b);
18        y = bigint_uadd(q,c);
19        free(t);
20        free(q);
21        t = bigint_copy(x);
22        q = bigint_copy(y);
23
24        if (bigint_ucompare(t,b) < 1)
25            break;
26    }
27
28    free(x);
29    free(y);
30    free(t);
31    free(c);
32    return q;
33}

```

Fig. 5.11 Low-level big integer division

So, how do we implement division with big integers? Thinking back to the simplest possible ways of working with numbers, we remember that the division of two integers is really counting the number of times the divisor can be subtracted from the dividend before we hit zero or go below zero if the division would leave a remainder. In this case, we ignore the remainder and leave implementing it as an exercise.

We implement division as a high-level function that handles the signs of the numbers and a low-level function that deals with the magnitude only. Figure 5.11 shows our “division by repeated subtraction” approach. We divide a by b , ignoring signs, by counting the number of times we can subtract b from a , updating a in the process before we hit zero or go negative. This is the quotient that we store in q . Lines 4 through 6 initialize the quotient and set it to zero. Likewise, lines 8 through 10 initialize a constant value of 1, added to q each time through the loop.

Line 12 copies a so we can subtract from it without altering it in memory. The loop of lines 14 through 26 do the work. We must manage memory carefully to

```

1 signed int *bigint_divide(signed int *a, signed int *b) {
2     int sa, sb;
3     signed int *c;
4
5     if ((bigint_iszero(b)) ||
6         (bigint_ucompare(a,b) == -1)) {
7         c = (signed int *)malloc(2*sizeof(signed int));
8         c[0] = 1;
9         c[1] = 0;
10        return c;
11    }
12
13    sa = (a[0] < 0) ? -1 : 1;
14    sb = (b[0] < 0) ? -1 : 1;
15    a[0] = abs(a[0]);
16    b[0] = abs(b[0]);
17
18    if (((sa<0) && (sb<0)) ||
19        ((sa>0) && (sb>0))) {
20        c = bigint_udiv(a,b);
21    } else {
22        c = bigint_negate(bigint_udiv(a,b));
23    }
24
25    a[0] = sa*a[0];
26    b[0] = sa*b[0];
27
28    return c;
29 }

```

Fig. 5.12 High-level big integer division

avoid leaks, so we use auxiliary pointers x and y to hold intermediate results. Line 17 subtracts the divisor in b from t , which is the portion of a remaining. Line 18 bumps the quotient counter by adding the constant 1 in c . Lines 21 and 22 copy the partial results of the loop to t and q . We do this to free memory before losing the pointer address. Languages with automatic garbage collection can avoid this sort of inelegance. Line 24 checks to see if we have reached a point where the remainder in t is less than b , the divisor. If so, we are done and break out of the loop. After cleaning up memory, the quotient is returned.

It is well known that this approach to division, while correct, is terribly inefficient, especially for fixed-length integers. Be that as it may, the algorithm is straightforward to understand. All we need to do now is account for the signs of the dividend and divisor. This is done in Fig. 5.12 through the `bigint_div` routine.

To handle the signs of a and b , it is necessary to check several conditions. Along the way, we check for situations that result in invalid outputs for the case of integer division or division by zero. Lines 5 through 11 do this. If the divisor is zero (`bigint_iszero(b) == 0`) or the dividend is less than the divisor, we return a constant value of zero. For the second case, this is correct, but division by zero should

instead signal an error. For clarity, we ignore the error and return the incorrect value of zero instead. This would be corrected if a complete big integer package is implemented.

We extract the signs of the arguments in lines 13 and 14. We then ensure that they are positive so we can pass the values to `bigint_udiv`. We will reset the signs of the arguments when done; see lines 25 and 26. We know from school that if the signs of `a` and `b` are the same, either both positive or both negative, the quotient must be positive. So, if this is the case, we execute line 20 to perform the division. If the signs of the arguments are opposite, either `a` positive and `b` negative or vice versa, we must also make the answer negative. In this case, line 22 is what gets executed. After restoring the proper signs of the arguments, we return the quotient in line 28.

Division by repeated subtraction is unsatisfying in the long run and can be incredibly inefficient. However, it completes our implementation of basic arithmetic for big integers and, therefore, is worth looking at even if it is not how things would be done if we were developing a big integer package for widespread use.

The algorithms we implemented above are adequate for simple purposes, but people have designed far better algorithms, especially for multiplication and division. In the next section, we look at improved algorithms without implementing them. This will give an appreciation for what goes into efficient algorithm implementation and design and will provide us with a background we will use when discussing particular big integer libraries that you may someday wish to use in your programs.

5.4 Alternative Multiplication and Division Routines

In this section, we review alternate multiplication and division algorithms. A proper big integer package would implement some, if not all, of these alternatives. By necessity, we are cherry-picking algorithms. A particularly nice treatment of big integer algorithms is found in Chap. 9 of Crandall and Pomerance [1], and readers interested in a more thorough mathematical treatment are directed there.

Multiplication. In the previous section, we implemented the school method for multiplying big integers. As we noted, this method runs $O(n^2)$ in the size of the inputs. While this is adequate for numbers with perhaps a thousand digits, it becomes intractable if we work with large numbers with tens of thousands, hundreds of thousands, or even millions of digits. We need to improve our multiplication algorithm to work with large big integers. Fortunately, this problem has been investigated, and several alternate multiplication algorithms exist. We will discuss three of them: Comba multiplication, Karatsuba multiplication, and FFT multiplication.

Our first alternative is the Comba algorithm [2]. Comba noted that the school method is geared towards humans and not computers. After each single-digit multiplication of a column, we carry to the next column. In Comba multiplication, we do not carry but store the entire product for each column position. We then repeat for the next digit to multiply, adding in any existing value in that column (from a previous

			2	3	7	
		×	1	4	8	
(a)			16	24	56	← 237×8
		8	12	28		← 237×4
	+	2	3	7		← 237×1
		2	11	35	52	56 ← column sums
		3	5	0	7	6 ← carries, right to left

			2	3	7	
		×	1	4	8	
(b)			16	24	56	← column 1
		8	28	52	56	← column 2
	2	11	35	52	56	← column 3
	3	5	0	7	6	← carries, right to left

Fig. 5.13 Comba column-wise multiplication of 237×148 in base 10. **a** Writing out the multiplication by hand, keeping each column without carries to the next. After all digits have been multiplied, we add the columns. Finally, we perform the carries from right to left. **b** The same would be implemented in a computer. In this case, a single output number stores the columns, adding the new value for that column to the previous one. Once all the digits of the multiplier have been visited the output is adjusted from right to left to achieve the same answer as in **a**

digit multiplication). When all single digit multiplications have been performed, the resulting vector of column values is adjusted from left to right by adding any carry to the next digit to the left and leaving the remainder in the column. When this is done, the vector will contain the output value. The process is described in Fig. 5.13 where we work in base 10 for clarity. If we were implementing this algorithm for our big integer representation, we would use base 10,000 instead.

In (a), we work out the multiplication using the school method, but instead of carrying after each multiplier digit multiplication, we keep the value in the proper column. When each multiplier digit has been used, we sum the results to get an intermediate answer that needs to be adjusted for the carries we did not perform in the previous steps. The last step is to move from right to left, replacing each column value, n , with $n\%10$, and adding to the next digit the result of $n/10$ using integer division. This handles all the carries we did not do in the lines above. In (b), we do the same as in (a), but each time we multiply a digit, we immediately add the result to the proper column of the single output value. Then, when done, we handle the carries.

Comba multiplication is generally believed to be about 30% more efficient than the school method. However, there are some things to keep in mind before using it. Depending upon the way digits of the big integer are stored, it is possible to overflow a particular digit value because a column may need to sum the single-

digit multiplication for n digits where n is the minimum number of digits in the multiplicand and multiplier. For that reason, Comba multiplication is limited in the number of digits in the product and is unsuitable for multiplying truly large numbers. Additionally, while a 30% improvement is helpful, the algorithm is still essentially the school method and is still $O(n^2)$. We can do better than this.

Karatsuba multiplication [3] improves on the school method by simplifying the multiplication. The essential insight involves simplifying the multiplication of two large integers, a and b , by replacing the multiplication with three multiplications of numbers with about half as many digits as a and b . If we choose m , $m < n$, where n is the number of digits, we can rewrite a and b as

$$\begin{aligned} a &= a_1 B^m + a_0 \\ b &= b_1 B^m + b_0 \end{aligned}$$

where a_0, a_1, b_0 , and b_1 are all less than B^m where B is the base of our numbers, for example, $B = 10,000$. If we then calculate ab , we get

$$ab = (a_1 B^m + a_0)(b_1 B^m + b_0) = a_1 b_1 B^{2m} + (a_1 b_0 + a_0 b_1) B^m + a_0 b_0$$

which we can rewrite as

$$\alpha B^{2m} + \beta B^m + \gamma$$

with

$$\begin{aligned} \alpha &= a_1 b_1 \\ \beta &= a_1 b_0 + a_0 b_1 \\ \gamma &= a_0 b_0 \end{aligned}$$

This is seemingly no improvement. Instead of the single multiplication ab we now have *four* multiplications to get α , β and γ , but we press on anyway and will soon see the benefit. We can remove one of the four multiplications by rewriting our expressions. First we look at $(a_1 + a_0)(b_1 + b_0)$,

$$\begin{aligned} (a_1 + a_0)(b_1 + b_0) &= a_1 b_1 + a_1 b_0 + a_0 b_1 + a_0 b_0 \\ &= (a_1 b_0 + a_0 b_1) + a_1 b_1 + a_0 b_0 \end{aligned}$$

where the first term on the right is β . This means we can determine β with one less multiplication,

One thing to note is that $(a_1 + a_0)(b_1 + b_0)$ can be calculated using Karatsuba multiplication, meaning this algorithm is recursive. This is one of its strong suits.

$$\begin{aligned}
 a_1b_0 + a_0b_1 &= (a_1 + a_0)(b_1 + b_0) - a_1b_1 - a_0b_0 \\
 \beta &= (a_1 + a_0)(b_1 + b_0) - \alpha - \gamma
 \end{aligned}$$

So, where is the benefit? It comes from the fact that the three multiplications we replaced ab with are numbers with roughly half the number of digits as a or b . While a formal complexity analysis is beyond the scope of this book, we see that the additions and subtractions are linear, $O(n)$, and as n grows, they become less critical. The recursive nature makes the analysis a bit tricky, but the final result is that the algorithm as a whole is $O(n^{\lg(3)})$ where $\lg(3)$ is the log base 2 of 3. Compare this to the $O(n^2)$ of the school method, and as Fig. 5.14 shows, this difference quickly becomes significant for large n .

If n is too small, say less than four digits, using the school method to multiply ab is more efficient, implying we can use the school method as the terminating condition of the recursive application of Karatsuba multiplication.

Our final multiplication algorithm is that of Schönhage and Strassen [4]. This algorithm is based on abstract algebra and Fourier transform theory, but we will describe it at a high level. Here, we discuss the variant presented as Algorithm 9.5.23 in [1].

Before diving into the algorithm itself, we need to address a few concepts related to the operation of the algorithm, namely, its connection to abstract algebra and a bit about convolution, multiplication, and Fourier transforms.

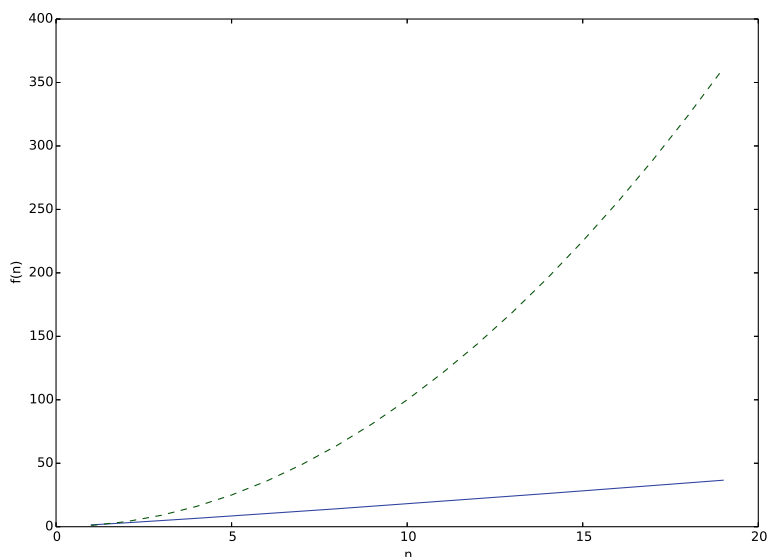


Fig. 5.14 $O(n^{\lg(3)})$ (solid blue line) versus $O(n^2)$ (dashed green line) growth as n increases. This plot shows that the Karatsuba method is a significant improvement over the school method for suitable input lengths (n)

The algorithm calculates multiplication modulo a specific integer, $2^n + 1$. This means that the result, which is an integer, is a number from $\mathbb{Z}_{2^n+1}$. If you are unfamiliar with the notation, $\mathbb{Z}$ refers to the set of integers, and $\mathbb{Z}_m$ refers to the set of integers modulo m . For example, $\mathbb{Z}_4 = 0, 1, 2, 3$ and operations that go above or below “wrap-around” as needed so that in $\mathbb{Z}_4$ we have $2 + 3 = 1$. In practice, this is not a problem as we select n so that $2^n + 1$ is larger than the product of our arguments, $2^n + 1 > xy$.

The algorithm operates on a *ring* [5] defined in $\mathbb{Z}_{2^n+1}$ and uses only integer operations. A ring is an enhancement of a group, which is itself a generalization of a binary operation acting on a set. In a ring, along with the first binary operation of the group, often denoted as $+$ even though it need not be addition, we add a second operation, $\bullet$, which need not be multiplication. Here is the abstract algebra connection. In our case, addition is addition, multiplication is multiplication, and the set is $\mathbb{Z}_{2^n+1}$, a (possibly very large) set of integers. If we select n so that $n \geq \lceil \lg x \rceil + \lceil \lg y \rceil$ then $2^n + 1 > xy$.

In the Comba algorithm for multiplication, we learned that multiplying the multiplicand by a digit of the multiplier, especially when we collected the resulting value without carrying, was essentially convolution. A convolution passes one value over another, so here we “pass” the single multiplier digit “over” the multiplicand digits and collect the product. Therefore, the multiplication of two numbers is related to convolution. And this is where the Fourier connection comes from.

We need not know anything about Fourier transforms at present beyond the fact that they map a sequence of values from one space to another space. We denote a Fourier transform of x , where x is a vector of values, by $\mathcal{F}(x)$ and recognize that this forward transform has an inverse transform, $\mathcal{F}^{-1}$, so that $\mathcal{F}^{-1}(\mathcal{F}(x)) = x$. In truth, the Fourier transform is such a powerful mathematical tool that all readers should be familiar with it. Please review Bracewell [6] or a similar text to familiarize yourself with this elegant addition to the mathematical toolbox.

One of the many properties of the Fourier transform is that multiplication in one space is equivalent to convolution in another. This is a key hint in understanding what the Schönhage and Strassen algorithm is doing. The algorithm will calculate the convolution of x and y through the Fourier transform and multiplication when in Fourier space. In essence, the algorithm does

$$\begin{array}{ll} X' = \mathcal{F}(X), Y' = \mathcal{F}(Y) & \leftarrow \text{map to Fourier space} \\ Z' = X'Y' & \leftarrow \text{calculate a product} \\ Z = \mathcal{F}^{-1}(Z') & \leftarrow \text{return to } X \text{ and } Y \text{ space} \end{array}$$

which at first seems backward but isn’t because the result in z is the convolution of X and Y , which is what we want. It is important to note that X and Y are not x and y , the numbers to be multiplied, but decompositions as performed in the algorithm’s first part. We are intentionally glossing over the fact that the Fourier transform referred to by $\mathcal{F}$ is not the one familiar to scientists and engineers but a generalization of it

1. [Initialize]
Choose an FFT size, $D = 2^k$ dividing n .
Set a recursion length, $n' \geq 2M + k$, $n = DM$ such that D divides n' , $n' = DM'$.
2. [Decomposition]
Split x and y into arrays A and B of length D such that $A = A_0, A_1, \dots, A_{D-1}$ and $B = B_0, B_1, \dots, B_{D-1}$ as residues modulo $2^{n'} + 1$.
3. [Prepare FFT]
Weight A and B by $(2^{jM'} A_j) \bmod (2^{n'} + 1)$ and $(2^{jM'} B_j) \bmod (2^{n'} + 1)$ respectively for $j = 0, 1, \dots, D-1$.
4. [Perform FFT]
 $A = \mathcal{F}(A)$ and $B = \mathcal{F}(B)$ done in-place.
5. [Dyadic stage]
 $A_j = A_j B_j \bmod (2^{n'} + 1)$ for $j = 0, 1, \dots, D-1$. Storing product in Fourier space in A .
6. [Inverse FFT]
 $A = \mathcal{F}^{-1}(A)$. A now contains the convolution of the decomposition of x and y . This is the product.
7. [Normalization and Sign Adjustment]
Scale A assigning to C to account for scale factor in Fourier transform.
8. [Composition]
Perform carry operations to C to give final sum: $xy \bmod (2^n + 1) = \sum_{j=0}^{D-1} C_j 2^{jM} \bmod (2^n + 1)$ which is the desired product.

Fig. 5.15 The Schönhage and Strassen big integer multiplication algorithm as presented in [1] with paraphrasing and explanatory text added

that operates via integer operations only. See Definition 9.5.3 of [1], particularly the integer-ring DFT variant.

We are now ready to examine the Schönhage and Strassen algorithm as presented in Fig. 5.15 with apologies to Crandall and Pomerance for the paraphrasing.

In step 1, we select a size for our Fourier transforms and n' . The selection of n' is not arbitrary. It is chosen so that the algorithm will work on smaller sets in step 5 should the multiplication there, which is also of the form $xy \bmod N$ where $N = 2^{n'} + 1$, be done by a recursive call to the algorithm itself. In step 2, we split the input values, x and y , into vectors A and B . These vector representations will be mapped to Fourier space to perform the convolution. Recall that the convolution in this space will mimic the school multiplication method.

In Step 3, we weight the vectors A and B to prepare for the FFT, remembering that the FFT here implements integer-only operations. The multiplication can be performed using only shifts in a binary representation of the numbers. Also, the modulo operation can likewise be implemented efficiently, as Crandall and Pomerance also show in [1].

Step 4 performs the actual FFT in-place to save memory, as these numbers may be enormous. Step 5 multiplies the components in Fourier space. Again, this is convolution in the original space, and this step requires something other than shifts and basic integer operations. Notice that the form of this multiplication matches what this algorithm does, so it is possible to make a recursive call here, which is the motivation for selecting n' properly. Other multiplication methods could also be used here, including the school method. Notice that the result is stored in A , again

to save memory. Step 6 performs the inverse FFT to return to the original space. Step 7 normalizes the result by applying a scale factor typical of the FFT, either split between the two transforms or applied in one direction. During this process, the vector C is created. Lastly, step 8 accounts for the carries (ala Comba above) and returns the desired product. A complexity analysis of this algorithm demonstrates that it runs in $O(n \log n \log \log n)$ which is significantly better than the $O(n^2)$ of the school method.

Division. For division, we look at two algorithms. The first is that found in Knuth [7] known as Algorithm D, which implements long division suitable for application to big integers. The second is the divide-and-conquer division algorithm of Burnikel and Ziegler [8].

In [7], Knuth gives a thorough description of his long division algorithm, which follows the school method except for one twist. The twist has to do with trial divisions based on the observation that assuming the quotient of an $n+1$ digit number x divided by an n digit number y to be the quotient of the top two digits of x by the leading digit of y means that your assumption will never be more than 2 away from the true value. Consider a few examples of this fact,

$$\begin{array}{rcl} 24356 / 5439 & = & 4 \mid 24 / 5 = 4 \mid 0 \\ 95433 / 6796 & = & 14 \mid 95 / 6 = 15 \mid 1 \\ 12345678 / 9876543 & = & 1 \mid 12 / 9 = 1 \mid 0 \\ 888888888 / 77777777 & = & 11 \mid 88 / 7 = 12 \mid 1 \end{array}$$

The rightmost column is the difference between the exact quotient and the trial quotient. It is important to note, however, that this fact is only true if the divisor is greater than or equal to the one half of the base; here, the base is 10, so the divisors are all 5 or larger. The normalization step of the algorithm takes care of this case by multiplying both dividend and divisor by a value, which can be done with a simple shift since the quotient will be the same, and the remainder can be unnormalized at the end.

The essence of the Knuth algorithm, paraphrased from [7], is given below:

1. [Normalize]

Scale inputs u and v by a single digit, d so that the leading digit of v , called v_1 , is $v_1 \geq \lfloor b/2 \rfloor$ where b is the base. Note that u and v are big-endian vectors representing the digits of the numbers in base b .

2. [Initialize]

Loop m times where m is the difference between the number of digits in u and v . If m is negative, the quotient is 0, and the remainder is v since $v > u$. Each pass through the loop is a division of the n digits of u starting at j , the loop index, by v .

3. [Calculate Trial Quotient]

Set $\hat{q} = \lfloor (u_j b + u_{j+1})/v_1 \rfloor$, the trial quotient. If $u_j = v_1$ set $\hat{q} = b - 1$. Knuth adds a test here to ensure that the trial quotient is never more than one greater than it should be.

4. [Multiply and Subtract]

Subtract $\hat{q}$ times v from the n digits of u , starting at position j , replacing those digits. This is the subtraction normally done in long division by hand. Since it is possible that $\hat{q}$ is too large, this result may be negative. If so, it will be handled in step 6.

5. [Test Remainder]

Set the j -th digit of the quotient to the trial value in $\hat{q}$. If the result of step 4 was negative, go to Step 6; otherwise, go to Step 7.

6. [Add Back]

Decrease the j -th digit of the quotient by one and add v back to u starting with the j -th digit. This accounts for the few times the trial quotient is too large.

7. [Loop]

Go back to Step 2 while $j \leq m$.

8. [Unnormalize]

The correct quotient is in q , and the remainder is u_{m+1} through u_{m+n} divided by d , the scale factor used in the first step.

The run time of this algorithm is $O(mn)$, which is adequate for smaller input sizes but becomes cumbersome if the inputs are very large.

Burnikel and Ziegler [8] implement two algorithms that work together to perform the division of big integers with specific numbers of digits. They describe how to apply these algorithms to arbitrary division problems. The first algorithm, which they name $D_{2n/1n}$, divides a number of $2n$ digits by a number of n digits. They do this by breaking the number up into super digits. If we wish to find A/B and use this algorithm we must break A into four digits, (A_1, A_2, A_3, A_4) and break B into two digits (B_1, B_2) so that each digit has a length of $n/2$. In other words,

$$\begin{aligned} A &= A_1 \beta^{3n/2} + A_2 \beta^n + A_3 \beta^{n/2} + A_4 \\ B &= B_1 \beta^{n/2} + B_2 \end{aligned}$$

With this separation of A and B into super digits we can examine the $D_{2n/1n}$ algorithm,

Algorithm $D_{2n/1n}$

1. [Given]
 $A < \beta^n B$, $\beta/2 \leq B < \beta^n$, n even, β is the base. Find $Q = \lfloor A/B \rfloor$ and remainder $R = A - QB$.
2. [High Part]
 Compute Q_1 as $Q_1 = \lfloor [A_1, A_2, A_3]/[B_2, B_1] \rfloor$ with remainder $R_1 = [R_{1,1}, R_{1,2}]$ using algorithm $D_{3n/2n}$.
3. [Low Part]
 Compute Q_2 as $Q_2 = \lfloor [R_{1,1}, R_{1,2}, A_4]/[B_2, B_1] \rfloor$ with remainder R using algorithm $D_{3n/2n}$.
4. [Return]
 Return $Q = [Q_1, Q_2]$ as the quotient and R as the remainder.

This algorithm is deceptively simple. The recursive nature of it becomes clear when we look at algorithm $D_{3n/2n}$ which in turn calls $D_{2n/1n}$ again. First we present $D_{3n/2n}$ and then we discuss them together,

Algorithm $D_{3n/2n}$

1. [Givens]
 $A = [A_1, A_2, A_3]$ and $B = [B_1, B_2]$ from $D_{2n/1n}$. Find $Q = \lfloor A/B \rfloor$ and remainder $R = A - QB$.
2. [Case 1]
 If $A_1 < B_1$ compute $\hat{Q} = \lfloor [A_1, A_2]/B_1 \rfloor$ with remainder R_1 using $D_{2n/1n}$.
3. [Case 2]
 If $A_1 \geq B_1$ set $\hat{Q} = \beta^n - 1$ and set $R_1 = [A_1, A_2] - [B_1, 0] + [0, B_1] = [A_1, A_2] - \hat{Q}B_1$.
4. [Multiply]
 Compute $D = \hat{Q}B_2$ using Karatsuba multiplication.
5. [Remainder]
 Compute $\hat{R} = R_1\beta^n + A_4 - D$.
6. [Correct]
 As long as $\hat{R} < 0$ repeat:
 - a. $\hat{R} = \hat{R} + B$
 - b. $\hat{Q} = \hat{Q} - 1$.
7. [Return]
 Return $\hat{Q}$ and $\hat{R}$.

The pair of algorithms, $D_{2n/1n}$ and $D_{3n/2n}$, work together on successively smaller inputs to return the final solution to the division problem. We know that $D_{2n/1n}$ requires A to have twice as many digits as B , and when it is called in the first [Case 1] of $D_{3n/2n}$ we are also passing it arguments that meet the $2n$ to n digit requirement so the recursion will end.

It is interesting to compare the steps in $D_{3n/2n}$ with those of the Knuth algorithm above. We get a “trial” quotient, $\hat{Q}$, from one of the cases and then determine the remainder in step 5. Then, in step 6, we correct the quotient in case we select an answer that is too large. The Knuth algorithm does precisely this in step 6. Here, we are structuring the problem so that the digits always match the desired pattern and use recursion to divide the work. In the Knuth algorithm, we move through the divisor digit by digit.

The school division method of Knuth is $O(mn)$, which would be $O(n^2)$ for the case we are considering here since the dividend has twice as many digits as the divisor. What about the Burnikel and Ziegler algorithm? As they discuss in [8], the performance of this algorithm depends upon the type of multiplication used in step 4 of $D_{3n/2n}$. If using Karatsuba multiplication, they determine the performance to be $2K(n) + O(n \log n)$ with $K(n)$ the performance of Karatsuba multiplication itself. As they note, if the school method of multiplication is used instead, we get $n^2 + O(n \log n)$, which is worse than long division itself.

We have examined, superficially, several alternatives to the basic arithmetic algorithms we developed for our big integer implementation. We learned that they outperform the naive algorithms and extend our abilities in working with big integers to those much larger than what we would be able to work with otherwise. Computer memory is inexpensive, so the limiting factor is now algorithm performance. Every small improvement counts. We might have examined many other algorithms but cannot for space considerations. There is ongoing research in this area, and tweaks and minor improvements are happening constantly. But, for now, we leave the algorithms and move on to more practical considerations, namely, implementations. It is fun to implement some of these basic algorithms ourselves, but the complex and highly performant ones are often tricky, and it is wise to use proven libraries when speed and accuracy count.

5.5 Implementations

Most programming languages do not define big integer operations as intrinsic to the language. This section will look at some that do, specifically Python and Scheme. We will also examine extensions to existing languages, such as C/C++, Java, and JavaScript, which give big integer functionality to developers using these languages. We begin with the extensions.

Libraries. Time is naturally a factor when discussing software libraries. Projects start and finish and are often abandoned as people move on to other work, making an orphan of the library, which becomes less and less relevant over time. Here we will look at one library that has considerable popular support and is a member of the Gnu Software suite, namely, the *GNU Multiple Precision Arithmetic Library* or GMP for short [9]. This library is open-source software distributed via the GNU LGPL and GNU GPL licenses. This allows the library to be used in its existing form in commercial code while enabling additions to the library to be passed on to other users. It has been in constant development since 1991, so we include it here, as it will likely be supported for some time.

GMP is a C and C++ library for various mathematical operations involving multi-precision numbers. At its core, it consists of several main packages: signed integer arithmetic functions (*mpz*), rational arithmetic functions (*mpq*), floating-point functions (*mpf*), and a low-level implementation (*mpn*). Additionally, a C++ class wrapper is available for the *mpz*, *mpq*, and *mpf* packages. Here, we will consider only the integer operations of the *mpz* package. The GMP library goes to great lengths to ensure efficiency and optimal performance.

The *mpz* package uses the same number format we used above. It defines a structure in `gmp.h` called `__mpz_struct` which is typedef-ed to `mpz_t`. The struct is

```
typedef struct
{
    int _mp_alloc;
    int _mp_size;
    mp_limb_t *_mp_d;
} __mpz_struct;
```

where `mp_limb_t` points to a “limb” or digit. Its actual size depends on the system the code is compiled for but is typically an unsigned long int, which is 4 bytes on the 32-bit system we assume in this book. The `_mp_alloc` field counts the number of limbs allocated while `_mp_size` is the number of limbs used. As we did above, the sign of the number is stored in the sign of `_mp_size`. If `_mp_size` is zero, the number is zero. This saves allocating a limb and setting it to zero.

Input of big integers is via strings, as in our code. Output is to file streams or strings depending on the function used. Integers must be created and initialized before being assigned but may be freely assigned afterward. When done, numbers must be cleared. The code below initializes a big integer, assigns it a value, and then prints the value to `stdout` in two different number bases,

```

1 | #include <stdio.h>
2 | #include <gmp.h>
3 |
4 | int main() {
5 |     mpz_t A;
6 |
7 |     mpz_init(A);
8 |
9 |     mpz_set_str(A, "1234567890123456789", 0);
10 |    mpz_out_str(NULL, 10, A);
11 |    printf("\n");
12 |    mpz_out_str(NULL, 2, A);
13 |    printf("\n");
14 |
15 |    mpz_clear(A);
16 | }

```

The GMP library is included in line 2 (linking with `-lgmp` is also required). A new big integer, `A`, is defined in line 5 and initialized in line 7. Note that `A` is not passed as an address. The definition of `mpz_t` is a one-element array of `__mp_struct` which passes it as an address already. In line 9, we assign `A` from a string, where 0 means use the string itself to determine the base, in this case, base 10. Lines 10 and 12 output `A` first as a base 10 number and then in binary. Finally, we release `A` in line 15.

Basic big integer functions in *mpz* include the expected addition (`mpz_add`), subtraction (`mpz_sub`), and multiply (`mpz_mul`). Also included are negation (`mpz_neg`) and absolute value (`mpz_abs`). There are also special functions that combine operations for efficiency, as it is sometimes better to combine them than perform them individually. For example, `mpz_addmul` adds the product of two numbers to an existing number, and `mpz_submul` subtracts the product.

For division *mpz* gives us many options. These options reflect various rounding methods and implement different algorithms, which may be much faster than general algorithms if certain conditions are met. If we select only the `tdiv` functions, which round towards zero, like C does, we have three functions:

```

mpz_tdiv_q - divide  $n$  by  $d$  returning only the quotient,  $q$ .
mpz_tdiv_r - divide  $n$  by  $d$  returning only the remainder,  $r$ .
mpz_tdiv_qr - divide  $n$  by  $d$  returning both quotient,  $q$ , and remainder,  $r$ .

```

There is also a general `mod` function which always returns a positive result (`mpz_mod`). If it is known in advance that the numerator n is exactly divisible by the denominator d , one may use `mpz_divexact`, which is much faster than the generic division routines. Rounding out the division options are `mpz_divisible` to check if n is exactly divisible by d and `mpz_congruent_p` to check if n is congruent to c modulo d meaning there is an integer q such that $n = c + qd$.

What algorithms are implemented by these top-level functions? For addition and subtraction, which are at the lowest level for Intel architecture implemented in assembly, we can expect the normal school method. What about multiplication? As we are now well aware, there are many options.

The GMP library uses one of seven multiplication algorithms for N digit by N digit multiplication (as of GMP version 6.0.0). Which is used depends upon the size of N and whether or not it exceeds a preset threshold. The algorithms are, in order of use (i.e., larger N uses algorithms further down the list),

1. School method. This is the algorithm we implemented above.
2. Karatsuba. The Karatsuba method is discussed above.
3. Toom-3. A three-way Toom extension to Karatsuba. See [7].
4. Toom-4. A four-way Toom algorithm.
5. Toom-6.5. A six-way Toom-like algorithm.
6. Toom-8.5. An eight-way Toom-like algorithm.
7. FFT. The Schönhage and Strassen method, as discussed above.

These algorithms are optimized by the authors of GMP and, at the lowest level, are somewhat different than might be found in the original papers. If the operands to the multiplication are of very different sizes, N digits by M digits, then below one threshold, the school method is used, while FFT is used for huge numbers. In between, the Toom algorithms are used based on heuristics given the sizes N and M .

For division, four main algorithms are in use. For numbers below a threshold (50 digits), the long division method of Knuth is used. Above this threshold, the divide-and-conquer method of Burnikel and Ziegler (modified) applies. Division by huge numbers uses an algorithm based on Barrett reduction [10]. Exact division uses Jebelean's method [11]. Switching between methods is automatic; the user only needs to call the top-level functions.

If GMP is for C and C++, what about other languages that lack intrinsic big integer functionality? Java uses the `BigInteger` class. This class comes with its own Java implementations for low-level operations that exactly match those found in GMP's *mpn* package. This was done to make it possible to easily replace the Java code with high-performance C code if desired.

For JavaScript, a common choice for big integer support is the `BigInteger.js` library, which is distributed via the MIT License and implements all functionality in pure JavaScript so it can run within a web browser. This implementation is very generic and uses a base of 10,000,000 to take advantage of the simplicity this provides input and output, just as we did with our implementation above. Addition and subtraction use the school method, as does multiplication. There is a comment in version 0.9 to add Karatsuba in the future. Division uses the Knuth algorithm.

Languages. We know that Python has intrinsic support for big integers. Let's examine what it is doing under the hood. The Python interpreter is written in plain C, and big integers are found in the source code file (referencing Python 2.7) `longobject.c`. This file contains implementations for all the operations supported by Python long

integers; it does not use the GMP or other libraries. For addition and subtraction, the expected school method is used. For multiplication, only two algorithms are available: the school method and Karatsuba if the number of digits exceeds 70. Division is long division *ala* Knuth. The numbers themselves use a base of 2^{15} . While the native implementations are adequate, the `gmpy` library is recommended for serious big integer work in Python.

Another language that supports big integers natively is Scheme [12]. Scheme is a dialect of Lisp, and has been used widely for instruction in computer programming. Indeed, it is used in the very popular *Structure and Interpretation of Computer Programs* by Abelson and Sussman [13] that has trained a generation of programmers from MIT, though it has fallen out of style in recent years. Regardless, the text comes highly recommended.

The version of Scheme we will consider is *Scheme48* available online from s48.org. This version of Scheme conforms to the R5RS version of the language [14] and includes big integer support for exact numeric computation. Section 6.2.3 of the R5RS report strongly encourages implementations to support “exact integers and exact rationals of practically unlimited size and precision”, which *Scheme48* does.

The Scheme interpreter is written in C and places big integer operations in the file `bignum.c`. Reviewing this file, we see that the implementations are based on earlier work from MIT. As expected, the addition and subtraction routines use the school method. The multiplication routine is also limited to the school method, regardless of the input size, thereby limiting the usefulness of Scheme for high-performance work with big integers unless an external library like GMP is used. We see that division is the Knuth algorithm, again, regardless of the size of the operands.

5.6 Rational Arithmetic with Big Integers

The ability to represent any integer p with an arbitrary number of digits leads one to consider representing arbitrary precision fractions, p/q . In this section, we explore rational arithmetic with big integer components. We will take advantage of the big integers provided by Python and use them to implement arbitrary precision rational arithmetic.

Elementary mathematics tells us how to implement the four basic arithmetic operations using fractions. They are

$$\text{Addition} \quad \frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd}$$

$$\text{Subtraction} \quad \frac{a}{b} - \frac{c}{d} = \frac{ad-bc}{bd}$$

$$\text{Multiplication} \quad \frac{a}{b} \times \frac{c}{d} = \frac{ac}{bd}$$

$$\text{Division} \quad \frac{a}{b} \div \frac{c}{d} = \frac{ad}{bc}$$

where a, b, c , and d are themselves big integers. However, there is one twist we must bear in mind. If we were to implement these operations directly, we would quickly find our numerators and denominators exploding in size. This is because, after every operation, we must reduce the resulting fraction to the lowest terms, as our grade school teachers repeatedly reminded us. Reducing a fraction involves finding the greatest common divisor (GCD) of the numerator and denominator. Thankfully, this is a straightforward task.

Storing rationals in Python. Python is a modern object-oriented language. To store a rational number, we must store two integers, the numerator and the denominator. As we are developing a rational arithmetic class, we will store the numerator and denominator as member variables `n` and `d`. Additionally, we will overload the basic math operations to work with rationals like other numbers. Input and output will be straightforward as we simply display the number by printing the numerator, the fraction symbol (`/`), and the denominator. We will adopt the convention that the sign of the rational is stored in the numerator.

A basic rational arithmetic class. The first part of our `Rational` class defines the class and the constructor, `__init__`. So far, then, our class looks like this

```
1 from types import *
2
3 class Rational:
4
5     def __init__(self, n,d=None):
6         if (d == None):
7             t = float(n).as_integer_ratio()
8             n = t[0]
9             d = t[1]
10        g = self.__gcd(n,d)
11        self.n = n//g
12        self.d = d//g
13        if (self.d < 0):
14            self.n *= -1
15            self.d = abs(self.d)
```

where we consistently use four spaces for indenting, as is the Python standard. In line 1 we include the `types` module, which is part of the standard Python library. We will use this to determine whether or not arguments are `Rational` objects or integers. Our constructor is in `__init__` and takes `self` as the first argument, as all Python methods do. `Self` refers to the object itself, allowing us to access member data and call other class methods. The constructor takes two arguments, `n` and `d`, which are the numerator and denominator of the new rational object. Lines 6 through 9 check to see if a single argument was given. If so, it is assumed to be a floating-point number and converted to a rational by calling the `as_integer_ratio` method of the `float` class. In line 10, we call the `__gcd` method to calculate the GCD of the given numbers. We will explore this method in more detail shortly. Note that we use the

standard Python naming convention of two leading underscores (`__`), which marks the method as private to our class.

Each method in our class representing an operation returns a new `Rational` object. This means that we only need to reduce the fraction when a `Rational` object is created. In lines 11 and 12, we set the local values for the numerator (`self.n`) and denominator (`self.d`) dividing each by the GCD so that the fraction is stored in the lowest terms. For simplicity, we ignore error states such as a denominator of zero. Lines 13, 14, and 15 ensure that the new fraction places its sign in the numerator, as is our convention. We have a new `Rational` object initialized with the given numerator and denominator in lowest terms. Now let's consider the `__gcd` method itself.

Reducing fractions using the GCD. In school, we learned that fractions are not unique. Therefore, when working with fractions, we were constantly encouraged to express our answers in lowest terms. When working with rational numbers on a computer, this is especially important to prevent our numerators and denominators from growing too large too quickly. To reduce the fraction, we need to find the GCD of p and q and divide p and q by it.

The iterative form of Euclid's GCD algorithm of Euclid [15] is an excellent way to calculate the GCD even for large numbers. Our implementation of it is

```
1 def __gcd(self, a,b):  
2     while b:  
3         a,b = b, a % b  
4     return abs(a)
```

Arithmetic and other methods. We are now ready to implement basic arithmetic operations. Python defines a set of methods that users can override in their classes to capture situations that syntactically look like more ordinary operations. For example, the seemingly simple statement

$$a + b$$

is equivalent to

$$a.__add__(b)$$

which means that a Python class that overrides the `__add__` method will be able to participate in mathematical expressions. We require this to make using the `Rational` class as natural as possible.

Python has many methods that we could override in our class. We will define some here and leave others for the exercises. In particular, we will override the following methods:

<code>__add__</code>	Addition (+)
<code>__sub__</code>	Subtraction (−)
<code>__mul__</code>	Multiplication (×)
<code>__truediv__</code>	Division (/)
<code>__str__</code>	String output for <code>str()</code> and <code>print</code>
<code>__repr__</code>	String output for <code>repr()</code>
<code>__float__</code>	Floating-point value for <code>float()</code>
<code>__int__</code>	Integer value for <code>int()</code>
<code>__long__</code>	Integer value for <code>long()</code>

In addition, we will override the “reverse” forms for the arithmetic operators. These are called if instead of `a+2` we write `2+a`. We only need to call the non-reversed form for operators that commute, like addition and multiplication. For division and subtraction, we have to be more careful.

So, how do we implement addition? Like so,

```

1 def __add__(self, b):
2     if isinstance(b, self.__class__):
3         n = self.n*b.d + self.d*b.n
4         d = self.d*b.d
5     elif isinstance(b, int):
6         n = self.n + self.d*b
7         d = self.d
8     return Rational(n,d)
9
10 def __radd__(self, b):
11     return self.__add__(b)

```

where we first look at the type of the argument, `b`, in line 2. We want to work not only with other rational numbers but also with integers. If the argument is another rational number, we move to line 4 and add to get the numerator followed by the denominator. Otherwise, if the argument is an integer, we move to line 7 to add the integer to the fraction. Line 9 returns the new rational answer. Recall that the constructor divides by the GCD, so it is not explicitly called here. Line 11 defines the reverse case method. Since addition is commutative, $a + b = b + a$, we simply fall back to the primary addition method.

Subtraction is virtually identical except for the reversed call,

```

1 def __sub__(self, b):
2     if isinstance(b, self.__class__):
3         n = self.n*b.d - self.d*b.n
4         d = self.d*b.d
5     elif isinstance(b, int):
6         n = self.n - self.d*b
7         d = self.d
8     return Rational(n,d)
9
10 def __rsub__(self, b):
11     if isinstance(b, self.__class__):
12         n = b*self.d - self.n
13         d = self.d
14     return Rational(n,d)

```

where `__sub__` is identical to `__add__` except for the change from `+` to `-`. However, as subtraction is not commutative, we must implement it explicitly for the reverse case. This is done in lines 13 through 16. Why is there no check for `InstanceType`? Because Python is smart enough to know that if both arguments are objects, the main method will be called instead.

Multiplication follows addition,

```

1 def __mul__(self, b):
2     if isinstance(b, self.__class__):
3         n = self.n * b.n
4         d = self.d * b.d
5     elif isinstance(b, int):
6         n = self.n * b
7         d = self.d
8     return Rational(n,d)
9
10 def __rmul__(self, b):
11     return self.__mul__(b)

```

while division follows subtraction in implementing the reverse call explicitly,

```

1 def __truediv__(self, b):
2     if isinstance(b, self.__class__):
3         n = self.n * b.d
4         d = self.d * b.n
5     elif isinstance(b, int):
6         n = self.n
7         d = self.d * b
8     return Rational(n,d)
9
10 def __rtruediv__(self, b):
11     if isinstance(b, int):
12         n = b * self.d
13         d = self.n
14     return Rational(n,d)

```

and in both cases, the proper steps for multiplication and division of fractions are performed.

The remaining methods are convenience methods,

```
1 def __str__(self):
2     if (self.n == 0):
3         return "0"
4     elif (self.d == 1):
5         return "%d" % self.n
6     else:
7         return "%d/%d" % (self.n, self.d)
8
9 def __repr__(self):
10     return self.__str__()
11
12 def __float__(self):
13     return float(self.n) / float(self.d)
14
15 def __int__(self):
16     return int(self.__float__())
17
18 def __neg__(self):
19     return Rational(-self.n, self.d)
20
21 def __abs__(self):
22     return Rational(abs(self.n), self.d)
```

which allows us to interact with Python more naturally. Use `__str__` to return a string representation of the fraction as the output of `str()` or `print`. This method checks for special cases like zero or an integer (a denominator of one). The `__repr__` method responds to the `repr()` function by returning exactly the same string `print` would receive. The `__float__` and `__int__` methods perform the necessary conversion to return the rational as a float or integer, respectively. These are used by the `float()` and `int()` functions. Finally, `__neg__` and `__abs__` respond to negation and `abs`.

Using the Rational class. We can use the `Rational` class in a quite natural way. The following examples are from an interactive session and illustrate how to use the class:

```
1 >>> from Rational import *
2 >>> a = Rational(2,3)
3 >>> b = Rational(355,113)
4 >>> c = a + b
5 >>> c
6 1291/339
7 >>> (a+b) * c
8 1666681/114921
9 >>> d = (a + b) * c
10 >>> d = 2*d
11 >>> d
12 3333362/114921
13 >>> float(b)
14 3.1415929203539825
15 >>> d / d
16 1
17 >>> 1/d
18 114921/3333362
```

where `>>` is the Python prompt, and replies from Python are lines without the prompt.

In line 1, we import the `Rational` class. Lines 2 and 3 define `a` and `b` to be $2/3$ and $355/113$, respectively. Line 4 adds `a` and `b` and assigns the result to `c` which is then displayed in line 6. Line 9 uses all three rationals in an expression and displays it. Note already how the numerator and denominator are growing. We define `d` in line 9 and multiply it by an integer in line 10. Line 13 shows us that $355/113$ is a good approximation of π by requesting the floating-point version of it. We use line 15 as a sanity check to ensure that a rational number acts rationally. Finally, line 17 checks that the inverse of a rational is what you get when you flip the numerator and the denominator.

5.7 When to Use Big Integers and Rational Arithmetic

The sections above demonstrate how to implement big integers, how to operate on them efficiently, and where to find performant big integer libraries, however, none of the above talks about *when* and *why* one might want to use big integers. We remedy that in this section.

In modern computing, the largest use of big integers is undoubtedly in cryptography. Cryptographic systems depend upon the fact that certain operations are easy to do in one direction but next to impossible to do in a reasonable period of time in the other direction. Most of these systems depend upon the following two facts:

1. The *fundamental theorem of arithmetic* states that all numbers are either prime or can be factored, uniquely, into a specific set of primes.
2. Factoring large integers is a tedious process, even though multiplying them is straightforward.

The facts imply that while it is easy to find $n = pq$ with p and q both integers, it is hard to find p and q such that $pq = n$. This is the second fact above. The first says that if p and q are prime then indeed the *only* way to factor n is to find the specific pair, p and q . If p and q are too small, trial and error will eventually find them, but if they are enormous, on the order of several hundred bits or about 100 decimal digits, then finding the factors of their product will take a very long time. This is the heart of modern cryptography. Let's briefly explore how multiplying large integers plays into it.

First, we consider classic Diffie-Hellman key exchange [16]. This system allows two parties, traditionally known as Alice and Bob, to exchange a secret number to be used as a key to encrypt a message shared between them. We are not concerned with the encryption but rather how they exchange the secret key so that an outsider (traditionally known as Carol or Eve) cannot learn what it is. Let's look at the steps, and then we will see where big integers play into it.

The heart of the exchange relies on the choice of p and g . We must choose p prime and g a primitive root modulo p . The exact definition of a "primitive root modulo p " does not concern us here; it is a number theory concept. What concerns us is that p and g are public and known by Alice, Bob, and Carol. Additionally, Alice and Bob each select a secret number only they know. We'll call Alice's number a and Bob's number b . These numbers are known only to the respective person. For our example, we will select $p = 37$ and $g = 13$, a primitive root of 37. With this setup, the steps necessary to exchange a secret key are

1. Alice chooses $a = 4$. Only Alice knows this number.
2. Bob chooses $b = 11$. Only Bob knows this number.
3. Alice sends Bob $A = 13^4 \bmod 37 = 34$. Carol sees A .
4. Bob sends Alice $B = 13^{11} \bmod 37 = 15$. Carol sees B .
5. Alice computes the secret key, $s = 15^4 \bmod 37 = 9$. Carol cannot compute s .
6. Bob computes the secret key, $s = 34^{11} \bmod 37 = 9$.
7. Alice and Bob both use s as the key for their encryption system.

Recall that a and b are secret but $p = 37$, $g = 13$, $A = 34$, and $B = 15$ are public. When the exchange is complete, Alice and Bob have both calculated $s = (g^a)^b \bmod 37 = (g^b)^a \bmod 37 = 9$, which they will use as the secret key for their encryption system.

Big integers come into this system with the choice of p , a , and b . If these are very large, hundreds of digits long, it becomes virtually impossible to determine s from the values that are known publicly. Even though A is known and it is known that $A = g^a \bmod p$, the reverse operation, known as the *discrete logarithm*, is very hard to do, so a remains unknown publicly. It is this difficulty that makes the key exchange system secure, and big integers are essential because, without them, the numbers involved would be too small to offer security against brute force attacks.

Another cryptographic system that relies on big integers is RSA public key encryption [17]. This system was published by Rivest, Shamir, and Adelman (hence RSA) of MIT in 1977. In this system, users generate two keys: a public key and a private

key. They then publish their public key, allowing anyone to encrypt a message using it. The holder of the corresponding private key, however, is the only one who can decrypt it. We ignore the extra twist of signing the message using the sender's private key for simplicity. The essence of the algorithm depends upon two primes, p and q ; their product, $n = pq$; a public key exponent, e ; and its multiplicative inverse, d . The person generating the keys keeps d as the private key and releases both n and e together to be the public key. With this public key, anyone can encrypt a message, but only the private key holder can decrypt it. The length of n is the key length, which is related to the algorithm's strength.

If Bob has released his public key, the pair n and e , then Alice can send Bob a secure message by turning her message into an integer, m ; the exact method for this is irrelevant but must be agreed upon by both Alice and Bob. Alice computes $c = m^e \bmod n$. She then sends c to Bob. Bob, in turn, recovers Alice's message by computing $m = c^d \bmod n$ using the private key, d . Since only Bob has d , he is the only one who can recover the message.

The security of the encryption depends upon choosing large primes for p and q . When these are very large numbers, recovering them from knowledge of just n becomes virtually impossible. The exponent, e , and private key, d , are calculated from p and q , so knowledge of p and q would break the entire algorithm. Fortunately, big integers come to the rescue and let us work with the huge primes that make the system secure.

Practical implementations of encryption algorithms that rely on big integers are found in *OpenSSL* [18], which implements the secure socket layer for Internet data transfer and *libgcrypt* [19] which is an open-source package implementing numerous algorithms, including RSA.

Big integers are of practical importance to cryptography, but mathematicians also use them for research purposes. For example, the open-source package *Pari* [20] supports arbitrary precision calculations for mathematicians interested in number theory and other disciplines. Additionally, sometimes, it is helpful to have the ability to restrict the precision of calculations to explore the effect of mathematical operations. In the paper *Chaos, Number Theory and Computers* [21] Adler, Kneusel, and Younger explored the effect of the precision used when calculating values for the logistic map, $x_{n+1} = Ax_n(1 - x_n)$, $A \leq 4$, $0 < x < 1$, which can lead to abnormal behavior. The software for this paper used the intrinsic big integer arithmetic found in the Scheme language to calculate logistic map values with a set precision.

What about rational arithmetic? As we've seen, adding rational arithmetic once big integers are available is straightforward. Many languages and libraries support rational arithmetic. For example, the `mpq` part of GMP implements rational functions and while Python supports big integers natively; Scheme supports rationals natively, including syntactically, so that `(+ 2/3 3/4)` is a perfectly valid function call yielding `17/12` as its answer.

5.8 Summary

In this chapter, we introduced the concept of a big integer, which means an integer larger than any native type supported by the computer. We implemented basic big integer operations in C and discussed improved algorithms for these basic operations, which are of more practical utility. Next, we looked at important implementations of big integer libraries and programming languages that intrinsically support big integers. We then examined rational numbers built from big integers and implemented a simple rational number class in Python. Lastly, we discussed some prominent uses of big integers to give examples of when big integers might be the tool of choice.

Exercises

5.1 Big integer libraries typically represent integers in *sign-magnitude* format as we did in this chapter. Another possibility would be to use a complement notation, calling it *B*'s-complement where *B* is the base, the analog of 2's-complement used almost exclusively for fixed-length integers. However, very few big integer packages do this. Why?

5.2 In Section 5.2, we gave a C function for `bigint_equal` to test if two big integers are equal. This function used `bigint_compare`. Write the remaining comparison functions for `<`, `>`, `≤`, `≥`, and `≠`. *

5.3 Figure 5.11 implements a function to perform big integer division. Extend this function to return the remainder as well. *

5.4 Add a new function to our library, `bigint_pow(a, b)`, which returns a^b . Be sure to handle signs and special cases. *

5.5 Add another new function to our library, `bigint_fact(a)`, which returns $a!$. Recall that the argument must be ≥ 0 . *

5.6 Using the big integer functions defined in this chapter, create a simple arbitrary precision desk calculator in C which takes as input expressions of the form “ $A \text{ op } B$ ”, as a string, where *A* and *B* are big integers and *op* is one of the four basic arithmetic operations and returns the result. If you wish, you may use postfix notation and a stack. In postfix notation, the operands come first, followed by the operation, “ $A \ B \text{ op}$ ”. You can use Python to check your answers. **

5.7 Extend the basic `Rational` Python class outlined above to compare rational numbers by overriding the `__lt__`, `__le__`, `__gt__`, `__ge__`, `__eq__`, and `__ne__` methods to implement comparison operators `<`, `≤`, `>`, `≥`, `==`, and `≠`. *

5.8 Extend the basic `Rational` Python class outlined above by adding `__neg__`, `__abs__` methods for negation and absolute value. *

5.9 Extend the basic `Rational` Python class outlined above by adding methods to handle shortcut operations. N.B., these, unlike all the others, destructively update the existing rational object. They do not return a new object. The method names are: `__iadd__`, `__isub__`, `__imul__`, and `__idiv__` corresponding to operators `+=`, `-=`, `*=`, and `/=`. Be careful when handling integer arguments, and know that the updated value may no longer be in the lowest terms. *

5.10 Use the `Rational` class to create a simple interactive rational desk calculator. This should take as input expressions of the form “*A op B*”, as a string, where *A* and *B* are rationals (with a “/” character) or integers and *op* is one of the four basic arithmetic operations and returns the result. If you wish, you may use postfix notation and a stack. **

References

1. Crandall R, Pomerance C (2005) Prime numbers: a computational perspective. Springer
2. Comba P (1990) Exponentiation cryptosystems on the IBM PC. IBM Syst J 29(4):526–536
3. Karatsuba A, Ofman Y (1962) Multiplication of many-digital numbers by automatic computers. Proc USSR Acad Sci 145:293–294. Translation in the Acad J Phys-Doklady 7:595–596 (1963)
4. Schönhage D, Strassen V (1971) Schnelle multiplikation grosser zahlen. Computing 7(3–4):281–292
5. Saracino D (1980) Abstract algebra: a first course. Addison-Wesley
6. Bracewell R (1986) The Fourier transform and its applications. McGraw-Hill, New York
7. Knuth D (2014) The art of computer programming, volume 2: seminumerical algorithms. Addison-Wesley Professional
8. Burnikel C, Ziegler J, Im Stadtwald D (1998) Fast recursive division
9. Granlund T (2014) gmp—GNU multiprecision library, version 6.0.0
10. Barrett P (2006) Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In: Advances in cryptology - CRYPTO’ 86. Lecture notes in computer science 263:311–323
11. Jebelean T (1993) An algorithm for exact division. J Symb Comput 15:169–180
12. Sussman G, Steele Jr G (1998) The first report on scheme revisited. Higher-Order Symb Comput 11(4):399–404
13. Abelson H, Sussman G (1996) Structure and interpretation of computer programs, 2nd edn. MIT Press
14. Kelsey R, Clinger W, Rees J (eds) (1998) Revised5 report on the algorithmic language scheme. Higher-Order Symb Comput 11(1)
15. Heath T (1956) The thirteen books of Euclid’s elements, 2nd ed. (Facsimile. Original Publication: Cambridge University Press, 1925). Dover Publications
16. Diffie W, Hellman M (1976) New directions in cryptography. IEEE Trans Inf Theory 22(6):644–654
17. Rivest R, Shamir A, Adleman L (1978) A method for obtaining digital signatures and public-key cryptosystems. Commun ACM 21(2):120–126

-
18. <http://www.openssl.org/>
 19. <http://www.gnu.org/software/libgcrypt/>
 20. The PARI Group, PARI/GP version 2.7.0, Bordeaux (2014). <http://pari.math.u-bordeaux.fr/>
 21. Adler C, Kneusel R, Younger B (2001) Chaos, number theory, and computers. *J Comput Phys* 166:165–172



Abstract

Fixed-point numbers implement floating-point operations using ordinary integers. This is accomplished through clever scaling, allowing systems without explicit floating-point hardware to work with floating-point values effectively. In this chapter, we explore how to define and store fixed-point numbers; how to perform signed arithmetic with fixed-point numbers; how to implement common trigonometric and transcendental functions using fixed-point numbers; and, lastly, discuss when one might wish to use fixed-point numbers in place of floating-point numbers, including an example of an emerging use case involving modern neural networks.

6.1 Representation (Q Notation)

Floating-point numbers are called “floating point” because the position of the decimal point “floats” as the exponent changes. A *fixed-point* number splits a collection of bits into two parts: an integer part on the left and a fractional part on the right. Between the two is an implied, fixed decimal point.

A collection of 8 bits may be interpreted in multiple ways. For example, each of the following is a valid interpretation of 8 bits:

00000000 ₂	an 8-bit integer, unsigned or two’s complement
000.00000 ₂	a fixed-point number with three integer bits and five fractional bits
00.000000 ₂	a fixed-point number with two integer bits and six fractional bits
00000.000 ₂	a fixed-point number with five integer bits and three fractional bits

Writing “a fixed-point number with three integer bits and five fractional bits” is quickly tiresome. Fortunately, *Q notation*, i.e., $Qm.n$, simplifies matters. In *Q notation*, m refers to the number of bits in the integer part, and n is the number in the fractional part. Unfortunately, there are two versions of *Q notation*: the original from Texas Instruments (TI) and another from ARM. The difference concerns including the sign bit in m (most fixed-point numbers are signed). The TI format does not, implying $Qm.n$ occupies $m + n + 1$ bits, while the ARM format does. We’ll use the TI format in this chapter [1].

According to TI, then, “a fixed-point number with three integer bits and five fractional bits” is denoted $Q2.5$ since $m + n + 1 = 2 + 5 + 1 = 8$. The integer part, using 3 bits, accommodates eight possible values interpreted as a two’s complement number:

bits	value	bits	value
000	0	100	-4
001	1	111	-1
010	2	110	-2
011	3	101	-3

Notice that the bit patterns on the right are the two’s complement of the pattern on the left (flip the bits and add one).

The fractional part is an n -bit number which we want to represent a fraction, $[0, 1)$. It is perhaps easiest to understand the fractional part as the numerator of a rational number with a denominator of 2^n . A fractional part of zero adds nothing to the integer part, while a fractional part of all ones (thinking of the bits) adds $2^n - 1/2^n = 1 - 1/2^n$.

Therefore, a $Qm.n$ fixed-point number is capable of representing real numbers in the range $[-2^m, 2^m - 2^{-n}]$ with a resolution of 2^{-n} . The resolution is the difference between any two adjacent fractional values. Increase n , and the resolution decreases, implying better fidelity to the real number we wish to represent. The resolution of a $Q2.5$ number is 2^{-5} or 0.03125, the smallest interval between any two $Q2.5$ fixed-point numbers.

Let’s consider another example. A $Q2.13$ number, which fits into a 16-bit integer, can represent numbers in $[-4, 3.999878]$ with a resolution of $2^{-13} \approx 0.000122$. A $Q7.8$ number also fits in a 16-bit integer with a $[-128, 127.9961]$ range. The $Q7.8$ number enables a larger numeric range, but there’s a price to pay; the resolution is only $2^{-8} \approx 0.0039$, 32 times greater. Figure 6.1 illustrates the difference in resolution. The lesson is clear: select a fixed-point representation with a range that is great enough to encompass the needed numbers. Anything else will reduce the precision of your results.

Now that we understand what a fixed-point number is and how to denote one, let’s begin piecing together a library of fixed-point functions in C. As we do so, any lingering confusion will disappear.

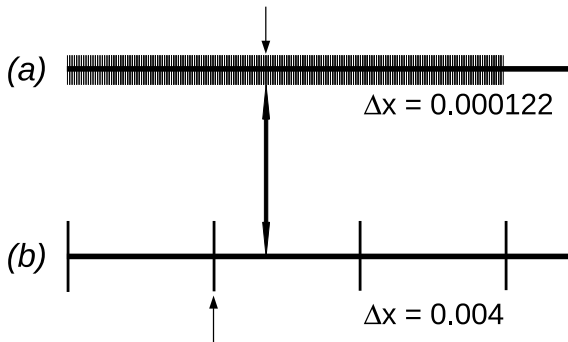


Fig. 6.1 The difference between numbers representable in (a) $Q2.13$ and (b) $Q7.8$. For $Q2.13$, each hash mark is 0.000122 apart while for $Q7.8$ the hash marks are 0.0039 apart. If we want to represent a floating-point value, indicated by the double arrow, we must select the closest hash mark in each subfigure (small arrows)

Existing fixed-point libraries are highly optimized to take full advantage of the performance found in small systems. To this end, they often implement more than one type of fixed-point number. For example, the open-source *libfixmath* project [2] implements only $Q15.16$ using signed 32-bit integers. This is a reasonable choice for a general-purpose library as it covers the range $[-32768, 32767.99998]$, which likely includes many of the values encountered in daily practice, especially in the embedded development world. The resolution is 2^{-16} or about 0.0000153, which is also quite reasonable. However, our library will allow a variable set of fixed-point numbers.

We restrict ourselves to signed 32-bit integers and initialize the library by specifying m explicitly with n coming along for the ride as $n = 32 - (m + 1)$. Library initialization will also calculate constants using the specified $Qm.n$. The constants are used to implement trigonometric and transcendental functions. Naturally, recalculating floating-point constants is a slight cheat if we use hardware that lacks floating point, but we can imagine calculating these ahead of time or a situation where the compiler uses slow, software-based floating point, and we wish to use faster fixed-point routines. The library is meant to be pedagogical, after all.

We begin with code to configure for a specified $Qm.n$ and to calculate necessary constants. Note that the code creates a new type, `fix_t`, to distinguish between ordinary integers and integers we mean to interpret as fixed-point numbers. There is also a `fixx_t` type using 64-bit integers for greater precision. The library initialization code is presented in Fig. 6.2.

Notice that `Q_init` accepts the number of integer bits in m . We check that m is reasonable in line 19, then set global `M` to it (line 9). Line 20 sets `N` so that we always use 32 bits for the entire number. Note that we are including two standard libraries (not listed). The *math.h* library is necessary for conversion between floating point and fixed point. To be explicit about integer types, we also include *stdint.h*. This latter

```

1  #define MASK_SIGN 0x80000000
2  #define DEG2RAD 0.0174532925
3  #define PI 3.141592653589793
4  #define EXP_ITER_MAX 50
5  #define EXP_TOLERANCE 0.000001
6
7  typedef int32_t fix_t;
8  typedef int64_t fixx_t;
9  int32_t M=0,N=31;
10 fix_t PIh, PId, d2r, r2d;
11 fix_t f90,f180,f270,f360;
12 fix_t st1,st2,st3,st4;
13 fix_t p0,p1,p2,p3,p4,p5;
14 fix_t sin_tbl[91]; // sine lookup table
15
16 void Q_init(int32_t m) {
17     int i;
18
19     M = ((m >= 0) && (m < 32)) ? m : 0;
20     N = 32 - (m+1);
21
22     for(i=0; i<91; i++)
23         sin_tbl[i] = Q_to_fixed(sin(i*DEG2RAD));
24
25     PId = Q_to_fixed(2*PI);
26     PIh = Q_to_fixed(PI/2);
27     d2r = Q_to_fixed(DEG2RAD);
28     r2d = Q_to_fixed(1.0/DEG2RAD);
29     f90 = Q_to_fixed(90*DEG2RAD);
30     f180= Q_to_fixed(180*DEG2RAD);
31     f270= Q_to_fixed(270*DEG2RAD);
32     f360= Q_to_fixed(360*DEG2RAD);
33     st1 = Q_to_fixed(1.0/6);
34     st2 = Q_to_fixed(1.0/120);
35     st3 = Q_to_fixed(1.0/5040);
36     st4 = Q_to_fixed(1.0/362880);
37     p0 = Q_to_fixed(1.17708643e-05);
38     p1 = Q_to_fixed(9.99655930e-01);
39     p2 = Q_to_fixed(2.19822760e-03);
40     p3 = Q_to_fixed(-1.72068031e-01);
41     p4 = Q_to_fixed(5.87493727e-03);
42     p5 = Q_to_fixed(5.79939713e-03);
43 }

```

Fig. 6.2 Initializing the fixed-point library

library defines the integer types in a consistent way so that `int32_t` is a signed `int` of 32 bits, while `uint32_t` is an unsigned `int` of 32 bits.

Several `#define` values are introduced. We'll use `MASK_SIGN` to mask values to access the sign of a number. Accessing the sign becomes handy when checking for overflow. `DEG2RAD` is a constant for converting an angle from degrees to radians. It's used in line 23 to build the sine lookup table. We define `PI` to be π . `EXP_ITER_MAX` and `EXP_TOLERANCE` will find use when implementing our exponential function.

The function `Q_init` assigns values to a small army of global constants using `Q_to_fixed`, which itself depends on `M` and `N`. The yet-to-be-described trigonometric and transcendental functions use the constants.

Specifically, `f90`, `f180`, `f270`, and `f360` are set to $\pi/2$, π , $3\pi/2$, and 2π radians. The fixed-point sine function uses these values. Variables `st1`, `st2`, `st3`, and `st4` are set to coefficients of the Taylor series expansion of $\sin x$. Lastly, `p0` through `p5` are the coefficients of the 5-th degree polynomial fit to $\sin x$. These values are determined empirically by curve fitting.

We'll add two convenience functions to the library for pedagogical purposes. The functions allow us to easily map a floating-point value (`C double`) to a fixed-point value and then back to a floating point. The functions also tell us how to find the $Q_{m,n}$ representation of a floating-point value by scaling it by the denominator, 2^n , and rounding to the nearest integer,

$$z_{fixed} = \lfloor 2^n z_{float} + 0.5 \rfloor$$

Similarly, to change a fixed-point number to a floating point, multiply by the reciprocal of the denominator,

$$z_{float} = 2^{-n} z_{fixed}$$

These functions are easy to implement in C,

```

1 | fix_t Q_to_fixed(double z) {
2 |     return (fix_t)floor(pow(2.0,N)*z + 0.5);
3 | }
4 |
5 | double Q_to_double(fix_t z) {
6 |     return (double)z * pow(2.0,-N);
7 | }
```

The `Q_init` function above makes frequent use of `Q_to_fixed`, as shall we during the experiments that follow later in the chapter.

Now, what about comparisons of fixed-point numbers (assuming the same $Q_{m,n}$)? We scaled by multiplying by a fixed power of two, implying normal integer comparisons still work as expected. In other words, fixed-point numbers compare as if they were ordinary integers. An example will illustrate. Consider an 8-bit signed integer representing fixed-point numbers as $Q_{3,4}$. To store 2.6875 in this format, we interpret each bit as follows:

$$\begin{array}{lcl}
 \text{bit value} \rightarrow & 0 & 0 \mid 1 \mid 0 \mid 1 \mid 0 \mid 1 \mid 1 \\
 \text{place value} \rightarrow & \text{sign} & 2^2 \mid 2^1 \mid 2^0 \mid 2^{-1} \mid 2^{-2} \mid 2^{-3} \mid 2^{-4}
 \end{array}$$

The stored integer is $2b_{16} = 43$. Now, store 2.3125 as a $Q3.4$ fixed-point number. Doing so gives us an integer with the value of 37. Clearly, $2.3125 < 2.6875$, and we also have $37 < 43$. If the Q notation is the same, the place value for each bit of the fixed-point number is the same, and we can compare bit by bit as is the case when interpreting the bits as two signed integers. Negative values are stored in two's complement format, so we again see that comparing two signed integers arrives at the same relationship as the two fixed-point values. We are now ready to implement basic fixed-point arithmetic.

6.2 Arithmetic with Fixed-Point Numbers

Assume x is a fixed-point representation of X , a floating-point number. In that case, $x = Xd$ where $d = 2^n$ is the scale factor in $Qm.n$, with rounding to the nearest integer. Similarly, for fixed-point number y , we have $y = Yd$. Therefore,

$$x + y = Xd + Yd = (X + Y)d$$

and

$$x - y = Xd - Yd = (X - Y)d$$

In other words, addition and subtraction of fixed-point numbers naturally result in a properly scaled answer. If the magnitude of $X + Y$ or $X - Y$ is too large to fit in m bits, the resulting number will overflow or underflow the representation and a possibly significant loss of precision will occur.

For multiplication, we have

$$xy = (Xd)(Yd) = XYd^2$$

where the answer is now scaled by d^2 instead of d , implying we must divide by d to return to a $Qm.n$ representation (remember that $d = 2^n$).

There are several things to remember when multiplying fixed-point numbers. First, multiplication may lead to an intermediate result requiring double the expected precision because the answer is scaled by d^2 instead of d . Second, a loss of fractional precision may likely happen when dividing by d to put the answer in $Qm.n$ form. This loss will be in the lowest bits of the fractional part.

We selected scaled factors of the form 2^n , powers of two. Here's where we benefit from this choice: dividing the result of a multiplication becomes nothing more than shifting the answer 2^n bits to the right.

However, if the integer part of the product has overflowed, meaning the result has an integer part that does not fit within m bits, there will be a catastrophic loss of precision. Therefore, it is critical to check for integer part overflow when working with the double-precision fixed-point product.

There is one other detail yet to mention. Dividing the double-precision product by the scale factor is a shift to the right of a specified number of bits. If we do this shift without concern rounding, we will introduce bias into our results. To avoid this, we take the extra step of adding a value equal to 2^{n-1} , which is the highest bit position that will be discarded by the right shift of n bits. Doing so is tantamount to adding 0.5 to a floating-point value before applying the *floor* operation to round to the nearest integer. Properly accounting for such rounding increases the precision in the long run by causing errors to average out over a series of calculations.

Therefore, the algorithm for the multiplication of two $Qm.n$ fixed-point numbers, x and y , stored in a b -bit integer is

1. Cast x to x' , a sign-extended $2b$ -bit integer. Similarly, cast y to y' , also a sign-extended $2b$ -bit integer.
2. Calculate $t' = x' \times y'$, a result scaled by $(2^n)(2^n) = 2^{2n}$ and stored in a $2b$ -bit integer.
3. Calculate $t' = t' + 2^{n-1}$ to minimize truncation bias.
4. Calculate $t' = t' \times 2^{-n}$, a right shift by n bits.
5. Cast t' to t keeping the lowest $m + n + 1$ bits of t' . This is the product in $Qm.n$ format. If any bits of t' above bits $m + n + 1$ are set, overflow has happened.

To divide two $Qm.n$ numbers,

1. Cast x to $x' = x \times 2^n$ where x' is a $2b$ -bit integer. Doing so scales x scaled by 2^{2n} .
2. Correct for truncation bias by adding $y/2$ to x' : $x' = x' + y/2$. Note that $y/2$ is also a $2b$ -bit integer. This addition adds $\frac{Y \times 2^n}{2} = Y \times 2^{n-1}$ to x' . We'll learn why this step is helpful below.
3. Calculate a quotient, $q' = x'/y'$ where y' is y properly sign-extended to $2b$ -bits. The initial scaling of x ensures the quotient is properly scaled, so we keep the lowest b bits to return the $Qm.n$ answer.

Let's look more closely at this algorithm. In the end, we want a number that is $(X/Y) \times 2^n$, the $Qm.n$ representation of the quotient, X/Y . By scaling x in step 1, we have $x' = x \times 2^n = X \times 2^{2n}$. The bias correction in step 2 seems particularly mysterious but we see that it adds $y/2 = (Y \times 2^n)/2 = Y \times 2^{n-1}$, meaning the division in step 3 is

$$x'/y = \frac{X \times 2^{2n} + Y \times 2^{n-1}}{Y \times 2^n} = (X/Y) \times 2^n + \frac{1}{2}$$

which is exactly the quotient we want correctly scaled to $Qm.n$ plus the bias correction of $1/2$.

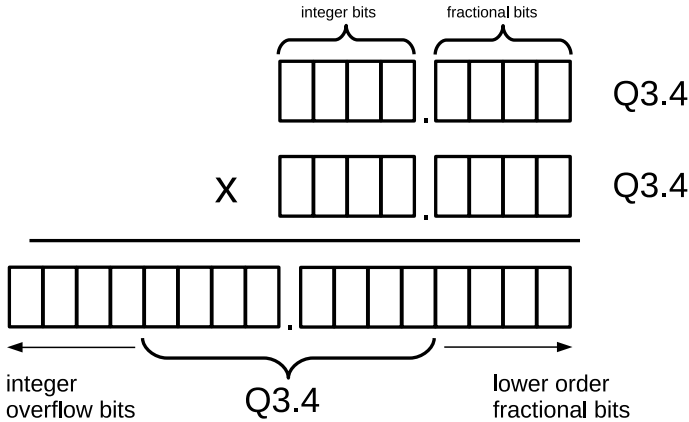


Fig. 6.3 Multiplication of two $Q3.4$ fixed-point numbers. The radix point separates the fractional bits from the integer bits (including the sign). The product requires 16 bits, of which the middle eight are the desired product. Fractional bits below this range are of lower order and are discarded when the product is shifted down $n = 4$ bits. If any bits above the indicated range are set, and not the extended sign of the answer, then overflow has happened

Let's add the arithmetic operations to our fixed-point library. We'll implement two versions of addition, subtraction, and multiplication, one that checks for overflow and a faster one that does not. We begin with the faster versions,

```

1 | fix_t Q_add(fix_t x, fix_t y) { return x+y; }
2 | fix_t Q_sub(fix_t x, fix_t y) { return x-y; }
3 |
4 | fix_t Q_mul(fix_t x, fix_t y) {
5 |     fixx_t ans = (fixx_t)x * (fixx_t)y;
6 |
7 |     ans += 1<<(N-1);
8 |     return (fix_t)(ans >> N);
9 | }
```

Addition and subtraction are as simple as can be.

Multiplication is nearly as straightforward. First, multiply the 32-bit fixed-point numbers, storing them in 64-bit `ans`. To complete the multiplication, we must add the bias correction (line 7) and then shift right before returning the product (line 8).

Pictorially, multiplication looks like Fig. 6.3, where we see the result of multiplying two 8-bit $Q3.4$ values using a 16-bit intermediate. The figure shows visually why the product must be shifted to the right to remove the extraneous lower order bits.

The caption of Fig. 6.3 refers to overflow. Let's add overflow checks to our library routines, beginning with addition and subtraction,

```
1  fix_t Q_add_over(fix_t x, fix_t y) {
2      uint32_t ux = x;
3      uint32_t uy = y;
4      uint32_t s = ux + uy;
5
6      if ((ux & MASK_SIGN) == (uy & MASK_SIGN)) {
7          if ((s & MASK_SIGN) != (ux & MASK_SIGN)) {
8              printf("Q_add_over: overflow!\n");
9              return 0;
10         }
11     }
12     return x+y;
13 }
14
15 fix_t Q_sub_over(fix_t x, fix_t y) {
16     uint32_t ux = x;
17     uint32_t uy = y;
18     uint32_t s = ux - uy;
19
20     if ((ux & MASK_SIGN) != (uy & MASK_SIGN)) {
21         if ((s & MASK_SIGN) != (ux & MASK_SIGN)) {
22             printf("Q_sub_over: overflow!\n");
23             return 0;
24         }
25     }
26     return x-y;
27 }
28
```

Now, before returning the result, we use unsigned 32-bit integers to examine the sign of the result. Why unsigned integers? Because overflow is undefined in the C standard for signed integers. This tip we borrow from the *libfixmath* project.

For addition, we know overflow happened if the signs of the arguments are the same, but the sign of the answer does not match the arguments. In line 6, we mask the unsigned representation, the same bit pattern as the signed representation, keeping only the highest bit, which we know reflects the number's sign. If these are the same for both arguments, we then check whether the sign of the result differs from the sign of the first argument (line 7). If it does, we know overflow happened. We report it in line 8 and return zero in line 9. Naturally, a complete implementation would handle the overflow error more gracefully.

The situation is similar for subtraction, except instead of checking for the same sign in the arguments, line 20 checks to see if the signs differ. If so, we check to see if the sign of the answer is the same as the sign of the first argument. If not, we know overflow has happened and report it in lines 22 and 23. Otherwise, return the difference in line 26 knowing that the two's complement representation will handle the signs correctly.

Multiplication with overflow detection becomes

```

1  fix_t Q_mul_over(fix_t x, fix_t y) {
2      fixx_t xx,yy,tt;
3      uint32_t ux = x;
4      uint32_t uy = y;
5
6      xx = (fixx_t)x;
7      yy = (fixx_t)y;
8      tt = xx * yy;
9      tt += 1<<(N-1);
10     tt >>= N;
11
12     if ((ux & MASK_SIGN) == (uy & MASK_SIGN)) {
13         if (((fix_t)tt & MASK_SIGN) != (ux & MASK_SIGN)) {
14             printf("Q_mul_over: overflow!\n");
15             return 0;
16         }
17     }
18     return (fix_t)tt;
19 }
```

where we multiply as before, but check our result starting at line 12. If the signs of the arguments are the same (line 12), but the signs of the result and first argument are different (line 13), we know overflow happened and report (lines 14 and 15). If not, return the product in line 18.

All that remains is division. Let's implement it now,

```

1  fix_t Q_div(fix_t x, fix_t y) {
2      fixx_t xx,yy;
3
4      xx = (fixx_t)x << N;
5      xx += (fixx_t)y / 2;
6      return xx / (fixx_t)y;
7  }
```

`Q_div` implements the division algorithm. We cast the dividend to a 64-bit integer and then scale it by N bits (line 4). Doing so sets `xx` to $x \times 2^N$ using 64 bits for the result. Line 5 implements the truncation bias correction term by adding a 64-bit version of `y` divided by 2. The division is in line 6, which returns a properly scaled quotient.

We've discussed the truncation bias correction term several times in this section. Let's consider its effect on repeated multiplications. To do this, we'll explore powers of $p = \pi - 3$ using `Q2.29`, which is capable of accurately representing value to nine decimal places. First, we'll calculate p^{10} using floating point, then p^{10} with truncation bias correction, and finally without to show the effect of a series of ten multiplications in a row,

$$\begin{aligned}
(\pi - 3)^{10} &= 0.00000000323897 && \text{floating-point value} \\
(\pi - 3)^{10} &= 0.00000000372529 && Q2.29, \text{ truncation bias correction} \\
(\pi - 3)^{10} &= 0.00000000186265 && Q2.29, \text{ no bias correction}
\end{aligned}$$

Notice how the truncation bias correction term keeps the final value reasonably close to the actual value, while without it, the error accumulates, and a severe loss of precision occurs. Truncation bias correction is an essential component of fixed-point arithmetic.

6.3 Trigonometric and Other Functions

Let's enhance our fixed-point library by adding trigonometric and transcendental functions (e.g., log and exp). We have several options when it comes to implementing these functions. We can use a lookup table, which is very fast but potentially inaccurate because the table is too small or too memory-hungry if accurate. We might approximate values with a polynomial. Lastly, we might sum terms of a Taylor series expansion. We'll examine each of these approaches for the sine function. Sine in hand, we get the cosine easily enough and, from them, the tangent. We'll ignore the inverse trig functions to keep things simple. The section closes with fixed-point square root, logarithm, and exponential functions.

Trigonometric functions. We want sine, cosine, and tangent, but we only really need sine. The other two can be defined in terms of sine, which, as mentioned, we'll implement in three different ways to compare each method's performance and accuracy. The cosine is easy to derive from the sine since

$$\cos \theta = \sin \left(\frac{\pi}{2} - \theta \right)$$

and, by definition,

$$\tan \theta = \frac{\sin \theta}{\cos \theta}$$

meaning we get everything we want once we have the sine. Let's implement the sine using a table, a Taylor series expansion, and a polynomial beginning with the table,

```

1 fix_t Q_sin_table(fix_t x) {
2     int32_t sign;
3     fix_t angle;
4
5     if (x < 0) x += PID;
6     sin_val_sign(&x, &sign);
7
8     angle = Q_mul(x, r2d);
9     angle += 1 << (N-1);
10    return sign*sin_tbl[angle >> N];
11 }

```

`Q_sin_table` uses `sin_tbl`, the array defined when `Q_init` is called. The function also uses a helper, `sin_val_sign`, which we describe below. All of our sine implementations use this function.

The function `sin_val_sign` accepts an angle in radians and updates it to be in the range $[0, \pi/2]$ along with a sign. This is equivalent to paying attention to which quadrant the angle puts us in and then using the appropriate angle and sign for that quadrant to give us the proper sine value. Recall that `Q_init` initializes `sin_tbl` with the sine for each degree from $[0, 90]$, meaning need to only consider input angles in that range.

If the argument to `Q_sin_table` is negative, add 2π to make it positive, implying the input to the function must be in the range $[-2\pi, 2\pi]$. This is line 5. Note that we use standard addition, thereby making an implicit assumption that we will not overflow (nor do we check for it). In line 6, we call `sin_val_sign` to adjust our input angle to be in the range $[0, \pi/2]$ and set `sign` accordingly. In line 8, we convert our adjusted input angle, assumed to be in radians, to degrees because we built the table by degrees and want to index it by degree. Line 9 adds 0.5 to the angle, a bias correction to mapping angle to the nearest degree.

In line 10, we truncate `angle`, dropping the fractional part by shifting it down N bits making `angle >> N` an integer in the range $[0, 90]$ degrees. With this integer, we index the sine table and, after multiplying by `sign`, return the final answer.

The `Q_sin_table` function is fast but has the overhead of building the table in the first place on each call to `Q_init`. Additionally, while most accurate for inputs with no fractional component, the function is not as precise as it could be for arbitrary inputs. The exercises at the end of the chapter challenge the reader to increase the accuracy by linear interpolation.

It's now time to examine `sin_val_sign`,

```

1 void sin_val_sign(int32_t *angle, int32_t *sign) {
2
3     if ((*angle >= 0) && (*angle <= f90)) {
4         // quadrant I
5         *sign = 1;
6     } else {
7         if ((*angle > f90) && (*angle <= f180)) {
8             // quadrant II
9             *sign = 1;
10            *angle = f180-*angle;
11        } else {
12            if ((*angle > f180) && (*angle <= f270)) {
13                // quadrant III
14                *sign = -1;
15                *angle = *angle-f180;
16            } else {
17                // quadrant IV
18                *sign = -1;
19                *angle = f360-*angle;
20            }
21        }
22    }
23 }

```

The function accepts as input via pointers the current angle and a sign value it will set. By comparing with the constants `f90`, `f180`, `f270`, and `f360`, each of which is set in `Q_init`, the function determines which quadrant angle falls in. It then sets `sign` appropriately for that quadrant and updates the value of `angle` so that the sine of the new angle, times `sign`, will be the proper sine for the original angle, meaning we only work with angles in the range $[0, \pi/2]$.

`Q_cos_table` and `Q_tan_table` come virtually for free once we have `Q_sin_table`,

```

1 fix_t Q_cos_table(fix_t x) {
2     return Q_sin_table(Q_sub(PIh,x));
3 }
4
5 fix_t Q_tan_table(fix_t x) {
6     return Q_div(Q_sin_table(x), Q_cos_table(x));
7 }

```

Note the use of the constant `PIh` for $\pi/2$. `PIh` is set in `Q_init` for the given *Qm.n* fixed-point representation.

Now, let's implement the sine using a Taylor series expansion. Please see any calculus book for a description of a Taylor series. Here, we simply use the well-known series expansion for $\sin x$,

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} + \cdots$$

after mapping the input angle to $[0, \pi/2]$ as before. In code, this becomes

```

1 fix_t Q_sin_taylor(fix_t x) {
2     fix_t x3,x5,x7,x9;
3     fix_t ans;
4     int32_t sign;
5
6     if (x < 0) x += PId;
7     sin_val_sign(&x, &sign);
8
9     x3 = Q_mul(Q_mul(x,x),x);
10    x5 = Q_mul(Q_mul(x3,x),x);
11    x7 = Q_mul(Q_mul(x5,x),x);
12    x9 = Q_mul(Q_mul(x7,x),x);
13
14    ans = x;
15    if (x3 != 0) ans -= Q_mul(st1,x3);
16    if (x5 != 0) ans += Q_mul(st2,x5);
17    if (x7 != 0) ans += Q_mul(st3,x7);
18    if (x9 != 0) ans += Q_mul(st4,x9);
19
20    return sign*ans;
21 }
```

Lines 6 and 7 check for a negative input, adding 2π if it is, and mapping the argument to $[0, \pi/2]$ while setting *sign*. Lines 9 through 12 calculate powers of the input, x . The series is implemented directly for a specific number of terms, so the powers are calculated in advance. If we did not include the truncation bias correction factor in *Q_mul*, it is unlikely this function would be of any value given so many multiplications. A little algebraic manipulation will show that we only need x and x^2 plus some well-placed parentheses. See the exercises for another challenge to the reader.

Line 14 sets *ans* to the first term in the series, while lines 15 through 18 add subsequent terms if that power of x is not zero, which may happen if $x < 1$ and we do not have enough precision in the n fractional bits of the $Q_{m.n}$ number to represent the power. The magic constants *st1*, *st2*, *st3*, and *st4* are the reciprocals of the factorials in the series representation and are calculated in advance for the given Q representation when *Q_init* is called. Finally, line 20 multiplies by the sign and returns the final estimate for $\sin x$. As before, we get $\cos x$ and $\tan x$ for free,

```

1 fix_t Q_cos_taylor(fix_t x) {
2     return Q_sin_taylor(Q_sub(PId,x));
3 }
4
5 fix_t Q_tan_taylor(fix_t x) {
6     return Q_div(Q_sin_taylor(x), Q_cos_taylor(x));
7 }
```

where the form of the functions matches the table versions above, but based on `Q_sin_taylor`.

Our final $\sin x$ implementation comes from fitting point calculated from a floating-point version of the sine to a 5th-degree polynomial.

Specifically, $\sin x$ was calculated for 100 equally spaced points in $[0, \pi/2]$ using the C library version of the function. Then, a 5th-degree polynomial was fit to these points to find polynomial coefficients p_0, p_1, p_2, p_3, p_4 , and p_5 which minimize the squared difference between $\sin x$ and $f(x)$ where

$$f(x) = p_0 + p_1x + p_2x^2 + p_3x^3 + p_4x^4 + p_5x^5$$

This is standard least-squares curve fitting via the Python NumPy package [3]; see the NumPy *polyfit* function.

The coefficients are mapped to the current *Q* notation in `Q_init` and set to the variables `p0, p1, p2, p3, p4`, and `p5`, respectively. With these parameters, we can approximate the sine in the range $[0, \pi/2]$ like so

```

1 fix_t Q_sin_poly(fix_t x) {
2     fix_t x2,x3,x4,x5;
3     fix_t ans;
4     int32_t sign;
5
6     if (x < 0) x += PID;
7     sin_val_sign(&x, &sign);
8
9     x2 = Q_mul(x,x);
10    x3 = Q_mul(x2,x);
11    x4 = Q_mul(x3,x);
12    x5 = Q_mul(x4,x);
13
14    ans = p0 + Q_mul(p1,x) + Q_mul(p2,x2) + Q_mul(p3,x3) +
15          Q_mul(p4,x4) + Q_mul(p5,x5);
16    return sign*ans;
17 }
```

whereas before, lines 6 and 7 adjust for negative arguments and map x to $[0, \pi/2]$ while setting `sign` appropriately. Lines 9 through 12 calculate powers of x for the polynomial. Lines 14 and 15 directly implement the polynomial using `Q_mul` to multiply with a proper intermediate number of bits and ordinary addition, thereby ignoring any potential overflow. Lastly, line 16 multiplies by `sign` and returns the answer. As before, we can write $\cos x$ and $\tan x$ using the polynomial sine,

```

1 fix_t Q_cos_poly(fix_t x) {
2     return Q_sin_poly(Q_sub(PTh,x));
3 }
4
5 fix_t Q_tan_poly(fix_t x) {
6     return Q_div(Q_sin_poly(x), Q_cos_poly(x));
7 }

```

Comparison of trigonometric functions. All three sets of trigonometric functions have a version of sine at their core. Therefore, we can compare them by looking at the accuracy of the sine functions. If we generate the sine, using the table, series expansion, and polynomial fit, for 100 randomly selected angles in $[0, 2\pi)$, we can compare the resulting values to the actual floating-point sine. Any deviation can be used to assess the quality of the sine approximation.

We generate the table with code like this

```

1 Q_init(7);
2 srand(time(NULL));
3
4 for(i=0; i<100; i++) {
5     f = 2*3.14159265*((double)rand()/RAND_MAX);
6     a = Q_to_fixed(f);
7     printf("%0.8f %0.8f %0.8f %0.8f %0.8f\n", f, sin(f),
8         Q_to_double(Q_sin_table(a)),
8         Q_to_double(Q_sin_taylor(a)),
9         Q_to_double(Q_sin_poly(a)));
10 }

```

where we include the `stdlib.h` C library and use `srand` to seed the random number generator (line 5) to set `f` to a random value in the range $[0, 2\pi)$. It is well known that `rand` is a poor pseudorandom number generator and that we have a very (very) small chance of selecting 2π as `rand` returns a number from $[0, RAND_MAX]$ and we are dividing by `RAND_MAX`. However, for our purposes, we accept the inefficiency and take the risk. The reader is encouraged, however, to develop an understanding of pseudorandom number generation, which is fascinating, by referring to one of the many good texts on the subject, for example, my *Random Numbers and Computers* (Springer) or Gentle's *Random Number Generation and Monte Carlo Methods* [4].

The code above produces a table of sine values from which we can calculate the delta between the floating-point value and the three approximations to report the absolute maximum deviation ($|\Delta|$), the mean ($\bar{x}$), and the standard deviation (σ). These give us a feel for how the functions perform over their entire range. The results of this analysis for $Q7.24$ numbers are in Table 6.1.

We see that while all three approximations are adequate with relatively small errors, on average, it is clear that `Q_sin_table` is the least accurate, as expected, because it forces the argument to the nearest degree, and that `Q_sin_poly` is to be preferred over `Q_sin_taylor` in terms of accuracy. If we count the number of

Table 6.1 Comparison of the difference between the floating-point sine and each of the fixed-point sine functions using summary statistics for 100 randomly selected angles over the entire range $[0, 2\pi)$

Function	$ \Delta $	$\bar{x}$	σ
Q_sin_table	8.4×10^{-3}	-6.5×10^{-5}	3.8×10^{-3}
Q_sin_taylor	8.2×10^{-3}	-7.8×10^{-6}	8.2×10^{-3}
Q_sin_poly	7.7×10^{-6}	-1.1×10^{-7}	3.9×10^{-6}

multiplications we see that `Q_sin_poly` is also more efficient than `Q_sin_taylor` with nine multiplications to as many as twelve.

Square root and transcendental functions. Let's add square root, natural logarithm, and exponential functions to our library, beginning with the square root.

Newton's method is perhaps the fastest way to implement a fixed-point square root function. It is an iterative method to find the roots of a function by solving $f(x) = 0$. The method selects an initial guess for the root, x_0 , and then iterates

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

until convergence where $f'(x)$ is the first derivative of $f(x)$. In this case, we want $f(x) = x^2 - n = 0$ to locate the value of x which, when squared, equals a specific number n . In other words, x will be the square root of n . If we work through the steps we have

$$\begin{aligned}
 f(x) &= x^2 - n \\
 f'(x) &= 2x \\
 x_{i+1} &= x_i - \frac{f(x_i)}{f'(x_i)} \\
 &= x_i - \frac{x_i^2 - n}{2x_i} \\
 &= x_i - \frac{1}{2} \left(x_i - \frac{n}{x_i} \right) \\
 &= \frac{1}{2} \left(x_i + \frac{n}{x_i} \right)
 \end{aligned}$$

This implies that we need only an initial guess followed by some number of iterations of the final equation above to generate a good approximation of the square root of n . It is known that this method converges quickly, so a handful of iterations will suffice.

Translating all of this into code gives

```

1 | fix_t Q_sqrt(fix_t n) {
2 |     fix_t a;
3 |     int32_t k;
4 |
5 |     if (n <= 0) return -(1<<N);
6 |     if (n == (1<<N)) return (1<<N);
7 |
8 |     a = n >> 1;
9 |
10 |    for(k=0; k < 10; k++)
11 |        a = Q_mul(1 << (N-1), Q_add(Q_div(n,a), a));
12 |
13 |    return a;
14 |}

```

We check for a negative argument and return -1 if one is found (line 5). Recall that our fixed-point numbers are binary fractions scaled by N bits, so shifting 1 up by N bits gives us a value equal to one in our current Q notation. Line 6 is a quick check to see if our argument is one. If so, we immediately return the square root (1). Line 8 is our initial guess, which we store in a . We are looking for a square root, so our initial guess is $n/2$, which is exactly what we get by shifting the argument 1-bit position to the right. The loop in line 10 performs the actual iteration. We fix the number of iterations at ten, though a more sophisticated approach would be sensitive to the number of bits in the fractional part of our numbers and adjust the iteration limit accordingly. Line 11 is a direct translation of our final equation above using our previously defined arithmetic functions. Note that for simplicity, we are not checking for any overflow conditions. Also note the use of $1 \ll (N-1)$ which is 0.5 for N bits in the fractional part. Finally, line 13 returns a , the approximate square root of n .

A random analysis similar to the one performed above for the sine functions lets us compare `Q_sqrt` to the floating-point `sqrt` function for 100 randomly selected values in $[1, 1000)$ using $Q_{10.21}$ format numbers,

$$\frac{|\Delta|}{3.6 \times 10^{-7}} \quad \bar{x} \quad \sigma \quad \frac{\sigma}{-1.3 \times 10^{-7} \quad 1.4 \times 10^{-7}}$$

Indicating that this approach returns accurate results.

The exponential function, e^x , is next. We'll need it to implement the natural logarithm efficiently. The Taylor series expansion of e^x is particularly well suited to implementation on a computer,

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots = \sum_{i=0}^{\infty} \frac{x^i}{i!}$$

Examining the series reveals that if the current term in the series is t_i , then the next term in the series is $t_{i+1} = t_i(\frac{x}{i})$, which means that we need not calculate the factorials completely but will get them by multiplying the current term by $\frac{x}{i}$

and adding it to a running total. If we accumulate terms until we locate a term that is below a given threshold, say too small to add anything meaningful to the total given the number of bits used for the fractional part of the fixed-point number, we have a particularly elegant implementation. Let's implement the series, then, terms, 0.000001 or the like, and adding a maximum number of iterations (`EXP_ITER_MAX = 50`) gives the following:

```

1 | fix_t Q_exp(fix_t x) {
2 |     int32_t i;
3 |     fix_t e, c, t, tol;
4 |
5 |     tol = Q_to_fixed(EXP_TOLERANCE);
6 |     i = 0;
7 |
8 |     c = 1 << N;
9 |     e = 1 << N;
10 |    t = x;
11 |
12 |    do {
13 |        e = Q_add(e,t);
14 |        c = Q_add(c, 1<<N);
15 |        t = Q_mul(t, Q_div(x,c));
16 |        i++;
17 |    } while ((abs(t) > tol) && (i < EXP_ITER_MAX));
18 |
19 |    return e;
20 |}
```

The code uses two constants: `EXP_TOLERANCE` and `EXP_ITER_MAX`. The former is a tolerance value below which additional terms are considered too small to add meaningfully to the series sum. The latter is a hard limit on the maximum number of terms we're willing to calculate. The tolerance is 0.000001, and the maximum number of iterations is 50.

In detail, line 5 sets the tolerance, and line 6 initializes the iteration counter. Line 8 sets `c` to one. This is the term counter and is set to one because the first term in the series (1) is used to initialize the accumulator, `e` in line 9. The current term is then the second term in the series, $x = x^1/1!$, and we initialize `t` with it in line 10.

The loop in lines 12 through 17 adds new terms to the accumulator (line 13) while bumping the term counter (line 14). Line 15 calculates the next term in the series from the previous and sets `t` accordingly. Line 16 increments the iteration counter, while line 17 checks to see if we can exit the loop by either calculating a term below the tolerance or by reaching the limit on iterations. At this point, `e` contains the sum of the series, which has converged on e^x , and it is returned in line 19.

Repeating the random value test for 100 random arguments in the range $[-4, 4)$ using $Q10.21$ format numbers gives

$$\frac{|\Delta|}{2.0 \times 10^{-5}} \quad \bar{x} \quad -9.2 \times 10^{-7} \quad \sigma \quad 4.5 \times 10^{-6}$$

showing that Q_exp works reasonably well.

The implementation of e^x enables adding our final library function, the natural log ($\log x$), which we also implement using Newton's method. In this case, the function is $f(x) = e^x - c = 0$, which is zero when x is the log of c . The first derivative of this function is $f'(x) = e^x$, meaning we iterate

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} = x_i - \frac{e^{x_i} - c}{e^{x_i}} = x_i - 1 + ce^{-x_i}$$

where we again set our initial guess to $x_0 = c/2$. Translating to C code gives

```

1 | fix_t Q_log(fix_t x) {
2 |     fix_t a;
3 |     int32_t k;
4 |
5 |     if (x <= 0) return -(1<<N);
6 |     if (x == (1<<N)) return 0;
7 |
8 |     a = x >> 1;
9 |
10 |    for(k=0; k < 7; k++) {
11 |        a = Q_add(a, Q_mul(x, Q_exp(-a)));
12 |        a = Q_sub(a, 1<<N);
13 |    }
14 |
15 |    return a;
16 |}
```

A sanity check is made in line 5 to disallow negative arguments (return -1). A quick check for a known value ($\log 1 = 0$) is in line 6. Line 8 sets a to our initial guess of $c/2$. Lines 10 through 13 are the iteration. Line 11 calls Q_exp as part of the update to a . Line 12 subtracts one from the sum in line 11. The number of iterations is hard-coded to seven, which gives good results in practice. A more sophisticated approach would be aware of the number of bits in the fractional part of our fixed-point numbers and set the loop limit accordingly. Line 15 returns a as the approximation of the natural log.

A final repeat of the random value test with 100 random arguments from $[0.5, 10)$ gives

$$\frac{|\Delta|}{1.3 \times 10^{-5}} \quad \bar{x} \quad \frac{\sigma}{4.4 \times 10^{-6}}$$

Again, this shows good agreement with the floating-point C library function. Our C library of fixed-point functions is now complete.

6.4 When to Use Fixed-Point Numbers

Perhaps the most obvious time to use fixed-point numbers is when working on a system that does not have floating-point capability in hardware. This includes many small systems, like microcontrollers and digital signal processors (DSP), and older computers lacking floating-point hardware. Using fixed-point numbers on these systems makes sense and is much faster than software emulating floating-point hardware. For example, older Macintosh computers based on the Motorola 68000 and later processors often used software to emulate floating-point hardware. SANE (Standard Apple Numerics Environment) implemented IEEE 754 floating-point arithmetic in software for early Macintosh computers that lacked the floating-point hardware found in the Motorola 68040 or did not use the 68881 floating-point coprocessor. The SANE library provided maximum numeric compatibility, ensuring that floating-point operations on one computer matched those of another that used hardware instead of software for the calculations. However, the simplicity of fixed-point arithmetic would have significantly improved performance on these early systems.

In 1993, Id Software released DOOM [5], one of the earliest first-person shooter (FPS) video games. FPS games give the user the illusion of being in a three-dimensional environment and, as such, require a lot of geometric graphics processing at high speed to enable realistic (limited for DOOM, granted) rendering of the environment. However, in 1993 only Intel 80486DX microprocessors had built-in hardware floating-point abilities. As processor speeds were very low compared to current standards, only tens of megahertz, calculations needed to be as fast as possible. Id's solution was to use fixed-point arithmetic for all graphics calculations. Specifically, they used $Q15.16$ format numbers which fit in a 32-bit signed integer.

Another reason to consider fixed-point math is power consumption. A processor's floating-point unit uses more power than the integer unit; hence, fixed-point arithmetic will require less power than the same calculations performed with hardware floating point.

Lastly, many popular audio and video codecs are designed to be implemented using fixed-point arithmetic so that simple devices lacking floating-point hardware may be used. For example, the open-source Shine project [6] is a fast fixed-point MP3 encoder suitable for simple computers.

6.5 Summary

This chapter introduced fixed-point numbers and the associated Q notation. Fixed-point numbers interpret an integer as a real number with a fixed number of fractional bits, i.e., a rational number. We learned how to convert between floating-point and fixed-point numbers.

We implemented a small C fixed-point library using signed 32-bit integers. Next, we developed trigonometric functions in three different ways and compared them for accuracy. We then added square root, natural logarithm, and exponential functions to round out the library. Lastly, we discussed when to use fixed-point numbers.

Exercises

6.1 The function `Q_sin_table` rounds its argument to the nearest degree and then returns the sine stored in the precomputed table for that degree. This can be made more accurate by linearly interpolating between the degrees that bound the input angle. For example, if the input value, as a fixed-point number in some $Qm.n$ representation, is equivalent to 45.4 degrees then `Q_sin_table` will round down to 45 degrees and return 0.70710678 as the sine. The actual sine of 45.4 degrees is 0.71202605, so the returned value underestimates the true value.

In linear interpolation, the values bounding the input, in this case $45 < 45.4 < 46$ degrees, are used to determine the line between the values, $y = mx + b$, with $m = (y_2 - y_1)/(x_2 - x_1)$ and $b = y_1 - mx_1$ where $x_1 = 45$, $x_2 = 46$, $y_1 = \sin(x_1)$, and $y_2 = \sin(x_2)$. Then, $\sin(45.4)$ is equal to $m(45.4) + b$.

In this case, then, the improved answer is $\sin(45.4) = m(45.4) + b = 0.71199999$ with $m = 0.01223302$ and $b = 0.15662088$ for a difference on the order of 2.6×10^{-5} compared with the true result. This is better than the difference of 4.9×10^{-3} by simply rounding to the nearest degree. Add linear interpolation to `Q_sin_table`. **

6.2 Add a function, `Q_ipow`, that takes two arguments. The first is a fixed-point number, x , and the second is an integer, n , which may be negative. The return value of the function is x^n . *

6.3 Add a function, `Q_pow`, that takes two arguments. The first is a fixed-point number, x , and the second is also a fixed-point number, y . The return value of the function is x^y . (Hint: $x^y = e^{y \log(x)}$) *

6.4 Add a function, `Q_asin`, that takes a sine value, positive or negative, and returns the angle for that sign, in radians. Use a Taylor series expansion,

$$\sin^{-1} x = x + \left(\frac{1}{2}\right)\left(\frac{x^3}{3}\right) + \left(\frac{1}{2}\right)\left(\frac{3}{4}\right)\left(\frac{x^5}{5}\right) + \left(\frac{1}{2}\right)\left(\frac{3}{4}\right)\left(\frac{5}{6}\right)\left(\frac{x^7}{7}\right) + \dots$$

where you may need terms to x^9 or higher. (Hint: The coefficient of term t_{i+1} can be created from t_i) **

6.5 Create a Python class, `Fixed`, that duplicates the arithmetic functionality of the C fixed-point library. Be sure to overload the operators for $+$, $-$, $/$, $*$ so that the fixed-point class can be used in expressions. **

6.6 The function `Q_sin_taylor` uses terms out to x^9 and calculates them explicitly for input x . Show via algebraic manipulation that the same truncated series sum can be found using only x and x^2 . Then, update `Q_sin_taylor` to use this form. Note that you must introduce new constants in place of the values used for `st1` through `st4`. Update `Q_init` to set these new values. **

References

1. TMS320C64x DSP Library Programmer's Reference Literature Number: SPRU565B, Texas Instruments, October 2003
2. <https://code.google.com/p/libfixmath/>
3. <http://www.numpy.org/>
4. Gentle JE (2003) Random number generation and Monte Carlo methods. Springer
5. Doom. Id Software (1993) Video game
6. <https://github.com/savonet/shine>
7. Courbariaux M, David J-P, Bengio Y (2014) Training deep neural networks with low precision multiplications. arXiv preprint [arXiv:1412.7024](https://arxiv.org/abs/1412.7024)
8. Gupta S et al (2015) Deep learning with limited numerical precision. CoRR, [arXiv:abs/1502.02551](https://arxiv.org/abs/1502.02551)



Abstract

Decimal floating point uses base 10 instead of base 2 to represent floating-point numbers. This chapter examines how IEEE 754 decimal floating-point numbers are stored. We then contemplate a C language software implementation of decimal floating point conforming to the IEEE standard and give some examples of its use. We follow this with examples using the Python `decimal` library and end with some thoughts on the utility of decimal floating point.

7.1 What is Decimal Floating Point?

Modern computers make extensive use of binary floating-point arithmetic. However, it is known that there are often accuracy issues with binary floating point because of rounding or lack of precision when converting decimal numbers to binary. For example, decimal 0.1 is an infinitely repeating binary fraction that must be truncated when stored in a finite number of bits.

Decimal floating point (DFP) was introduced to address these issues. In DFP, the base is 10, not 2, so decimal numbers may be represented exactly (to within the number of digits in the significand). The IEEE standard that defines binary floating point also defines decimal floating-point formats and operations. This is the standard we will refer to in this chapter. There is currently limited hardware support for DFP, meaning we will focus exclusively on software implementations.

A binary floating-point number is represented as

$$\pm d_0.d_1d_2d_3 \dots d_{p-1} \times 2^e, \quad 0 \leq d_i < 2$$

where $d_0.d_1d_2d_3 \dots d_{p-1}$ is the p digit significand (or mantissa), 2 is the base, and e is the integer exponent. Similarly, a decimal floating-point number is represented as

$$\pm d_0.d_1d_2d_3 \dots d_{p-1} \times 10^e, \quad 0 \leq d_i < 10$$

where the digits of the significand are decimal digits, and the base is 10. Storing DFP numbers on a computer implies an encoding for the significand so the digits 0 . . . 9 can be stored efficiently. This is not necessary in a binary floating-point number.

7.2 The IEEE 754 Decimal Floating-Point Format

This section introduces decimal floating point using the IEEE 754 format (most recent version 2019). We will describe how DFP numbers are stored using the fixed *decimal32*, *decimal64*, and *decimal128* storage formats, and cover special values such as zero, infinity, and Not-a-Number (NaN). These values have direct analogs in binary floating point.

Storage Formats. The IEEE 754 standard specifies DFP numbers as a sign, coefficient, and exponent so that the actual number is

$$n = (-1)^{sign} \times coefficient \times 10^{exponent}$$

with the sign a single bit, 0 if positive and 1 for negative, and the exponent in excess binary notation. Excess notation means that the actual exponent is found by subtracting the bias from the stored value. There are two equivalent storage formats for the coefficient field, one as a binary integer and the other as a packed decimal format. In this chapter, we will work with a software library using the packed decimal format. The few existing hardware implementations of DFP use the packed format most often as well.

While the DFP value is specified by the sign, coefficient, and exponent, the format by which DFP numbers are stored in memory is somewhat complicated. For example, the *decimal64* format consists of four fields

S	CF	BXCF	CCF
1	5	8	50

where S is the sign (1 bit), CF is the combination field (5 bits), $BXCF$ is the biased exponent continuation field (8 bits), and CCF is the coefficient continuation field (50 bits). For a *decimal32* number, the $BXCF$ is 6 bits and the CCF is 20 bits. For *decimal128*, these fields are 12 bits and 110 bits, respectively.

Let’s start by decoding an example of a *decimal64* number, in this case 0.1. While 0.1 is infinitely repeating when expressed in binary, in decimal it is a single digit. Encoding 0.1 as a DFP number uses eight bytes of memory (64 bits),

0.1 → 22 34 00 00 00 00 00 01

from which we see that the fields are, in binary,

S = 0
CF = 01000
BXCF = 10001101
CCF = 0000000000 0000000000 0000000000 0000000000 0000000001

where we have written the *CCF* as five groups of ten binary digits. We’ll understand why momentarily. For now, let’s consider the other fields. The sign is straightforward enough. It is zero, implying the number is positive since $(-1)^0 = 1$. The *CF* field requires explanation. It is a 5-bit field that encodes the two leftmost binary digits of the bias exponent and the coefficient’s leftmost digit. There are ten possible coefficient digits and three different values of the leftmost 2 bits of the exponent for a total of 30 possible 5-bit combinations. As $2^5 = 32$, this means that 30 of the possible bit patterns encode the values we want, with two left over for other uses. Table 7.1 shows all the bit patterns for the *CF* field and how to interpret them to extract the exponent and coefficient digit values.

For our example, we have 01000 which corresponds to exponent bits 01 and a leftmost coefficient digit of 0.

The remainder of the 10-bit exponent is in the *BXCF* field. Combining this with the decoded *CF* value gives $0110001101_2 = 18D_{16} = 397$ for the biased exponent. The bias value for *decimal64* is 398 so the unbiased exponent is $397 - 398 = -1$.

Table 7.1 Decoding the *CF* field. Locate the *CF* bit pattern to read off the leftmost 2 bits of the biased exponent (column) and the leftmost digit of the coefficient (row). After [2]

Digit	00	01	10
0	00000	01000	10000
1	00001	01001	10001
2	00010	01010	10010
3	00011	01011	10011
4	00100	01100	10100
5	00101	01101	10101
6	00110	01110	10110
7	00111	01111	10111
8	11000	11010	11100
9	11001	11011	11101

The bias value for *decimal32* is 101 while the bias for *decimal128* is 6176. The minimum and maximum exponents for each of the three encodings are, therefore,

	<i>minimum</i>	<i>maximum</i>
<i>decimal32</i>	−101	90
<i>decimal64</i>	−398	398
<i>decimal128</i>	−6176	6111

The last part we need to decode is the coefficient field itself. We already know the leftmost digit of the coefficient field from the *CF* field. It is zero. The remainder of the coefficient is encoded in the *CCF* field. This field encodes the decimal digits of the coefficient in *densely packed decimal* or DPD. This format uses 10 bits to encode three decimal digits. DPD is more efficient than classical binary-coded decimal, which would require 12 bits to encode three decimal digits. See Chap. 2 for an explanation of binary-coded decimal.

The DPD encoding bits, labeled *pqr stu v wxy*, map to three decimal digits represented in binary as *abcd efgh ijkm* where each letter is a binary digit (*l* is skipped to avoid confusion with 1). This group of 10 bits is called *declet* and can be decoded via Table 7.2.

For our example, we have five declets corresponding to 15 decimal digits, three digits per declet, plus one additional leading (leftmost) digit of zero. This means that a *decimal64* number has 16 digits of precision. For *decimal32* the precision is 7 digits and 34 for *decimal128*. Four of the five declets in our example are zero mapping to three zero digits. The rightmost declet is 0000000001, which means that *v* in Table 7.2 is zero corresponding to the first row of the table. Therefore, the three decimal digits of this last declet are

0pqr 0stu 0wxy
0000 0000 0001

Table 7.2 Unpacking a declet (*pqr stu v wxy*) by translating it into three groups of binary digits representing three decimal digits (*abcd efgh ijkm*). After [2]

<i>vwst</i>	<i>abcd</i>	<i>efgh</i>	<i>ijkm</i>
0 − − −	0pqr	0stu	0wxy
100 −	0pqr	0stu	100y
101 −	0pqr	100u	0sty
110 −	100r	0stu	0pqy
11100	100r	0pqu	100y
11110	0pqr	100u	100y
11111	100r	100u	100y

or 001 in decimal. We have decoded the entire DPF number: sign, coefficient, and exponent. The final value is, as we expect,

$$n = (-1)^0 \times 0000000000000001 \times 10^{-1} = 1 \times 10^{-1} = 0.1$$

It should be noted that, unlike IEEE binary floating point, DFP does not have a single exponent for each number. This means multiple representations of the same number are possible since $1 \times 10^{-1} = 10 \times 10^{-2}$, etc. The standard defines a preferred exponent based on an operation’s arguments to produce consistent output. The set of representations mapping to the same number is called its *cohort*.

Packing numbers into a DPD declet is straightforward. For the example above, we produced three BCD digits (4 bits each; the value in binary is the value of the digit) and called the bits of each *abcd*, *efgh*, and *ijklm*. Using these labels for the bits, we can take any set of three BCD digits and encode them into a DPD declet using Table 7.3. Recall that the 10 bits of the declet are labeled *pqr stu v wxy*, which are read from the table by finding the row matching the *aei* bit values from the input BCD numbers. We will not work an example here, but Table 7.3 is necessary for the exercises.

Special Values. Just as IEEE 754 defines special values for binary floating point, so it does for decimal floating point. These are zero, which may be signed or unsigned, infinities, and Not-a-Number (NaN). Let’s consider each of these.

The eight bytes representing +0 are 22 38 00 00 00 00 00 00 for a *decimal64* number. Parsing gives *S* of 0, *CF* of 01000, and *BXCF* of 10001110. As before, the *CF* implies that the first 2 bits of the exponent field are 01 while the first digit of the coefficient field is zero. Glancing at the remaining digits of the bytes representing zero makes it clear that the entire coefficient field is zero. What about the exponent? Combining the 01 from the *CF* field with the bits of the *BXCF* field gives

$$e = 0110001110_2 = 18E_{16} = 398$$

Table 7.3 Packing three BCD numbers (*abcd efgh ijklm*) into a declet (*pqr stu v wxy*). Locate the row by matching the first bit of each BCD number to the value under the *aei* column. Then, the bits of the declet are read from the remaining columns of that row. After [2]

aei	pqr	stu	v	wxy
000	bcd	fgh	0	jkm
001	bcd	fgh	1	00m
010	bcd	jkh	1	01m
011	bcd	10h	1	11m
100	jkd	fgh	1	10m
101	fgd	01h	1	11m
110	jkd	00h	1	11m
111	00d	11h	1	11m

which unbiased is $e = 398 - 398 = 0$. So, zero is represented in DFP as $(-1)^0 \times 0 \times 10^0$, as we expect. What about -0 ? The only value that changes is the sign giving $(-1)^1 \times 0 \times 10^0$, which is still zero.

A special value of infinity is used if a number, regardless of sign, is too large to be represented. To signal infinity, the *CF* field is set to one of the two 5-bit values not used to encode the exponent. In this case, the *CF* field becomes 11110. The sign bit determines whether the value is $+\infty$ or $-\infty$. Other bits are ignored. So, if we try to represent 1×10^{1000} with a *decimal64* number, which is too large since the largest allowed exponent value is 369; we instead get infinity,

78 00 00 00 00 00 00 00

with

$$\begin{aligned} S &= 0 \\ CF &= 11110 \end{aligned}$$

and *BXCF* and *CCF* of zero. Negative infinity is the same with *S* set to 1.

Decimal floating point defines NaN with a special *CF* field value of 11111. For example, attempting to calculate the logarithm of a negative number results in NaN. In addition to the bits of the *CF* field, the leftmost bit of the *BXCF* field is used to determine whether or not the NaN is signaling (bit set to 1) or quiet (bit set to 0). A signaling NaN will throw a floating-point exception, while a quiet NaN will not.

When a NaN is generated, the IEEE 754 standard allows implementations to use the *CCF* field to encode information about the source of the NaN, i.e., to define a payload. The standard ensures that subsequent operations involving a NaN will propagate the payload to the result of the operation. It appears, in practice, that this ability is seldom used. Instead, the appearance of the NaN is taken to mean there is an error in the expression or code without using the payload to provide any additional information.

Rounding Modes. The IEEE standard defines multiple rounding modes. The standard assumes that operations are performed with “infinite” precision and then fit into the storage format at hand. Doing so requires rounding the results to the proper number of output digits. Users seldom modify these modes, but there are times when it is beneficial to do so (interval arithmetic is one such instance). Five rounding modes apply to decimal floating-point numbers,

Name	Description
<i>roundTowardsPositive</i>	Round the result towards $+\infty$.
<i>roundTiesToAway</i>	Round the result to the closest representable value. Round ties to the larger in magnitude.
<i>roundTiesToEven</i>	Round the result to the closest representable value. Round ties towards the even value.
<i>roundTowardsZero</i>	Round the result towards zero.
<i>roundTowardsNegative</i>	Round the result towards $-\infty$.

The default action is *roundTiesToEven*.

Storage Order. The examples in this section implicitly assume that the bytes of the DFP number are stored in memory with the most significant byte first, i.e., in big-endian format. Big-endian matches how the bits are labeled in the standard diagrams. The library explored later in the chapter uses little-endian. The difference is reversing the order of the bytes in memory. Our examples will continue to use big-endian.

An Example. Before exploring the open-source decimal floating-point library, let's use what we've covered in this section to write a simple routine to parse a DFP number, stored as eight bytes, and output the value as an ASCII string. We will assume the storage order to be big-endian.

The main function, `dfp_parse()`, is listed in Fig. 7.1. It accepts a big-endian *decimal64* number as an array of bytes and fills a given string with the output value. The string array is assumed to be large enough.

The sign of the output string is set (line 4). If the number is not negative, the sign will be overwritten. Lines 5 through 7 extract the sign bit, *CF* field, and *BXCF* field. The *BXCF* field consists of the last 2 bits of the first byte and all of the second. Two quick checks ask if the number is infinity or a NaN. If either, copy the proper string to the output and return (lines 9 and 10). Line 12 calls a helper function, `dfp_parse_exp()`, to extract the exponent and first output digit from the combined *CF* and *BXCF* fields,

```

1 | int dfp_parse_exp(int cf, int bxcf, int *d) {
2 |     static int digits[] = {0,1,2,3,4,5,6,7,
3 |                           0,1,2,3,4,5,6,7,
4 |                           0,1,2,3,4,5,6,7,
5 |                           8,9,8,9,8,9};
6 |     static int bits[] = {0,0,0,0,0,0,0,0,
7 |                         1,1,1,1,1,1,1,1,
8 |                         2,2,2,2,2,2,2,2,
9 |                         0,0,1,1,2,2};
10 |
11 |     *d = digits[cf];
12 |     return ((bits[cf] << 8) + bxcf) - 398;
13 | }
```

```

1 void dfp_parse(unsigned char *b, unsigned char *s) {
2     int sign, cf, bxcf, d0,d1,d2, dec, exponent, i=0;
3
4     s[0] = '-';
5     sign = (b[0] & 0x80) != 0;
6     cf = (b[0] >> 2) & 0x1f;
7     bxcf = ((b[0] & 0x3) << 6) + (b[1] >> 2);
8
9     if (cf==0x1e) { strcpy(&s[sign],"inf"); return; }
10    if (cf==0x1f) { strcpy(s,"NaN"); return; }
11
12    exponent = dfp_parse_exp(cf, bxcf, &d0);
13    i += sign;
14    s[i++] = d0 + '0';
15
16    dec = ((b[1] & 0x3) << 8) + b[2];
17    dfp_parse_declet(dec, &d0, &d1, &d2);
18    s[i++] = d0 + '0';
19    s[i++] = d1 + '0';
20    s[i++] = d2 + '0';
21    dec = (b[3] << 2) + ((b[4] >> 6) & 0x3);
22    dfp_parse_declet(dec, &d0, &d1, &d2);
23    s[i++] = d0 + '0';
24    s[i++] = d1 + '0';
25    s[i++] = d2 + '0';
26    dec = ((b[4] & 0x3f) << 4) + ((b[5] & 0xf0) >> 4);
27    dfp_parse_declet(dec, &d0, &d1, &d2);
28    s[i++] = d0 + '0';
29    s[i++] = d1 + '0';
30    s[i++] = d2 + '0';
31    dec = ((b[5] & 0xf) << 6) + ((b[6] & 0xfc) >> 2);
32    dfp_parse_declet(dec, &d0, &d1, &d2);
33    s[i++] = d0 + '0';
34    s[i++] = d1 + '0';
35    s[i++] = d2 + '0';
36    dec = ((b[6] & 0x3) << 8) + b[7];
37    dfp_parse_declet(dec, &d0, &d1, &d2);
38    s[i++] = d0 + '0';
39    s[i++] = d1 + '0';
40    s[i++] = d2 + '0';
41
42    s[i++] = 'E';
43    sprintf(&s[i], "%d", exponent);
44 }

```

Fig.7.1 A function to parse a *decimal64* number

which is an encoding of Table 7.1 allowing the 5-bit *CF* value to be used to look up the proper digit and bits. The bits are combined with the *BXCF* to produce the biased exponent, which is then reduced by subtracting the bias term (line 12).

Once we have the exponent and first significand digit, we can put it in the output (line 14) after setting up our index (line 13), which causes the initial minus sign to be overwritten if the number is positive. Lines 16 through 40 appear daunting at first, but they are merely five repetitions of the same pattern. First, we extract the 10 bits of a declet (lines 16, 21, 26, 31, and 36). Next, we recover the three decimal digits from this declet by calling the helper function `dfp_parse_declet()`. We then place the digits in the output string. The lines defining the declet are somewhat cryptic. They pull the 10 bits of the declet from the bytes of the input DFP number using standard bit manipulation to piece the declet together. A helpful exercise is to write out all the bits for one of the hexadecimal examples above, say the one for 0.1, and then mark off the five declets. This will make clear the origin of the particular incantations used in this function. The helper function itself is an implementation of Table 7.2, and is given in Fig. 7.2 with `dfp_parse_bits()` defined as

```

1 | int dfp_parse_bits(int a, int b, int c, int d) {
2 |     return 8*a + 4*b + 2*c + d;
3 | }
```

The helper function assigns the bits from the input declet (lines 4 through 8) and then compares them to the values from the table in lines 10 through 44. When a match occurs, the three decimal numbers are extracted by setting the proper bits according to Table 7.2.

After line 40 in Fig. 7.1, all that remains is to copy the exponent to the output string (lines 42 and 43).

This example is the simplest approach to converting a DFP number to a decimal string. The output is given without attempting to remove leading zeros of the significand or to scale the number to a more standard range. For example, if the function `dfp_parse()` is given 22 18 00 00 19 43 65 65 as input (in hexadecimal) it produces

0000000314159265E-8

as output. A more complete implementation would adjust the output to be closer to the more familiar 3.14159265. We leave a *decimal32* version of the function to the exercises.

```

1 void dfp_parse_decllet(int dec, int *d0, int *d1, int *d2) {
2     int p,q,r,s,t,u,v,w,x,y;
3
4     y = (dec & 0x1) != 0;    x = (dec & 0x2) != 0;
5     w = (dec & 0x4) != 0;    v = (dec & 0x8) != 0;
6     u = (dec & 0x10) != 0;   t = (dec & 0x20) != 0;
7     s = (dec & 0x40) != 0;   r = (dec & 0x80) != 0;
8     q = (dec & 0x100) != 0;  p = (dec & 0x200) != 0;
9
10    if (v==0) {
11        *d0 = dfp_parse_bits(0,p,q,r);
12        *d1 = dfp_parse_bits(0,s,t,u);
13        *d2 = dfp_parse_bits(0,w,x,y);
14    }
15    if ((v==1) && (x==0) && (w==0)) {
16        *d0 = dfp_parse_bits(0,p,q,r);
17        *d1 = dfp_parse_bits(0,s,t,u);
18        *d2 = dfp_parse_bits(1,0,0,y);
19    }
20    if ((v==1) && (x==0) && (w==1)) {
21        *d0 = dfp_parse_bits(0,p,q,r);
22        *d1 = dfp_parse_bits(1,0,0,u);
23        *d2 = dfp_parse_bits(0,s,t,y);
24    }
25    if ((v==1) && (x==1) && (w==0)) {
26        *d0 = dfp_parse_bits(1,0,0,r);
27        *d1 = dfp_parse_bits(0,s,t,u);
28        *d2 = dfp_parse_bits(0,p,q,y);
29    }
30    if ((v==1) && (x==1) && (w==1) && (s==0) && (t==0)) {
31        *d0 = dfp_parse_bits(1,0,0,r);
32        *d1 = dfp_parse_bits(0,p,q,u);
33        *d2 = dfp_parse_bits(1,0,0,y);
34    }
35    if ((v==1) && (x==1) && (w==1) && (s==1) && (t==0)) {
36        *d0 = dfp_parse_bits(0,p,q,r);
37        *d1 = dfp_parse_bits(1,0,0,u);
38        *d2 = dfp_parse_bits(1,0,0,y);
39    }
40    if ((v==1) && (x==1) && (w==1) && (s==1) && (t==1)) {
41        *d0 = dfp_parse_bits(1,0,0,r);
42        *d1 = dfp_parse_bits(1,0,0,u);
43        *d2 = dfp_parse_bits(1,0,0,y);
44    }
45
46    return;
47 }

```

Fig.7.2 A function to parse a DPD decllet into three BCD digits

7.3 Decimal Floating Point in Software

In this section, we examine DFP implementations in C and Python. DFP operations are complex enough that beyond the example above to parse a DFP number, it does not make sense to attempt an implementation of our own.

DFP in C. DFP implementations in hardware are not yet common. However, IBM has released, as open source, a DFP library which is compatible with the IEEE 754 standard. We will be using this library in this section. The *decNumber* library is available here

<http://speleotrove.com/decimal/decnumber.html>

along with some documentation on its use. We are using version 3.68, but later versions should operate similarly. See the files in the `decNumber` directory to save downloading it. There is no Makefile or installation; the library is simply a set of source code files in C, along with documentation and a few examples. The IEEE-compliant portion is implemented in `decimal32.c`, `decimal64.c`, and `decimal128.c`. We will focus on the *decimal64* implementation.

The library uses little-endian storage order. A useful example is `example5.c`, which we can trivially modify to output in big-endian order. In a text editor, open `example5.c` and change line 30 from

```
printf(&hexes[i*3], "%02x ", a.bytes[i]);
```

to

```
printf(&hexes[i*3], "%02x ", a.bytes[7-i]);
```

to output with the highest order byte first. Compile the program,

```
gcc example5.c -o bin/example5 decimal64.c decNumber.c
decContext.c
```

and then test the program,

```
./example5 0.1
```

which should produce

```
0.1 => 22 34 00 00 00 00 00 01 => 0.1
```

matching the hexadecimal numbers we decoded in the previous section.

Fig. 7.3 A program to iterate the logistic map, $x_{i+1} = rx_i(1 - x_i)$, using IEEE 754 binary floating point

```

1 | #include <stdio.h>
2 |
3 | int main(int argc , char *argv[]) {
4 |     double a,b;
5 |     double x=0.25, r=3.8;
6 |     int i;
7 |
8 |     for(i=0; i < 8; i++) {
9 |         a = 1.0 - x;
10 |        b = x * a;
11 |        x = r * b;
12 |        printf("%0.16f\n", x);
13 |    }
14 |
15 |    for(i=0; i < 20000000; i++) {
16 |        a = 1.0 - x;
17 |        b = x * a;
18 |        x = r * b;
19 |    }
20 |
21 |    printf("\n");
22 |
23 |    for(i=0; i < 8; i++) {
24 |        a = 1.0 - x;
25 |        b = x * a;
26 |        x = r * b;
27 |        printf("%0.16f\n", x);
28 |    }
29 |
30 |    return 0;
31 | }
```

The *decNumber* library is far more capable than our examples will illustrate. We will use the IEEE 754 compliant functions in *decDouble.c* to ensure our results match IEEE 754 decimal floating-point hardware. To compare results and performance between DFP and binary floating point, we will again use the logistic map we first encountered in Chap. 4. The map iterates values between $[0, 1]$ to demonstrate the bifurcation route to chaotic dynamic behavior. For the present, it is sufficient to know that this map has the form

$$x_{i+1} = rx_i(1 - x_i)$$

for some initial x_0 , which we set to 0.25, and a fixed r of 3.8, which is in the chaotic region.

First, we will implement this map in C using variables of type `double`. We will then implement the map using the *decNumber* library and compare outputs and execution time. Doing so illustrates how to use library functions to define a context, initialize variables, and perform arithmetic.

Figure 7.3 shows the plain C version. This version will initialize x and then output the first eight iterates. It then runs 20 million more iterates before displaying the final eight. Note that the calculation is split into individual operations (lines 9 to 11) so that the DFP implementation matches the order in which the expressions are evaluated.

Figure 7.4 presents the *decNumber* library version. Before the main program, we define a helper function, `pp()`, which will print a *double64* number by first converting it to a string. The function uses the `decDoubleToString()` library function. The constant `DECDOUBLE_String` is large enough to hold the biggest possible string. The argument is a pointer to a `decDouble`, the base type for a *decimal64* number. The type is a structure that does not use heap memory, meaning it can be created, used, and ignored like a standard C variable.

The program begins in line 12 by defining the temporary variables for the logistic map calculation. Line 13 declares a context that will be initialized to handle a *decimal64* number in line 17. The context is an argument to many library functions. The context includes the rounding mode, which we leave unchanged. Rounding modes are adjusted later in the chapter.

Line 14 defines our variable, x , as well as r and a constant *one*. Initial values are set from strings in lines 19 through 21. Since this is base ten, the value set is exact. Lines 23 through 28 iterate the equation eight times, printing the value of x after each iteration. The logistic equation is implemented in a way mirroring the binary implementation but using the `decDoubleSubtract()` and `decDoubleMultiply()` library functions. There are similarly named functions for addition and division, plus many other operations. The final eight iterates are printed with lines 38 through 43. Compile the programs with,

```
gcc logistic.c -o logistic -O2
gcc logistic64.c -o logistic64 decContext.c
decDouble.c decQuad.c -O2
```

where `logistic64.c` needs both `decDouble.c` and `decQuad.c` so that computations can be performed with 128 bits and then rounded to 64 bits.

Let's compare the output of the two programs, Figs. 7.3 and 7.4. The first eight iterates for `logistic` and `logistic64` are

logistic	logistic64
0.7124999999999999	0.71250
0.7784062500000002	0.77840625000
0.6554618478515620	0.6554618478515625
0.8581601326777955	0.8581601326777949
0.4625410135688510	0.4625410135688527
0.9446679324750937	0.9446679324750942
0.1986276333476368	0.1986276333476352
0.6048638471497435	0.6048638471497399

Fig. 7.4 A program to iterate the logistic map, $x_{i+1} = rx_i(1 - x_i)$, using IEEE 754 decimal floating point

```

1 | #include "decDouble.h"
2 | #include <stdio.h>
3 |
4 | void pp(decDouble *x) {
5 |     char s[DECDOUBLE_String];
6 |
7 |     decDoubleToString(x, s);
8 |     printf("%s", s);
9 | }
10 |
11 | int main(int argc, char *argv[]) {
12 |     decDouble a,b;
13 |     decContext set;
14 |     decDouble x,r,one;
15 |     int i;
16 |
17 |     decContextDefault(&set, DEC_INIT_DECDOUBLE);
18 |
19 |     decDoubleFromString(&x, "0.25", &set);
20 |     decDoubleFromString(&r, "3.8", &set);
21 |     decDoubleFromString(&one, "1", &set);
22 |
23 |     for(i=0; i < 8; i++) {
24 |         decDoubleSubtract(&a, &one, &x, &set);
25 |         decDoubleMultiply(&b, &x, &a, &set);
26 |         decDoubleMultiply(&x, &r, &b, &set);
27 |         pp(&x); printf("\n");
28 |     }
29 |
30 |     for(i=0; i < 20000000; i++) {
31 |         decDoubleSubtract(&a, &one, &x, &set);
32 |         decDoubleMultiply(&b, &x, &a, &set);
33 |         decDoubleMultiply(&x, &r, &b, &set);
34 |     }
35 |
36 |     printf("\n");
37 |
38 |     for(i=0; i < 8; i++) {
39 |         decDoubleSubtract(&a, &one, &x, &set);
40 |         decDoubleMultiply(&b, &x, &a, &set);
41 |         decDoubleMultiply(&x, &r, &b, &set);
42 |         pp(&x); printf("\n");
43 |     }
44 |
45 |     return 0;
46 | }
```

We already see that the two sequences begin to diverge after a few iterations. Note that the decimal result is exact, at least for the first two iterations, while the binary result already shows rounding errors. The final eight iterations are

logistic	logistic64
0.6689935885076239	0.3563448084581058
0.8414764347646003	0.8715801065836345
0.5068966091017391	0.4253272526869202
0.9498192597750116	0.9288111270731242
0.1811180074347505	0.2512598657334523
0.5635942443069413	0.7148877132992272
0.9346319339459104	0.7745284285575106
0.2321613115788636	0.6636097392722264

where there is no longer any relationship between the two sequences. This is after 20, 000, 000 iterations. In this case, the departure is not fatal due to the shadowing theorem, which states that for any chaotic trajectory, the numerically computed trajectory using that initial condition (here 0.25) ultimately follows another chaotic trajectory with a slightly different initial condition. This theorem means that chaotic trajectories generated by computers are real.

The Python rational arithmetic class developed in Chap. 5 indicates that the first few iterates of the logistic map are exactly

57/80

24909/32000

3355964661/5120000000

112480764910343946501/13107200000000000000

= 0.7125000000000000

= 0.7784062500000000

= 0.6554618478515625

= 0.8581601326777950

which matches the decimal floating-point output more closely than the binary.

Naturally, there is a significant difference in performance between `logistic` and `logistic64` as the former is using floating-point hardware. `Logistic64` is approximately 36x slower than `logistic`, so DFP’s precision must be balanced by the runtime hit.

We conclude this introduction to *decNumber* by examining how rounding modes affect calculations. Rounding modes are set using `decContextSetRounding()`. A simple example will suffice. Figure 7.5 illustrates all IEEE decimal rounding modes using the *decNumber* library. The selected value is such that the first digit beyond the limit of a *decimal64* number is 5 to enable a tie between representable numbers. The output of Fig. 7.5 is

```
1 int main(int argc , char *argv[]) {
2     decDouble a;
3     decContext set;
4
5     decContextDefault(&set, DEC_INIT_DECDOUBLE);
6
7     decContextSetRounding(&set, DEC_ROUND_HALF_EVEN);
8     decDoubleFromString(&a, "0.12345678901234565", &set);
9     pp(&a); printf("\n");
10    decContextSetRounding(&set, DEC_ROUND_CEILING);
11    decDoubleFromString(&a, "0.12345678901234565", &set);
12    pp(&a); printf("\n");
13    decContextSetRounding(&set, DEC_ROUND_UP);
14    decDoubleFromString(&a, "0.12345678901234565", &set);
15    pp(&a); printf("\n");
16    decContextSetRounding(&set, DEC_ROUND_DOWN);
17    decDoubleFromString(&a, "0.12345678901234565", &set);
18    pp(&a); printf("\n");
19    decContextSetRounding(&set, DEC_ROUND_FLOOR);
20    decDoubleFromString(&a, "0.12345678901234565", &set);
21    pp(&a); printf("\n\n");
22    decContextSetRounding(&set, DEC_ROUND_HALF_EVEN);
23    decDoubleFromString(&a, "-0.12345678901234565", &set);
24    pp(&a); printf("\n");
25    decContextSetRounding(&set, DEC_ROUND_CEILING);
26    decDoubleFromString(&a, "-0.12345678901234565", &set);
27    pp(&a); printf("\n");
28    decContextSetRounding(&set, DEC_ROUND_UP);
29    decDoubleFromString(&a, "-0.12345678901234565", &set);
30    pp(&a); printf("\n");
31    decContextSetRounding(&set, DEC_ROUND_DOWN);
32    decDoubleFromString(&a, "-0.12345678901234565", &set);
33    pp(&a); printf("\n");
34    decContextSetRounding(&set, DEC_ROUND_FLOOR);
35    decDoubleFromString(&a, "-0.12345678901234565", &set);
36    pp(&a); printf("\n");
37 }
```

Fig. 7.5 A program to illustrate the effect of IEEE 754 decimal rounding modes

Mode	Value
<i>roundTiesToEven</i>	0.1234567890123456
<i>roundTowardsPositive</i>	0.1234567890123457
<i>roundTiesToAway</i>	0.1234567890123457
<i>roundTowardsZero</i>	0.1234567890123456
<i>roundTowardsNegative</i>	0.1234567890123456
<i>roundTiesToEven</i>	-0.1234567890123456
<i>roundTowardsPositive</i>	-0.1234567890123456
<i>roundTiesToAway</i>	-0.1234567890123457
<i>roundTowardsZero</i>	-0.1234567890123456
<i>roundTowardsNegative</i>	-0.1234567890123457

The last digit of the converted number demonstrates the IEEE rounding rules.

DFP in Python. Python supports decimal floating point via its `decimal` module. A quick example shows it in action,

```

1 >>> from decimal import *
2 >>> "
3     '0.100000000000000000'
4 >>> "
5     '0.099999999999999999'
6 >>> setcontext(Context(prec=16))
7 >>> print(Decimal("0.1"))
8     0.1
9 >>> print(Decimal("1.95") - Decimal("1.85"))
10    0.10

```

The first two examples (lines 2 and 4) illustrate why decimal floating point is desirable. The output should be identical, 0.1, but it is not because of rounding and because 0.1 is a repeating value in binary.

Line 6 sets the `decimal` library's context to 16 digits to match the `decDouble` library used earlier and to correspond to an IEEE *decimal64* number. Both libraries support arbitrary precision, but we restrict ourselves to IEEE-sized numbers here.

The `Decimal` class defines a DFP number, typically via a string. Lines 7 and 9 repeat lines 2 and 4 to demonstrate that there is no rounding error. Additionally, line 9 returns “0.10” instead of “0.1” to indicate that the result is accurate to two digits.

Figure 7.6 presents a Python implementation of the logistic example given earlier. The output is identical to that of `logstic64`.

Fig. 7.6 A Python version of the logistic map using the `decimal` module

```

1 from decimal import *
2
3 def main():
4     setcontext(Context(prec=16))
5
6     x = Decimal("0.25")
7     r = Decimal("3.8")
8     one = Decimal("1")
9
10    for i in range(8):
11        a = one - x
12        b = x * a
13        x = r * b
14        print(x)
15
16    for i in range(20000000):
17        a = one - x
18        b = x * a
19        x = r * b
20
21    print()
22
23    for i in range(8):
24        a = one - x
25        b = x * a
26        x = r * b
27        print(x)
28
29 main()

```

7.4 Thoughts on Decimal Floating Point

Decimal floating point has not been adopted as rapidly as it might (as of 2024). Humans work in base ten almost universally, so it seems reasonable for computers to use base ten. One reason for the slow adoption of DFP is that hardware binary floating point is still much faster. The few hardware implementations of DFP are slower than their binary cousins but not terribly so. Still, even a 10 percent reduction in performance may not be acceptable in some circumstances.

The precision of DFP calculations is ideal for commercial or financial work. Many, if not most, databases are using decimal storage for monetary values, often as character strings in base ten, so that the possible application of DFP to these databases is obvious.

Is DFP useful for scientific programming? I would think so if it were fast enough. The logistic example above clearly shows that DFP results are sometimes “truer” than binary floating point. Preserving the knowledge that there are meaningful trailing zeros in a value is also potentially significant. The Python example returning “0.10” as the answer instead of “0.1” is helpful as it tells users that the digit in the hundredth place is truly zero and not simply unknown. A scientific result might require much effort to know that digit is precisely zero.

Hopefully, this chapter has demonstrated the potential utility of decimal floating point. Perhaps the thought should not be “Should I use decimal floating point?” but rather “Is there any reason why I shouldn’t use decimal floating point?”

7.5 Summary

This chapter reviewed the IEEE standard for storing decimal floating-point numbers. We learned how to decode such numbers to produce the corresponding decimal value as a string. We then examined an IEEE-compliant C library for decimal floating point and learned how to use it. We used the logistic map as an example of how decimal floating point can be more accurate than binary floating point for iterated calculations. We then examined the decimal floating-point library for Python and demonstrated that it produces the same results on the logistic map as the C version. We concluded with a brief discussion of situations where decimal floating point might best be used.

Exercises

7.1 Parse the following *decimal64* numbers. Each evaluates to a well-known mathematical constant.

```
29 ff 98 42 d2 e9 64 45
25 ff 18 0c ce ef 26 1f
25 fe 14 44 ee 27 cc 5b
```

7.2 The function `dfp_parse()` in Fig. 7.1 only works with *decimal64* numbers. Create a version that works with *decimal32* numbers. Test it with the following inputs:

```
A2 10 C6 15 → -3.1415
F8 00 00 00 → -inf
22 40 00 01 → 0.1
```

7.3 Encode the number 123.456 as a *decimal64* number. First, decide on the significand and exponent to use, then encode the exponent and first digit of the significand to create the *CF* and *BXCF* fields. Finally, use Table 7.3 to encode the significand, mapping the remaining 15 decimal digits to 5 declets. ***

References

1. IEEE Standards Association (2008) Standard for floating-point arithmetic. IEEE 754-2008
2. Duale A et al (2007) Decimal floating-point in z9: an implementation and testing perspective. IBM J Res Develop 51(1, 2):217–227



Abstract

Floating-point numbers cannot represent real numbers with infinite precision. Additionally, there are sometimes uncertainties in our knowledge of the true value of a quantity. Both of these situations can be addressed with interval arithmetic that keeps bounds on the possible value of a number while a calculation is in progress. In this chapter, we describe interval arithmetic; implement basic operations for interval arithmetic in C; discuss functions as they pertain to intervals; examine interval implementations for C and Python; and, finally, offer advice on when to use interval arithmetic.

8.1 Defining Intervals

A number like π is mathematically exact, though it is impossible to write it as a decimal. However, in the real world, there is rarely complete certainty as to the value of a quantity since most are measurements made by finite devices with finite precision. A scale may only be accurate to the ounce or gram. A ruler may only be accurate to a millimeter. Scientists associate uncertainties with measured quantities to account mathematically for the inexactness of our dealings with reality. Similarly, since computers use a finite number of bits to store floating-point numbers, we know it is impossible to store just any real number in a computer. In most situations, we can only store the floating-point number we deem to be “nearest to” the actual real number.

Mathematics has ways to deal with uncertainty through probability and statistics or error analysis and propagation. The latter are mathematical concepts which involve a function, f , of input variables, say, x and y , and how to determine the uncertainty in $f(x, y)$, denoted as σ_f , given x , y and their uncertainties, σ_x and σ_y . The propagation of errors determines how to calculate σ_f . Error propagation is an essential technique

for science as a whole, and the reader is directed to the presentation in Bevington [1] for examples and advice on how to calculate the uncertainty of expressions involving variables with uncertainties. For our purposes, we need to only consider what σ_x means for a particular x and how to represent that meaning in a computer using floating-point numbers. We will always assume that any floating-point numbers we use are IEEE 754 compliant.

The scientific literature is full of measured quantities, along with some uncertainty as to their actual value. This uncertainty may be estimated, the standard deviation of a set of measurements, the mean of a set of measurements along with the standard error of the mean, etc. For example, it might be reported that the temperature of a reaction was 74.0 ± 1.3 C, where the ± 1.3 portion is the uncertainty. This means that, with some confidence level, we know that the actual temperature was in the interval $[72.7, 75.3]$ C because this is 1.3 C less and more than the first value reported.

Instead of tracking mean values and uncertainties, we might track the interval itself. This is the essence of *interval arithmetic*. We write intervals as $t = [72.7, 75.3]$ where the first number is the lower bound on the interval and the second is the upper bound (the bounds are included in the interval). Symbolically, we write $t = [\underline{t}, \bar{t}]$ where $\underline{t}$ is the lower bound and $\bar{t}$ is the upper.

We should slow down a bit here. The previous paragraphs mentioned the concept of “the propagation of errors” without describing it and discussing why one might choose interval arithmetic over propagation uncertainty. Without deriving it, we give the general formula for determining the uncertainty of the value of a function (σ_f) from the uncertainties of its arguments. We assume $f(x, y)$ with argument uncertainties of σ_x and σ_y , but the formula holds for any number of arguments by using a similar term for each additional variable. The formula is

$$\sigma_f^2 = \sigma_x^2 \left(\frac{\partial f}{\partial x} \right)^2 + \sigma_y^2 \left(\frac{\partial f}{\partial y} \right)^2$$

under the assumption that x and y are independent (i.e., not correlated with each other so that the covariance terms disappear) and that x and y represent random variables from Gaussian distributions.

The point to remember is that the formula used for error propagation is based on assumptions about how the values behave or, rather, the environment in which they were produced. And, as is always the case, an assumption may be incorrect. With intervals, there is no assumption beyond the statement that the true value lies within the given interval.

Below, we learn how to calculate intervals, but let’s consider an example comparing intervals to error propagation. In this example, we want to calculate $f(x, y) = x + y$ and find the uncertainty or interval containing $f(x, y)$. We use a C program that uses interval arithmetic to add x and y and to calculate σ_f from the formula above. The program produces the following:

```
propagation of errors = [23.344, 23.564]
interval arithmetic = [23.337, 23.571]
```

for $x = [12.12, 12.34]$ and $y = [11.217, 11.231]$. Notice that, while not exact, the two methods produce comparable answers, so we know at the least that interval arithmetic might be a useful way to account for uncertainty.

In some sense, the origin of interval arithmetic stretches back to antiquity. Archimedes, in his *On the measurement of the circle* estimated the value of π by using a two-sided bound, which is, in effect, an interval since he showed that $\underline{x} \leq \pi \leq \bar{x}$ for the bounds $[\underline{x}, \bar{x}]$ placed on the value of π .

Intervals were discussed by Young [2] (1931) and extended by Dwyer [3] (1951). Sunaga again extended the algebra of intervals [4] (1958). Dwyer discussed computing with approximate numbers, but it was Moore who first wrote about implementing interval arithmetic on digital computers [5] (1959). Moore refers to Dwyer and then extends intervals to floating-point numbers representable on the computer. Indeed, [5] is more than simply interval arithmetic and is an interesting historical work discussing how to represent floating-point numbers on digital computers. Note that Moore calls interval arithmetic “range arithmetic” after Dwyer and then extends this to “digital range arithmetic”, meaning the interval arithmetic of this chapter.

The remainder of the chapter discusses interval arithmetic over basic operations ($+$, $-$, $\times$, $/$, comparison) with examples in C and Python. Following that comes a discussion of functions and intervals. The discussion includes elementary functions like sine, cosine, and exponential, along with general functions consisting of expressions involving variables that are themselves intervals. We will learn that general functions lead to undesirable consequences and discuss the “dependency effect”, which is the biggest drawback to interval arithmetic. After this, we examine intervals in programming. The chapter concludes with a brief discussion.

8.2 Basic Operations

Basic arithmetic operations are well suited to intervals, but before we begin our library of functions, we must consider how to define intervals using floating-point numbers. Mathematically, the bounds of an interval are exact real numbers. Computers do not operate on real numbers but instead use floating-point numbers, which are, of necessity, members of a finite set. Therefore, the result of an operation is often not a valid floating-point number and must be rounded to a member of the finite set according to the current rounding mode. This is true even when the operands are already floating-point numbers. In other words, floating-point arithmetic is not closed on the set of floating-point numbers.

If our interval arithmetic functions round to the nearest floating-point number, we will introduce a bias affecting the intervals. Over time, we will no longer be confident that the actual result lies within the specified interval. We can remedy this, however, by using the IEEE 754 non-default rounding modes. Specifically, we must round the lower bound towards $-\infty$ and the upper bound towards $+\infty$ [6]. Doing so ensures that the interval does not shrink during the calculation.

Let's begin our library,

```

1  #include <fenv.h>
2  #include <stdio.h>
3  #include <math.h>
4
5  typedef struct {
6      double a;
7      double b;
8  } interval_t;
9
10 char *pp(interval_t x) {
11     static char s[100];
12     sprintf(s, "[%0.16g, %0.16g]", x.a, x.b);
13     return s;
14 }
15
16 interval_t interval(double a, double b) {
17     interval_t ans;
18     ans.a = a;
19     ans.b = b;
20     return ans;
21 }

```

The function `pp()` displays an interval, a variable of type `interval_t`. To maximize precision, we use doubles with `a` the lower bound and `b` the upper. The function `interval()` builds an interval from two floating-point numbers representing the bounds.

Note the inclusion of not only `stdio.h` and `math.h` but also `fenv.h`. The last defines constants and functions related to floating point, including those necessary to change the rounding mode. Including `fenv.h` is not enough on its own, however. We must also compile our programs with the `-frounding-math` compiler switch (for `gcc`); otherwise, the function calls changing the rounding mode will be ignored.

Addition and Subtraction. We're ready to add two intervals, $x = [\underline{x}, \bar{x}]$ and $y = [\underline{y}, \bar{y}]$. The result is not surprising,

$$x + y = [\underline{x}, \bar{x}] + [\underline{y}, \bar{y}] \equiv [\underline{x} + \underline{y}, \bar{x} + \bar{y}]$$

However, we must remember to round the lower bound towards $-\infty$ and the upper bound towards $+\infty$. Intuitively, it makes sense that addition sums the lower and upper bounds independently. So far, so good. Here's the code,

```

1 interval_t int_add(interval_t x, interval_t y) {
2     interval_t sum;
3
4     fesetround(FE_DOWNWARD);
5     sum.a = x.a + y.a;
6     fesetround(FE_UPWARD);
7     sum.b = x.b + y.b;
8     fesetround(FE_TONEAREST);
9     return sum;
10 }

```

Lines 5 and 7 add the lower and upper bounds. Line 4 sets the floating-point rounding mode towards $-\infty$ just before adding the lower bounds, while line 6 sets the rounding mode towards $+\infty$ for the upper bounds in line 7. Lastly, line 8 returns to rounding towards nearest before returning the new sum in line 9. Recall that rounding towards nearest is the IEEE default.

Subtraction is similar in form to addition but with a twist. Interval subtraction is defined to be

$$x - y = [\underline{x}, \bar{x}] - [\underline{y}, \bar{y}] \equiv [\underline{x} - \bar{y}, \bar{x} - \underline{y}]$$

which at first glance might seem a bit odd. One way to understand why subtraction is defined like so is to consider what the negation of an interval might mean. To negate a value is to change its sign, $x \rightarrow -x$. The operation is the same for intervals, but not only are the signs of the bounds changed, but also their order. If the order were not changed, the lower bound would suddenly be larger than the upper bound. Therefore, negation is

$$-[\underline{x}, \bar{x}] \equiv [-\bar{x}, -\underline{x}]$$

where we have swapped the order of the bounds. How does this help in understanding subtraction? By remembering that subtraction is simply the addition of a negative value,

$$[\underline{x}, \bar{x}] - [\underline{y}, \bar{y}] = [\underline{x}, \bar{x}] + [-\bar{y}, -\underline{y}] \equiv [\underline{x} - \bar{y}, \bar{x} - \underline{y}]$$

In code, we get

```

1 interval_t int_sub(interval_t x, interval_t y) {
2     interval_t diff;
3
4     fesetround(FE_DOWNWARD);
5     diff.a = x.a - y.b;
6     fesetround(FE_UPWARD);
7     diff.b = x.b - y.a;
8     fesetround(FE_TONEAREST);
9     return diff;
10 }

```

Again, we must set the rounding modes properly.

Multiplication. The goal of interval arithmetic is to ensure that the interval represents the entire range in which the actual value may be found. This means that the lower bound must be the smallest such value, and the upper bound must be the largest. When adding two intervals, the lower bound must be the sum of the lower bounds, and the upper bound must be the sum of the upper bounds. The same is true for subtraction. However, multiplication is a bit messier,

$$[\underline{x}, \bar{x}] \times [\underline{y}, \bar{y}] \equiv [\min\{\underline{x}\underline{y}, \underline{x}\bar{y}, \bar{x}\underline{y}, \bar{x}\bar{y}\}, \max\{\underline{x}\underline{y}, \underline{x}\bar{y}, \bar{x}\underline{y}, \bar{x}\bar{y}\}]$$

which reduces to

$$[\underline{x}, \bar{x}] \times [\underline{y}, \bar{y}] = [\underline{x}\underline{y}, \bar{x}\bar{y}]$$

if the lower bounds of x and y are greater than zero. The definition considers all combinations to determine the smallest and largest products of the bounds. Let's add multiplication to the library,

```

1 interval_t int_mul(interval_t x, interval_t y) {
2     interval_t prod;
3     double t;
4
5     fesetround(FE_DOWNWARD);
6     prod.a = x.a * y.a;
7     t = x.a * y.b;
8     if (t < prod.a) prod.a = t;
9     t = x.b * y.a;
10    if (t < prod.a) prod.a = t;
11    t = x.b * y.b;
12    if (t < prod.a) prod.a = t;
13
14    fesetround(FE_UPWARD);
15    prod.b = x.a * y.a;
16    t = x.a * y.b;
17    if (t > prod.b) prod.b = t;
18    t = x.b * y.a;
19    if (t > prod.b) prod.b = t;
20    t = x.b * y.b;
21    if (t > prod.b) prod.b = t;
22
23    fesetround(FE_TONEAREST);
24    return prod;
25 }
```

We accept two input intervals in `x` and `y` and return `prod` as their product. Lines 5 through 12 set the lower bound, so we first set the IEEE rounding mode to $-\infty$ (line 5). Line 6 sets the lower bound of `prod` to `x.y`. This will be our lower bound for the product unless one of the other component products is smaller. We set the auxiliary variable `t` to the next candidate (line 7), and in line 8, check to see if this is smaller than our current lower bound. If so, we update the lower bound. Lines 9 through 12

continue this process for the remaining component products. After line 12, `prod.a` is set to the smallest value. Line 14 sets the rounding mode to $+\infty$ and repeats the process for the upper bound, keeping the largest component product in `prod.b`. Line 23 then restores the default IEEE rounding mode and returns the product.

Reciprocal and Division. Just as subtraction is the addition of a negative value, so it is that division is multiplication by the reciprocal. Therefore, to implement division, we must implement a reciprocal. A first thought might be to simply take each interval component's reciprocal, $x \rightarrow 1/x$. However, just as the negation of an interval required swapping the bounds, so will the reciprocal because $1/\bar{x} < 1/\underline{x}$. Therefore, the reciprocal of an interval is

$$1/[\underline{x}, \bar{x}] \equiv [1/\bar{x}, 1/\underline{x}]$$

However, there is an important caveat to bear in mind. Division by zero is mathematically undefined, implying we must be mindful of the case where the interval contains zero. If the interval does not contain zero, i.e., $\underline{x} > 0$ or $\bar{x} < 0$, we proceed normally because the reciprocal is defined over the entire interval. In other words, we must add a qualifier to our original definition,

$$1/[\underline{x}, \bar{x}] \equiv [1/\bar{x}, 1/\underline{x}], \underline{x} > 0 \text{ or } \bar{x} < 0$$

to make explicit the fact that we are not allowed to consider intervals that contain zero.

For scalars, we simply say that division by zero is undefined and leave it at that. Can we say something more in the case of intervals? One approach is to split the interval into two parts at zero,

$$\begin{aligned} 1/[\underline{x}, 0] &\rightarrow [-\infty, 1/\underline{x}] \\ 1/[0, \bar{x}] &\rightarrow [1/\bar{x}, \infty] \end{aligned}$$

following the definition above and writing $\pm\infty$ for $1/0$. Doing so results in two overlapping intervals representing the reciprocal of our original interval, x , which contained zero. There is no information in writing the reciprocal of an interval containing zero, but we continue to work through an expression involving such an interval if we use both the intervals above. This is one reason why interval arithmetic is not more widely used, though it isn't the main one (we learn that reason later in the chapter). For our convenience, and because we are trying to be pedagogical and not obfuscate things, we will declare reciprocals of intervals containing zero to be illegal and ignore them.

We are now ready to calculate x/y . We do this by first calculating the reciprocal, $y' \equiv 1/y$, and then $x \times y'$. The code becomes

```

1 interval_t int_recip(interval_t x) {
2     interval_t inv;
3
4     if ((x.a < 0.0) && (x.b > 0.0)) {
5         inv.a = -exp(1000.0);
6         inv.b =  exp(1000.0);
7     } else {
8         fesetround(FE_DOWNWARD);
9         inv.a = 1.0 / x.b;
10        fesetround(FE_UPWARD);
11        inv.b = 1.0 / x.a;
12        fesetround(FE_TONEAREST);
13    }
14
15    return inv;
16 }
17
18 interval_t int_div(interval_t x, interval_t y) {
19     return int_mul(x, int_recip(y));
20 }

```

where `int_mul()` performs the multiplication applying proper rounding modes.

The function `int_div()` is straightforward: simply multiply the first argument by the reciprocal of the second (line 19). All the action is in `int_recip()`, which first checks to see if zero is in the interval (line 4). If it is, we declare the reciprocal undefined and return $[-\infty, +\infty]$ by attempting to calculate the exponential of a large number (lines 5 and 6). Line 8 sets rounding towards $-\infty$ and then calculates the lower bound in line 9. The upper bound is in lines 10 and 11 with rounding set towards $+\infty$. Line 12 resets the rounding mode, and line 15 returns the new interval.

Powers. We can compute x^n for floating point x and integer power, n . If we want x^4 , we multiply x with itself four times: $x^4 = x \times x \times x \times x$. Integer powers of intervals are not as straightforward. The result of raising an interval to an integer power depends on the evenness or oddness of the exponent and the signs of the interval components.

It's easiest to present the formulas and then assess why they are as they are. For an odd integer exponent, $n = 2k + 1$, $k = 0, 1, \dots$, exponentiation works as expected

$$[\underline{x}, \bar{x}]^n = [\underline{x}^n, \bar{x}^n], \quad n = 2k + 1, \quad k = 0, 1, \dots$$

Life becomes more interesting when the exponent is even. In that case, we compute

$$\begin{aligned}
 \underline{x}, \bar{x}^n &= [\underline{x}^n, \bar{x}^n], \quad \underline{x} \geq 0, \quad n = 2k, \quad k = 1, 2, \dots \\
 &= [\bar{x}^n, \underline{x}^n], \quad \bar{x} < 0 \\
 &= [0, \max\{\underline{x}^n, \bar{x}^n\}], \quad \text{otherwise}
 \end{aligned}$$

For an odd exponent, it is straightforward to understand why the formula takes the form it does. If the interval has negative bounds, then an odd power of those bounds will still be negative, and the order of the original components will not change. In other words, for an odd exponent $\underline{x}^n < \bar{x}^n$, implying no special cases.

However, when the exponent is even, we must pay close attention to the signs of the bounds. If $\underline{x} \geq 0$, we know that both bounds are positive and that $\underline{x} < \bar{x}$. Therefore, the order of the bounds need not change when exponentiating as $\underline{x}^n < \bar{x}^n$.

What if both bounds are negative? Since $a^n = |a|^n$ if n is even and $a < 0$ it follows that $\underline{x}^n > \bar{x}^n$ when $\underline{x}, \bar{x} < 0$ so we must swap the order of the bounds.

The last case concerns intervals that include zero. Because x^n is always positive when n is even (and a positive integer), we know that the lower bound of the result must be zero. The upper bound is set to the larger of the two original bounds after raising them to the n -th power.

What if n is a negative integer? If $n < 0$, we use the rule that $x^{-n} = 1/x^n$. Therefore, first, determine $y = x^n = [\underline{x}, \bar{x}]^n$ and then calculate the result as $[\underline{x}, \bar{x}]^{-n} = [1/\bar{y}, 1/\underline{y}]$ with suitable warnings about y containing zero.

Adding `int_pow()` to our library is straightforward, if a little tedious because of the special cases:

```

1 interval_t int_pow(interval_t x, int n) {
2     interval_t ans;
3
4     if ((n
5         fesetround(FE_DOWNWARD);
6         ans.a = pow(x.a,n);
7         fesetround(FE_UPWARD);
8         ans.b = pow(x.b,n);
9     } else {
10        if (x.a >= 0) {
11            fesetround(FE_DOWNWARD);
12            ans.a = pow(x.a,n);
13            fesetround(FE_UPWARD);
14            ans.b = pow(x.b,n);
15        } else {
16            if (x.b < 0) {
17                fesetround(FE_DOWNWARD);
18                ans.a = pow(x.b,n);
19                fesetround(FE_UPWARD);
20                ans.b = pow(x.a,n);
21            } else {
22                ans.a = 0.0;
23                fesetround(FE_UPWARD);
24                ans.b = pow(x.a,n);
25                if (pow(x.b,n) > ans.b) ans.b = pow(x.b,n);
26            }
27        }
28    }
29
30    fesetround(FE_TONEAREST);
31    return ans;
32 }
```

Line 4 checks to see if the exponent is even or odd. If the remainder after dividing by 2 is 1, then n is odd, and we calculate the lower bound in line 6 and the upper in line 8 after appropriately setting the floating-point rounding mode. If n is even, we move to line 10 and check the three cases above. If the lower bound is greater than or equal to zero (line 10), we calculate the new bounds in lines 12 and 14. If the upper bound is less than zero (line 16), we swap the results (lines 18 and 20). Finally, if the interval includes zero, we set the lower bound to zero (line 22) and the upper bound to the larger of the two exponentiated initial bounds (lines 24 and 25). Line 30 restores the default rounding mode, and line 31 returns our answer.

It is time for some examples using `int_pow()` for the powers and `int_mul()` for the multiplications. First, an all-negative interval:

```
x          = [ -2.7182818284590451, -2.6182818284590454]
x3        = [-20.0855369231876679, -17.9493685483622478]
x × x × x  = [-20.0855369231876679, -17.9493685483622443]
x4        = [ 46.9965055024911891, 54.5981500331442291]
x × x × x × x = [ 46.9965055024911749, 54.5981500331442433]
```

Second, an all positive interval:

```
x          = [ 2.7182818284590451, 2.8182818284590452]
x3        = [20.0855369231876644, 22.3848022077206359]
x × x × x  = [20.0855369231876608, 22.3848022077206359]
x4        = [54.5981500331442220, 63.0866812956689813]
x × x × x × x = [54.5981500331442149, 63.0866812956689884]
```

Finally, an interval including zero:

```
x          = [ -2.7182818284590451, 2.8182818284590452]
x3        = [-20.0855369231876679, 22.3848022077206359]
x × x × x  = [-21.5905309612583913, 22.3848022077206359]
x4        = [ 0.0000000000000000, 63.0866812956689813]
x × x × x × x = [-60.8482010748969273, 63.0866812956689884]
```

The output uses a format specifier of `%0.16f` decimal point. Repeated multiplications lead to intervals that are a little too large compared to exponentiation. In particular, note that multiplying an interval including zero by itself an even number of times results in an interval including a negative lower bound, making the interval much wider than it needs to be.

Other Operations. Let's add a few more operations to our library. The new operations are: negation, absolute value, comparison for equality, comparison for less than or equal, and a concept of "distance" between two intervals. Code comes first, followed by examples beginning with negation:

```

1 interval_t int_neg(interval_t x) {
2     interval_t ans;
3     fesetround(FE_DOWNWARD);
4     ans.a = -x.b;
5     fesetround(FE_UPWARD);
6     ans.b = -x.a;
7     fesetround(FE_TONEAREST);
8     return ans;
9 }

```

The code implements $-\underline{x}, \bar{x}] = [-\bar{x}, -\underline{x}]$ which is always true because the lower bound of an interval is, by definition, less than (or equal) to the upper bound. Therefore, changing the sign of these values means that their order must be reversed to keep the lower bound less than the upper.

The absolute value of an interval depends on the signs of the bounds. Mathematically, we have

$$\begin{aligned}
 |x| &= [\min\{|\underline{x}|, |\bar{x}|\}, \max\{|\underline{x}|, |\bar{x}|\}], \text{ if } \underline{x}\bar{x} \geq 0 \\
 &= [0, \max\{|\underline{x}|, |\bar{x}|\}], \text{ if } \underline{x}\bar{x} < 0
 \end{aligned}$$

where we maximize the interval if the signs of the bounds are the same ($\underline{x}\bar{x} \geq 0$) or we set the lower bound to zero if the interval includes zero ($\underline{x}\bar{x} < 0$). In code,

```

1 interval_t int_abs(interval_t x) {
2     interval_t ans;
3     if (x.b * x.a >= 0) {
4         fesetround(FE_DOWNWARD);
5         ans.a = (fabs(x.b) < fabs(x.a)) ? fabs(x.b) : fabs(x.a);
6         fesetround(FE_UPWARD);
7         ans.b = (fabs(x.b) < fabs(x.a)) ? fabs(x.a) : fabs(x.b);
8     } else {
9         ans.a = 0.0;
10        fesetround(FE_UPWARD);
11        ans.b = (fabs(x.b) < fabs(x.a)) ? fabs(x.a) : fabs(x.b);
12    }
13    fesetround(FE_TONEAREST);
14    return ans;
15 }

```

Line 3 compares the signs of the bounds. If they are the same, the minimum is used for the lower bound (line 5), and the maximum is used for the upper bound (line 7) with proper rounding modes. If the interval includes zero, we move to line 9 and set the output lower bound to zero and the upper bound to the maximum (line 11). We reset to the default rounding mode and return the answer in line 14.

So far, our interval library contains only arithmetic operations. Let's add comparisons, namely, tests for equality and less than or equal. Equality is straightforward: two intervals are equal if their bounds are equal. Symbolically, this is represented as

$$[\underline{x}, \bar{x}] = [\underline{y}, \bar{y}] \text{ iff } (\underline{x} = \underline{y}) \wedge (\bar{x} = \bar{y})$$

where $\wedge$ is conjunction or “logical and”.

For less than or equal, we compare both the bounds for less than or equal like so,

$$[\underline{x}, \bar{x}] \leq [\underline{y}, \bar{y}] \text{ iff } (\underline{x} \leq \underline{y}) \wedge (\bar{x} \leq \bar{y})$$

Why implement just equal and less than or equal? These relations are enough to define the others as needed and are called out explicitly in [7], which points towards a standard for interval arithmetic.

The distance between two scalars, x and y , is the absolute value of their difference, $|x - y|$. The distance between two intervals is

$$\text{dist}(x, y) = \max\{|\underline{x} - \underline{y}|, |\bar{x} - \bar{y}|\}$$

where $\text{dist}(x, y)$ measures the degree to which two intervals “overlap”. The larger this value, the more dissimilar the intervals. In code, this becomes

```

1 unsigned char int_eq(interval_t x, interval_t y) {
2     return (x.a==y.a) && (x.b==y.b);
3 }
4
5 unsigned char int_le(interval_t x, interval_t y) {
6     return (x.a <= y.a) && (x.b <= y.b);
7 }
8
9 double int_dist(interval_t x, interval_t y) {
10     double a,b;
11     a = fabs(x.a - y.a);
12     b = fabs(x.b - y.b);
13     return (a>b) ? a : b;
14 }
```

Line 2 compares the bounds for equality, and line 6 compares them for less than or equal. The function `int_dist()` first computes $|\underline{x} - \underline{y}|$ (line 11) and $|\bar{x} - \bar{y}|$ (line 12) and then returns the larger of the two in line 13.

It's time for some quick examples before moving on to Python. Consider two overlapping intervals:

```

x = [-2.7182818284590451, 2.8182818284590452]
y = [ 2.5182818284590449, 2.9182818284590453]
```

our new library functions return:

```

int_neg(x)      = [-2.8182818284590452, 2.7182818284590451]
int_abs(x)      = [ 0.0000000000000000, 2.8182818284590452]
int_eq(x,y)     = 0 (false)
int_le(x,y)     = 1 (true)
int_dist(x,y)   = 5.2365636569180900

```

The Python Version. Let's implement an interval class in Python. We will only implement the basic arithmetic operations and leave extending the class to the exercises. The key to interval arithmetic is controlling the floating-point rounding mode. This is easily accomplished in C by including the `fenv.h` header file and compiling with the `-frounding-math` option. However, the Python interpreter is already compiled and does not offer direct access to the constants and functions defined in `fenv.h`. Fortunately, everything is not lost; we can still access the underlying C library containing these functions and use them to set the rounding mode as necessary. For this trick, we thank Rafael Barreto for his blog post [8], which describes how to do this to set the rounding mode. Jumping in, then, we begin with

```

1 from math import exp
2 from ctypes import cdll
3 from ctypes.util import find_library
4
5 class Interval:
6     libc = cdll.LoadLibrary(find_library("m"))
7     FE_TOWARDZERO = 0xc00
8     FE_DOWNWARD = 0x400
9     FE_UPWARD = 0x800
10    FE_TONEAREST = 0
11
12    def __init__(self, a,b=None):
13        if (b == None):
14            self.a = a
15            self.b = a
16        else:
17            self.a = a
18            self.b = b

```

Lines 2 and 3 import standard Python libraries, giving us access to the C library to call the rounding functions. Line 1 imports the exponential function, which we will use to generate infinity. The class starts on line 5. Line 6 sets a class-level variable, `libc`, to the C library, which on Linux systems can be found by searching for “m” (hence `-lm` when compiling). We also define the rounding modes as class-level variables. The specific values were determined by a short C program which displayed them after including `fenv.h`. The constructor for the class starts on line 12 and takes up to two arguments. If only one is given, the argument is assumed to be a real number and an interval is created from this number. Doing this simplifies the class by removing the need to consider mixed interval and float operations. If both

arguments are given, they are assumed to be the lower (a) and upper (b) bounds of the interval.

Next, we need methods to set the rounding mode:

```
1 def __RoundDown(self):
2     self.libc.fesetround(self.FE_DOWNWARD)
3
4 def __RoundUp(self):
5     self.libc.fesetround(self.FE_UPWARD)
6
7 def __RoundNearest(self):
8     self.libc.fesetround(self.FE_TONEAREST)
```

where `__RoundDown()`, `__RoundUp()`, and `__RoundNearest()` are wrappers for calls to the C `fesetround()` function. Note the double underscore prefix as Python notation for “private”. We will call these methods whenever we need to change the rounding mode.

Let’s define addition and subtraction by overloading `+` and `-`,

```
1 def __add__(self, y):
2     self.__RoundDown()
3     a = self.a + y.a
4     self.__RoundUp()
5     b = self.b + y.b
6     self.__RoundNearest()
7     return Interval(a,b)
8
9 def __sub__(self, y):
10    self.__RoundDown()
11    a = self.a - y.b
12    self.__RoundUp()
13    b = self.b - y.a
14    self.__RoundNearest()
15    return Interval(a,b)
```

The methods, `__add__()` and `__sub__()`, are called when objects of the `Interval` class are encountered in arithmetic expressions involving `+` and `-`. We are not implementing mixed operations, so both operands of `+` or `-` must be `Interval` objects. Notice that the rounding modes are adjusted as necessary.

Multiplication is next (`__mul__`). The definition is a direct port of the C version. As with `+` and `-`, the result is a new `Interval` object:

```
1 | def __mul__(self, y):
2 |     self.__RoundDown()
3 |     a = min([self.a*y.a, self.a*y.b, self.b*y.a, self.b*y.b])
4 |     self.__RoundUp()
5 |     b = max([self.a*y.a, self.a*y.b, self.b*y.a, self.b*y.b])
6 |     self.__RoundNearest()
7 |     return Interval(a,b)
```

Line 2 sets rounding towards $-\infty$ before calculating all the products between the two intervals. It then selects the smallest of these for the lower bound. Line 4 changes to rounding towards $+\infty$ and repeats the calculation, this time selecting the maximum value. We must perform the multiplications twice because we changed the rounding mode. Line 6 returns to default rounding, and the new object is returned in line 7.

Division, reciprocals, and powers are all that remain to be implemented. Division is multiplication by the reciprocal, as before

```
1 | def recip(self):
2 |     if (self.a < 0.0) and (self.b > 0.0):
3 |         return Interval(-exp(1000.0), exp(1000.0))
4 |     self.__RoundDown()
5 |     a = 1.0 / self.b
6 |     self.__RoundUp()
7 |     b = 1.0 / self.a
8 |     self.__RoundNearest()
9 |     return Interval(a,b)
10 |
11 | def __truediv__(self, y):
12 |     return self.__mul__(y.recip())
```

The reciprocal method, not associated with an operator but not private, is used to return the reciprocal of the interval. If the interval contains zero (line 2), we return $[-\infty, +\infty]$ (line 3). Otherwise, we round down and calculate the lower bound, round up, calculate the upper bound, and finally return the new reciprocal `Interval`.

Our final method is raising an interval to a positive integer power. Again, we mimic the C code:

```

1      def __pow__(self, n):
2          if (n%2) == 1:
3              self.__RoundDown()
4              a = self.a**n
5              self.__RoundUp()
6              b = self.b**n
7          elif (self.a >= 0):
8              self.__RoundDown()
9              a = self.a**n
10             self.__RoundUp()
11             b = self.b**n
12          elif (self.b < 0):
13              self.__RoundDown()
14              a = self.b**n
15              self.__RoundUp()
16              b = self.a**n
17          else:
18              a = 0.0
19              self.__RoundUp()
20              b = self.a**n
21              t = self.b**n
22              if (t > b):
23                  b = t
24          self.__RoundNearest()
25          return Interval(a,b)

```

with each condition checked as before. Note that `**` is exponentiation in Python.

Let's review some examples using an interactive session where `>>>` is the Python prompt followed by user input. Python output does not have the prompt and has been slightly intended for clarity:

```

>>> from Interval import *
>>> from math import sqrt
>>> x = Interval(-sqrt(2), sqrt(2)+0.1)
>>> x; x*x; x**2
[-1.4142135623730951, 1.5142135623730952]
[-2.1414213562373101, 2.2928427124746200]
[0.0000000000000000, 2.2928427124746200]
>>> x*x*x; x**3
[-3.2425692603699230, 3.4718535316173851]
[-2.8284271247461903, 3.4718535316173846]
>>> x = Interval(sqrt(2), sqrt(2)+0.01)
>>> y = x**2
>>> x/y
[0.6972118559681739, 0.7121067811865476]
>>> y/x
[1.4042837765619229, 1.4342842730512142]

```

We first import the library and the square root function. We then define an interval, x , and show it along with $x \times x$ and x^2 . Notice that the former gives an interval that is too large in this case because the interval contains zero. The more correct interval is

the one returned by x^2 . Similar code multiplies x by itself three times and compares the result to calling the exponential. The multiplication leads to an interval that is too large. Re-defining x as positive allows division.

This section explored and implemented basic interval arithmetic in C and Python. Next, we consider functions of intervals. Here is where we encounter the biggest reason interval arithmetic is not widely used.

Properties of Intervals. Let's discuss interval properties. The first few are familiar and mimic those of normal arithmetic. However, things become more interesting when we get to the distributive property.

For any three intervals, $x = [\underline{x}, \bar{x}]$, $y = [\underline{y}, \bar{y}]$, and $z = [\underline{z}, \bar{z}]$, the following are true:

$$\begin{aligned}
 (x + y) + z &= x + (y + z) && (\text{associative}) \\
 (x - y) - z &= x - (y - z) \\
 x + y &= y + x && (\text{commutative}) \\
 x \times y &= y \times x \\
 x + [0, 0] &= x && (\text{additive identity}) \\
 x \times [1, 1] &= x && (\text{multiplicative identity})
 \end{aligned}$$

However, the distributive property causes problems. In general, for intervals,

$$x \times (y + z) \neq xy + xz$$

For example, let $x = [1, 2]$, $y = [1, 2]$, and $z = [-2, -1]$, then

$x \times (y + z)$	$xy + xz$
$[1, 2] \times ([1, 2] + [-2, -1])$	$[1, 2] \times [1, 2] + [1, 2] \times [-2, -1]$
$[1, 2] \times [-1, 1]$	$[1, 4] + [-4, -1]$
$[-2, 2]$	$[-3, 3]$

The results are unexpected. This property is called *subdistributivity* and implies that for intervals, we have

$$x \times (y + z) \subseteq x \times y + x \times z$$

which is seen clearly in the example because $[-2, 2] \subseteq [-3, 3]$ as it is a tighter bound. Therefore, in practice, we must be careful with the form of the expression we intend to evaluate.

8.3 Functions and Intervals

Now that we know how to implement intervals, we can use them to ensure tight, accurate bounds on our calculations. Or not. We have yet to contemplate general functions of intervals. As we'll soon learn, there is a large elephant in the living room.

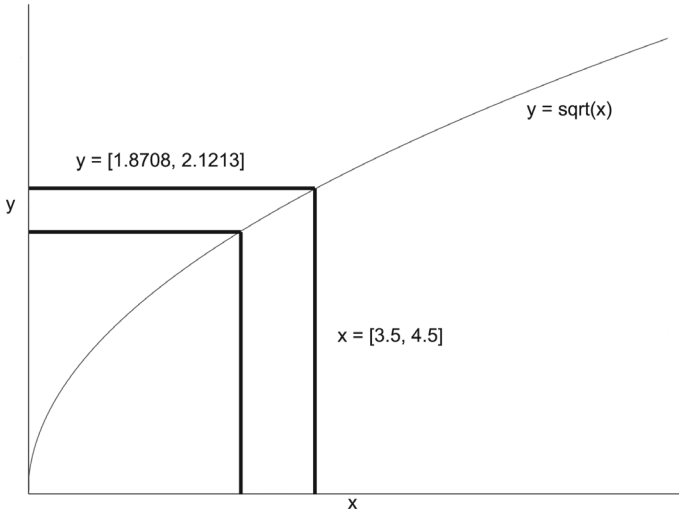


Fig. 8.1 Since the function $y = \sqrt{x}$ is always increasing for a positive input, the bounds on any output interval are simply the function values at the lower and upper bounds of the input interval, as shown for the interval $x = [3.5, 4.5]$. This leads to the proper output interval of approximately $[1.8708, 2.1213]$. If the function were monotonically decreasing over this interval, the output would be the same, but with the upper and lower bounds swapped so that the interval has the right ordering

If a function of one variable is monotonic over a range, the function, $f(x)$, is either increasing or decreasing over the range. If this is the case, then interval arithmetic will work as expected because, for the interval range, the function will maintain (or reverse) the ordering of the bounds. For example, consider the square root of positive intervals. Since the square root is monotonically increasing over any positive interval, the output of $\sqrt{x}$ for an interval x will also increase, as shown in Fig. 8.1.

Because of this consistent increase, the tightest output interval is simply $[\sqrt{\underline{x}}, \sqrt{\bar{x}}]$. If the function is monotonically decreasing over the interval, the result is similar but with the bounds flipped to maintain a proper ordering. Therefore, in general, for monotonically increasing functions of one variable ($f(x)$), we have

$$f(x) = f([\underline{x}, \bar{x}]) = [f(\underline{x}), f(\bar{x})]$$

while for monotonically decreasing functions ($g(x)$) we have

$$g(x) = g([\underline{x}, \bar{x}]) = [g(\bar{x}), g(\underline{x})]$$

It should be noted that many elementary functions are of this form, including the exponential function, roots (e.g., cube root), and logarithms. For exponentials and logarithms, we can define the output explicitly

$$\begin{aligned} a^{[\underline{x}, \bar{x}]} &= [a^{\underline{x}}, a^{\bar{x}}], \quad a > 1 \\ \log_a [\underline{x}, \bar{x}] &= [\log_a \bar{x}, \log_a \underline{x}] \end{aligned}$$

However, what about functions that are not monotonically increasing or decreasing? In this case, life becomes more difficult. Consider the sine function. It alternates between increasing and decreasing. How do we work with functions like this?

One possibility is to use a Taylor series expansion. This is the solution presented in Daumas et al. [9] in terms of bounded functions using a parameter n to control the bounds where in the limit $n \rightarrow \infty$ the bounds tighten to the actual value of the function. Specifically, for the sine function [9] defines the output interval to be

$$[\sin(x)]_n = [\underline{\sin}(x, n), \overline{\sin}(x, n)], \quad x \subseteq \left[\frac{-\pi}{2}, \frac{\pi}{2} \right]$$

for a specified value of n . We'll learn shortly what a good value of n might be. The functions $\underline{\sin}(x, n)$ and $\overline{\sin}(x, n)$ are themselves defined in terms of Taylor series parametrized by n and operating on scalar values (the bounds of x). The series expansions are

$$\underline{\sin}(x, n) = \sum_{i=1}^m (-1)^{i-1} \frac{x^{2i-1}}{(2i-1)!}$$

$$\overline{\sin}(x, n) = \sum_{i=1}^{m+1} (-1)^{i-1} \frac{x^{2i-1}}{(2i-1)!}$$

where $m = 2n + 1$, $x \geq 0$, otherwise, $m = 2n$. In [9], the value of π itself is defined as an interval, but for our purposes, we will treat it as an exact quantity and note that the argument to our interval sine is restricted to $[-\frac{\pi}{2}, \frac{\pi}{2}]$.

Examining the series expansions and the dependency upon the summation limit as a function of n reveals the cleverness in the definition. The bounds of the sine come from the number of terms used in the summation, with the lower bound set by $m(n)$ and the upper set by $m(n) + 1$. Additionally, as $n \rightarrow \infty$, it becomes clear that the series will converge to the sine of x .

Let's add this function to our interval library. The implementation is straightforward, requiring a top-level function:

```

1 interval_t int_sin(interval_t x) {
2     int n=8;
3     interval_t ans;
4
5     fesetround(FE_DOWNWARD);
6     ans.a = int_sin_lo(x.a, n);
7     fesetround(FE_UPWARD);
8     ans.b = int_sin_hi(x.b, n);
9     fesetround(FE_TONEAREST);
10    return ans;
11 }
```

The function `t_int_sin` follows our standard format of setting the rounding mode appropriately before calculating a bound. Lines 6 and 8 set the output value by calling

helper functions to compute the series value for the lower and upper bounds. Note that line 2 sets n to eight. This value can be adjusted, but eight gives good results. The helper functions are

```

1 double fact(int n) {
2     return (n==1) ? 1.0 : n*fact(n-1);
3 }
4
5 double int_sin_lo(double x, int n) {
6     int i, m = (x < 0) ? 2*n : 2*n+1;
7     double ans=0.0;
8
9     for(i=1; i <= m; i++) {
10         ans += pow(-1,i-1) * pow(x,2*i-1) / fact(2*i-1);
11     }
12     return ans;
13 }
14
15 double int_sin_hi(double x, int n) {
16     int i, m = (x < 0) ? 2*n : 2*n+1;
17     double ans=0.0;
18
19     for(i=1; i <= m+1; i++) {
20         ans += pow(-1,i-1) * pow(x,2*i-1) / fact(2*i-1);
21     }
22     return ans;
23 }
```

where `int_sin_lo()` and `int_sin_hi()` call `fact()` to compute the factorial. We use the recursive algorithm for simplicity, noting that we require less than a dozen terms in the series. The functions themselves are a literal translation of the formulas with m set appropriately (lines 6 and 16). Note also that for `int_sin_hi()`, we add one to m in the loop limit. Why do we not combine these two functions? If `int_sin_hi()` needs to calculate only one more term in the series expansion than `int_sin_lo()` why calculate everything twice? The answer lies in the guarantee implicit in intervals implemented in floating point, namely, that the bounds on the interval will be certain to include the correct answer. When `int_sin_lo()` is called from `int_sin()` the floating-point rounding mode has been set towards $-\infty$. Therefore, all calculations round in that direction, assuring us that rounding errors will not shrink the interval. Likewise, when `int_sin_hi()` is called the rounding mode is towards $+\infty$.

Let's compare the output of our sine function to the "gold standard" present in the C math library. Let,

$$x = [0.7852981625, 0.7854981625]$$

where the difference between the bounds is underlined. Taking the sine gives

$$\sin(x) = [0.7070360663383567, 0.7071774876944870] \text{ (interval sine)}$$

while the `sin()` function from `math.h` produces the interval:

$$\sin(x) = [0.70703606633835\underline{76}, 0.70717748769448\underline{62}] \text{ (math.h sine)}$$

Examining the underlined digits of each interval reveals that our sine function does indeed produce an interval that is sure to encompass the gold standard value.

The formulas for the interval cosine are

$$[\cos(x)]_n = [\underline{\cos}(x, n), \overline{\cos}(x, n)], \quad x \subseteq [0, \pi]$$

and

$$\begin{aligned} \underline{\cos}(x, n) &= 1 + \sum_{i=1}^{m+1} (-1)^i \frac{x^{2i}}{(2i)!} \\ \overline{\cos}(x, n) &= 1 + \sum_{i=1}^m (-1)^i \frac{x^{2i}}{(2i)!} \end{aligned}$$

where, again, $m = 2n + 1$, $x \geq 0$, otherwise, $m = 2n$. The implementation in C and Python is left to the exercises.

We have examined interval versions of elementary functions. What about general functions of x ? Here's where the elephant shows itself. Applying interval arithmetic to general functions leads to the *dependency problem* (or *dependency effect*), which is the bane of intervals.

The problem reveals itself when applying intervals to general functions where the interval variable appears more than once in the expression. The most commonly cited example is evaluating the expression $x - x^2$ for $x = [0, 1]$. Another way to write this expression is $x(1 - x)$. Let's compare both approaches:

$x(1-x)$	$x-x^2$
$[0,1] \times (1 - [0,1])$	$[0,1] - [0,1]^2$
$[0,1] \times [0,1]$	$[0,1] - [0,1]$
$[0,1]$	$[-1,1]$

We have arrived at differing answers. We also have seen that the interval for $x(1-x) \subseteq x-x^2$ as expected due to subdistributivity. However, while neither answer is technically incorrect, since the proper answer lies inside each interval, neither is satisfactory because of the dependency problem. The form of the answer depends on the form of the expression, even though both are mathematically equivalent.

The expression we are evaluating is a downward-facing parabola with a maximum value of 0.25 at $x = \frac{1}{2}$ and zeros at $x = 0$ and $x = 1$. Therefore, over the given range, $x = [0, 1]$, we would expect the output interval to be $[0, \frac{1}{4}]$. While this interval is

within the range of both intervals above, we do not find the narrow interval unless we remove the dependency problem by rewriting the expression so that x appears only once. Completing the square gives $\frac{1}{4} - (\frac{1}{2} - x)^2$, where x appears only once. Evaluating this expression over $x = [0, 1]$ returns

$$\begin{aligned}\frac{1}{4} - \left(\frac{1}{2} - x\right)^2 &= 0.25 - (0.5 - [0, 1])^2 \\ &= 0.25 - [0, 0.25] \\ &= [0, 0.25]\end{aligned}$$

which is the narrowest interval in this case.

The dependency problem is likely the primary reason interval arithmetic is not widely used. It is often impossible to rewrite an expression so the variable appears only once. When this is the case, it is possible to introduce auxiliary intervals whose union results in the original interval and then apply the expression to each. This *interval splitting* will result in less of a dependency problem. For example, [9] includes the expression $2x - x$ for $x = [0, 1]$. If we evaluate the expression directly, we get $[-1, 2]$ as the answer when the narrowest interval is naturally $[0, 1]$. The observation key to interval splitting is that while $x = [0, 1]$ does not lead to the narrowest interval, we can split the interval into two parts, $x_1 = [0, 0.5]$ and $x_2 = [0.5, 1]$, and evaluate each of these in turn. In that case, we arrive at $[-0.5, 1]$ for x_1 and $[0, 1.5]$ for x_2 . The union of these two intervals is $[-0.5, 1.5]$, which is closer to the expected answer of $[0, 1]$. As the intervals are further subdivided, the resulting union approaches $[0, 1]$. While interval splitting can help with the dependency problem, it introduces significant computational overhead.

8.4 Implementations

This section contemplates two implementations of interval arithmetic, one for C and the other for Python. Perhaps the most widely used library for C is the Multiple Precision Floating-point Interval (MPFI) library [10].

Using MPFI requires us to install GMP (<http://gmplib.org/>) and MPFR (<http://www.mpfr.org/>). Prerequisites in place, download MPFI (<http://perso.ens-lyon.fr/nathalie.revol/software.html>) and install it.

Let's begin with a simple example that defines two intervals and then prints them. Doing so will teach us how to define and initialize intervals and how to print them. Please note that there is an extensive API for this library and that we are only using a small part of it. Our initial program is

```

1  #include <stdio.h>
2  #include <mpfr.h>
3  #include <mpfi.h>
4  #include <mpfi_io.h>
5
6  int main() {
7      mpfi_t x, y;
8
9      mpfr_set_default_prec(64);
10
11     mpfi_init(x);
12     mpfi_set_str(x, "[-1.0, 1.0]", 10);
13
14     mpfi_init_set_str(y, "[0,2]", 10);
15
16     printf("x = ");
17     mpfi_out_str(stdout, 10, 0, x);
18     printf("\n");
19
20     printf("y = ");
21     mpfi_out_str(stdout, 10, 0, y);
22     printf("\n");
23
24     mpfi_clear(x);
25     mpfi_clear(y);
26 }

```

Lines 1 through 4 include necessary header files. Note that the documentation for MPFI recommends including `stdio.h` before the remaining header files. Line 7 declares two interval variables, x and y . They are declared but not initialized.

Line 9 sets the default precision to 64 bits to mimic a `double`. Note that this is a call to the MPFR library. Lines 11 and 12 show one way to initialize an interval variable and set its value using a string. We will use strings to set values, though the API has many options. For `mpfi_set_str()`, the first argument is the variable, the second is a string that defines the interval in the way we have been displaying them throughout this chapter, and the third argument is the base in which the interval is defined (2 through 36). Line 14 combines lines 11 and 12 into a single initialize and set function. The arguments are the same as line 12. We then display the intervals using `mpfi_out_str()` in lines 17 and 21. The first argument is the output file stream, here `stdout`, the second is the base, the third is the number of digits (0 means use the precision of the variable), and the fourth is the interval variable. Finally, lines 24 and 25 clear the memory used by x and y . While not necessary in this case because the program is about to exit, it is necessary before re-assignment.

Compile the program with:

```
gcc mpfi_ex.c -o mpfi_ex -lmpfi -lmpfr
```

assuming `mpfi_ex.c` to be the file name. Note that MPFI installs itself in `/usr/local/lib`, which might require adding that path to the `LD_LIBRARY_PATH` environment variable.

We know how to initialize, set, and display intervals in MPFI. Let's examine basic arithmetic functions. These are

```
int mpfi_add(mpfi_t ROP, mpfi_t OP1, mpfi_t OP2)
int mpfi_sub(mpfi_t ROP, mpfi_t OP1, mpfi_t OP2)
int mpfi_mul(mpfi_t ROP, mpfi_t OP1, mpfi_t OP2)
int mpfi_div(mpfi_t ROP, mpfi_t OP1, mpfi_t OP2)
```

where `ROP` is the variable to receive the result and `OP1` and `OP2` are the operands. Note that division using an interval containing zero will result in $[-\infty, +\infty]$. The return value indicates whether or not the endpoints of the interval are exact.

Let's revisit the dependency problem example as implemented in MPFI. The example computes $x(1 - x)$ and $x - x^2$ for $x = [0, 1]$:

```
1 int main() {
2     mpfi_t x, one, t, y;
3
4     mpfr_set_default_prec(64);
5     mpfi_init_set_str(x, "[0, 1]", 10);
6     mpfi_init_set_str(one, "[1,1]", 10);
7
8     mpfi_init(t);
9     mpfi_sub(t, one, x);
10    mpfi_init(y);
11    mpfi_mul(y, t, x);
12
13    printf("y = ");
14    mpfi_out_str(stdout, 10, 0, y);
15    printf("\n");
16
17    mpfi_clear(y);
18    mpfi_clear(t);
19    mpfi_clear(one);
20
21    mpfi_init(t);
22    mpfi_mul(t, x, x);
23    mpfi_init(y);
24    mpfi_sub(y, x, t);
25
26    printf("y = ");
27    mpfi_out_str(stdout, 10, 0, y);
28    printf("\n");
29 }
```

where we notice that it is necessary to preserve the intermediate temporary variables so that they can be released properly without leaking memory. Specifically, line 5 initializes x while line 6 initializes the constant interval, `one`. The expression $x(1 - x)$ is calculated in lines 8 through 11. First, $t = 1 - x$ in line 9, and then the final answer

as $y = t \times x$ in line 11 and output in line 14. Lines 17 through 19 release memory, allowing us to reuse y and t to calculate $x - x^2$. This expression is in lines 21 through 24 and output in line 27. Running the code produces

```
y = [0,1.00000000000000000000000000000000]
y = [-1.00000000000000000000000000000000,1.00000000000000000000000000000000]
```

This is precisely what we expect for $x(1 - x)$ and $x - x^2$.

Interval arithmetic is available in Python via the `mpmath` [11] library. This library is implemented in pure Python and is quite powerful. As with MPFI, we will only contemplate the simplest of use cases. To install `mpmath` on Ubuntu use the command:

```
pip3 install mpmath
```

The following console session demonstrates `mpmath`'s basic interval abilities:

```
1 >>> from mpmath import mpi
2 >>> x = mpi('0', '1')
3 >>> print(x)
4     [0.0, 1.0]
5 >>> print(x*(1-x))
6     [0.0, 1.0]
7 >>> print(x-x**2)
8     [-1.0, 1.0]
9 >>> y = mpi('-1', '1')
10 >>> print(y**2)
11     [0.0, 1.0]
12 >>> print(y*y)
13     [-1.0, 1.0]
14 >>> print(1/y)
15     [-inf, +inf]
16 >>> print(1/x)
17     [1.0, +inf]
```

Line 1 imports the interval module (`mpi`) from the larger `mpmath` library. Line 2 defines a new interval and stores it in x . Note that the bounds are two separate arguments and may be given as strings or numbers. If given as numbers, they are interpreted as Python `floats` (64 bits), while strings are interpreted as exact decimals. Line 5 prints $x(1 - x)$ with the expected output and line 7 prints $x - x^2$, again with the expected output. Line 9 defines y as an interval containing zero. Lines 10 through 15 display the results of several simple calculations on y . Note that y^2 (line 10) returns an interval different than $y \times y$ (line 12) but positive, as in our C library above. Note, also, that line 14 returns $[-\infty, +\infty]$ as the reciprocal of y since it contains zero, while line 16 prints the reciprocal of x , which has zero as a lower bound. In this case, the resulting interval is $[0, +\infty]$.

The bounds of an interval are acquired by the `a` and `b` properties on the object. Additionally, there are also `mid` and `delta` properties for the midpoint of an interval and the difference between the endpoints:

```
1 >>> from mpmath import mpi
2 >>> x = mpi('0.41465', '0.5')
2 >>> x.a
2      mpi('0.41464999999999996', '0.41464999999999996')
2 >>> x.b
2      mpi('0.5', '0.5')
2 >>> x.mid
2      mpi('0.45732499999999998', '0.45732499999999998')
2 >>> x.delta
2      mpi('0.085350000000000037', '0.085350000000000037')
```

Use `x.a` to return the lower bound and `x.b` to return the upper. Use `x.mid` for the middle position between the lower and upper bounds and `x.delta` for the interval range. Notice that the return values are intervals even if the lower and upper bounds are the same.

8.5 Thoughts on Interval Arithmetic

Interval arithmetic is most useful when uncertainty is part of the data. With intervals and proper rounding, we have assurances that the correct answer lies within a particular range. Intervals work well for basic computations, even if they may involve significantly more low-level calculations than floating-point numbers. For that reason, unless truly necessary, intervals are generally not recommended in situations where high performance is required.

Floating-point arithmetic is standardized via the IEEE 754 standard. Interval arithmetic is standardized via IEEE 1788-2015. A summary of interval arithmetic, along with a discussion of its uses in scientific and mathematical research, may be found in [13].

Because of the difficulty in adequately applying interval arithmetic to general functions and expressions, one must be careful and thoughtful when using intervals to avoid falling into a situation where the uncertainty grows too large. When that happens, the intervals are so broad as to be virtually meaningless. Recall that, unlike standard floating-point computation, the interval $[-\infty, +\infty]$ is a valid return value for any interval expression, albeit not a particularly useful one.

8.6 Summary

This chapter took a practical look at interval arithmetic. We defined what we mean by intervals and interval arithmetic, summarized the history of intervals, and explained basic operations on intervals. We implemented basic interval libraries in C and Python. We then discussed the dependency problem with examples and mentioned interval splitting as a technique for minimizing the dependency problem. Lastly, we perused popular implementations of interval arithmetic and offered some thoughts on its utility.

Exercises

8.1 Add `int_lt()`, `int_gt()`, `int_ge()`, and `int_ne()` functions to test for $<$, $>$, $\geq$, and $\neq$ using `int_eq()` and `int_le()`. **

8.2 Add exponential (e^x), square root ($\sqrt{x}$), and logarithm functions (any base) to the C interval library. **

8.3 The Python interval class contains basic operations. Complete the class by adding the missing operations found in the C library. **

8.4 Extend the Python interval library to support mixed scalar and interval arithmetic for $+$, $-$, $\times$, and $/$. (Hint: Look at how methods like `__radd__()` might be used) **

8.5 Extend the Python `__pow__()` method to properly handle negative integer exponents.

8.6 Extend the C library by adding cosine using the formulas in Sect. 8.3. *

8.7 Extend the Python class by adding sine and cosine using the formulas in Sect. 8.3. *

8.8 Add interval splitting to the Python interval class to minimize the dependency problem. Automatically split intervals into n parts and evaluate a function on each of those parts, reporting their union when complete. ***

References

1. Bevington PR, Robinson DK (1969) Data reduction and error analysis for the physical sciences, vol 336. McGraw-Hill
2. Young RC (1931) The algebra of multi-valued quantities. *Math Ann* 104:260–290
3. Dwyer PS (1951) Linear computations. Wiley, New York
4. Sunaga T (2009) Theory of interval algebra and its application to numerical analysis. In: Research association of applied geometry (RAAG) memoirs, Ggujutsu Bunken Fukuy-kai. Tokyo, Japan, 1958, vol 2, pp 29–46 (547–564); reprinted in *Japan J Indust Appl Math* 26(2–3):126–143
5. Moore RE (1959) Automatic error analysis in digital computation. Technical report space div. Report LMSD84821, Lockheed Missiles and Space Co
6. Hickey T, Ju Q, van Emden MH (2001) Interval arithmetic: from principles to implementation. *J ACM (JACM)* 48(5):1038–1068
7. Bohlender G, Kulisch U (2011) Definition of the arithmetic operations and comparison relations for an interval arithmetic standard. *Reliable Comput* 15:37
8. Barreto R (2014) Controlling FPU rounding modes with Python. <http://rafaelbarreto.wordpress.com/2009/03/30/controlling-fpu-rounding-modes-with-python/> (Accessed 07 Nov 2014)
9. Dumas M, Lester D, Muoz C (2009) Verified real number calculations: a library for interval arithmetic. *IEEE Trans Comput* 58(2):226–237
10. Revol N, Rouillier F (2002) Multiple precision floating-point interval library. <http://perso.ens-lyon.fr/nathalie.revol/software.html> (Accessed 15 Nov 2014)
11. Johansson F et al (2014) mpmath: a Python library for arbitrary-precision floating-point arithmetic (version 0.14), February 2010. <http://code.google.com/p/mpmath> (Accessed 16 Nov 2014)
12. Kearfott R (2013) An overview of the upcoming IEEE P-1788 working group document: standard for interval arithmetic. IFSA/NAFIPS
13. Kearfott R (1996) Interval computations: introduction, uses, and resources. *Euromath Bull* 2(1):95–112



Arbitrary-Precision Floating Point

9

Abstract

Floating-point numbers use a fixed number of bits, limiting the precision with which they can represent real numbers. In this chapter, we examine arbitrary precision, also called multiple-precision, floating-point numbers. We learn how they are represented in memory and how basic arithmetic works. We end with an exploration of arbitrary-precision floating-point libraries and offer some thoughts on when to consider arbitrary-precision floating point.

9.1 What is Arbitrary-Precision Floating Point?

By “arbitrary-precision floating point”, we mean a floating-point representation that can vary the level of precision to suit any calculation. Unlike arbitrary-sized integers, which can grow without bound (as long as there is memory), arbitrary-precision floating point generally fixes the precision before calculations begin. The “arbitrary” label is appropriate, however, because we can set the precision as necessary to maintain any level of accuracy (again, assuming memory allows).

9.2 Representing Arbitrary-Precision Floating-Point Numbers

There are two main approaches to representing arbitrary-precision floating-point numbers: extend the sign-significand-exponent approach to a large number of digits in the significand or use a fixed-point representation with a large number of digits in the fractional part. In both cases, the user can set the precision, either the number of digits in the significand or in the fractional part. The base can be binary (most

efficient) or decimal (better accuracy, slower). The libraries that we will examine below use the exponent approach and represent numbers as

$$s \times b_{n-1}b_{n-2} \dots b_0 \times 2^e$$

where s is the sign, e is an integer exponent, potentially large, and b is the binary significand of a preset precision (n bits) with an implied leading “1” digit. In this case, arithmetic proceeds as in IEEE 754 floating point but in software so that the significand can be arbitrarily large.

Our small implementation of arbitrary-precision floating point in Python will use a fixed-point representation and make copious use of the fact that Python natively supports infinite precision integers. We will also freely abuse the term “floating point” and use it in an expanded sense, including fixed-point representations as a form of floating point.

As we learned in Chap. 6, fixed-point representations make excellent use of integer arithmetic. Specifically, we will represent floating-point numbers in base 10 with a given number of digits in the decimal part. The integer part can be arbitrarily large and will be handled automatically by Python. We will set the number of decimal digits in the fractional part before beginning calculations to simplify the code and then proceed using that precision. So, if we set the precision to 8 digits, we will represent numbers as

$$\begin{array}{rcl} 10.93210032 & \rightarrow & 10\ 93210032 \\ -10.9320122356 & \rightarrow & -10\ 93201224 \\ 0.003423 & \rightarrow & 342300 \end{array}$$

We store the sign separately from the magnitude and have added a space on the right to differentiate between the integer portion and the scaled fractional part. Note also that since the precision is 8 digits after the decimal, the second example, with 10 digits after the decimal, is rounded. To return to the actual floating-point number, we need to divide the stored value by 10^d where d is the precision of the fractional part. For the example above, $d = 8$. In practice, we will convert to a float by using the string representation of the integer value with sign and insert the decimal point in the proper place (d digits from the end of the string representation).

Let’s begin our Python class (`APFP.py`):

```

1  from types import *
2
3  class APFP:
4      dprec = 23
5
6      def __toFP(self, arg):
7          sign = 1
8          arg = arg.strip()
9          if (arg[0] == "."):
10             arg = "0" + arg
11          if (arg[0] == "-"):
12             sign = -1
13             arg = arg[1:]
14          if (arg[0] == "+"):
15             arg = arg[1:]
16             idx = arg.find(".")
17             ipart = arg[0:idx]
18             fpart = arg[(idx+1):]
19             flen = len(fpart)
20             rounding = 0
21
22             if (flen <= APFP.dprec):
23                 fpart += "0"*(APFP.dprec-flen)
24             else:
25                 if (int(fpart[APFP.dprec]) >= 5):
26                     rounding = 1
27                 fpart = fpart[0:APFP.dprec]
28
29             self.n = sign*(int(ipart) * 10**APFP.dprec + int(fpart) + rounding)
30
31      def __init__(self, x="0"):
32          self.terms = 0
33          self.iters = 0
34          if isinstance(x, str):
35              if "." in x:
36                  self.__toFP(x)
37              else:
38                  self.__toFP(f"{x}.0")
39          elif isinstance(x, float):
40              self.__toFP(f"{x:0.18f}")
41          elif isinstance(x, int):
42              self.__toFP(f"{x}.0")
43          else:
44              raise ValueError("unsupported value")

```

We store the decimal precision in the class variable `dprec` (line 4). Set this to the desired precision before using the class. A more sophisticated library would track the precision per value and scale accordingly. Here, the default precision is randomly set to 23 decimal digits. We start with two methods, `__init__` and `__toFP`, the constructor which accepts a value to store in `x`, and a private method to process that value. The `__init__` method (line 31) uses the type of the argument to make a string representation and passes that to `__toFP` (line 6). The code in `__init__` creates a string of the argument with a decimal point in the proper place. Note that the conversion on line 40 uses 18 digits for the decimal, so a Python float (internally a C double) is represented with complete precision.

The `__toFP` method performs string checks and processing (lines 8 through 15) and sets the sign (`sign`) accordingly where 1 is used for a positive number and -1 is used for a negative number. Finally, lines 17 and 18 extract the integer and fractional

parts of the input string (i.e., left of the decimal point and right of the decimal point). Note that these are kept as strings. The variable `rounding` (line 20) is set to zero. It will be set to one if it is necessary to truncate the fractional part of the input, and the $d + 1$ -th digit is ≥ 5 . Adding one to the scaled input is equivalent to adding 10^{-d} , the smallest value that can be represented.

Lines 22 through 27 manipulate the fractional part to handle the case when the fractional part is less than d digits by adding trailing zeros (line 23). If the fractional part is longer than d digits, we check if the first digit to be dropped is ≥ 5 . If it is, increment `rounding` (line 26). The first d digits of the fractional part are then extracted.

Finally, the scaled integer value corresponding to the input is calculated (line 29). The integer part is scaled by 10^d before adding the d -digit integer value representing the fractional part. Notice that `rounding` is also added. The fully assembled integer is then multiplied by `sign`.

For display, we overload the Python methods for generating a string version of an object along with the `float` and `int` functions:

```

1 def __str__(self):
2     if (self.n == 0):
3         return "0.0"
4
5     s = str(self.n)
6     if (len(s) <= APFP.dprec):
7         s = "0"*(APFP.dprec-len(s)) + s
8         ans = "0." + s
9     else:
10        ip = len(s) - APFP.dprec
11        ans = s[0:ip] + "." + s[ip:]
12
13        k = len(ans)-1
14        while (ans[k] == "0"):
15            k -= 1
16        ans = ans[0:(k+1)]
17        if (ans[-1] == "."):
18            ans += "0"
19        if (self.n < 0):
20            ans = "-" + ans
21
22        return ans
23
24 def __repr__(self):
25     return self.__str__()
26
27 def __float__(self):
28     return float(self.__str__())
29
30 def __int__(self):
31     ans = self.__str__()
32     idx = ans.find(".")
33     return int(ans[0:idx])

```

where the main method is `__str__`, which returns a string representing the number. If zero, return zero (line 3). Otherwise, convert the scaled integer to a string (line 5). If this string is less than the precision, the integer part is zero, so return a value

starting with zero after prepending the fractional part with enough leading zeros to match the precision (lines 7 and 8). Otherwise, insert the decimal point before the fractional part (line 11). When line 13 is reached `ans` contains a string representation of the number, but it also has all the trailing zeros necessary to reach the fractional precision. Lines 13 through 15 locate the first nonzero digit scanning right to left to make the return value look nicer. Line 16 then truncates the answer to drop the trailing zeros. Lines 17 and 18 add a final zero after the decimal point if the number has no fractional part. Lines 19 and 20 prepend the sign if the number is negative. Finally, the exact representation is returned (line 22).

The `__repr__` method, called by the `repr` function, simply returns the string representation (line 25). Similarly, to convert to a Python float, the `__float__` method attempts to interpret the string representation as a floating-point number. If it is too large in magnitude, it will return positive or negative infinity. Lastly, `__int__` finds the integer portion of the string representation and returns it as an integer. Because of Python's big integer support, this integer will be exact (without the fractional part, which is truncated not rounded).

Let's experiment with the class in its current form:

```
>>> from APFP import *
>>> APFP.dprec = 6
>>> x = APFP("3.14159265")
>>> x
3.141593
>>> print(x)
3.141593
>>> str(x)
'3.141593'
>>> repr(x)
'3.141593'
>>> float(x)
3.141593
>>> int(x)
3
```

We set the decimal precision to six, then set `x` to π . Notice that π is truncated with rounding to use exactly six digits, 3.141593. The overloaded `__str__` method then ultimately handles all the calls that follow. The `float` call returns the exact representation because six digits easily fit in a C double. Naturally, the integer part is simply three.

9.3 Basic Arithmetic with Arbitrary-Precision Floating-Point Numbers

Let's add basic arithmetic to our library. We follow the methods used in Chap. 6 for addition, subtraction, multiplication, and division of fixed-point numbers. Addition and subtraction are particularly simple. Since each number is stored internally as a signed integer that is properly scaled by 10^d where d is the decimal precision, we add or subtract as normal:

```

1 def __add__(self, b):
2     z = APFP()
3     z.n = self.n + b.n
4     return z
5
6 def __sub__(self, b):
7     z = APFP()
8     z.n = self.n - b.n
9     return z

```

where we create a new object (lines 2 and 7) and then set it to the sum or difference of the current number and the given number (lines 3 and 8). We return the new object and are done. Why does this work? An example will illustrate

$$\begin{array}{r|l}
 123 & 123\ 0000000 \\
 +\ 34 & +\ 34\ 0000000 \\
 \hline
 157 & 157\ 0000000
 \end{array}$$

for $d = 7$ and added space after the integer part on the right side. Similarly, for subtraction, we have

$$\begin{array}{r|l}
 123.549 & 123\ 5490000 \\
 -\ 34.067 & -\ 34\ 0670000 \\
 \hline
 89.482 & 89\ 4820000
 \end{array}$$

where it is plain that the scaled versions of the numbers lead to exact results since there is no shifting of the implied decimal point. Overflow and underflow are not issues in this case because we use infinite precision integers.

What about multiplication and division? They are nearly as straightforward, but we must be aware of what is happening to rescale the results properly. For multiplication, we have

$$XY = (x \times 10^d) \times (y \times 10^d) = (xy) \times 10^{2d}$$

where X and Y are the scaled versions of x and y . In other words, multiplying xy returns the product scaled by 10^{2d} , not 10^d . We can fix this by dividing by 10^d , but if we do only that, we will introduce a bias by truncating digits without rounding. We must add 5 to the highest digit to be truncated, then divide.

The smallest value we can represent when the entire number is scaled by 10^{2d} is 10^d . Therefore, adding $5 \times 10^{d-1}$ is equivalent to adding five to the first truncated digit, which will round the next digit, the smallest digit we are keeping, to the proper value. We did much the same in Chap. 6 for our binary fixed-point library. The implementation is shorter than the description:

```

1 def __mul__(self, b):
2     z = APFP()
3     z.n = (self.n * b.n + 5*10**(APFP.dprec-1)) // 10**APFP.dprec
4     return z

```

where z holds the answer set in line 3 by first multiplying the two numbers ($\text{self.n} * \text{b.n}$), adding our bias correction ($5*10^{d-1}$), and then dividing by 10^d (10^{d-1}) to rescale. The signs of the two numbers are handled automatically by the multiplication.

Division is similarly straightforward. Following the derivation of Chap. 6, we understand that the division of decimal fixed-point numbers is best represented by

$$\frac{x \times 10^{2d} + 5 \times y \times 10^{d-1}}{y \times 10^d} = \frac{x}{y} \times 10^d + \frac{5}{10}$$

where $\frac{5}{10} = \frac{1}{2}$ is added to round properly and avoid bias due to truncation effects. For example, if $d = 8$ and the bias correction is present, $1/1120 = 0.00089286$ because the actual result is closer to 0.00089285714 . . . where the next digit, 7, requires rounding the previous to 6. Division, then, is easily implemented,

```

1 def __truediv__(self, b):
2     z = APFP()
3     z.n = (self.n * 10**APFP.dprec + (5*b.n) // 10) // b.n
4     return z

```

where the result (line 2) is set to the properly scaled and bias corrected quotient (line 3). This line directly translates the left-hand side of the equation above. Note that $(5*b.n) // 10$ is equivalent to $5 \times y \times 10^{d-1}$ since b.n is already scaled by 10^d .

Let's again experiment with the class in its current form:

```

1 >>> from APFP import *
2 >>> x = APFP(11.09); y = APFP(1120.7768)
3 >>> x+y
4     1131.8668000000000093718
5 >>> x = APFP("11.09"); y = APFP("1120.7768")
6 >>> x+y
7     1131.8668
8 >>> x-y
9     -1109.6868
10 >>> x*y
11     12429.414712
12 >>> x/y
13     0.00989492287848927636618
14 >>> APFP.dprec
15     23
16 >>> APFP.dprec = 64
17 >>> x = APFP("11.09"); y = APFP("1120.7768")
18 >>> x/y
19     0.00989492287848927636617745834853112591195677854859236
        9149682613

```

The module is imported in line 1. Line 2 defines two variables, and line 3 adds them, producing line 4. This seems mysterious at first; why the extra bit at the end? It has to do with how the variables were defined in line 2. The values given to the constructor were Python floats and, as such, were subject to the imprecision of all 64-bit IEEE 754 numbers. In line 5, we redefine x and y using strings. In this case, the sum shown in line 7 is exact, as expected. The same is true for subtraction (line 9), multiplication (line 11), and division (line 13), which uses all the available decimal digits for the default precision of 23 digits (shown in line 15). Line 16 changes the precision to 64 digits, line 17 redefines x and y in this new precision, and line 19 produces the division accurate to 64 digits.

So far, so good. The library works and is already useful, but we can improve it. We are missing comparison operators and a few other “nice to have” functions, to say nothing of trigonometric and other transcendental functions.

9.4 Comparison and Other Methods

Python offers us many methods that we can overload to work with existing operators. This includes all the expected comparison operators: $>$, $<$, $\geq$, $\leq$, $==$, and $!=$ which, courtesy of the magic of fixed-point arithmetic, become one-liners:

```
1 def __lt__(self, b):
2     return self.n < b.n
3
4 def __gt__(self, b):
5     return self.n > b.n
6
7 def __le__(self, b):
8     return self.n <= b.n
9
10 def __ge__(self, b):
11     return self.n >= b.n
12
13 def __eq__(self, b):
14     return self.n == b.n
15
16 def __ne__(self, b):
17     return self.n != b.n
```

Since the same value scales each number, direct comparison of the integer representation is equivalent to a floating-point comparison. Notice that this statement is only valid if both values use the same fixed-point precision.

Absolute value and unary negation are similarly easy:

```
1 def __abs__(self):
2     z = APFP()
3     z.n = abs(self.n)
4     return z
5
6 def __neg__(self):
7     z = APFP()
8     z.n = -self.n
9     return z
```

where, in each case, we return a new `APFP` instance and apply the operation directly to the integer representation.

The operation of these new methods is self-explanatory, so we will dispense with the examples and move directly to the more interesting issue of trigonometric and transcendental functions. We'll throw square root in there as well, just for fun.

9.5 Trigonometric and Transcendental Functions

A complete arbitrary-precision library must include the basic trigonometric functions, `sin`, `cos`, and `tan`, as well as transcendental functions `log` (natural log), `exp` (e^x), and for good measure, a fast square root. Let's add these functions to our library. We will either adapt the cookbook examples given in the Python decimal module documentation [1], which are already designed for arbitrary precision, or use New-

ton's method to determine the proper value iteratively. The cookbook examples of the trigonometric functions and `exp` add terms to a series representation until the value is no longer changing. We use Newton's method for square root and log as we did in Chap. 6 for binary fixed-point arithmetic.

Sine and cosine are as follows, with tangent defined as $\tan \theta = \frac{\sin \theta}{\cos \theta}$:

```

1 def sin(self):
2     i, lasts, fact, sign = [APFP(j) for j in [1,0,1,1]]
3     s = self
4     num = self
5
6     while (s != lasts):
7         lasts = s
8         i += APFP("2")
9         fact *= i * (i - APFP("1"))
10        num *= self * self
11        sign *= APFP("-1")
12        s += num / fact * sign
13    return s
14
15 def cos(self):
16     i, lasts, s, fact, num, sign = [APFP(j) for j in [0,0,1,1,1,1]]
17
18     while (s != lasts):
19         lasts = s
20         i += APFP("2")
21         fact *= i * (i - APFP("1"))
22         num *= self * self
23         sign *= APFP("-1")
24         s += num / fact * sign
25    return s
26
27 def tan(self):
28    return self.sin() / self.cos()

```

where `sin` and `cos` are implementations of the Taylor series expansions for `sin` and `cos`:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \dots$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \dots$$

and add terms to the result until it no longer changes (`s == lasts`). This approach adapts the methods to any precision at the expense of potentially requiring hundreds of terms. The tangent is defined in line 28.

Adding e^x is similarly accomplished by a Taylor's series expansion summing terms until they no longer affect the result:

```

1 def exp(self):
2     i, lasts, s, fact, num = [APFP(j) for j in [0,0,1,1,1]]
3
4     while s != lasts:
5         lasts = s
6         i += APFP("1")
7         fact *= i
8         num *= self
9         s += num / fact
10    return s

```

In this case, the series is

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \dots$$

We switch from summing a series to iterating using Newton's method to implement square root and natural logarithm. As shown in Chap. 6, iterating,

$$x_{i+1} = \frac{1}{2} \left(x_i + \frac{n}{x_i} \right)$$

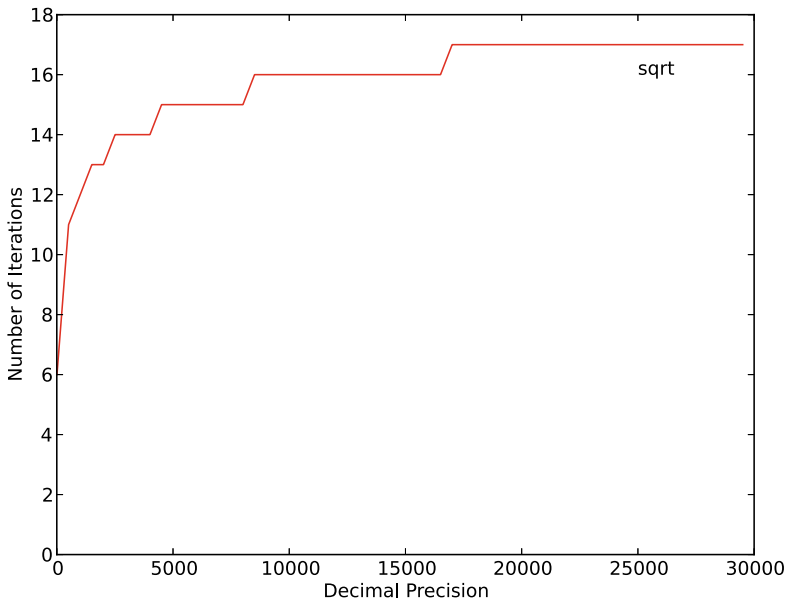
will result in $x \rightarrow \sqrt{n}$. In code, this becomes

```

1 def sqrt(self):
2     if (self < APFP(0)):
3         return APFP(-1)
4     if (self == APFP(1)):
5         return APFP(1)
6
7     s = self / APFP(2)
8     lasts = APFP(0)
9
10    while (s != lasts):
11        lasts = s
12        s = APFP("0.5") * (s + self / s)
13    return s

```

where we check for negative and zero arguments (lines 2 through 5) and then iterate an initial guess, $x_0 = n/2$, in line 7 until it no longer changes ($s == lasts$). It is known that this method for approximating the square root converges very rapidly, which we can see if we plot the number of iterations required as a function of the precision:



Our last transcendental function is $\log$ which, again following the derivation in Chap. 6, is found by iterating

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} = x_i - \frac{e^{x_i} - c}{e^{x_i}} = x_i - 1 + ce^{-x_i}$$

with $x_0 = c/2$ and $x \rightarrow \log(c)$. Unlike square root, this function might, for certain precisions, result in a set of repeating values so that simply checking for equality with the last estimate is insufficient. Therefore, we add a maximum iteration term to ensure that the function will return eventually. In code, this becomes

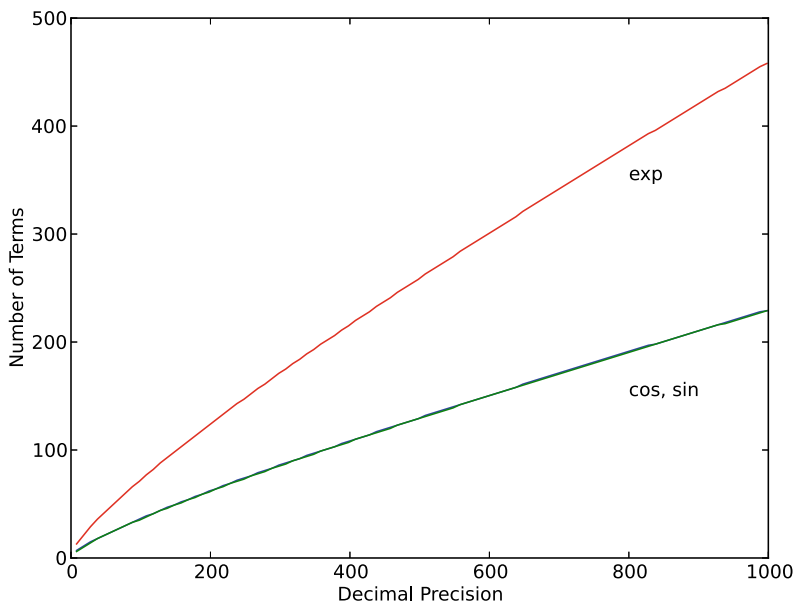
```

1 def log(self, imax=20):
2     if (self <= APFP(0)):
3         return APFP(-1)
4     if (self == APFP(1)):
5         return APFP(0)
6
7     s = self / APFP(10)
8     lasts = APFP(0)
9     i = 0
10
11    while (s != lasts) and (i < imax):
12        lasts = s
13        s += self * (-s).exp()
14        s -= APFP(1)
15        i += 1
16    return s

```

where, unlike the other methods, we allow a keyword to set the maximum number of iterations before returning (`i_max`). Lines 2 through 5 check for bad values or simple values. The loop (line 11) runs until we get convergence or max out the number of iterations.

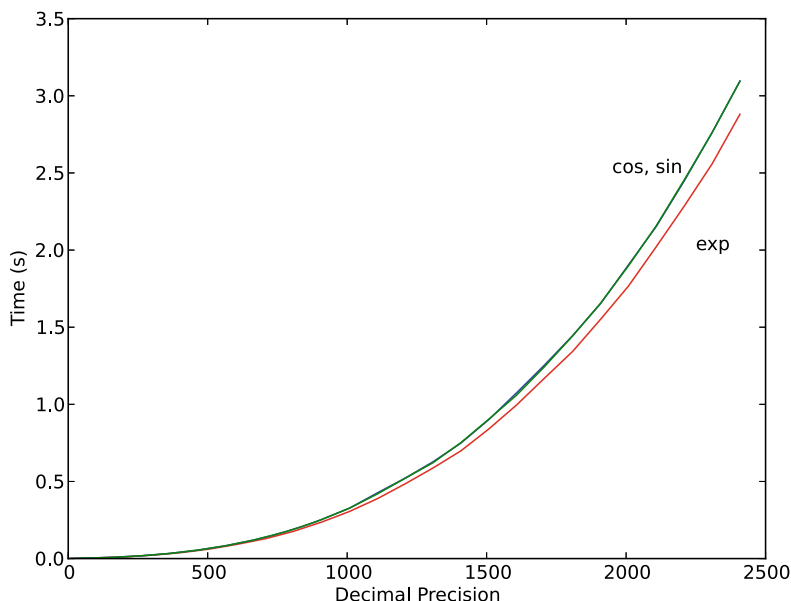
Before we move on to the next section, let's consider the performance of `sin`, `cos`, and `exp` by number of terms required as a function of precision. Plotting these gives



where we see that `sin` and `cos` are virtually indistinguishable, as we might expect, but `exp` requires approximately twice as many terms for any given precision.

We've made no attempt to optimize the library, but it's instructive to examine performance in terms of runtime. The arithmetic operations ($+$, $-$, $\times$, $\div$) will be virtually instantaneous relative to the trigonometric and transcendental functions because they use the optimized, C-based routines of the Python interpreter. Python makes use of Karatsuba multiplication for large integers, which is a good choice offering $O(n^{\log_2 3})$ performance, better than the grade school method, which is $O(n^2)$. See Chap. 5, Sect. 5.4 for a description of the Karatsuba algorithm.

Let's plot the runtime time of `sin`, `cos`, and `exp` as a function of precision:



where we see that our implementations are far from stellar, appearing to be of $O(n^2)$.

We created a simple but effective implementation of arbitrary-precision floating-point numbers in Python via fixed-point numbers and have added more than just basic arithmetic. We must remember that the library is meant for illustration purposes only and understand, especially when considering high-precision performance, that we should investigate more advanced implementations for serious work. Coincidentally, this is precisely what the next section contemplates.

9.6 Arbitrary-Precision Floating-Point Libraries

Most major programming languages support arbitrary-precision floating point via libraries. We are focusing here on Python and C, so we will review two such libraries, one for Python (`mpmath`) and one for C (GNU MPFR). The former is implemented in pure Python, like the `APFP` class we defined above, while the latter is pure C and has been wrapped many times to make it available to different programming languages.

We will discuss how to install the libraries and then demonstrate the main functionality of each, including how the libraries store and manipulate arbitrary-precision numbers.

mpmath Our first library is `mpmath` [2], which describes itself as “*a free (BSD licensed) Python library for real and complex floating-point arithmetic with arbitrary precision. It has been developed by Fredrik Johansson since 2007, with help from many contributors*”. The library is installed `mpmath` easily enough:

```
pip3 install mpmath
```

We can check that it is installed with

```
>>> import mpmath
>>> mpmath.__version__
'1.3.0'
```

where your version should be at least 1.3.0 or higher.

The `mpmath` library provides the basic functionality of our `APFP` class and much more. We can test that `mpmath` installed correctly and get a flavor for the plethora of functionality it provides by running its internal tests:

```
>>> import mpmath
>>> mpmath.runtests()
```

where the test results should be `'ok'` across all the tested areas. If the tests fail, you likely need to do `pip3 install pytest` first. We will do a great disservice to this elegant library by considering only the functionality matching our `APFP` class.

To set the precision:

```
1 >>> from mpmath import mp
2 >>> mp.mpf('1') / mp.mpf('1120')
3     mpf('0.00089285714285714283')
4 >>> mp.pretty = True
5 >>> mp.mpf('1') / mp.mpf('1120')
6     0.000892857142857143
7 >>> mp.dps = 20
8 >>> mp.mpf('1') / mp.mpf('1120')
9     0.00089285714285714285714
10 >>> mp.dps = 50
11 >>> mp.mpf('1') / mp.mpf('1120')
12     0.00089285714285714285714285714285714285714285714285714
```

Line 1 imports `mpmath`. In particular, it imports the `mp` context. Several contexts are supported; we will use only the `mp` context. Notice that this line is different from the typical NumPy import line of `import numpy as np`, but the effect is similar; we prefix `mpmath` calls with `mp`. Line 2 evaluates the expression $1/1120$ by defining two `mpmath` numbers using `mp.mpf()`, which creates an `mpmath` floating-point scalar. Line 3 shows the response. We are using the Python interpreter to get the output of `repr()`. Line 4 sets an internal flag so that repeating the calculation (line 5) gives us a prettier output (line 6).

The library's default precision is 15 digits (line 6). Ignore the leading zeros—we'll see why when discussing how `mpmath` stores numbers internally below. Line 7 ups the precision to 20 digits. Lines 10 and 12 do the same for 50 digits. Notice that the `mpf` constructor also accepts strings, integers, and floats, ala our `APFP` class.

Lastly, `mpmath` supports the free mixing of its numbers with native Python integers and floats. We didn't add this to the `APFP` class to keep the implementation simple. Feel free to remedy that shortcoming in your copious free time.

If we want to compare `mpmath` with `APFP`, we should be aware of the difference in meaning each library assigns to the word “precision”. For example,

```
>>> mp.dps = 15
>>> APFP.dprec = 15
>>> mp.mpf("1") / mp.mpf("1120")
0.000892857142857143
>>> APFP("1") / APFP("1120")
0.000892857142857
```

The example sets the precision of `mpmath` and `APFP` to 15 digits. However, calculating the same quantity with each produces different output because `mpmath` uses 15 significant digits, while `APFP` uses 15 digits after the decimal point, including leading zeros. With this distinction in mind, a few arithmetic examples demonstrate that `mpmath` and `APFP` give virtually identical results, but with some interesting differences:

```
1 >>> mp.dps = 23
2 >>> APFP.dprec = 23
3 >>> mp.mpf("12.9432") * mp.mpf("-0.043225")
4 -0.55946982
5 >>> APFP("12.9432") * APFP("-0.043225")
6 -0.55946982
7 >>> mp.mpf("12.9432") / mp.mpf("-0.043225")
8 -299.43782533256217466744
9 >>> APFP("12.9432") / APFP("-0.043225")
10 -299.43782533256217466743783
11 >>> mp.mpf("12.9432") * mp.mpf("43225094543.33")
12 559471043693.228856
13 >>> APFP("12.9432") * APFP("43225094543.33")
14 559471043693.228856
15 >>> mp.mpf("129435433.2095345332") * mp.mpf("43225094543.33")
16 5594858837739005819.5525
17 >>> APFP("129435433.2095345332") * APFP("43225094543.33")
18 5594858837739005819.552518723556
```

Lines 1 and 2 set the precision of both libraries to 23 decimals. Lines 3 and 5 illustrate that each library gives the same result for an operation that fits within the allowed precision. Lines 9 and 11, on the other hand, reveal the difference between the libraries' interpretation of the word “precision”, namely, that `mpmath` use 23 decimal digits for the entire number, while `APFP` uses 23 digits in the fractional part, meaning where `mpmath` ends with 744, `APFP` ends with **743783**. The bold digits are used to

get to 23 decimal digits in the fractional part and are also correctly rounded in the `mpmath` result. The other calculations continue in this way.

The number of trigonometric and transcendental functions `mpmath` supports is more than several dozen. Here, we will perform a few quick comparisons between `APFP` and `mpmath` for `sin` and `exp` to understand how an optimized implementation can improve over our naïve approach.

If we measure the computation time for `sin(1)` and $e^{0.5}$ for each library, we get numbers like these

	sin(1)		$e^{0.5}$	
precision	APFP	mpmath	APFP	mpmath
100	0.004277	0.000201	0.003466	0.000073
1000	0.301725	0.002486	0.245470	0.001851
2000	1.758381	0.008085	1.442545	0.006231
3000	5.005423	0.016561	4.142346	0.013261
4000	10.866939	0.028612	9.002207	0.023152
5000	20.118882	0.044388	16.673306	0.036576

demonstrating that our naïve `APFP` implementation, while accurate, is orders of magnitude slower than it could be.

How does `mpmath` achieve such good performance? An examination reveals finely tuned code employing clever tricks, including caching, symmetries, and preset precision thresholds where different approaches are used. If the precision is high enough, for example, the Taylor series expansion is used, but it is also finely tuned and far more sophisticated than the simple algorithms in `APFP`. Interestingly, one of the optimizations used by `mpmath` is to switch internally to a fixed-point representation in some cases. To explore these optimizations, see the file `libelefun.py` in the `mpmath/libmp` directory of the `mpmath` distribution.

So, how does `mpmath` store numbers internally? Whereas `APFP` uses a scaled arbitrary precision integer, `mpmath` uses a true floating-point format stored internally as a tuple. Specifically, `mpmath` stores numbers as 4-tuples: (*sign*, *significand*, *exponent*, *bit count*). The *sign* is zero (positive) or one (negative). The *significand* is a Python integer, as in `APFP`, but it represents the significand in this case. The *exponent* is also a Python integer. The *bit count* is the size of the significand in bits. Therefore, the number expressed by a 4-tuple is

$$x = (-1)^{\text{sign}} \times \text{significand} \times 2^{\text{exponent}}$$

where `mpmath` ensures that the *significand* is always odd. Notice that the base is 2, not 10.

Clearly, `mpmath` is a solid, well-optimized library, all the more impressive because it is pure Python.

GNU MPFR We discussed the `mpmath` library in some detail because it was directly comparable to our `APFP` class. Here, we take a quick look at the popular MPFR library [3] that forms the basis of many other libraries and tools.

The GNU MPFR arbitrary-precision floating-point library is part of a loose suite of tools that implement arbitrary-precision integers (GMP) and intervals (MPFI). MPFR is written in C but has easy access from C++. The library is also usable in MATLAB, Python (via the `bigfloat` module), and other languages. We will show simple examples in C.

First, install MPFR from <http://www.mpfr.org/> by following the directions on that site. Then, to test the installation, let's run the sample program from the website:

```
1 #include <stdio.h>
2 #include <gmp.h>
3 #include <mpfr.h>
4
5 int main (void)
6 {
7     unsigned int i;
8     mpfr_t s, t, u;
9
10    mpfr_init2 (t, 200);
11    mpfr_set_d (t, 1.0, MPFR_RNDD);
12    mpfr_init2 (s, 200);
13    mpfr_set_d (s, 1.0, MPFR_RNDD);
14    mpfr_init2 (u, 200);
15    for (i = 1; i <= 100; i++)
16    {
17        mpfr_mul_ui (t, t, i, MPFR_RNDU);
18        mpfr_set_d (u, 1.0, MPFR_RNDD);
19        mpfr_div (u, u, t, MPFR_RNDD);
20        mpfr_add (s, s, u, MPFR_RNDD);
21    }
22    printf ("Sum is ");
23    mpfr_out_str (stdout, 10, 0, s, MPFR_RNDD);
24    putchar ('\n');
25    mpfr_clear (s);
26    mpfr_clear (t);
27    mpfr_clear (u);
28    mpfr_free_cache ();
29    return 0;
30 }
```

which can be compiled with

```
$ gcc mpfr_ex1.c -o mpfr_ex1 -lmpfr -lgmp
```

assuming the code is stored in `mpfr_ex1.c`. Note the inclusion of the MPFR library via `-lmpfr`. We also include the GMP library via `-lgmp`. The program produces

```
Sum is 2.7182818284590452353602874713526624977572470936999595749669131e0
```

which is a reasonably accurate value for e . The program implements

$$e = \sum_{k=0}^{\infty} \frac{1}{k!}$$

The result is “reasonably” accurate because the last three digits (131) are incorrect. They should be, according to `APFP`, 676. This is exactly what is returned by changing the program to increase the precision from 200 in lines 10, 12, and 14 to 400.

Much is happening in the sample program. Let’s review it carefully as our introduction to fundamental MPFR concepts. First, note that lines 2 and 3 import `gmp.h` and `mpfr.h`. Line 8 creates several variables of `mpfr_t` type. These store MPFR numbers. Lines 10 through 14 are

```
10 | mpfr_init2 (t, 200);
11 | mpfr_set_d (t, 1.0, MPFR_RNDD);
12 | mpfr_init2 (s, 200);
13 | mpfr_set_d (s, 1.0, MPFR_RNDD);
14 | mpfr_init2 (u, 200);
```

which initialize and set MPFR variables. The variable `t` is initialized (line 10) by calling `mpfr_init2`. This function initializes the given variable, setting its precision to the given number of *bits*, not decimal digits like `APFP` or `mpmath`.

The number itself is set to NaN, indicating that MPFR supports the expected floating-point concepts of NaN and $\pm\infty$. Line 11 sets `t` to 1.0 via `mpfr_set_d`, which assigns an IEEE double-precision value to an MPFR variable. The third argument of this call, `MPFR_RNDD`, rounds the value towards $-\infty$. Therefore, MPFR supports rounding modes as well. Lines 12 and 13 do the same for `s`. What if we want to assign a variable to a high-precision constant? We can call `mpfr_set_str`, which takes a string argument representing the value. Line 14 initializes `u` but does not assign it a value.

Since MPFR uses bits for precision, how do we know the number of decimal digits? The relationship is $digits \approx bits/3.32$, implying 200 bits is about 60 decimal digits.

The main loop implements the series summation with

```

15 | for (i = 1; i <= 100; i++)
16 | {
17 |     mpfr_mul_ui (t, t, i, MPFR_RNDU);
18 |     mpfr_set_d (u, 1.0, MPFR_RNDD);
19 |     mpfr_div (u, u, t, MPFR_RNDD);
20 |     mpfr_add (s, s, u, MPFR_RNDD);
21 | }

```

where different MPFR functions are called to perform basic arithmetic. Each function follows this form:

```
mpfr_add(c, a, b, rnd)
```

which implements $c \leftarrow a + b$ with rounding mode `rnd`. Line 17 multiplies `t` by `i` rounding towards positive infinity. In this case, `mpfr_mul_ui` is called to multiply an MPFR number by an unsigned integer. Likewise, lines 19 and 20 implement $u \leftarrow u/t$ and $s \leftarrow s + u$, respectively, with rounding towards negative infinity. The loop runs for 100 iterations, which is sufficient to converge when 200 bits of precision are used.

The result is displayed by calling `mpfr_out_str` (line 23) to write a decimal representation of `s` to `stdout`. The third argument (`o`) selects the number of digits to output based on the precision. Note here that a specific rounding mode is given, in this case, towards negative infinity.

MPFR variables must be cleared to release memory as in lines 25 through 27 by calling `mpfr_clear`. Line 28 calls `mpfr_free_cache` to explicitly free memory associated with predefined constants.

Let's walk through a second MPFR example. Chapter 4 contains an exercise iterating the logistic equation written in different algebraically equivalent forms to demonstrate that round-off error quickly dominates and causes diversion in calculations that should produce identical results. Let's repeat that exercise using MPFR. The logistic equation can be written in these ways:

$$x_{i+1} = rx_i(1 - x_i) = rx_i - rx_ix_i = x_i(r - rx_i)$$

where $x = [0, 1]$ and $r < 4$. The map's behavior becomes chaotic if r is above a certain value. We'll set $r = 3.8$ in the chaotic region and iterate each of the three forms above. In an ideal world, the values will track identically, but because of round-off error in floating point, even with arbitrary precision, the values will eventually diverge after a sufficient number of iterations. The code we need is

```

1 void pp(mpfr_t x0, mpfr_t x1, mpfr_t x2) {
2     printf("    x0: "); mpfr_out_str(stdout,10,0,x0,MPFR_RNDN); printf("\n");
3     printf("    x1: "); mpfr_out_str(stdout,10,0,x1,MPFR_RNDN); printf("\n");
4     printf("    x2: "); mpfr_out_str(stdout,10,0,x2,MPFR_RNDN); printf("\n\n");
5 }
6
7 int main (int argc, char *argv[]) {
8     unsigned int i, p, n;
9     mpfr_t x0,x1,x2,u,v, r, one;
10
11     p = (unsigned int)atoi(argv[1]);
12     n = (unsigned int)atoi(argv[2]);
13     mpfr_init2(x0, p); mpfr_init2(x1, p);
14     mpfr_init2(x2, p); mpfr_init2(r, p);
15     mpfr_init2(one,p); mpfr_init2(u, p); mpfr_init2(v, p);
16     mpfr_set_d(one,1, MPFR_RNDN); mpfr_set_d(r, 3.8, MPFR_RNDN);
17     mpfr_set_d(x0, 0.25, MPFR_RNDN); mpfr_set_d(x1, 0.25, MPFR_RNDN);
18     mpfr_set_d(x2, 0.25, MPFR_RNDN);
19
20     printf("Initial values:\n");
21     for(i=0; i < 8; i++) {
22         mpfr_sub(u, one, x0, MPFR_RNDN); // x0 = r*x0*(1.0 - x0)
23         mpfr_mul(x0, x0, u, MPFR_RNDN);
24         mpfr_mul(x0, r, x0, MPFR_RNDN);
25         mpfr_mul(u, r, x1, MPFR_RNDN); // x1 = r*x1 - r*x1*x1
26         mpfr_mul(v, r, x1, MPFR_RNDN);
27         mpfr_mul(v, v, x1, MPFR_RNDN);
28         mpfr_sub(x1, u, v, MPFR_RNDN);
29         mpfr_mul(u, r, x2, MPFR_RNDN); // x2 = x2*(r - r*x2)
30         mpfr_sub(v, r, u, MPFR_RNDN);
31         mpfr_mul(x2, x2, v, MPFR_RNDN);
32     }
33     pp(x0,x1,x2);
34     for(i=0; i < n-16; i++) {
35         mpfr_sub(u, one, x0, MPFR_RNDN); // x0 = r*x0*(1.0 - x0)
36         mpfr_mul(x0, x0, u, MPFR_RNDN);
37         mpfr_mul(x0, r, x0, MPFR_RNDN);
38         mpfr_mul(u, r, x1, MPFR_RNDN); // x1 = r*x1 - r*x1*x1
39         mpfr_mul(v, r, x1, MPFR_RNDN);
40         mpfr_mul(v, v, x1, MPFR_RNDN);
41         mpfr_sub(x1, u, v, MPFR_RNDN);
42         mpfr_mul(u, r, x2, MPFR_RNDN); // x2 = x2*(r - r*x2)
43         mpfr_sub(v, r, u, MPFR_RNDN);
44         mpfr_mul(x2, x2, v, MPFR_RNDN);
45     }
46     printf("\n\nFinal values:\n");
47     for(i=0; i < 8; i++) {
48         mpfr_sub(u, one, x0, MPFR_RNDN); // x0 = r*x0*(1.0 - x0)
49         mpfr_mul(x0, x0, u, MPFR_RNDN);
50         mpfr_mul(x0, r, x0, MPFR_RNDN);
51         mpfr_mul(u, r, x1, MPFR_RNDN); // x1 = r*x1 - r*x1*x1
52         mpfr_mul(v, r, x1, MPFR_RNDN);
53         mpfr_mul(v, v, x1, MPFR_RNDN);
54         mpfr_sub(x1, u, v, MPFR_RNDN);
55         mpfr_mul(u, r, x2, MPFR_RNDN); // x2 = x2*(r - r*x2)
56         mpfr_sub(v, r, u, MPFR_RNDN);
57         mpfr_mul(x2, x2, v, MPFR_RNDN);
58     }
59     pp(x0,x1,x2);
60     return 0;
61 }

```

where `#include` statements are ignored to save space. The listing is three repetitions of a simple block to implement the three different forms of the logistic equation:

```

1  for(i=0; i < n-16; i++) {
2      mpfr_sub(u, one, x0, MPFR_RNDN); // x0 = r*x0*(1.0 - x0)
3      mpfr_mul(x0, x0, u, MPFR_RNDN);
4      mpfr_mul(x0, r, x0, MPFR_RNDN);
5      mpfr_mul(u, r, x1, MPFR_RNDN); // x1 = r*x1 - r*x1*x1
6      mpfr_mul(v, r, x1, MPFR_RNDN);
7      mpfr_mul(v, v, x1, MPFR_RNDN);
8      mpfr_sub(x1, u, v, MPFR_RNDN);
9      mpfr_mul(u, r, x2, MPFR_RNDN); // x2 = x2*(r - r*x2)
10     mpfr_sub(v, r, u, MPFR_RNDN);
11     mpfr_mul(x2, x2, v, MPFR_RNDN);
12 }
```

Lines 2 through 4 iterate x_0 , lines 5 through 8 iterate x_1 , and lines 9 through 11 iterate x_2 . The values after the first eight iterations are displayed along with the final set of values. Compile the program and call it with two command line arguments: precision and number of total iterations. Chapter 3 told us that after eight iterations using standard floating point, the values were already diverging. Here, we also see this deviation, but only for the last few digits. Let's fix the number of total iterations at 10,000. Doing this with 113 bits of precision (equivalent to an IEEE 754 quadruple precision float), we get the following output for the final iteration:

```

x0: 9.49404460849763073594113685574416764e-1
x1: 4.57638502447496138812013997759474251e-1
x2: 7.98660115295355269211918348596331193e-1
```

The values are clearly no longer in sync with each other. This begs the question, for a fixed iteration limit of 10,000, what precision do we need to get final values that are equal to some level of precision? Some experimentation using a manual binary search shows that when we set the precision to 6250 bits, we get the following:

```

x0: 6.794658940724743462e-1
x1: 6.726466428639511824e-1
x2: 6.738777979703301964e-1
```

where the first two decimals are now equal, and we are showing the precision we'd expect from an IEEE 754 double (53 bits). This implies that we need nearly 118 times the precision of a C double to even approach uniformity when calculating 10,000 iterations of the logistic map!

What precision will let us calculate 10,000 iterates and maintain the 53-bit precision of a C double? Some more searching reveals that 6315 bits of precision is the value we seek. At that precision, the final values are $x_0 = x_1 = x_2 =$

0.7061911661282795065. We begin to see how it is possible that mathematical research could require very high precision to be convinced results are legitimate. Fortunately for us, the real world seldom requires such high precision, but it is a bit disconcerting that the bulk of floating-point calculations use so few bits.

Let’s take a closer look at the rounding modes supported by MPFR. These mirror the four rounding modes supported by IEEE 754 along with a mode not supported by IEEE:

Mode	Description
MPFR_RNDU	Round the result up towards $+\infty$.
MPFR_RNDD	Round the result down towards $-\infty$.
MPFR_RNDZ	Round the result up or down towards zero.
MPFR_RNDN	Round the result to the nearest representable number. Ties to even.
MPFR_RNDA	Round away from zero, up or down. Not in IEEE 754.

Note that in IEEE, round to nearest is the assumed default, while MPFR requires the rounding mode to be given explicitly. In general, round to nearest (MPFR_RNDN) is usually sufficient.

How does MPFR store numbers internally? If we examine `mpfr.h`, we see that `mpfr_t` is a structure:

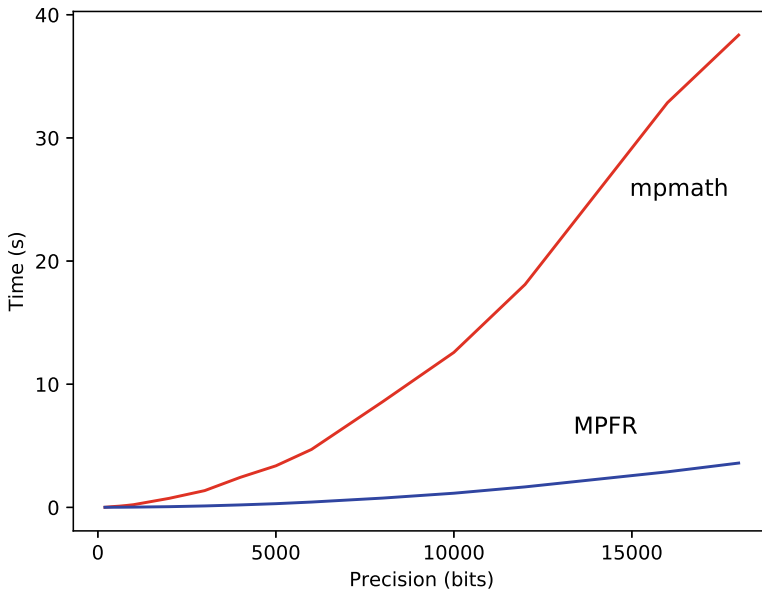
```
1 typedef struct {
2     mpfr_prec_t  _mpfr_prec;
3     mpfr_sign_t  _mpfr_sign;
4     mpfr_exp_t   _mpfr_exp;
5     mp_limb_t    *_mpfr_d;
6 } __mpfr_struct;
```

where it is evident that MPFR is using a format similar to `mpmath` with separate sign, exponent, and a collection of “digits” (`_mpfr_d`). The `mpfr.h` header tells us how to generate the represented number:

$$x = s \times \left(\frac{d_{k-1}}{B} + \frac{d_{k-2}}{B^2} + \cdots + \frac{d_0}{B^k} \right) \times 2^e$$

where s is the sign (± 1), $k = \lceil p/g \rceil$, $B = 2^g$, p is x ’s precision in bits, e is the exponent, and g is a constant used by the underlying *GMP* library. On our test system, $g = 32$ bits. Just as IEEE 754 requires that the highest order bit of the significand be one, MPFR requires that the highest order data value of the significand be $\geq B/2$.

How does MPFR stack up against `mpmath`? This is a somewhat unfair question as `mpmath` is pure Python, and MPFR is written in C, but it is illustrative to make a quick comparison to understand when it might be acceptable to use `mpmath` for a calculation or when one might want to go through the extra work of using MPFR. To this end, I measured the runtime for 1000 calculations of $\sin(1)$ using both libraries as a function of the binary precision. The timing only measured the actual calculations and no other overhead:



For precisions much above 1000 bits (about 300 decimal digits), we will want to use MPFR for serious calculations. Actually, the plot demonstrates just how well optimized `mpmath` is in that it can effectively compete with a purely compiled library at precisions below 1000 bits. A single comparison is not a fair evaluation, but it does provide a ballpark estimate of relative performance.

It is possible to use MPFR directly from Python. First, install `gmpy2`,

```
pip3 install gmpy2
```

then use it like so,

```
>>> gmpy2.get_context().precision = 100
>>> mpfr('3.14159265') / mpfr('2.71828182')
mpfr('1.1557273520668287440483268213891',100)
>>> print(mpfr('3.14159265') / mpfr('2.71828182'))
1.1557273520668287440483268213891
```

I leave it as an exercise for the reader to compare MPFR from Python with `mpmath` and `ARFP`.

9.7 Thoughts on Arbitrary-Precision Floating Point

What should we think of arbitrary-precision floating point? If working simply, say as a sort of “desk calculator”, almost any library, including the `MPFR` library developed in this chapter, will work nicely. However, if developing calculation-heavy software, we want the optimized libraries.

But is arbitrary precision ever really needed? The cheeky answer is “it depends”. High precision is helpful for some scientific calculations to avoid round-off effects or because the values are known with more than the precision expressed by an IEEE 754 float. A glance at the list of software using MPFR reveals places where arbitrary precision is desirable. For example, paraphrasing select packages from the “Software Using MPFR” section of the main MPFR website gives the following list (as of July 2016):

ALGLIB.NET	Implements multiple-precision linear algebra using MPFR
BNCPack	The numerical analysis library, which can be compiled with MPFR
CGAL	Computational Geometry Algorithms Library
DateTime-Astro	Functions for astronomical calendars
FLINT	Fast Library for Number Theory
FractalNow	A fractal generator
GCC	In GFortran, then in the middle-end phase as of GCC 4.3, to resolve math functions with constant arguments
libieeep1788	C++ implementation of the preliminary IEEE P1788 standard for interval arithmetic
Macaulay 2	A software system devoted to supporting research in algebraic geometry and commutative algebra
Magma	A computational algebra system
MATLAB	Multiprecision Computing Toolbox for MATLAB
Protea	Software devoted to protein-coding sequences identification
TIDES	To integrate numerically ordinary differential equations in arbitrary precision
TRIP	A general computer algebra system dedicated to celestial mechanics
ZKCM	A C++ library for multiprecision complex-number matrix calculations

9.8 Summary

In this chapter, we developed a Python class implementing arbitrary-precision “floating-point” calculations using fixed-point arithmetic. We demonstrated how to use the class and explored its limitations. We then introduced a sophisticated Python library for arbitrary-precision floating point and compared it to our class, revealing the strengths and weaknesses of both. Next, we examined a high-performance C library for arbitrary-precision floating point and explored the level of precision necessary to maintain complete precision for calculations that, naïvely, should be equivalent. We ended with thoughts about arbitrary-precision floating point and when it might be helpful.

Exercises

9.1 Add a `frac` method to the APFP class that returns the fractional part of a number.

9.2 Add `ceil` and `floor` methods to the APFP class that return $\lceil x \rceil$ and $\lfloor x \rfloor$, respectively. Then add `round` that returns the closest integer to x .

9.3 Add modulo to the APFP class by overloading the modulo operator (%) using the `__mod__` method.

9.4 Add x^y to the APFP class. Overload the `__pow__` method so `x**y` works properly. Distinguish between two cases: x^y where both x and y are arbitrary-precision numbers and x^n where n is an ordinary Python integer, positive or negative. (Hint: Recall that for real numbers, $x^y = e^{y \log x}$).

9.5 Make a plot of the number of iterations for the following series approximations of e as a function of the precision. Which series converges in the fewest number of terms? Test precisions from 8 to 4500. Look at the implementation of the APFP `exp` method for hints on implementing the series. The series are

$$\begin{aligned}
 e_0 &= \frac{1}{2} \sum_{k=0}^{\infty} \frac{k+1}{k!} \\
 e_1 &= 2 \sum_{k=0}^{\infty} \frac{k+1}{(2k+1)!} \\
 e_2 &= \sum_{k=0}^{\infty} \frac{(3k)^2 + 1}{(3k)!} \\
 e_3 &= \sum_{k=1}^{\infty} \frac{k^7}{877(k!)}
 \end{aligned}$$

References

1. <https://docs.python.org/2/library/decimal.html>. (Accessed 10 Jul 2016)
2. <http://mpmath.org/>. (Accessed 14 Jul 2016)
3. <http://www.mpfr.org/>. (Accessed 15 Oct 2014)

Abstract

Posits are a family of floating-point number formats claimed to offer improved precision and increased dynamic range compared to standard IEEE floating point. This chapter explores posits and tests these claims.

10.1 Why Posits?

The IEEE 754 floating-point format is an entrenched standard, so why contemplate a new approach to representing floating-point numbers? In other words, what's wrong with IEEE 754?

On the one hand, there is nothing wrong with IEEE 754. Chapter 3 describes IEEE floats in detail. They come in different sizes using, typically, 64, 32, or 16 bits or less for modern AI. They represent numbers as a binary significand multiplying 2 to some power, a reasonable thing to do. They also handle special values like “infinity” and “not a number” along with an extended range of minimal numbers (subnormals). The decades of use behind IEEE floats justify their existence. Still, there's room for improvement.

IEEE floats admit two representations of zero (signed and unsigned). Also, subnormal numbers are cumbersome and typically not implemented in hardware, thereby causing a discontinuity in terms of performance. Finally, while eminently reasonable, using a binary significand to multiply a power of two means that the spacing between successive IEEE floats doubles every time the exponent changes, again causing discontinuities, this time in representation space.

Posits [1] use a clever approach to representing floating-point numbers that taper the spacing between successive floats continuously. Posits, for a bit-width matching IEEE floats, show a greater concentration of values around zero, thereby covering smaller number ranges with increased density. However, the posit format is suffi-

ciently flexible to allow greater dynamic range, if helpful. The quality of the representation can change during a calculation if the underlying posit system (hardware or software) is capable of adjusting on the fly.

This chapter explores posits. Let's begin by understanding posits and how they are represented in memory.

10.2 Understanding Posits

Posits are described in detail in [1]. Of particular importance is Figure 4 and the interpolation rules given prior to the figure defining how to construct posits of a given bit width and exponent size. Here, we focus on the posit storage format and how to interpret it, thereby building a practical understanding of posits. First, we discuss the general format of a posit and how to understand its range. Second, we restrict ourselves to standard posits as defined by the 2022 draft standard (posithub.org/docs/posit_standard-2.pdf) to compare posit representations to those of similar-sized IEEE 754 floats.

Before proceeding, however, an apology is required. The draft posit standard includes posits and their associated quires, an extended format for each standard posit intended as a high-resolution accumulator for iterated operations. Quires enable repeated calculations without round-off error (e.g., a dot product). Quires are an essential component of a complete posit implementation. That said, we'll ignore them here to provide a direct comparison with IEEE floats.

Posits are defined by their width (n) and number of exponent bits (es), which may be zero. The posit standard uses es for the size of the exponent field and e for its value. Figure 10.1 presents the structure of a general n -bit posit consisting of a sign bit (1 = negative), regime, exponent, and fraction bits. Note that the exponent and fraction are optional, and that the number of regime bits can vary up to the remaining $n - 1$ bits after the sign.

IEEE floats have a sign, a fraction (significand), and exponent bits as well. What sets posits apart is the introduction of the variable-width regime field. The regime field enables posits to focus on high precision around zero or to increase the dynamic range, as required. For example, Fig. 10.2 presents three different $n = 8$ bit posits with $es = 0, 1$, and 2 . Notice that the same bit pattern is used for each.

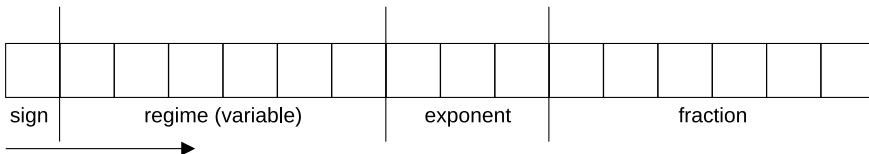
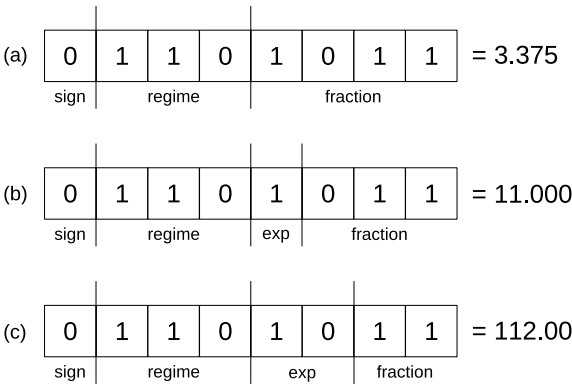


Fig. 10.1 A general posit with sign, regime, exponent, and fraction bits indicated. Note that the exponent and fraction fields are optional. The arrow indicates the parsing direction

Fig. 10.2 Three different $n = 8$ bit posits with **a** $es = 0$, **b** $es = 1$, and **c** $es = 2$



The arrow in Fig. 10.1 is our first clue to interpreting a posit. Interpretation moves from left to right (high bit to low bit), beginning with the sign, which is always the high bit. The bit after the sign begins a variable-length regime field. The regime field uses runlength encoding where the first bit after the sign specifies the marker and the opposite bit (or end of the posit) the stop condition. Positive regime values use 1 bits while negative use 0 bits. By design, any field (and corresponding bits) not present in a posit are 0. Therefore, a regime field beginning with a 1 bit can run for the remainder of the posit, but a regime field beginning with a 0 bit must end with a 1 bit. For example, the following indicates the largest positive and largest negative regime fields for an $n = 8$ posit:

$$\begin{array}{c} 0 \mid 111111 \\ 0 \mid 000001 \end{array}$$

The first posit’s regime field is $111111_2 = 7 \rightarrow k = 6$ while the second’s is $000001_2 = 1 \rightarrow k = -1$. In general, the regime field value is $-k$ if the first regime bit is 0 and $k - 1$ if 1. We’ll learn how to use k momentarily.

The fact that the regime field may extend to all the remaining bits informs us as to why the exponent and fraction fields are optional. If $es = 0$, any bits remaining (i.e., to the right of the end of the regime field) are considered the fraction. If $es > 0$, then the following es bits are interpreted as an unsigned integer. Unlike IEEE floats, posit exponents do not have an excess. Also, the “missing bits are zero” rule applies to exponents. So, an $n = 8$ posit with $es = 2$ and a bit pattern of

$$0 \mid 000001 \mid 1$$

has an exponent of $e = 10_2 = 2$ because there must be 2 bits in the exponent, and parsing has run out of posit bits, implying that the low-order bit of the exponent is 0 (along with the fraction). If the final bit were 0, the exponent would be $e = 00_2 = 0$.

The fraction, if any, is interpreted as an unsigned integer divided by 2^{fs} where fs is the number of bits in the fraction.

We have most of what we need to interpret a posit. Three elements remain: how to use the regime value, special posit values, and what to do if the posit is negative.

The regime value, k , is an exponent on a field often denoted as *useed*. Posits use two exponents, one on *useed*, and the other on 2. Together, these set the value of the posit with the fraction, if any, acting as a fine adjustment to the base value derived from k and e . To calculate *useed*, we must know es :

$$useed = 2^{2^{es}}$$

Therefore, for an $es = 2$ posit: $useed = 2^{2^{es}} = 2^{2^2} = 2^4 = 16$. If $es = 3$, then $useed = 2^{2^3} = 2^8 = 256$ and $es = 1$ leads to $useed = 2^{2^1} = 2^2 = 4$. Finally, if $es = 0$: $useed = 2^{2^0} = 2^1 = 2$. Let's find the value of Fig. 10.2c with $es = 2$ implying $useed = 16$. Moving from left to right, we first see that the number is positive ($s = 0$). The regime is also positive because the first bit is 1. There are two 1 bits followed by a 0 bit; therefore, the regime field is 2 and $k = 2 - 1 = 1$. The exponent is $e = 10_2 = 2$, and there are 2 bits in the fraction ($fs = 2$) with a value of $f = 11_2 = 3$. The general formula for a positive ($s = 0$) posit is

$$v = (useed^k)(2^e) \left(1 + \frac{f}{2^{fs}} \right) \quad (10.1)$$

Substituting the values for Fig. 10.2c gives

$$(16^1)(2^2) \left(1 + \frac{3}{2^2} \right) = (1)(16)(4)(1 + 3/4) = (1)(16)(4)(1.75) = 112.0$$

as expected. A similar analysis for Fig. 10.2a returns

$$(2^1)(2^0) \left(1 + \frac{11}{2^4} \right) = (1)(2)(1)(1 + 11/16) = 3.375$$

because $e = 0$. Finally, for (b)

$$(4^1)(2^1) \left(1 + \frac{3}{2^3} \right) = (1)(4)(2)(1 + 3/8) = 11.0$$

Posits admit two special values the first of which is: if every bit is zero, the posit is zero. The second special value is NaR (“not a real”), which posits use for anything IEEE calls infinity (positive or negative) or NaN (“not a number”). NaR is more general and captures the concept that the value cannot be represented as a finite real number. The posit where the sign is 1, and all other bits are 0, is NaR. Therefore, for $n = 8$,

$$00000000_2 \rightarrow 0$$

$$10000000_2 \rightarrow \text{NaR}$$

Notice that NaR does not accommodate a payload like IEEE NaNs. This isn't a drawback in practice, as NaN payloads are virtually unused. Indeed, many developers seem unaware of their existence.

What if the posit is negative ($s = 1$)? In that case, first determine the two's complement of the posit interpreted as a signed integer, followed by negating the floating-point value calculated from the complement. For example, assume $n = 8$ and $es = 0$, and the posit bits are 11101011_2 . The sign is negative, so calculate the two's complement:

$$11101011_2 \rightarrow 00010100_2 + 1 \rightarrow 0|001|0101_2$$

I added separators for the sign, regime, and fraction ($e = 0$ always in this case). Equation 10.1 then informs us that

$$v = (useed^k)(2^e) \left(1 + \frac{f}{2^{fs}}\right) = (2^{-2})(2^0) \left(1 + \frac{5}{2^4}\right) = 0.328125$$

meaning the original posit represents -0.328125 . Notice that the two's complement trick also avoids the signed zero annoyance of IEEE floats and maps NaR to NaR, but don't take my word for it; prove it to yourself.

10.3 Standard Posits

The draft posit standard defines expected posit types: *posit8*, *posit16*, and *posit32*. A 64-bit posit is also indicated, but we'll restrict ourselves to 32-bit posits in this chapter to use 64-bit IEEE floats as a "gold standard" for comparing posit and IEEE representations. The number in the name tells us n . As we now appreciate, we must also know es to interpret a posit properly. The standard posits use $es = 0$, $es = 1$, and $es = 2$ for *posit8*, *posit16*, and *posit32*, respectively. Figure 10.3 presents C code to parse a *posit8*.

The code includes requisite libraries (compile with `-lm`) and defines a `posit8_t` type as an alias for an unsigned character (`uint8_t`). The `bit` helper function returns the value of a requested bit.

The `parse8` function accepts the posit (`p`) and checks for zero (line 17) or NaR (line 18). If NaR, IEEE positive infinity is returned. Naturally, other options are available (e.g., a NaN). Lines 20 and 21 examine the sign bit and apply the two's complement if the posit is negative.

Lines 23 through 26 determine k by reading along the regime encoding. If the encoding does not encounter a flipped 1 bit (i.e., a 0) before the end of the posit, there is no fractional part. Since $es = 0$, there is no exponent ($e = 0$), and if there is no fractional part, line 27 returns the posit value using s to set the sign properly. The second element of the expression in line 27, `pow(2, k)`, raises $useed = 2^{2^{es}} = 2^{2^0} = 2^1 = 2$ to the k -th power with an implied $2^0 = 1$ exponent.


```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <math.h>
4
5  typedef uint8_t posit8_t;
6
7  uint8_t bit(uint8_t v, uint8_t b) {
8      return (v>>b) & 0x1;
9  }
10
11 double parse8(posit8_t p) {
12     int s,f,k,i,b;
13     uint8_t v;
14     double ans;
15
16     v = (uint8_t) p;
17     if (v==0x00) return 0.0;
18     if (v==0x80) return exp(1000);
19
20     s = bit(v,7);
21     if (s) v = (uint8_t)((0x100-(int)v) & 0xFF);
22
23     b = bit(v,6);
24     k=1; i=5;
25     while ((bit(v,i)==b) && (i>=0)) k++,i--;
26     k = (b) ? k-1 : -k;
27     if (i==-1) return pow(-1,s)*pow(2,k);
28
29     f = v & ((1<<i)-1);
30     return pow(-1,s)*pow(2,k)*(1.0 + (double)f/pow(2,i));
31 }

```

Fig. 10.3 Parsing a posit8. See `posit8.c`

Table 10.1 Characteristics of the standard posits

	<i>n</i>	<i>es</i>	minPos	maxPos
posit8	8	0	0.015625	64.0
posit16	16	1	3.725290298461914e-09	268435456.0
posit32	32	2	7.52316384526264e-37	1.329227995784916e+36

Finally, lines 30 and 31 extract the fraction and produce the full posit value according to Eq. 10.1. Notice again that $e = 0$ is implied.

Table 10.1 characterizes the standard posits. Notice how increasing n (and es) increases the dynamic range ($\pm \text{maxPos}$) while simultaneously decreasing the smallest positive representable value (minPos).

The experiments later in the chapter use standard posits. However, before we proceed, let's examine the relationship between posit ranges (as a function of es) and the spacing between successive posit values.

10.4 Posit Range and Spacing

An n -bit posit implies the availability of 2^n different bit patterns. The precise interpretation of the bit patterns depends on es , but the following statements are always true:

- The all-zero bit pattern represents zero.
- There is only one NaR representation, the bit pattern with a leading 1 and all other bits 0.
- Because 2^n is always even and there is only one NaR bit pattern, there are $2^n - 1$ valid n -bit posits (an odd number).
- The negation of a posit is found by taking the two's complement of the posit bits and then interpreting the value with the opposite sign.
- The two's complement of zero is zero, implying negative and positive posit values are symmetric about zero with a spacing determined by es .

For fixed n , counting in binary from zero moves along the positive number line. For any value, v , as an integer, the next largest positive posit is $v + 1$ up to $v = 2^{n-1} - 1$, which is $maxPos$. Additionally, $v = 2^{n-1}$ is NaR, $v = 2^{n-1} + 1$ is $-maxPos$, and $v = 1$ and $v = 2^n - 1$ are $\pm minPos$, respectively.

Figure 10.4 presents *posit8* values over the entire ± 64 range (top), to ± 4 (middle), and ± 1 (bottom). What appear to be solid black rectangles are illusions; the posit values are too close together to be resolved at that scale. The plots graphically illustrate tapered precision. Notice that the largest *posit8* values have very low precision.

For example, the three largest positive *posit8* values are 24, 32, and 64 (top row of Fig. 10.4). Therefore, the best *posit8* representation of 48 is 32, while 49 is best represented as 64. Clearly, *posit8* is not a good choice for values in this range. However, for small values, tapered precision comes into play:

$$\begin{aligned} 1.0 &\rightarrow 1.0 \\ 0.5 &\rightarrow 0.5 \\ 0.1 &\rightarrow 0.09375 \\ 0.01 &\rightarrow 0.015625 \end{aligned}$$

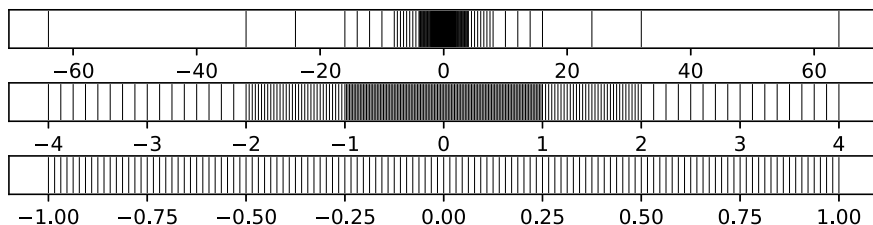


Fig. 10.4 The distribution of *posit8* values: $-4 \leq x \leq 4$ (87.9%), $-1 \leq x \leq 1$ (50.4%)

Table 10.2 Interpreting $n = 8$ posits as es changes

<i>Bits</i>	$es = 0$	$es = 1$	$es = 2$	$es = 3$	$es = 4$
00000001	0.015625	0.000244141	5.96046e-8	3.55271e-15	1.26218e-29
00000010	0.031250	0.000976563	9.53674e-7	9.09495e-13	8.27181e-25
00000011	0.046875	0.001953125	3.81470e-6	1.45519e-11	2.11758e-22
00000100	0.062500	0.003906250	1.52588e-5	2.32831e-10	5.42101e-20
01111100	16	256	65536	4.29497e+09	1.84467e+19
01111101	24	512	262144	6.87195e+10	4.72237e+21
01111110	32	1024	1048576	1.09951e+12	1.20893e+24
01111111	64	4096	16777216	2.81475e+14	7.92282e+28

where 1.0 and 0.5 are exact, and 0.1 is off by 0.00625. The absolute error for 0.01 is larger because 0.015625 is *minPos*, the smallest positive *posit8* value.

Notice that of the 255 valid *posit8* values, 50.4 percent are within ± 1 and 87.9 percent within ± 4 . Therefore, *posit8* numbers are most precise when restricted to those ranges.

The book’s GitHub contains text files with names like *posit8_es_0_table.txt* listing $n = 8$ posit values for $es = 0$ through $es = 4$. The tables were generated by the JavaScript applet at posithub.org/widget/lookup. Table 10.2 lists several bit patterns and how they are interpreted as es changes.

Table 10.2 is informative. The first row is *minPos* and the last *maxPos*. The dynamic range increases dramatically with es , but the difference between the largest representable values changes by orders of magnitude for $es > 1$. At the same time, values close to *minPos* are nearly identical in practical terms.

The table hints that $es = 0$ and $es = 1$ might offer a useful range of values, especially if restricted to ± 1 . However, plotting both over $[0, 1]$ reveals why the draft standard selected $es = 0$ for *posit8*; see Fig. 10.5.

For $n = 8$, the posit formula delivers 64 values in $[0, 1]$ for both $es = 0$ and $es = 1$. The upper plot ($es = 0$) shows a virtually even distribution of posits, while

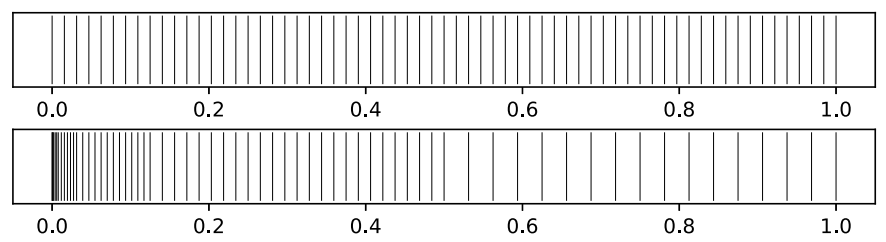


Fig. 10.5 The distribution of $n = 8$ posits over $[0, 1]$ for $es = 0$ (top) and $es = 1$ (bottom)

the lower plot demonstrates that $es = 1$ posits are concentrated near zero with uneven $[0, 1]$ coverage.

What about *posit16*? In that case, there are 65535 valid posit values, 50 percent of which cover ± 1 , growing to 89 percent for ± 5 . It's worth noting that many neural networks operate with weights in the ± 6 range or less, implying that *posit16* is worthy of consideration as an alternative to IEEE formats like *binary16* (float16) for model training or inference. Chapter 11 discusses AI-friendly floating-point formats. As of this writing, however, posits are not typically included in that collection.

10.5 Experiments

Let's use experiments to compare posits with the same bit-width IEEE floats. We'll restrict ourselves to standard posits, understanding that there are situations where other posit configurations might be more advantageous. We'll work in Python using two helpful libraries: *SoftPosit* and *SoftFloat*. The first implements standard posits for us, and the latter 32- and 16-bit IEEE floats. Python's native `float` is a 64-bit IEEE float serving as a gold standard representation.

First, install the necessary libraries. On Linux use:

```
> pip3 install softfloat
> pip3 install softposit
```

For other platforms, refer to the appropriate GitLab pages:

```
https://gitlab.com/cerlane/SoftPosit-Python
https://gitlab.com/cerlane/SoftFloat-Python
```

Test the installation with the Python console session in Fig. 10.6.

The session demonstrates how to import the libraries and define *posit16* and *float16* values. Two helpful methods are also presented: `fromBits` and `toBinaryFormatted`. The former lets us assign specific bit patterns to posits and IEEE floats. The latter displays the bit pattern using color (lost in print). For example, `a` and `b` are presented as

$$a = 0 \mid 1110 \mid 1 \mid 0101101011_2 \text{ (posit16)}$$

$$b = 0 \mid 11101 \mid 0101101011_2 \text{ (float16)}$$

This example is our first demonstration of posit precision. The *posit16* representation of 0.1 is more accurate than the *float16* version. The former deviates from exact by 6.1×10^{-6} , while the latter by 2.4×10^{-5} .

Fig. 10.6 Working with posits in Python

```
>>> import softposit as sp
>>> import softfloat as sf
>>> a = sp.posit16(0.1)
>>> b = sf.float16(0.1)
>>> a
0.100006103515625
>>> b
0.0999755859375
>>> a.fromBits(int('0111010101101011',2))
>>> a
43.34375
>>> b.fromBits(int('0111010101101011',2))
>>> b
22192.0
>>> a.toBinaryFormatted()
01110101 01101011
>>> b.toBinaryFormatted()
01110101 01101011
```

10.5.1 Comparing Posit and IEEE Float Ranges

The file *posit_range.py* generates all possible *posit16* and *float16* values by bit pattern ($2^{16} = 65536$ possible) before reporting how many are valid and how many are within specific ranges. The output is Table 10.3.

Table 10.3 Comparing *posit16* and *float16*

```
Number of invalids:
  posit16:    1  (65535 valid)
  float16: 2048 (63488 valid)
Range:
  posit16: +/- 268435456.000000
  float16: +/- 65504.000000
-1 <= n <= 1:
  posit16: 32769 (50.002%)
  float16: 30722 (48.390%)
-4 <= n <= 4:|
  posit16: 49153 (75.003%)
  float16: 34818 (54.842%)
-100 <= n <= 100:
  posit16: 62017 (94.632%)
  float16: 44162 (69.560%)
```

The first noticeable difference between *posit16* and *float16* lies in the number of bit patterns reserved for invalid floats. Posits, regardless of n and es , have at most a single invalid bit pattern, the one representing NaR. Therefore, *posit16* can use each of the remaining 65,535 bit patterns as valid floating-point numbers. On the other hand, IEEE *float16* reserves 2048 of the bit patterns for infinity and NaNs. This seems quite wasteful, given how rarely NaN payloads are used in practice.

A *posit16*'s dynamic range far exceeds that of a *float16*. However, as shown earlier in the chapter, the spacing between posit values grows rapidly as *maxPos* is approached. Therefore, while the difference between *posit16* and *float16* seems excessive, the posit's upper range is effectively useless for lack of precision.

The remainder of the output counts the number of valid floats in ± 1 , ± 4 , and ± 100 . Notice that 95 percent of *posit16* numbers are in ± 100 compared to 70 percent of *float16* values. This reinforces the previous paragraph's claim that the upper portion of a posit's range is effectively useless. In practice, posits work best with magnitudes closer to zero. *Float16* assigns 30 percent of its valid bit patterns to floats > 100 .

10.5.2 Comparing Posit and IEEE Float Precision

The file *posit_sum.py* compares *posit32* and *float32* by summing a user-specified number of random values in $[0, 1]$ multiplied by a given scale factor. Naïve summation is used for simplicity, knowing that Kahan summation is often more precise. The relevant part of the code is

```

1 random.seed(6502)
2 f64 = [rand(scale) for i in range(N)]
3 random.seed(6502)
4 f32 = [sf.float32(rand(scale)) for i in range(N)]
5 random.seed(6502)
6 p32 = [sp.posit32(rand(scale)) for i in range(N)]
7 sf64, sf32, sp32 = sum(f64), sum(f32), sum(p32)
8 bf32 = sum([float(v) for v in f32])
9 bp32 = sum([float(v) for v in p32])

```

Lines 1 through 6 generate three (potentially) identical lists of pseudorandom numbers multiplied by *scale*, which is read from the command line along with *N*. The first, *f64*, is the gold standard 64-bit reference. The other two (*f32* and *p32*) use 32-bit representations.

Line 7 sums each list, while lines 8 and 9 sum the 32-bit versions by transforming them to the closest 64-bit IEEE float.

Three runs using 100,000 samples for scale factors 1, 0.1, and 0.05 produced Table 10.4.

In each case, the *posit32* sum is closer to the 64-bit sum than the *float32* sum is. Moreover, as the scale factor decreases from 1 to 0.05, the absolute error diverges by up to three orders of magnitude between the posit and IEEE float version (0.0005 versus 0.02). Summing by mapping first to 64-bit floats reduces the deviation even more,

Table 10.4 Summing *float64*, *float32*, and *posit32* arrays

```

Summation results:
f64 : 50068.8225276773
p32 : 50068.6718750000 (error 0.1503906250)
f32 : 50069.1250000000 (error 0.3007812500)
p32 as f64: 50068.8225274990 (error 0.0000001783)
f32 as f64: 50068.8225257497 (error 0.0000019277)

Summation results:
f64 : 5006.8822527678
p32 : 5006.8879394531 (error 0.0056152344)
f32 : 5006.8427734375 (error 0.0395507812)
p32 as f64: 5006.8822526927 (error 0.0000000751)
f32 as f64: 5006.8822530728 (error 0.0000003050)

Summation results:
f64 : 2503.4411263839
p32 : 2503.4416503906 (error 0.0005493164)
f32 : 2503.4213867188 (error 0.0197753906)
p32 as f64: 2503.4411263668 (error 0.0000000171)
f32 as f64: 2503.4411265364 (error 0.0000001525)

```

which we now understand is due to posits' penchant for concentrating representable values around zero.

Sometimes, only a handful of iterations are required to cause a catastrophic loss of precision. For example, consider this sequence of calculations:

$$\begin{aligned}
 p &\leftarrow p^3 + 33p^2 + 1 \\
 p &\leftarrow p - 1 \\
 p &\leftarrow p - 33\pi^2 \\
 p &\leftarrow p/\pi^2
 \end{aligned}$$

If $p = \pi$ initially, p should be π at the end of the sequence. Mathematically, it will be. But computers cannot represent π , so round-off error will creep in. If the sequence is iterated, round-off error accumulates rapidly. Will posits fare better than IEEE floats? The file *posit_roundoff.py* runs the test, the essence of which is in Fig. 10.7.

The number of iterations ($\mathbb{N}$) is read from the command line along with a second argument to use $p = \pi$ or $p = 1/\pi$. *Float64*, *posit32*, and *float32* versions of p

```

1 N = int(sys.argv[1])
2 if (sys.argv[2].lower() == "pi"):
3     p = 3.141592653589793
4 else:
5     p = 1.0 / 3.141592653589793
6
7 p0,p1,p2 = p, sp.posit32(p), sf.float32(p)
8 P0,P1,P2 = p, sp.posit32(p), sf.float32(p)
9
10 for i in range(N):
11     p0 = p0*p0*p0 + 33*p0*p0 + 1; p0 = p0 - 1
12     p0 = p0 - 33*P0*P0; p0 = p0/(P0*P0)
13     p1 = p1*p1*p1 + 33*p1*p1 + 1; p1 = p1 - 1
14     p1 = p1 - 33*P1*P1; p1 = p1/(P1*P1)
15     p2 = p2*p2*p2 + 33*p2*p2 + 1; p2 = p2 - 1
16     p2 = p2 - 33*P2*P2; p2 = p2/(P2*P2)

```

Fig. 10.7 Iterating a sequence of calculations using *float64*, *float32*, and *posit32*

come next (lines 8 and 9). The loop in line 10 performs the sequence of calculations for each float type before reporting the final value of p (not shown).

For example, running *posit_roundoff.py* with $N = 1$ and $p = \pi$ returns:

```

original : 3.1415926536
float64   : 3.1415926536 (error 0.0000000000)
posit32   : 3.1415928900 (error 0.0000002364)
float32   : 3.1415934563 (error 0.0000008027)

```

indicating that *posit32* is more precise than *float32*. Round-off error grows as N increases. For $N = 3$:

```

original : 3.1415926536
float64   : 3.1415926536 (error 0.0000000000)
posit32   : 3.1417281628 (error 0.0001355092)
float32   : 3.1420571804 (error 0.0004645268)

```

And for $N = 5$ and 7:

```

original : 3.1415926536
float64   : 3.1415926542 (error 0.0000000006)
posit32   : 3.2197590470 (error 0.0781663934)
float32   : 3.4098403454 (error 0.2682476918)

original : 3.1415926536
float64   : 3.1415929908 (error 0.0000003372)
posit32   : 65.0932788849 (error 61.9516862313)
float32   : 392.3526916504 (error 389.2110989968)

```



```

1 def lfit_posit16(x,y):
2     N = sp.posit16(len(x))
3     sumx = sp.posit16(0); sumy = sp.posit16(0)
4     sumx2 = sp.posit16(0); sumxy = sp.posit16(0)
5     for i in range(len(x)):
6         sumx = sumx + sp.posit16(x[i])
7         sumy = sumy + sp.posit16(y[i])
8         sumxy = sumxy + sp.posit16(x[i])*sp.posit16(y[i])
9         sumx2 = sumx2 + sp.posit16(x[i])*sp.posit16(x[i])
10    d = N*sumx2 - sumx*sumx
11    m = (1/d)*(N*sumxy - sumx*sumy)
12    b = (1/d)*(sumx2*sumy - sumx*sumxy)
13    return m,b

```

Fig. 10.8 Least-squares fit to a line using *posit16*

Catastrophic round-off error occurs after seven iterations, but before that, the *posit32* value is more precise. I leave it as an exercise to run *posit_roundoff.py* using *ip* as the second command line argument to test $p = 1/\pi$.

The next example compares *posit16* and *float16* for a simple linear least-squares fit of random data to a line. The code is in *posit_lfit.py*. It implements a basic linear least-squares fit ala Bevington [2]; see Fig. 10.8.

The program runs in two regimes depending on whether or not a command line argument exists (any value will do). No command line argument selects *x* and *y* via:

```

m = -3 + 6*random.random()
b = -2 + 4*random.random()
x = [-4+8*random.random() for i in range(30)]
y = [m*i+b+(random.random()-0.5) for i in x]

```

where *m* and *b* are the nominal slope and intercept. Adding a command line argument divides all constants by 10 to push the data into a region bounded by ± 1 .

For example, a run with no command line argument produced:

```

Fit:
float64: y = 1.55385679x + 0.39708240
posit16: y = 1.53759766x + 0.40161133 (0.01625913, 0.00452893)
float16: y = 1.55273438x + 0.39624023 (0.00112241, 0.00084216)

```

In this case, *float16* produced more accurate results according to the absolute deviation from the *float64* gold standard (values in parentheses).

We understand the reason: the scale of the problem is such that the values involved, especially when summed and squared as required by the fit algorithm, are outside of the range of the bulk of the *posit16* values, but within a range where *float16* values were still relatively dense.

Adding a command line argument scales the problem and alters the results:

```
Fit:
float64: y = -0.09616701x + 0.42379393
posit16: y = -0.09600830x + 0.42376709 (0.00015871, 0.00002684)
float16: y = -0.09637451x + 0.42407227 (0.00020751, 0.00027834)
```

As expected, the *posit16* results are now more accurate. Repeated runs for each condition demonstrate the overall conclusions, though at times the posit values will be less accurate compared to the IEEE floats and vice versa for each range.

10.5.3 Comparing Posit and IEEE Compressibility

This experiment compares the compressibility of files stored on disk as posits or floats, both 16 and 32 bit. Compression is a proxy measure for the entropy of a dataset. The question asked is: Are posit bit patterns structured in a way that leads to lower overall entropy?

The experiment itself is split between two source code files ending in *part1.py* and *part2.py*. The SoftPosit and SoftFloat libraries are wrappers around an underlying C++ library which, when asked to output different representations as either binary or hexadecimal, uses `stdout` requiring capture of the output outside of Python itself. This is the job of *posit_compress_part1.py*. The captured output file then becomes the input to the second part of the experiment.

Figure 10.9 presents the part 1 code. It accepts three command line parameters: the source file name (`sname`), floating-point bit width (16 or 32, `typ`), and a scale parameter, α (alpha). The file is processed byte by byte, with each byte multiplied by the floating-point scale factor. As explained, the output of *posit_compress_part1.py* must be captured at the command line level.

```
1 | sname = sys.argv[1]
2 | typ = int(sys.argv[2])
3 | alpha = float(sys.argv[3])
4 |
5 | d = open(sname, "rb").read()
6 | d64 = []
7 | for v in d:
8 |     d64.append(alpha*v)
9 |
10 | for v in d64:
11 |     x = sp.posit32(v) if (typ==32) else sp.posit16(v)
12 |     x.toHex()
13 | for v in d64:
14 |     x = sf.float32(v) if (typ==32) else sf.float16(v)
15 |     x.toHex()
```

Fig. 10.9 Representing files as posits and floats (16 and 32 bit)

The second part of the experiment, *posit_compress_part2.py*, expects two arguments: 16 or 32 to specify the bit width used by part 1 and the name of the text file containing part 1's output. For example, to process a file named *cafe.png*, use a sequence of steps:

```
> python3 posit_compress_part1.py cafe.png 16 1 >hex.txt
> python3 posit_compress_part2.py 16 hex.txt
1417946 1400515
```

The output is the compressed file size for *posit16* and *float16* versions of the original file. In this case, $\alpha = 1$ and we see that the *posit16* version compressed less, implying greater entropy or more variation in the bits needed to represent the original bytes of the file. A second run using 32-bit floating point and $\alpha = 0.1$ returned:

```
> python3 posit_compress_part1.py cafe.png 32 0.1 >hex.txt
> python3 posit_compress_part2.py 32 hex.txt
1748711 1787906
```

In this case, the *posit32* representation was more compressible than the *float32* version. Scaling with $\alpha = 0.1$ effectively reduced the data range from $[0, 255]$ to about $[0, 25]$. In this range, *posit32* is more compressible, perhaps because of the regime bits. Please review the file *posit_compress_part2.py* to follow the process mapping *hex.txt* to the returned compressed file sizes.

The script *posit_compress_test* applies the two-step compression test to each of the files in the *data* directory ($\alpha = 1$). The files are of two kinds: PNG compressed images and text files of classic literary works. The script's output was captured and incorporated into the file *posit_compress_test.py*. Executing this file produces Table 10.5.

The table is split between 16-bit and 32-bit floating-point values. The numbers are the gzip compressed file sizes with posit on the left and float on the right. The < or > sign facilitates determining the relationship. An interesting effect appears between 16 bit and 32 bit.

When using *posit16*, the text files are better compressed than the corresponding *float16* versions, while the relationship is flipped for the image files. The 32-bit results show the opposite effect.

Why might this be? The text files are plain ASCII files. They use a narrow range of byte values, about $[32, 126]$, ignoring line-ending characters. Conversely, the PNG image files are approximately evenly distributed across all $[0, 255]$ byte values with a slight excess of zeros.

The *posit16* format might be representing the character range with a consistent high-order byte, thereby giving gzip more to work with when determining redundancy in the file compared to how that range is represented in *float16*. The converse might be valid for the image data using all available byte values.

Table 10.5 Comparing posit and IEEE float compression by floating-point bit width

Compressed file size:			
16:			
alice.txt:	76791	<	80919
dracula.txt:	426701	<	450674
frankenstein.txt:	214596	<	227330
pride.txt:	355778	<	379892
romeo.txt:	79757	<	83502
scarlet.txt:	257213	<	272363
tale.txt:	381327	<	401486
cafe.png:	1417946	>	1400515
mona.png:	913422	>	904521
nothingness.png:	1044550	>	1032416
vase.png:	1361577	>	1343501
walker.png:	1004986	>	991856
woman.png:	1043004	>	1030414
32:			
alice.txt:	106082	>	97616
dracula.txt:	586026	>	539432
frankenstein.txt:	302241	>	274117
pride.txt:	504948	>	460096
romeo.txt:	106000	>	99619
scarlet.txt:	358009	>	327306
tale.txt:	520982	>	478613
cafe.png:	1678935	<	1696017
mona.png:	1081026	<	1091563
nothingness.png:	1238896	<	1250459
vase.png:	1604582	<	1618213
walker.png:	1186016	<	1196879
woman.png:	1237867	<	1248874

The *float32* representation of the character data is possibly more redundant than the *posit32* representation over the same range, thereby explaining the reversal with the opposite effect over the entire $[0, 255]$ range, giving *posit32* a slight edge.

These results, along with the brief example using $\alpha = 0.1$, serve to reinforce the notion that for certain numeric ranges, posits might be more efficient when it comes to minimizing file size, at least for the gzip algorithm.

Let's continue testing posits. This time, by using them with neural networks.

10.6 Posits and Neural Networks

Modern AI is a natural target for posits. This section serves as a superficial exploration of how posits fare when representing the weights and biases of a trained neural network. In general, neural networks are trained using relatively high-precision floating-point values, say *float32* or, at times, *float64*. After training, networks are often modified to use lower precision representations for inference (using the network for something). Chapter 11 explores representations currently in common use for this purpose. Posits are not presently included in that set.

Our experiment first trains a simple multilayer perceptron (MLP), a classical neural network architecture, to classify small 14x14 pixel images, either handwritten digits (MNIST) or articles of clothing (Fashion MNIST or FMNIST). Image classification uses convolutional networks, but these datasets are simple enough that a classical model can do reasonably well. To use a classical MLP, the images are unraveled by laying rows end to end. Therefore, the train and test datasets consist of $14 \times 14 = 196$ element vectors. We'll train the models first using *float64* precision. Then, we'll map the resulting weights and biases, the parameters of the network sought by training, to *float32*, *float16*, *posit32*, *posit16*, and *posit8* to learn how each representation alters the expected test set performance, if at all.

10.6.1 Training the Models

Training a neural network means conditioning the model's parameters to perform well on novel, unseen data in the wild. Conditioning involves presenting the model repeatedly with known data, data for which we already know the desired output we wish from the model, like a class label to help us differentiate between, say, cats and dogs or, for us, any of the digits, 0 through 9, or any of ten types of clothing and shoes. This is supervised learning and is the most common use case in practice.

A traditional MLP processes data layer by layer. The input vector is mapped to the first hidden layer via a weight matrix and an associated bias vector. The output of the hidden layer is mapped to the next hidden layer, if any, or the output layer by another weight matrix and bias vector pair. The goal of training is to find values for the elements of these vectors and matrices to make the model perform as well as possible (what that means precisely is task-specific).

We'll train a model with a 196-element input vector, $\mathbf{x}$, two hidden layers of 100 and 50 nodes, and a 10-node output layer. The output becomes a 10-element vector, each representing the strength of the model's belief that the input represents the corresponding class (digit or article of clothing).

Conditioning the model follows AI's standard training algorithm: stochastic gradient descent with backpropagation. Gradient descent uses the gradient of a loss function (the model's errors using its current set of parameters) to adjust the parameter values to minimize the loss. The word "stochastic" relates to training with small subsets of the available training data for each gradient descent step. This is helpful to avoid local minima traps early in the training process. Backpropagation is a glorified

application of the chain rule for derivatives to supply $\partial L / \partial w$ for each weight and bias in the network. Knowing the derivatives permits the gradient descent step.

Fortunately, we do not need to know more than this to run the code. It requires NumPy and Scikit-Learn. The former to endow Python with array processing abilities and the latter to supply ready-made neural network code that greatly simplifies the experiment. On Linux systems, the following should install both libraries:

```
> pip3 install numpy
> pip3 install scikit-learn
```

The code we need is in *posit_train.py*. It expects a single command line argument, *mnist* or *fmnist*, to train an instance of `sklearn`'s `MLPClassifier` class from which the weights and biases are extracted and stored on disk as Python `pickle` files. Should you wish to jump directly to the tests, the book's GitHub contains the necessary `pickle` files, thereby alleviating the need to run *posit_train.py*:

```
posit_fmnist_weights.pkl
posit_mnist_weights.pkl
```

The model uses three weight matrices and three bias vectors. The first maps the input vector to the argument of the first hidden layer's activation function (a nonlinearity necessary for the model to learn). The second maps the output of the first hidden layer to the second hidden layer, and the third the second layer's output to the model's output layer. The activation function is a rectified linear unit (ReLU) that sets negative input elements to zero while outputting positive elements unchanged. In other words, ReLU applies $\max(0, b)$ to every element, b , of a vector.

Mathematically, the operation of a trained MLP during inference is a straightforward set of matrix–vector operations. Let $\mathbf{x}$ be an input vector (an image scaled $[0, 1]$ and unraveled), $\mathbf{W}$ a weight matrix, $\mathbf{b}$ a bias vector, and h the ReLU activation function (accepting and emitting vectors). Then,

$$\begin{aligned} \mathbf{a}_0 &= h(\mathbf{W}_0 \mathbf{x} + \mathbf{b}_0) \\ \mathbf{a}_1 &= h(\mathbf{W}_1 \mathbf{a}_0 + \mathbf{b}_1) \\ \mathbf{y} &= h(\mathbf{W}_2 \mathbf{a}_1 + \mathbf{b}_2) \end{aligned} \tag{10.2}$$

for $\mathbf{a}_0$ and $\mathbf{a}_1$ the activation vectors produced by the hidden layers and $\mathbf{y}$ the model output, a 10-element vector of logits. It is typical to apply a softmax operation to the logits, but that is not strictly necessary to select a class label during inference, and doing so requires an exponential function, something neither `softposit` or `softfloat` supply.

The MNIST dataset is in:

```
mnist_14x14_xtrn.npy, mnist_14x14_ytrn.npy
mnist_14x14_xtst.npy, mnist_14x14_ytst.npy
```

with corresponding `fmnist` versions for Fashion MNIST. The files are NumPy arrays where `xtrn` and `xtst` are train and test images with corresponding integer labels, `ytrn` and `ytst`. The training set conditions the model, and the test set is used to evaluate the model's performance.

Training the model is straightforward:

```
clf = MLPClassifier(hidden_layer_sizes=(100,50),
                    tol=1e-6, batch_size=64, verbose=1)
clf.fit(xtrn, ytrn)
```

The model instance is created (`clf`) and then trained by calling `fit`. The arguments are the training matrix `xtrn`, a $10,000 \times 196$ matrix of approximately 1000 samples of each digit or article of clothing. The test set, `xtst`, is a $3,000 \times 196$ matrix.

The neural network has two hidden layers of 100 and 50 nodes. The tolerance specifies how Scikit-Learn will decide when to stop training (up to a default of 200 epochs, which it calls iterations). The `batch_size` parameter sets the number of samples used to calculate a gradient descent step. This is the minibatch size in stochastic gradient descent. Setting `verbose` outputs the loss function value as the model trains. The smaller the loss, to a point, the better trained the model is likely to be. Make the loss too small, however, and the model will likely begin to overfit to the minute details of the training set and miss the general features useful for inference in the wild. Scikit-Learn's default behavior based on the tolerance (to lack of change in the loss function) will, in this instance, keep things reasonable. A state-of-the-art model is not our goal; good enough is just fine in this case.

The file `posit_train.py` tests the model on the full test set, producing both a confusion matrix and an overall accuracy (number correctly classified / number of test samples). The confusion matrix shows how often a true class (row) is labeled as each of the other classes by the model (columns). A perfect model makes no mistakes, resulting in a purely diagonal confusion matrix. For example, the MNIST model in the provided Python `pickle` file generates this confusion matrix:

$$\begin{bmatrix} 264 & 0 & 1 & 0 & 0 & 0 & 4 & 0 & 1 & 1 \\ 0 & 334 & 2 & 0 & 0 & 0 & 3 & 0 & 1 & 0 \\ 3 & 0 & 296 & 0 & 1 & 0 & 3 & 4 & 4 & 2 \\ 0 & 1 & 5 & 297 & 0 & 3 & 0 & 4 & 4 & 2 \\ 0 & 0 & 1 & 0 & 300 & 0 & 5 & 2 & 1 & 9 \\ 2 & 0 & 0 & 4 & 2 & 267 & 4 & 2 & 2 & 0 \\ 2 & 3 & 2 & 0 & 4 & 2 & 255 & 1 & 3 & 0 \\ 1 & 4 & 5 & 2 & 3 & 0 & 0 & 286 & 0 & 5 \\ 4 & 0 & 3 & 4 & 4 & 1 & 0 & 4 & 264 & 2 \\ 3 & 1 & 1 & 4 & 4 & 5 & 1 & 3 & 0 & 273 \end{bmatrix}$$

meaning class 1 (digit 1) was correctly labeled 334 times, but there were 2 instances that the model called a "2", 3 it called a "6", and 1 an "8".

The overall accuracy was 0.9453 when using `float64`. The overall FMNIST accuracy was 0.8413, as FMNIST is a more challenging classification problem by design.

Therefore, the best accuracy achievable for any tested floating-point precision on FMNIST is 0.8413, at least in theory; we'll learn otherwise momentarily.

We have the models. Let's run the test set through them after representing the weights and biases with different precision IEEE floats and posits.

10.6.2 Testing the Models

Testing different float types means implementing Equation 10.2, where each vector and matrix uses a specified float. It's essential to ensure that arithmetic operations are performed using just the desired float and not by casting to Python's native `float` first (*float64*).

Fortunately, NumPy supports vectors and matrices of objects, and, as luck would have it, `softposit` and `softfloat` produce objects. Therefore, we need to only create NumPy arrays containing the necessary float objects, then perform the calculations of Equation 10.2. NumPy, courtesy of Python's "dunder" methods (methods supporting arithmetic operations), will ensure that calculations proceed as required.

The code we need is in *posit_test.py*. The only command line argument is the name of the model (`mnist` or `fmnist`). Equation 10.2 is in the `Predict` function; see Fig. 10.10.

```

1 def Predict(xtst, ytst, weights, biases, mode):
2     X = Float(xtst, mode)
3     pred = []
4     value = {}
5     for c in range(10):
6         value[c] = []
7     for i in range(len(ytst)):
8         x = np.maximum(0,
9             np.dot(Float(X[i], mode),
10                  Float(np.array(weights[0]), mode))
11                 + Float(np.array(biases[0]), mode))
12         x = np.maximum(0,
13             np.dot(Float(x, mode),
14                  Float(np.array(weights[1]), mode))
15                 + Float(np.array(biases[1]), mode))
16         x = np.maximum(0,
17             np.dot(Float(x, mode),
18                  Float(np.array(weights[2]), mode))
19                 + Float(np.array(biases[2]), mode))
20         c = np.argmax(x)
21         v = float(x[c])
22         pred.append(c)
23         value[c].append(float(x[c]))
24     return ConfusionMatrix(pred, ytst), value

```

Fig. 10.10 Implementing Equation 10.2 in code

Lines 8 through 20 implement Equation 10.2 using `np.maximum` for ReLU and `np.dot` for matrix multiplication. The `for` loop in line 7 is over each test sample (feature vector). The weights and biases are lists requiring `np.array` to transform them into NumPy matrices and vectors, respectively.

The `Float` function returns the given matrix or vector as a new object matrix or vector where each element is an instance of a specific floating-point type according to `mode`:

```

1 def Float(x, mode):
2     if (len(x.shape) == 1):
3         ans = np.array([F(v,mode) for v in x])
4     else:
5         row, col = x.shape
6         ans = np.array([[F(0,mode) for j in range(col)]
7                           for i in range(row)])
8         for i in range(row):
9             for j in range(col):
10                ans[i,j] = F(x[i,j],mode)
11     return ans

```

The function reconstructs the input matrix or vector applying `F` to each *float64* value. The returned NumPy array will be of type object with the exception of mode *float64* which uses Python's native *float* type. Specifically,

```

1 def F(x,mode):
2     if (mode == "float32"): return sf.float32(float(x))
3     elif (mode == "float16"): return sf.float16(float(x))
4     elif (mode == "posit32"): return sp.posit32(float(x))
5     elif (mode == "posit16"): return sp.posit16(float(x))
6     elif (mode == "posit8") : return sp.posit8(float(x))
7     else: return float(x)

```

converts a *float64* value (`x`) into a new representation according to `mode`. The conversion happens on demand in `Predict` to ensure that the values manipulated by NumPy are of the proper type.

It's known that neural networks are generally not overly sensitive to the precision of their weights [8], so we have some reason to hope that moving from 64-bit floats to smaller floats will preserve most of the model's performance. Running *posit_test.py* for MNIST and then FMNIST produces output consisting of confusion matrices, overall accuracy on the test set, and the mean logit value per class over the test set, along with the standard deviation. You'll find the output for both models captured in the files:

```

posit_mnist_test_results.txt
posit_fmnist_test_results.txt

```

For example, consider the FMNIST and *float32* results:

Table 10.6 Overall accuracy on the test set by model and floating-point type

Model	float64	float32	float16	posit32	posit16	posit8
MNIST	0.9453	0.9453	0.9453	0.9453	0.9453	0.9453
Fashion MNIST	0.8413	0.8413	0.8420	0.8413	0.8410	0.8263

```
float32: 0.84133333
13.79, 22.99, 11.28, 12.84, 11.73, 21.13, 12.08, 16.27, 23.50, 21.62
7.50, 7.22, 4.42, 5.80, 4.26, 10.31, 6.04, 6.44, 11.91, 8.10
```

which I’ve presented here in compact form.

The overall *float32* accuracy was 84.1 percent. There are ten classes, the mean logit value of which is the second row with the corresponding standard deviation in the third row. The class labels are unimportant, but comparing the logits as precision changes is. First, however, let’s examine the overall accuracy for each model; see Table 10.6.

The MNIST model’s accuracy does not change regardless of floating-point type or precision. This isn’t surprising as the MNIST dataset is known to be particularly easy for models to learn well. The result also reinforces the notion that neural networks are, to a degree, robust to “noise” introduced by lower precision calculations. The fact the model’s chosen target label has not changed with precision, however, does not mean that the values produced by the model are unaltered, as we’ll learn momentarily.

The Fashion MNIST results are more realistic and aligned with what we might encounter with a real-world dataset. Notice that the 32-bit floats, posit or IEEE, perform as well as the original 64-bit data. Changes appear when moving to 16- and 8-bit floats.

First, *float16*’s accuracy is slightly higher than the original model. Looking across the diagonals of the confusion matrices for *float64* and *float16* gives deltas between them of

$$+1, -1, +1, +1, -1, 0, +1, 0, 0, 0 \rightarrow +2$$

implying that the *float16* calculations, by chance, altered the logits enough to make the model select the correct target label slightly more often than with the original precision. I think of this as a mutation in biological evolution: mutation is often neutral or detrimental, but, on occasion, helpful. It would be incorrect to believe that *float16* is somehow superior to the other representations based on this single observation.

The *posit16* result is slightly worse, as might be expected by random chance. The *posit8* result is less precise, but, in practice, it might be a tolerable trade-off with the lower power required to run *posit8* hardware versus *float64* hardware, to say nothing of the 4x reduction in memory footprint.

Overall, the accuracies in Table 10.6 make sense and hint at what might happen if we continue this test with many other datasets. A full investigation would also transition from classical neural networks to convolutional neural networks. It’s natural to wonder if the sensitivity to floating-point representation carries over to different neural network architectures, and if so, does sensitivity increase or decrease?

Selecting a class label means, for each test sample, locating the largest of the ten logit values produced by the model. The index of the largest logit is the assigned class. Therefore, as floating-point representation and precision change, the assigned label remains the same as, along as the corresponding logit is still the maximum of the output vector. Here’s where the mean per-class logits over the test set come into play.

Visually, we see that the logits do change slightly at first until we get to *posit8*, where the values are substantially different from those in the *float64* case. One way to rank the effect of floating-point type is to use the mean logit vectors as points in a 10-dimensional space and calculate the Euclidean distance between each float type and *float64*. The smaller the distance, the more similar the logits when using that type of float; see Table 10.7.

Table 10.7 implies that the *posit8* logits were significantly affected by the altered precision. Unfortunately, the `softfloat` library does not implement 8-bit IEEE-style floating point, so we cannot directly compare such with 8-bit posits. However, we’ve learned enough in this chapter to suspect that *posit8* might be more accurate than an 8-bit IEEE float. Chapter 11 explores other floating-point representations currently in use in AI, including what we might call *float8*.

Table 10.7 Euclidean distance between mean per-class logits using *float64* as the gold standard for MNIST (top) and FMNIST (bottom)

	float64	float32	float16	posit32	posit16	posit8
float64	0.000					
float32	0.000	0.000				
float16	0.043	0.043	0.000			
posit32	0.000	0.000	0.043	0.000		
posit16	0.008	0.008	0.043	0.008	0.000	
posit8	13.278	13.278	13.272	13.278	13.277	0.000
	float64	float32	float16	posit32	posit16	posit8
float64	0.000					
float32	0.000	0.000				
float16	0.075	0.075	0.000			
posit32	0.000	0.000	0.075	0.000		
posit16	0.013	0.013	0.065	0.013	0.000	
posit8	12.706	12.706	12.723	12.706	12.706	0.000

Table 10.7 also reinforces the belief that neural networks are somewhat robust to the precision of their weights because even though the mean *posit8* MNIST logit vector is much further from the gold standard *float64* vector compared to the other representations, there was no effect on the assigned class labels. Further, the FMNIST results, while slightly worse overall, were still close to the *float64* results, again justifying claims that reduced floating-point precision (and associated power savings) is worth considering in practice.

Finally, remember that this exercise trains the model using maximum precision and then uses lower precision weights during inference. This approach is natural and common. Less common is training the model with lower precision floats. Both approaches are active research areas and have different use cases. The first takes advantage of the fact that training is a one-time exercise (typically) followed by deployment (inference) in some environments that may include edge devices with limited computational abilities. The second is for large, computationally expensive models, e.g., LLMs, or for “in the wild” situations where a device like a robot must adapt to new situations on the fly by assembling data and building a model.

10.7 Discussion

What are we to make of posits? An analogy comes to mind: the old debate between VHS and Betamax video cassettes. VHS (Video Home System) was a video cassette format introduced in the 1970s. Betamax, by Sony, was a competing format. It was widely acknowledged that Betamax’s picture quality was superior, but nonetheless, VHS carried the day and became the industry standard. IEEE 754 is ubiquitous and unlikely to disappear anytime soon. Its weaknesses are known, and even though this chapter’s experiments demonstrate posits’ claims of improved accuracy and dynamic range, IEEE 754’s long head start, plus the “devil you know” thought process, implies a marginalized future for posits, which is unfortunate.

However, there is a potential bright spot: AI systems. Multiple studies have demonstrated the utility of posits for inference and even training on edge devices. For example, see [3] [4] [5] [6] [7]. We discuss AI number formats in the next chapter.

Exercises

10.1 Write `posit16` to parse a standard $n = 16$, $es = 1$ posit with a 1-bit exponent. Do not forget the “missing bits are zero” rule. Use any programming language you find convenient. **

10.2 An n -bit posit allows $0 \leq es \leq n - 3$ because all posits require a sign bit and at least two regime bits (10_2 or 01_2). Fix $n = 16$. Write a program to generate multiple look-up tables for all possible values of es . What are the densest regions of

the resulting posits as a function of es ? Do these regions or ranges seem useful for specific applications where the values are typically within that range? How do these ranges compare to the *posit16* standard using $es = 1$?

10.3 Repeat the compression exercise, replacing *gzip* with *xz* or *zip*. Appropriately edit the file *posit_compress_part2.py*. Do the results follow the *gzip* results? If not, what might be happening? What is the relationship between the *gzip* file sizes and the *xz* sizes? Review how each algorithm works to learn if that understanding clarifies your results.

10.4 Repeat the neural network exercise of Sect. 10.6 with other datasets suitable for a small MLP with two hidden layers of 100 and 50 nodes and 10 output labels. Options include CIFAR-10, a 10-element subset of CIFAR-100, and various alternative forms of MNIST. All of these datasets are easily located online. Preprocessing includes unraveling images to become vectors and dividing by the maximum possible pixel value (typically 255) to map the vectors to $[0, 1]$. Do the observations made in the chapter persist? Are there types of data more or less susceptible to floating-point representation? ***

References

1. Gustafson JL, Yonemoto IT (2017) Beating floating point at its own game: posit arithmetic. *Supercomput Front Innovat* 4(2):71–86
2. Bevington PR, Robinson DK (1969) Data reduction and error analysis. McGraw-Hill, New York
3. Murillo R, Del Barrio AA, Botella G (2020) Deep PeNSieve: a deep learning framework based on the posit number system. *Digit Signal Process* 102:102762
4. Langroudi SH, Pandit T, Kudithipudi D (2018) Deep learning inference on embedded devices: Fixed-point vs posit. In: 2018 1st workshop on energy efficient machine learning and cognitive computing for embedded applications (EMC2). IEEE, pp 19–23
5. Langroudi HF, Karia V, Gustafson JL, Kudithipudi D (2020) Adaptive posit: parameter aware numerical format for deep learning inference on the edge. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops, pp 726–727
6. Raposo G, Tomás P, Roma N (2021) Positnn: Training deep neural networks with mixed low-precision posit. In: ICASSP 2021–2021 IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, pp 7908–7912
7. Carmichael Z, Langroudi HF, Khazanov C, Lillie J, Gustafson JL, Kudithipudi D (2019) Deep positron: a deep neural network using the posit number system. In: 2019 Design, automation and test in Europe conference and exhibition (DATE). IEEE, pp 1421–1426
8. Hubara I, Courbariaux M, Soudry D, El-Yaniv R, Bengio Y (2018) Quantized neural networks: training neural networks with low precision weights and activations. *J Mach Learn Res* 18(187):1–30



Abstract

The explosion of artificial intelligence (AI) in recent years, powered by neural networks, has led to the development of numerous new number formats. This chapter explores the most common formats to understand why they exist and how they are used in AI. Experiments aid the discussion to illustrate the effects of model quantization, the process by which full-precision model weights and biases are mapped to reduced-precision formats for storage.

11.1 New Number Formats?

IEEE floating point is the industry standard and, though not perfect, has worked well for decades, so a natural question to ask is: Why consider alternative floating-point sizes for AI? The short answer is memory.

Modern neural networks consist of large collections of weights and biases, the magic numbers a trained network uses to do what it's trained to do. In this chapter, when I refer to the “weights” of a network, I'm referring collectively to the weights and biases. The following section defines the terms in more detail, but it is fair to consider a neural network as a function mapping an input, typically a vector or matrix, to a set of outputs, typically class likelihoods. Mathematically, we might write

$$\mathbf{y} = \mathbf{f}(\mathbf{x}; \boldsymbol{\theta}) \quad (11.1)$$

where $\mathbf{x}$ is the input vector, $\mathbf{y}$ is the set of outputs, and $\boldsymbol{\theta}$ is a collection of parameters associated with the network architecture and learned during network training. The architecture implements $\mathbf{f}$, a function accepting and returning a vector. A curve fitting analogy is helpful: in $y = f(x; \boldsymbol{\theta})$, f is the fit function, and $\boldsymbol{\theta}$ is the set of function parameters determined from the dataset being fit.

A simple neural network uses thousands to tens of thousands of parameters. The convolutional networks often used in computer vision tasks bump that number into the millions. Generative AI models, like the large language models (LLMs) used in Chap. 12, reach into the billions for small LLMs and the trillions for state-of-the-art models like GPT-4 as its successors. Efficient memory management is critical. An IEEE *float64* value uses 8 bytes of memory. If a model can get by with *float32* precision, memory use is halved. A model that works well with *float16* weights uses 4x fewer bytes, and, if possible, a model using single-byte weights reduces naïve memory use by a factor of eight. Small savings add up quickly when θ contains billions or trillions of values. Therefore, it isn't surprising that AI researchers investigate reduced-precision representations.

Memory isn't the only motivating factor, however. It has been shown and will be again later in the chapter that neural networks are insensitive to the exact precision of their weights. For an early example of such, see [1]. The observation is in accordance with a recent study of the human brain that arrived at a memory capacity of about 4.7 bits per synapse [2].

Memory (storage) requirements necessitate contemplation of reduced-precision model weights. As luck would have it, such a reduction in precision isn't a fatal blow to the utility of neural networks. Many of the new (and continually emerging) AI number formats are discussed in the following sections. However, before doing so, we should spend some time understanding neural network training and inference. The process dictates how and when reduced-precision values are applicable; it isn't a one-size-fits-all proposition.

11.2 Model Training and Inference

A neural network is a model, a model representable as f in Eq. 11.1. While it is always necessary to remember British statistician George Box's adage that "all models are wrong, but some are useful" it is true that the model learned by a neural network, solely from example data, is often extremely useful and accurate. This section describes the training process for a neural network and points out where reduced-precision representations have a role to play. A complete understanding of neural networks is beyond our ken, but interested readers are humbly directed to [3,4].

Building a neural network consists of selecting an arrangement of nodes (neurons) implementing an architecture, an appropriate nonlinear activation function (we'll use the rectified linear unit or ReLU here), and hyperparameters related to how the training process unfolds. As we're most interested in representing numbers on computers, we focus on the network weights, the quantity and arrangement of which are determined by the architecture.

A training dataset is used as a proxy for the data-generating parent distribution that creates the set of possible network inputs, the inputs the model encounters when used in the wild. Supervised training uses pairs of inputs and known outputs, along with a stochastic gradient descent process informed by per-weight derivatives determined by the backpropagation algorithm (think the chain rule for derivatives from calculus), to arrive at a set of weights that minimizes a loss function capturing something related to how we wish the model to perform. The generative AI models of Chap. 12 are similarly trained, but classic supervised learning is all we need in this chapter.

Training with an eye towards reducing the precision and thereby memory use of the weights often involves *mixed-precision* calculations. Typically, weights are stored in reduced precision, mapped to a higher precision on demand when needed, and, during the backward pass that updates the weights, mapped back to a lower precision to be used on the next forward pass. In the end, the reduced-precision weights are stored for downstream use.

Future use of a trained model is known as *inference*. Inference is the goal, in a sense, because why train a model if you don't intend to use it somewhere? Inference uses model weights as they are without modifying them. The curve fit analogy is again helpful: once the parameters, θ , are known, the y for any new x follows immediately from $y = f(x; \theta)$.

Training a model using a mixture of precisions is an ongoing research area. It's essential, but we'll ignore it here to focus instead on the effects introduced by reducing the precision of a trained model's weights, effects seen during inference when the model is used.

During inference, the reduced-precision (*quantized*) weights are mapped to higher precision (*dequantized*) for calculations. Neural networks are typically arranged in layers, where the output of one layer becomes the input to the next. The inference calculations for a layer produce higher precision outputs, which are maintained as the input to the following network layer. Weights are dequantized on demand.

Training with mixed precision to produce quantized model weights is likely the best approach in terms of building a model that has a memory footprint small enough for an edge device (think smartphone), but there is definite utility in taking a model trained traditionally and quantizing the weights after the fact. Modern deep learning toolkits, like TensorFlow, provide tools for such *post-training quantization* to process a model with high-precision weights (e.g., FP32) to a lower precision TensorFlow Lite model. The process generally produces a model using a scaled integer format like those discussed in Sect. 11.4.

Inference-time quantization presents in one of two forms: reduced-precision floating point or scaled integer. Let's review each, beginning with reduced-precision floating point. We follow with an experiment exploring the effect of floating-point precision on a simple neural network trained using FP64 and close with a brief discussion.

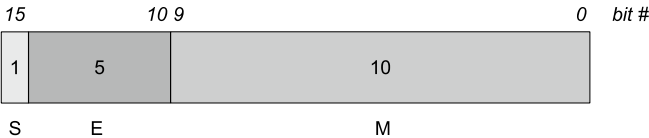
11.3 Reduced-Precision Floating-Point Formats

Chapter 3 discusses the IEEE floating-point format in detail. The reduced-precision floating-point formats currently supported for AI hardware like NVIDIA GPUs follow IEEE for the most part but alter the number of bits used for the exponent and mantissa (significant). Compare this to the posit format of Chap. 10.

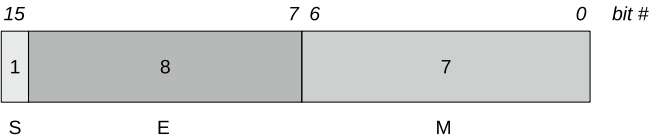
AI treats *float64* and *float32* as full precision. IEEE 754 *binary16*, denoted here as *float16*, is similarly widely used and supported by AI hardware. This section discusses the most commonly encountered formats and new approaches involving tiny floats (FP4) tailored to neural network training.

11.3.1 Half-Precision (FP16) Formats

Half-precision (FP16) floats use 16 bits for storage. IEEE *float16*, as presented in Chap. 3, is structured as



to reserve 10 bits for the mantissa and 5 bits for the excess-15 exponent. Google Brain’s 16-bit brain floating-point format (*bfloat16*), on the other hand, increases the exponent range at the expense of the mantissa:



where the mantissa is now only 7 bits, but the 8-bit exponent (excess-127) matches that of an IEEE *float32* value.

The altered exponent fields affect the overall range:

Name	Minimum	Maximum
<i>float16</i>	$2^{-14} \approx 6.104 \times 10^{-5}$	$(2 - 2^{-10}) \times 2^{15} = 65504.0$
<i>bfloat16</i>	$2^{-126} \approx 1.175 \times 10^{-38}$	$(2^8 - 1) \times 2^{-7} \times 2^{127} \approx 3.390 \times 10^{38}$

far exceeding that of *float16* but with only a few decimal digits of accuracy.

IEEE *float16* supports all the expected special numbers ($\pm$ infinity, NaN) and subnormal numbers. The *bfloat16* format also supports $\pm$ infinity and NaN but lacks subnormal numbers. For example, infinity is sign plus all exponent bits set and all mantissa bits cleared:

Name	Positive infinity				Negative infinity			
<i>float16</i>	0	11111	0000000000	1	11111	0000000000		
<i>bfloat16</i>	0	11111111	0000000	1	11111111	0000000		

NaNs, for both *float16* and *bfloat16*, follow the same format with at least 1 bit of the mantissa nonzero. Finally, zero is all exponent and mantissa bits zero regardless of the sign, thereby perpetuating the signed zero of IEEE 754.

A comparison between *float16* and *bfloat16* is in order. Fortunately, Python supports both formats. The first via NumPy, which we used in Chap. 10, and the second via the *bfloat16* package. Both are easily installed from the command line like so:

```
> pip3 install numpy
> pip3 install bfloat16
```

allowing *fp16_test.py* to run to produce a small table of the powers of π :

exp	float16	bfloat16	float64
0.25:	1.33105469	1.32812500	1.3313353638003897
0.50:	1.77246094	1.77343750	1.7724538509055159
0.75:	2.35937500	2.35937500	2.3597304924146969
1.00:	3.14062500	3.14062500	3.1415926535897931
1.25:	4.18359375	4.18750000	4.1825133983795988
1.50:	5.56640625	5.56250000	5.5683279968317079
1.75:	7.41406250	7.40625000	7.4133119794218363
2.00:	9.86718750	9.87500000	9.8696044010893580
2.25:	13.14062500	13.12500000	13.1397533658902272
2.50:	17.50000000	17.50000000	17.4934183276248625
2.75:	23.29687500	23.25000000	23.2896064533208502
3.00:	31.00000000	31.00000000	31.0062766802998162

Notice that while there are powers for which *float16* and *bfloat16* match, it is generally the case that the *float16* result is closer to the *float64* result than *bfloat16*. For the range of values often encountered in a trained neural network, *bfloat16*'s increased range at the expense of precision appears to be more of a liability than an aid.

The code in *fp16_test.py* is a straightforward application of NumPy and *bfloat16*:

```
1 import numpy as np
2 from bfloat16 import bfloat16
3 p = np.pi
4 print(" exp          float16      bfloat16                float64")
5 for x in np.linspace(0.25,3,12):
6     a,b,c = np.float16(p**x), bfloat16(p**x), p**x
7     print("%0.2f:  %11.8f  %11.8f  %19.16f" % (x,a,b,c))
```

Even with the difference in precision illustrated by this example, the range and number of mantissa bits in half-precision floats is likely sufficient in many cases where the precision loss is compensated for by the factor of 2 or 4 reduction in memory use when mapping from FP32 or FP64. However, using even two bytes for each weight is often too much to allow a single GPU to run a modern network. Let's push things to investigate using a mere 8 or fewer bits to represent the weights.

11.3.2 FP8 and FP4 Formats

FP8 and newer FP4 formats follow the IEEE convention ala *float16* and *bfloat16*, for the most part. We discuss two versions of each in this section: E4M3 and E5M2 for FP8 and E2M1 for FP4, along with the novel NF4 format that takes advantage of the fact that the weights in many neural network layers are approximately normally distributed. The nomenclature expresses the number of bits in the exponent (E) and mantissa (M), with the highest order bit understood as the sign. We begin with FP8.

11.3.2.1 FP8 Formats

FP8 numbers have a sign, exponent, and mantissa, as expected. There are two primary manifestations. The first, E5M2, reserves 5 bits for the exponent (excess-15) and 2 for the mantissa. The second, E4M3, shifts one of the exponent bits (excess-7) to the mantissa.

FP4 numbers likewise have a sign, exponent, and mantissa with E2M1 the format supported in industry. E2M1 uses two exponent bits (excess-1) and a single mantissa bit to represent $2^{-1} = 0.5$. Recent research proposes a shift to E3M0 [6], but we will not work with this format here. Figure 11.1 illustrates the storage format for each FP8 and FP4 type.

The FP8 formats support the expected special values, including signed zero, $\pm$ infinity, and NaNs, except for E4M3, which lacks infinities. The formats also support subnormal numbers ala *float16*, but we will ignore such numbers in the following experiments. See Table 11.1 for ranges and special values [5].

Fig. 11.1 FP8 formats (top) and FP4 formats (bottom)

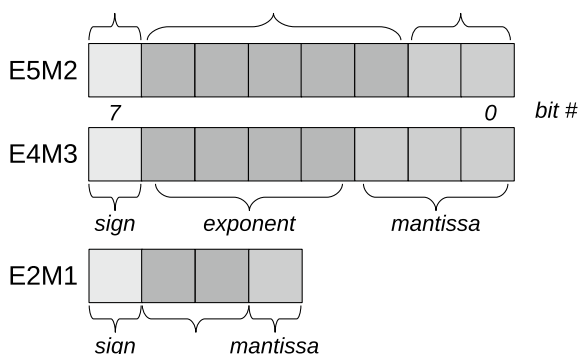


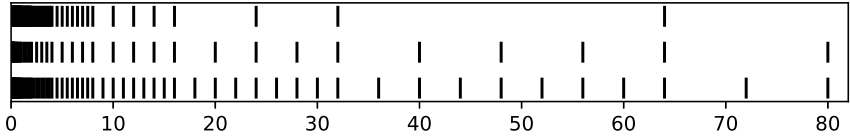
Table 11.1 E4M3 and E5M2 ranges and special values. Note that for E5M2, $xx = 01\ 10\ 11$

	E4M3	E5M2
Infinity	-n/a-	S 11111 00
NaN	S 1111 111	S 11111 xx
Zero	S 0000 000	S 00000 00
Max Normal	S 1111 110 = $1.75 \times 2^8 = 448$	S 11110 11 = $1.75 \times 2^{15} = 57,344$
Min Normal	S 0001 000 = 2^{-6}	S 00001 00 = 2^{-14}

The table indicates that E4M3 reserves no bit patterns for $\pm$ infinity and allows only two NaN values depending upon the sign (i.e., no payload). The E5M2 format supports six possible NaNs using the mantissa bits of 01_2 , 10_2 , and 11_2 in combination with the sign. Therefore, it is possible for an E5M2 NaN to include a payload as is the case with standard IEEE formats. The E5M2 maximum exponent with zero mantissa is reserved for $\pm$ infinity.

The E4M3 format’s smallest positive normal number is $2^{-6} = 0.015625$; the maximum is 448.0. The E5M2 format expands in both directions for a smallest value of 2^{-14} and a maximum of 57,344, achieved at the expense of one mantissa bit. Compare these to a standard *posit8* value with the same minimum as E4M3 but a maximum of only 64.0.

The ranges are symmetric about zero, but not uniformly distributed. For example, plotting the location of valid (not subnormal) numbers in $[0, 80]$ for E4M3 (bottom), E5M2 (middle), and *posit8* (top) gives



Both FP8 formats cover the range > 10 more densely than *posit8* while, at the scale of the plot, E4M3 and *posit8* are roughly equivalent < 10 . E5M2, on the other hand, is sparser in $[0, 80]$ but benefits from the ability to represent larger values than either E4M3 or *posit8*.

The plot is produced from the output generated by *e4m3_e5m2_posit8.py*, which maps each of the 256 8-bit patterns to E4M3, E5M2, and *posit8*. Table 11.2 shows the start of the resulting table indicating that E4M3 supports seven positive subnormal numbers, E5M2 three, and *posit8* none, as discussed in Chap. 10.

Table 11.2’s *posit8* output comes from the *softposit* Python library. For E4M3 and E5M2, we must roll our own functions to convert an integer in $[0, 255]$ to the corresponding FP8; see Fig. 11.2.

Both *e5m2* and *e4m3* functions operate similarly. The first three lines parse the 8-bit integer argument (ϵ) to extract the sign, exponent, and mantissa bits. Checks for special bit patterns follow. The E5M2 format supports zero, $\pm$ infinity, subnormals, and multiple NaNs, as does E4M3, except for infinity. The final three lines of each

Table 11.2 Bit patterns and the E4M3, E5M2, and *posit8* numbers they represent

pattern	E4M3	E5M2	posit8
000: 00000000	0.000000	0.000000	0.000000
001: 00000001	sub	sub	0.015625
002: 00000010	sub	sub	0.031250
003: 00000011	sub	sub	0.046875
004: 00000100	sub	0.000061	0.062500
005: 00000101	sub	0.000076	0.078125
006: 00000110	sub	0.000092	0.093750
007: 00000111	sub	0.000107	0.109375
008: 00001000	0.015625	0.000122	0.125000

```
1 def e5m2(f):
2     s = (f>>7) & 0x1
3     e = (f>>2) & 0x1F
4     m = f & 0x3
5     if (e==31) and (m==0):
6         return "-inf" if (s==1) else "+inf"
7     if (e==0) and (m==0): return "%0.6f" % 0.0
8     if (e==31): return "NaN"
9     if (e==0): return "sub"
10    v = (-1)**s*(1 + m / 2**2) * 2**(e-15)
11    return "%0.6f" % v
12
13 def e4m3(f):
14     s = (f>>7) & 0x1
15     e = (f>>3) & 0xF
16     m = f & 0x7
17     if (e==0) and (m==0): return "%0.6f" % 0.0
18     if (e==15) and (m==7): return "NaN"
19     if (e==0): return "sub"
20     v = (-1)**s*(1 + m / 2**3) * 2**(e-7)
21     return "%0.6f" % v
```

Fig. 11.2 Interpreting bit patterns in [0, 255] as E5M2 and E4M3 numbers

function calculate the represented floating-point value by first subtracting the excess from the exponent, 15 for E5M2 and 7 for E4M3, before multiplying the correctly scaled mantissa by the appropriate power of two.

Let’s put the FP8 formats to the same powers of π test we examined in the previous section; see *fp8_test.py*. The resulting output is in Table 11.3.

It is immediately evident that neither FP8 format is particularly effective at tracking the powers of π , but it should be remembered that these formats are intended for neural networks, not general-purpose use. The extra precision available to E4M3 makes it the better choice in this case, but *posit8* is still more accurate, as we might

Table 11.3 Powers of π using FP8, *posit8*, and *float64*

exp	E5M2	E4M3	posit8	float64
0.25:	1.25000000	1.25000000	1.34375000	1.33133536
0.50:	1.75000000	1.75000000	1.78125000	1.77245385
0.75:	2.00000000	2.25000000	2.37500000	2.35973049
1.00:	3.00000000	3.00000000	3.12500000	3.14159265
1.25:	4.00000000	4.00000000	4.00000000	4.18251340
1.50:	5.00000000	5.50000000	5.50000000	5.56832800
1.75:	7.00000000	7.00000000	7.50000000	7.41331198
2.00:	8.00000000	9.00000000	10.00000000	9.86960440
2.25:	12.00000000	13.00000000	14.00000000	13.13975337
2.50:	16.00000000	16.00000000	16.00000000	17.49341833
2.75:	20.00000000	22.00000000	24.00000000	23.28960645
3.00:	28.00000000	30.00000000	32.00000000	31.00627668

Table 11.4 All possible E2M1 floating-point values

Bits	Value
S 00 0	$(0.0 + 0.0) \times 2^0 = \pm 0.0$
S 00 1	$(0.0 + 0.5) \times 2^0 = \pm 0.5$
S 01 0	$(1.0 + 0.0) \times 2^{1-1} = \pm 1.0$
S 01 1	$(1.0 + 0.5) \times 2^{1-1} = \pm 1.5$
S 10 0	$(1.0 + 0.0) \times 2^{2-1} = \pm 2.0$
S 10 1	$(1.0 + 0.5) \times 2^{2-1} = \pm 3.0$
S 11 0	$(1.0 + 0.0) \times 2^{3-1} = \pm 4.0$
S 11 1	$(1.0 + 0.5) \times 2^{3-1} = \pm 6.0$

expect, given how posits focus on maximizing precision in the vicinity of zero, taking the scale of the posit into account.

Using a single byte to represent a floating-point neural network weight saves considerable memory. Further savings are possible by packing multiple weights into a single byte. Let's now consider the extreme presented by FP4 formats.

11.3.2.2 FP4 Formats

The FP4 format supported by industry, E2M1, is defined in [7] and enumerated in Table 11.4 as there are only eight unique values, each signed ($S=1$) or unsigned ($S=0$). Notice that there are no special values (infinity or NaN) and that a zero exponent field indicates either zero if the mantissa is zero or 0.5 if the mantissa is 1 (the only subnormal).

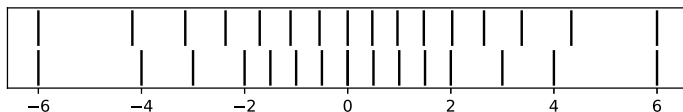


Fig. 11.3 The distribution of representable E2M1 (bottom) and NF4 (top) values. N.B., NF4 scaled to $[-6, 6]$

Conversion from a 4-bit pattern to a floating-point value is accomplished in practice by a few lines of code:

```

1 def e2m1_to_float(b):
2     s,e,m = b>>3, (b>>1)&3, b&1
3     if (e==0) and (m==0): return 0.0
4     if (e==0) and (m==1): return (-1)**s*0.5
5     return (-1)**s*(1+m/2.0)*2**(e-1)

```

where the argument (b) is a 4-bit unsigned integer. Line 2 extracts the sign, exponent, and mantissa. Lines 3 and 4 check for zero (signed or unsigned) and the subnormal 0.5. Finally, line 5 calculates the normal floating-point value.

The E2M1 format is exceedingly simple, but from Table 11.4 it is clear that representable floats are well positioned around zero to favor slightly better coverage than evenly spaced values. The $[-6, +6]$ range is adequate and typical of the weights found in many trained neural networks. Let's turn to NF4, the *NormalFloat* type introduced in [8]. This format splits 16 values across $[-1, +1]$ by first estimating $2^4 + 1$ quantiles from a standard normal distribution, $N(\mu = 0, \sigma = 1)$ scaled to $[-1, +1]$. Then, in use, the resulting values are scaled by a block factor. Here, and for the experiment later in the chapter, we fix this scale factor to 4.0. Plotting the resulting distribution of representable values for E2M1 and NF4 leads to Fig. 11.3.

It's evident from the figure that both formats reasonably cover the range. The slight differences between them should be expected, in practice, to lead to occasions where one outperforms the other, but, naively, there is no *a priori* reason to believe one will be consistently superior to the other. Bear in mind, however, that our approach to NF4 is for representation and after-the-fact quantization of an already trained model as opposed to [8], which uses NF4 in conjunction with a learned scale factor to fine-tune a model by adjusting its weights.

Reduced-precision floating-point formats are not the only game in town for improving the memory efficiency of large neural networks. Let's now transition to consider standard scaled integer formats.

11.4 Scaled Integer Formats

An alternative to squeezing complete floating-point numbers into ever smaller amounts of memory involves what amounts to the separation of the mantissa and the exponent. For example, it is conceivable to replace the mantissa with a signed integer value, which is then multiplied by a specified power of two to arrive at an approximation of a given float value:

$$(1 + m/2^M) \times 2^e \approx n \times 2^z$$

for integer mantissa m of bit width M , signed integer n , and exponents e and z .

At first blush, such a transformation seems unlikely to be helpful if our goal is to compress the weights of an extensive neural network. However, an entire block of weights may often be similarly transformed and approximated by a set p_i of signed integers multiplied by a common scale factor, $x = 2^z$. In that case, factoring the exponent from the weights results in memory savings, especially if the block size, k , is large enough.

In other words, a set of k FP16 weights, each occupying two bytes of memory, might be replaced by a set of k signed integer bytes and a single, common scale factor x , itself an E8M0 float (excess-127 and no mantissa), thereby moving from $2k$ bytes of storage to $k + 1$ bytes. Therefore, larger block sizes imply greater savings, so long as the scaled integer values in the block multiplied by the shared scale factor faithfully represent the original weights. The Microscaling Formats (MX) Specification [7] is a proposed industry standard specifying block (p_i) formats along with block scaling. One such a format is MXINT8, a combination of $k = 32$ INT8 (signed bytes) multiplied by an E8M0 scale factor. The format is commonly referred to as INT8, the presence of the scale factor being understood. A similar INT4 format was used and supported by NVIDIA GPUs until the Hopper generation, which dropped INT4 support.

As an example, consider the code in `int8_test.py`, which quantizes ten randomly selected values in $[-v, v]$ for v , a user-specified value, to INT8 format, then dequantizes them to illustrate the loss of precision. For example, a run with $v = 4.0$ produces

```
-3.72234702 ==> -119 ==> -3.71875000 (0.00359702)
-2.26746655 ==> -73 ==> -2.28125000 (0.01378345)
-1.59599161 ==> -51 ==> -1.59375000 (0.00224161)
-0.74526519 ==> -24 ==> -0.75000000 (0.00473481)
 0.47492373 ==>  15 ==>  0.46875000 (0.00617373)
-0.52608454 ==> -17 ==> -0.53125000 (0.00516546)
-2.89355445 ==> -93 ==> -2.90625000 (0.01269555)
-0.69930905 ==> -22 ==> -0.68750000 (0.01180905)
-2.25670552 ==> -72 ==> -2.25000000 (0.00670552)
 2.40789580 ==>  77 ==>  2.40625000 (0.00164580)
```

```
scale factor 122 ==> 2** -5 = 0.03125000
```


The INT8 values occupy a single byte, as does the scale factor, 122. For example, the scale becomes $2^{122-127} = 2^{-5}$ making the first dequantized value $-119 \times 2^{-5} = -3.71875$, as shown.

The final column is the absolute difference between the initial and final values. In most cases, the final value is accurate to about two decimals, a good sign if we intend to use INT8 with neural networks.

The quantize and dequantize code is only a few lines to generate a NumPy vector of random values (fp32), find the exponent of the scale factor (s), map from $[-4, 4]$ to $[-127, 127]$ (int8), and back (x32):

```
1 | fp32 = np.float32(2*v*(np.random.random(10)-0.5))
2 | S = np.round(np.log2(v/127))
3 | int8 = np.round(fp32/2.0**S).astype("int8")
4 | x32 = np.float32(int8*2.0**S)
```

Notice that NumPy functions and methods are used where necessary to ensure proper FP32 and INT8 data types.

Let's walk through the code. Line 1 creates a 10-element NumPy vector of values in $[-v, v]$. Line 2 finds the exponent of the scale factor. The goal is to map $[-127, 127]$ (integer) to $[-v, v]$. To do that, we must multiply by the scale factor, $v/127$, but the scale factor itself is stored as an E8M0 number, an exponent on 2, implying that we need

$$2^e = v/127 \rightarrow e = \log_2(v/127) \approx \lfloor \log_2(v/127) \rfloor$$

because s must be an integer (round to the nearest integer).

Line 3 converts the FP32 values by dividing each by 2^S and casting to a signed 8-bit integer. There are 10 elements in `int8`, meaning the block size is 10, and each is scaled by the same scale factor. Finally, line 4 dequantizes by multiplying each INT8 value by 2^S .

If you wish, experiment with `int8_test.py` to track how accuracy changes with v . Notice that a signed 8-bit integer's range is $[-128, 127]$, yet we use only $[-127, 127]$. Mapping in this way is symmetric and simplifies the implementation. It is possible to map asymmetrically from $[a, b]$ for $a \neq b$, but it introduces an offset to represent zero exactly. This offset somewhat complicates the arithmetic, working against the desire to reduce memory use while speeding inference by eliminating unnecessary operations. Losing a maximum negative value is a small price for easing the computational burden.

A signed INT4 value implies mapping $[-15, 15]$ to $[-v, v]$. Beyond that, the operation is the same as INT8 when using an E8M0 scale factor per block. A run or two of `int4_test.py` is in order. The code is identical to the INT8 version but replaces $[-127, 127]$ with $[-15, 15]$. It's worth noting that [7] is not restricted to MXINT8. The INT8 portion of a block of weights may be replaced by E5M2, E4M3, and E2M1 values to create mixed-precision representations MXFP8 and MXFP4.

Further, the specification defines FP6 formats (E3M2 or E2M3) along with E8M0 scale factors. In all cases, $k = 32$, making the total number of bits required to store a block always a multiple of 8:

$$8 + 32 \times 8 = 264, \quad 8 + 32 \times 6 = 200, \quad 8 + 32 \times 4 = 136$$

for blocks occupying 33, 25, and 17 bytes, respectively.

We've discussed many formats. Let's put them to the test with an experiment using the neural networks from Chap. 10.

11.5 An Experiment

Chapter 10 explored posits and their effect on traditional (non-convolutional) neural networks trained to recognize small images of handwritten digits (MNIST) or articles of clothing (FMNIST); see Sect. 10.6. In this section, we use the weights from those models to study the effect on test set accuracy when mapping the FP64 weights to the formats introduced in this chapter.

The pretrained weights are in two files on the book's GitHub site:

```
posit_fmnist_weights.pkl
posit_mnist_weights.pkl
```

The files are inputs to *ai_fp_test.py*, which I recommend you review before continuing.

The code expects two command line arguments: the weights file and the quantization mode. The supplied FP64 weights are appropriately quantized before passing the test set images through the model. The resulting overall accuracy and confusion matrix are then returned. For example,

```
> python3 ai_fp_test.py posit_mnist_weights.pkl float
[[264  0  1  0  0  0  4  0  1  1]
 [ 0 334  2  0  0  0  3  0  1  0]
 [ 3  0 296  0  1  0  3  4  4  2]
 [ 0  1  5 297  0  3  0  4  4  2]
 [ 0  0  1  0 300  0  5  2  1  9]
 [ 2  0  0  4  2 267  4  2  2  0]
 [ 2  3  2  0  4  2 255  1  3  0]
 [ 1  4  5  2  3  0  0 286  0  5]
 [ 4  0  3  4  4  1  0  4 264  2]
 [ 4  1  1  4  4  5  0  3  0 273]]
Overall accuracy: 0.94533
```

evaluates the MNIST test set using full-precision weights, which we take to be the gold standard. A corresponding run for FMNIST produces an overall accuracy of 0.84133.

The confusion matrix represents true sample labels (rows) and the model’s assigned label (columns). A perfect model makes no mistakes, producing a purely diagonal confusion matrix. The MNIST dataset is known to be “easy”, even for simple neural networks, explaining the relatively high accuracy.

Let’s run the experiment for each of this chapter’s number formats, including a new one that is nothing more than the sign of the original FP64 weight: -1 if negative and $+1$ if positive. Then explore some of the code used to map from FP64 to a reduced-precision number and back.

11.5.1 Quantization and Model Accuracy

Repeated executions of `ai_fp_test.py` for each model and quantization type produce Table 11.5, which we take to be the primary result of this chapter. The figure is sectioned by the precision of the numbers involved. It is important to remember that this is a post-training experiment, which merely quantizes the weights of a model trained at full precision. Additionally, all computations are performed at full precision; it is only the contribution of the reduced-precision weights influencing the results.

Table 11.5 is revealing. For both MNIST and FMNIST, model accuracy is effectively unperturbed by mapping the original weights to 32, 16, and even 8 bits. Further,

Table 11.5 MNIST and Fashion MNIST test set inference accuracy by quantization type

Precision	Encoding	MNIST	Fashion MNIST
FP64	<i>float64</i>	0.94533	0.84133
FP32	<i>float32</i>	0.94533	0.84133
	<i>posit32</i>	0.94533	0.84133
FP16	<i>float16</i>	0.94533	0.84133
	<i>bfloat16</i>	0.94500	0.84200
	<i>posit16</i>	0.94533	0.84100
FP8	<i>posit8</i>	0.94533	0.82633
	E5M2	0.94567	0.83833
	E4M3	0.94633	0.83733
	INT8	0.94567	0.84100
FP4	INT4	0.92200	0.76067
	NF4	0.88267	0.72067
	E2M1	0.76300	0.62667
1-bit	Sign only	0.57200	0.22933

the bold entries indicate cases where the test set performance *improves* when using reduced precision, most noticeably for MNIST and FP8 formats. We'll return to this observation momentarily.

Switching to FP4 formats introduces a roughly 20 percent loss of accuracy for both models, except for INT4, which retains most of the accuracy found with higher precision for MNIST and similarly, though less pronounced, for FMNIST. Recall that INT4 uses integer weights with block scaling, which is selected here to cover $[-4, 4]$ with the same scaling applied for all model weights.

The code in the following section will show that the NF4 $[-1, 1]$ range was scaled to $[-4, 4]$ to match INT4. In absolute value, the largest weights for both models are within this range, but the best representation will be for weights about zero. The INT4 results have an advantage over E2M1 and NF4 due to the E8M0 scale factor allowing for an exponent that is likely better suited to the original weights. In other words, block scaling dramatically expands the range that the 16 possible INT4 weights can cover. Hence, it is reasonable to believe that a suitable exponent might be located to better represent the actual range of the weights, thereby leading to the better performance observed.

That the NF4 results are superior to E2M1 lends credence to the claim in [8] that normally distributed weights are a better approach to low-precision quantization. The MXFP4 format described in [7] relies on E2M1 weights multiplied by a block scale factor. These admittedly limited results hint that replacing E2M1 weights with NF4 weights, perhaps scaled to cover the same range as E2M1 ($[-6, 6]$), might result in even better overall performance.

The last row in Table 11.5 reveals what happens when every weight is replaced by its sign. We might call this an INT1 format using 0 for -1, but no block scale factor is applied. We might expect model accuracy to fall to the level of random guessing, 0.1, as there are ten classes per model, but neither does. Indeed, MNIST accuracy is nearly six times that of random guessing. What might an MXINT1 format be capable of?

The results in Table 11.5 rely on single instances of each neural network. Training a neural network is, at its simplest, an optimization problem, namely, first-order stochastic gradient descent. Gradient descent begins at some location in the space to be optimized and steps along the negative of the gradient of the optimization function, known in machine learning as the loss function. Therefore, the weights of a neural network, which are the parameters to be optimized, must be initialized to correspond to some initial location in the loss space. Typically, initialization is random using heuristics learned over the decades (with some mathematical support).

All of this means that the single instances are not the only possible models for each dataset, and the selected architecture. Each random initialization leads to slightly different locations in the loss space corresponding to slightly different models. Might the observed improvement of the models for certain quantizations be merely an artifact of the trained instance? To partially answer the question, I trained the MNIST model ten times using ten different random initializations then calculated the overall accuracy on the test set for FP64, E5M2, and E4M3 weights; see Table 11.6.

Table 11.6 Overall test set accuracy for ten trainings of the MNIST model

	float	e5m2	e4m3
0	0.95067	0.95000	0.95033
1	0.94767	0.94800	0.94800
2	0.94867	0.94800	0.94667
3	0.95133	0.94967	0.95100
4	0.95033	0.94933	0.95133
5	0.94267	0.94233	0.94233
6	0.94276	0.94133	0.94333
7	0.95133	0.95033	0.95000
8	0.94700	0.94500	0.94600
9	0.94833	0.94800	0.95033

The results show some cases where FP8 weights lead to improved performance. To know if there is a consistent effect requires performing paired hypothesis tests, as the E5M2 and E4M3 results are based on the FP64 model. Doing so gives

	<i>Paired t-test Wilcoxon</i>	
FP64 vs E5M2	0.0031	0.0039
FP64 vs E4M3	0.7063	0.6250
E5M2 vs E4M3	0.0839	-n/a-

indicating that there is, statistically, no meaningful difference between the FP64 and E4M3 results across many trainings of the same model ($p = 0.71$) but that there is a significant difference between E5M2 and FP64 ($p = 0.003$) and near significance for E5M2 and E4M3 ($p = 0.084$), if relying upon $p < 0.05$ as a meaningful threshold.

Training an additional ten models ($n = 20$, total) improves the comparison between E5M2 and E4M3, leading to a paired t-test p-value of $p = 0.036$ and a nonparametric Wilcoxon p-value of $p = 0.011$ when adding random values on the order of 10^{-5} to the E4M3 results to break ties. The net result is an improved belief that there is a slight advantage to using E4M3 versus E5M2, at least for MNIST and the selected architecture.

A few comments on the FP4 results before we examine the code. The worst FP4 quantization was E2M1 with accuracies of 0.7630 for MNIST and 0.6267 for FMNIST, both quite a bit lower than the other 4-bit encodings (NF4 and INT4). A Nearest Centroid classifier, arguably the simplest of machine learning classifiers, produces accuracies of 0.7577 and 0.6630, respectively. My take from this single example is that on its own, E2M1 is likely insufficient without adding block-wise scaling factors, as is the case with MXFP4(E2M1) [7]. On the other hand, INT4 and NF4 (adequately scaled) are helpful if reduction to 4-bit weights is imperative.

11.5.2 Exploring the Code

In this section, I focus on the functions to convert Python floats (C doubles) to and from different reduced-precision encodings. The `main` function loads the supplied weights file and interprets the desired encoding before calling `Quantize` to interpret each weight and bias of the model as a float rendered with the requested precision; see Fig. 11.4.

The `convert` dictionary returns pairs of functions to map floats to the desired encoding and from the encoding back to floats, thereby explaining the loop in lines 5 and 6. The remainder of the function transforms the new values in `ans` to the shape of the argument (`arg`).

The new weight matrices and bias vectors are then used to pass the test set data through the model, followed by displaying the resulting confusion matrix (`cm`) and overall accuracy (`acc`); see Fig. 11.5.

The loop over test samples in `xtst`, defined appropriately for the input MNIST or FMNIST weights, is straightforward. The quantized values have been returned to FP64 format, so the matrix calculations and vector additions proceed normally.

The bulk of the file implements the conversion routines. Some are straightforward, like `sign` or `float` to `float` (for convenience), or use existing Python libraries like `NumPy` or `bfLOAT`. Others are homegrown, like those in Fig. 11.6.

Converting a float to E4M3 involves a quick check for zero (line 2), extraction of the sign (line 3), and magnitude bounds checks (lines 5 and 6). Lines 7 and 8 extract the exponent and mantissa (fractional part). Lines 9 and 10 reassign the exponent and mantissa appropriately for an E4M3 number (excess 7) and 3-bit mantissa. The return value is then all three components (sign, exponent, mantissa) placed in the proper bits of the 8-bit integer representing the E4M3 value. Conversion back to a float

Fig. 11.4 Quantizing FP64 arrays

```

1 def Quantize(arg, mode):
2     v = arg.ravel()
3     ans = np.zeros(len(v))
4     A,B = convert[mode]
5     for i in range(len(v)):
6         ans[i] = B(A(v[i]))
7     if (arg.ndim == 2):
8         r,c = arg.shape
9         ans = np.array(ans).reshape((r,c))
10    return ans

```

Fig. 11.5 Passing test samples through the network

```

1 pred = []
2 for x in xtst:
3     x = np.maximum(0, np.dot(x, W0) + b0)
4     x = np.maximum(0, np.dot(x, W1) + b1)
5     x = np.maximum(0, np.dot(x, W2) + b2)
6     pred.append(np.argmax(softmax(x)))
7 cm,acc = ConfusionMatrix(pred, ytst)

```

Fig. 11.6 Converting to and from E4M3 format

```

1 def float_to_e4m3(v):
2     if (v == 0.0): return 0
3     s = 0 if (v>0) else 1
4     v = abs(v)
5     if (v > 448): v = 448.0
6     if (v < 0.015625): v = 0.015625
7     e = int(np.floor(np.log2(v)))
8     m = v / 2**e - 1
9     e = e + 7
10    m = int(m * 2**3)
11    return (s<<7) | (e<<3) | m
12 def e4m3_to_float(f):
13    s = (f>>7) & 0x1
14    e = (f>>3) & 0xF
15    m = f & 0x7
16    v = (1 + m / 2**3) * 2**(e-7)
17    return -v if (s==1) else v

```

parses the E4M3 value (lines 13 through 15) and then calculates the corresponding value remembering to subtract 7 from the exponent.

11.6 Discussion

The use of reduced-precision weights is growing as the scale of neural network models increases. Therefore, selected representations must be up to the task required of them. The examples and experiments of this chapter demonstrate the utility of different formats, but only for raw representation as a post-training adaptation. What is missing is a focus on using such formats during training. As we might expect, a large body of active research is engaged in this kind of mixed-precision training. For example, NVIDIA supplies a basic introduction here:

docs.nvidia.com/deeplearning/performance/mixed-precision-training

which focuses on practical applications using primary deep learning toolkits like TensorFlow and PyTorch. Further, see [9] for a recent survey of mixed-precision approaches and [10] for much-cited early work.

Exercises

11.1 Add FP6 formats E3M2 and E2M3 to *ai_fp_test.py* and compare their performance to the results in Table 11.5.

11.2 The MX formats defined in [7] scale block weights by a power of two in E8M0 format. Create an MXNF4 format using the scale factor and a block size, which, in this case, is equal to the number of weights in the models. Leave the NF4 values in their default $[-1, 1]$ range. Compare with Table 11.5. **

11.3 Repeat Exercise 2, or use any other precision used in this chapter, but alter the block size to be each hidden layer of the networks (100 and 50 nodes each). Does this improve overall performance? **

References

1. Morgan N (1991) Experimental determination of precision requirements for back-propagation training of artificial neural networks. In: Proceedings of the second international conference microelectronics for neural networks. Citeseer, pp 9–16
2. Bartol Jr TM, Bromer C, Kinney J, Chirillo MA, Bourne JN, Harris KM, Sejnowski TJ (2015) Nanoconnectomic upper bound on the variability of synaptic plasticity. *elife* 4:e10778
3. Kneusel RT (2024) Practical deep learning: a Python-based introduction, 2nd edn. No Starch Press
4. Kneusel RT (2021) Math for deep learning: what you need to know to understand neural networks. No Starch Press
5. Micikevicius P, Stosic D, Burgess N, Cornea M, Dubey P, Grisenthwaite R, Ha S et al (2022) FP8 formats for deep learning. arXiv preprint [arXiv:2209.05433](https://arxiv.org/abs/2209.05433)
6. Sun X, Wang N, Chen CY, Ni J, Agrawal A, Cui X, Venkataramani S, El Maghraoui K, Srinivasan VV, Gopalakrishnan K (2020) Ultra-low precision 4-bit training of deep neural networks. *Adv Neural Inf Process Syst* 33:1796–1807
7. Rouhani BD, Garegrat N, Savell T, More A, Han K-N, Zhao M, Hall R, Klar J, Chung E, Yuan Yu, Schulte M, Wittig R, Bratt I, Stephens N, Milanovic J, Brothers J, Dubey P, Cornea M, Heinecke A, Rodríguez A, Langhammer M, Deng S, Naumov M, Micikevicius P, Siu M, Verrilli C (2023) OCP Microscaling (MX) specification. Open Compute Project
8. Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L (2024) QLoRA: Efficient finetuning of quantized LLMs. *Adv Neural Inf Process Syst* 36
9. Rakka M, Fouda ME, Khargonekar P, Kurdahi F (2024) A review of state-of-the-art mixed-precision neural network frameworks. *IEEE Trans Pattern Anal Mach Intell*
10. Micikevicius P, Narang S, Alben J, Diamos G, Elsen E, Garcia D, Ginsburg B et al (2017) Mixed precision training. arXiv preprint [arXiv:1710.03740](https://arxiv.org/abs/1710.03740)



Abstract

Large language models are generative neural networks that predict a sequence of tokens (words or parts of words) when given an initial text prompt. Trained on vast amounts of text, such models have surprised researchers with emergent abilities enabling the models to achieve human-level performance in various subjects, including those where creativity plays a role. In this chapter, we ask a large language model (GPT-4) to design novel computer number systems.

12.1 How Large Language Models Work

This chapter is an experiment. We aim to coax a large language model (LLM), namely, OpenAI's GPT-4, into developing novel computer number formats for us. Before we do that, however, it's worth some effort to understand the essential operation of a trained LLM. In particular, to understand the operation of the transformer layer at its heart. Doing so builds intuition about how such a model can generate meaningful responses to our queries. LLMs work by accepting an input prompt from which the model repeatedly generates output tokens (words, characters, or parts of words) until a stop token is generated. For example, if the prompt is:

Finish the rhyme: Roses are red. Violets are blue. I have a secret

the model might respond with

...and I'm telling it to you!

where the sequence of tokens is ... followed by the individual words, parts of words, or characters: *and, I'm, tell, ing, it, to, you, !* This example is courtesy of Meta's

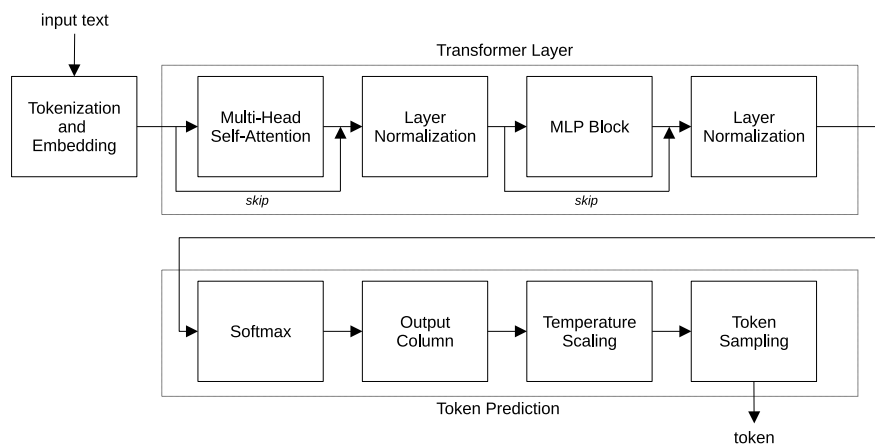


Fig. 12.1 A block diagram of a basic LLM during inference (from *Practical Deep Learning: A Python-Based Introduction*, 2nd ed, No Starch Press)

LLaMa 3 model running locally on a small desktop computer. The model was given the full input prompt, to which it responded with the token Next, the model, in effect, appended the first token to the prompt to generate a new input:

Finish the rhyme: Roses are red. Violets are blue. I have a secret ...

which caused the model to produce the next token, *and*, which was itself added to the input context and then passed through the model to produce *I'm*. The process repeated, token by token, until a final stop token was generated after *!* to conclude the model's reply.

Figure 12.1 presents a simplified block diagram of a basic LLM during inference. It shows the input (prompt), a single transformer layer, and token prediction [3]. The diagram is from Chapter 20 of my book *Practical Deep Learning: A Python-Based Introduction*, 2nd ed (No Starch Press, 2024). The leftmost block, tokenization and embedding, transforms the input string of characters into a sequence of tokens, elements of a finite set of tokens that the model can emit, and then into a matrix of embeddings, one per token. Embeddings are high-dimensional floating-point vectors learned by the model during training, one per possible token. The embeddings capture essential characteristics of the meaning of the token, perhaps in relation to other tokens.

LLMs process a context, a fixed-sized number of input tokens. If there are n possible input tokens, each represented by an m -dimensional embedding vector, then the output of the tokenization and embedding process is an $n \times m$ matrix where the number of tokens in the prompt, say c , is such that $c < n$. During inference, the LLM aims to produce an output matrix where the $c + 1$ -th column represents a probability distribution over all possible tokens.

The transformer block first applies multi-head self-attention, the novel mechanism developed in [3] that enabled models to learn how to mimic the process of paying attention to separate parts of the input token embedding matrix. Older recurrent neural networks attempted to process sequences iteratively, like the sequence of tokens in a prompt. The transformer layer's self-attention mechanism processes all tokens at once, with suitable tricks to preserve the order of the tokens. During training, the internal parts of the multi-head self-attention layer become conditioned to learn how to best focus the model's attention on portions of the input. In a real sense, the structure of the self-attention mechanism, itself based on earlier work involving notions of memory in neural networks, provides the architecture or environment from which self-attention suitable to the task might emerge. Neural networks are traditionally black boxes. Transformer layers, and by extension LLMs, which in practice have many such layers, are even more inscrutable.

The output of the self-attention block is normalized using a standardization akin to subtracting the mean and dividing by the standard deviation. The standardized matrix is then passed through a traditional multilayer perceptron block, which applies the same traditional neural network to each token vector. A second layer normalization and additional processing produces a final $n \times m$ output matrix as input to the token prediction block.

As mentioned, the token prediction block's goal is to make the $c + 1$ -th column of the matrix a probability distribution over the n possible tokens the model knows. The final block, token sampling, selects the output token according to this distribution, which is subject to conditioning by a temperature that alters the relative probabilities of the tokens. Most LLMs become deterministic if the temperature is zero, meaning the output distribution is such that the highest token probability is selected. In this mode, the LLM repeatedly produces the same output sequence for the same input prompt. In practice, many commercial LLMs, including GPT-4, set the temperature to about 0.7, thereby altering the output probability distribution so that some varieties of tokens are emitted, but such that the model's output sequence is still relevant to the prompt. If the temperature is too high, the emitted token distribution becomes flatter, implying increased randomness in the model's replies and a lower likelihood of the reply being appropriate to the input prompt. Once selected, the output token is, essentially, appended to the context, and the process repeats until the stop token is generated.

Now that we have a notion of how an LLM functions let's experiment with prompts to learn if we can coax GPT-4 into presenting us with novel computer number formats.

12.2 Experimenting with Prompts

Recent work proposed a model for the emergence of complex skills in LLMs [1]. Essentially, LLM training can be thought of as producing a bipartite graph of skills and text such that the number of combinations of the two far exceeds similar possible pairings in the human mind. This allows LLMs to form connections over a wide range

of skills and topics such that the output produced becomes novel, or at least has the potential to become novel. We seek to exploit this emergence of complex skills in this chapter by querying GPT-4 to elicit new number formats for computers.

LLMs respond to text prompts from users. This chapter's experiments used GPT-4's web interface, which is well suited to conversational discussions but lacks some of the flexibility of the API. In particular, the web interface does not allow adjustment of the model's temperature, which, as discussed, modifies the token probability distribution to make the model more deterministic (temperature approaching zero) or more likely to select a novel output token during the generation process (temperature near one or above).

I queried GPT-4 with two kinds of prompts. The first was a straightforward prompt asking the model to produce a novel number format as a conservative extension of existing formats. The second kind of prompt urged the model to be creative and even wild in imagining its reply. The following prompt was typical of the conservative approach:

Think deeply about all the computer number formats you know, especially those capable of supporting at least integers, signed or unsigned. Then, with this background in mind, create a novel computer number format, one that can be implemented on existing computers. Detail your thought process and design decisions.

The prompt encourages the model to consider (however accomplished within the transformer framework) existing computer number formats and then go beyond them with a format that can still be implemented on existing computers. Asking the model to detail its thought process and design decisions is akin to asking it to proceed step by step. Asking LLMs to think step by step is known to encourage correct output as opposed to requesting an answer without forcing the model to defend its actions [2]. This particular prompt led to the Mixed Precision Integer Float (MPIF) format explored in Sect. 12.4. Similar prompts, with one or two follow-up queries, led to the remainder of the chapter's formats: FlexInt: A Hybrid Integer Number Format (Sect. 12.3) and MetaInt (Sect. 12.5). All three formats use the names given by GPT-4. The prompt leading to the MetaInt format instructed the model to think deeply about known formats while simultaneously encouraging it to be creative and wild:

Think deeply about all the computer number formats you know, especially those capable of supporting integers, signed or unsigned. Then, with this background in mind, create a novel computer number format for integers, one that can be implemented on existing computers. Be creative! Be wild! Don't be one of the herd!

This prompt produced a reasonably sensible number format.

The second type of prompt often led to unrealizable number formats, though in GPT-4's defense, the prompt asked for something well beyond convention. For example,

I want you to think outside of the box, to be wild and creative. I want you to design a new integer number format for computers. Don't be part of the herd. Branch out. The sky is the limit!

led to the Fractal Integer Representation (FIR) which GPT-4 described as

The Fractal Integer Representation (FIR) is a radical departure from the binary, decimal, or even base-n representations traditionally used in computing. It leverages the recursive, infinitely scalable nature of fractals to encode integers in a way that can be visually intuitive yet mathematically complex.

A second session with the same prompt produced the Spectrum Integer Format (SIF):

Conceptual Foundation:

- *Wavelength Encoding*: Each integer is represented not by a sequence of bits but by a unique “wavelength” within a predefined spectrum. This spectrum is not limited to the visible light spectrum; it can extend into infrared, ultraviolet, and beyond, allowing for a vast range of unique representations.
- *Frequency Modulation*: The value of the integer can be modulated not only by its primary wavelength but also by subtle variations in frequency, akin to the concept of overtones in music. This allows for a nuanced, layered representation of integers.
- *Phase Shifts*: To incorporate negative integers and additional complexity, we introduce phase shifts into the wavelength representation, similar to phase shifts in waves.

In a similar vein, the prompt:

I want you to think outside of the box, to be wild and creative. I want you to design a new floating-point number format for computers. Don't be part of the herd. Branch out. The sky is the limit!

which is now asking for a floating-point format, produced the Quantum Chroma Float (QCF):

The Quantum Chroma Float (QCF) format is inspired by the principles of quantum mechanics and color theory. The core idea is to encode floating-point numbers using a combination of quantum states and color wavelengths, leading to an extremely high-density and parallel processing-capable number format.

and, in a separate session, evoked the Quantum-Flux Floating Point (QFFP):

Core Principles:

1. *Quantum Bit Encoding*: Unlike traditional binary that represents 0 and 1, QFFP uses qubits for encoding the mantissa and exponent. Each qubit can exist in multiple states simultaneously, allowing for a denser representation of data.
2. *Flux Capacitance Storage*: Inspired by the fictional concept of flux capacitance, this mechanism allows for dynamic adjustment of the precision and range of the floating-point number based on its quantum state, enabling it to store values across an immensely broad spectrum without losing precision.

3. *Temporal Resolution Adjustment*: Incorporates the ability to adjust calculations based on the temporal resolution, allowing for future-state predictions and retroactive adjustments based on quantum entanglement with past and future states of the number.

which is, with its reference to *Flux Capacitance Storage*, no doubt a number format near and dear to Doc Brown's heart (and DeLorean).

The following three sections implement the more reasonable storage formats imagined by GPT-4. All three are Python classes to take advantage of Python's "dunder methods" (special method names beginning with double underscores), thereby allowing objects of the class to participate in arithmetic expressions.

Each section explains the number format and any assumptions made before implementing it. GPT-4's inventions are expressed in binary ala IEEE 754. Therefore, the classes store information internally as a list of bits. Examples illustrate each format in action, including storing numbers on disk.

12.3 FlexInt: A Hybrid Integer Number Format

The file `FlexInt.pdf` contains a full transcript of my conversation with GPT-4. The model's initial reply to my prompt was

Inventing a new integer number format requires considering the limitations and challenges of existing formats, such as fixed bit-width integers (like 32-bit or 64-bit integers) and arbitrary-precision (bigint) formats. These formats balance fixed memory usage (and thus performance) and the flexibility to handle large numbers. Inspired by these considerations, let's propose a new integer number format named "FlexInt".

GPT-4 begins reasonably by noting that existing fixed width and arbitrary-precision formats have advantages before proposing FlexInts. GPT-4 continues

Design Overview:

FlexInt is a hybrid integer format designed to automatically adjust its memory footprint based on the magnitude of the number it represents. It combines the performance benefits of fixed-width integers for small to moderately large values with the flexibility of arbitrary-precision integers for very large values.

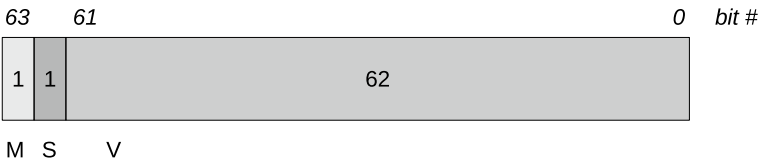
Structure:

- **Prefix Bit:** A single bit at the beginning of the structure indicates the mode: fixed-width (0) or arbitrary precision (1).
- **Length Field:** For arbitrary-precision mode, the next few bits (size of this field can be dynamic or fixed) indicate the length of the number in bytes.
- **Data:** The actual integer value, stored in a variable-length sequence of bytes. The length is either fixed (for fixed-width mode) or specified by the length field (for arbitrary-precision mode).

The remainder of the transcript shows GPT-4 offering further description, design justifications, and implementation considerations. I also challenged the model slightly concerning ambiguity about how signed integers are represented between fixed-width and arbitrary-precision modes.

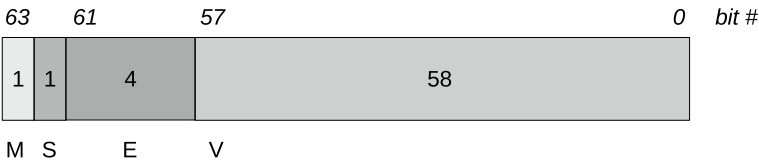
Eventually, I settled on the following: a fixed-width integer using 64 bits or an extended (but not arbitrary) length integer using multiple 64-bit integers. For both forms, bit 63 is the mode (0=fixed, 1=extended), and bit 62 is the sign (0=positive, 1=negative). Values themselves are stored as big-endian positive integers.

Specifically, fixed-width integers are represented as:

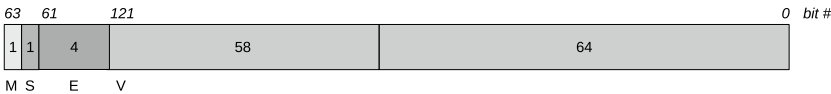


with M the mode bit, S the sign bit, and V a 62-bit unsigned integer representing the magnitude of the number. A fixed-width FlexInt number, therefore, occupies 64 bits and is capable of representing integers in the range $[-2^{62}+1, 2^{62}-1]$. The maximum magnitude is $2^{62}-1$ explaining why the lower limit is $-(2^{62}-1) = -2^{62}+1$.

Extended-length integers consist of one or more contiguous 64-bit integers. The first 64-bit integer contains the mode flag (0), sign, a 4-bit extended field, and the top 58 bits of the extended positive integer value in big-endian order. Therefore, the first 64 bits are



The M , S , and V fields are as before. The new 4-bit E field contains the number of additional 64-bit words needed to store the extended integer minus one. In other words, an E field of 0000_2 implies one additional 64-bit word beyond the initial. For example, an extended integer that requires more than 58 bits but less than $58+64=122$ bits fits in two 64-bit words like so:



with an E field of 0000_2 implying one additional 64-bit word beyond the first.

A maximally extended FlexInt has an E field of 1111_2 for an additional sixteen 64-bit words, providing a V field of up to $58+16(64)=1082$ bits for a magnitude range of $[2^{62}, 2^{1082}]$ because values less than 2^{62} are stored as fixed-width integers. Extended FlexInts are not arbitrary, but 2^{1082} is a number with 326 digits, which

is likely sufficient for most purposes, bearing in mind that many digits make for a highly accurate fixed-point number, if desired.

Let's implement `FlexInts` in Python knowing that, ideally, they would be implemented in hardware at some future point (should they prove worthy).

12.3.1 Implementation

A review of the file *FlexInt.py* is recommended before proceeding. At a high level, the goal is to build a Python class that overloads the dunder methods to allow `FlexInts` to participate in arithmetic expressions. Additionally, we should be able to extract the binary representation of a `FlexInt` to write to disk and to restore a `FlexInt` from a binary representation.

The implementation, as far as possible, buries details in two internal methods: `__encode` and `__decode`. The specifics of each number format exist in these methods so that the remainder of the class is substantially the same for each.

As this is the first, I'll describe things in some detail. Later, number formats will gloss over what hasn't changed or is trivially generated from tweaks to the code described in this section. We begin with the class definition and constructor, followed by the encoding and decoding methods, before finishing with the remainder of the class.

12.3.1.1 Definition and Constructor

The `FlexInt` class and constructor begin like so:

```

1 class FlexInt:
2     FIXED_BITS = 62
3     FLEX_BITS = 1082
4     FIXED_MAX = (1 << FIXED_BITS) - 1
5     FLEX_MAX = (1 << FLEX_BITS) - 1
6
7     def __init__(self, n, base=10):
8         if isinstance(n, bytes):
9             v = int.from_bytes(n, byteorder='big')
10            b = len(n)*8
11            self.fx = list(f'{v:0{b}b}')
12        elif isinstance(n, str):
13            self.fx = self.__encode(int(n, base))
14        else:
15            self.fx = self.__encode(int(n))

```

The class-level constants define the largest fixed-width integer (`FIXED_MAX`) and extended `FlexInt` integer (`FLEX_MAX`). The constructor accepts either a set of bytes expected to be the exact representation of a `FlexInt`, an integer, or any string that can be turned into an integer assuming a base understood by Python's `int` function. For example,


```
>>> from FlexInt import FlexInt
>>> A = FlexInt(1234)
>>> B = FlexInt('1234', base=5)
>>> C = FlexInt('1234', base=6)
>>> D = FlexInt('1234', base=16)
>>> E = FlexInt('1234', base=23)
>>> A, B, C, D, E
(1234, 194, 310, 4660, 13298)
```

imports the `FlexInt` class and defines five instances using a standard base-10 integer followed by the same set of digits in bases 5, 6, 16, and 23.

The constructor calls `__encode` when the argument is not a byte array. The return value is a list of bits as characters (strings in Python), representing the binary form of the `FlexInt` as outlined by GPT-4. When the argument is a byte array, Python's `int` class is used to convert first to an integer (recall that Python supports arbitrary-precision integers), knowing that the argument is in big-endian format. The resulting integer in `v` is transformed into a binary string, then split into a list of binary digits as characters before assigning to the `FlexInt` member variable `fx`.

12.3.1.2 Encoding

Encoding an integer in `FlexInt` format is the most difficult part of the implementation. The `__encode` method first examines the magnitude of the integer, then decides which `FlexInt` format to use:

```
1 def __encode(self, n):
2     v = abs(n)
3     sign = 1 if (n < 0) else 0
4     if (v <= self.FIXED_MAX):
5         fx = ['0']*64
6         fx[1] = str(sign)
7         b = 62
8         fx[2:] = list(f'{v:0{b}b}')
9     else:
10        if (v > self.FLEX_MAX):
11            raise ValueError("Magnitude overflow")
12        excess = v.bit_length() - 58
13        ex = excess // 64
14        fx = ['0']*(64 + (ex+1)*64)
15        fx[0] = '1'
16        fx[1] = str(sign)
17        b = 4
18        fx[2:6] = list(f'{ex:0{b}b}')
19        b = v.bit_length()
20        bits = list(f'{v:0{b}b}')
21        fx[-len(bits):] = bits
22    return fx
```

If n is within $[-2^{62} - 1, 2^{62} - 1]$, it fits into a single 64-bit word (line 5). The 64-element list in `fx` is initialized to “0”, which sets the mode correctly. The sign is set next in line 6. All that remains is to place the bits of the magnitude of the argument into `fx` beginning with bit 62. Recall that FlexInts use big-endian format, and Python’s f-string conversion will return first a string in that format (the way we usually write numbers). Passing the string to `list` completes the conversion.

If the argument exceeds the fixed-width range, an extended FlexInt is created, assuming n fits at all (line 10). Line 12 determines the number of excess bits beyond the 58 that fit in the first 64-bit word. Lines 13 and 14 use this value to determine how many words are needed and to define `fx` appropriately. Lines 15 and 16 set the mode (1=extended) and sign, and lines 17 and 18 set the *E* field. Finally, lines 19 through 22 set the proper elements of `fx` to the bits of v , the magnitude of the argument n . The now fully encoded binary FlexInt is returned.

12.3.1.3 Decoding

Using a FlexInt requires decoding its binary form. The `__decode` method does this to return a Python integer and is used by all the methods that follow so that arithmetic is performed on standard Python integers before encoding the result as a new FlexInt object. This cheat allows us to focus on the storage format without the (exceedingly) tedious code necessary to perform arithmetic operations in binary. The `__decode` method is

```

1 def __decode(self):
2     mode = int(self.fx[0])
3     sign = int(self.fx[1])
4     idx = 2 if (mode==0) else 6
5     val = int("".join(self.fx[idx:]), 2)
6     return (-1)**sign * val

```

Decoding a FlexInt stored as a list of binary digits requires extracting the `sign` and the magnitude (`val`). The latter is pulled directly from the list and turned into a string via `join` using the mode to set the beginning of the magnitude bits: bit 2 (from the left) for fixed-width FlexInts or bit 6 if extended (to skip the *E* field) (line 5). Line 6 reconstructs the full integer value and returns it.

12.3.1.4 Methods

To use FlexInts within Python arithmetic expressions requires supporting the dunder methods Python will automatically attempt to call when they exist. All dunder methods begin and end with double underscore characters. The `FlexInt` class supports many, though only representative methods will be outlined in this section along with several utility methods like these:

```

1 def __str__(self):
2     return "%d" % self.__decode()
3 def __repr__(self):
4     return self.__str__()
5 def __int__(self):
6     return self.__decode()
7 def __bytes__(self):
8     v = int("".join(self.fx), 2)
9     b = len(self.fx) // 8
10    return v.to_bytes(b, byteorder='big')
11 def bin(self):
12    return "".join(self.fx)

```

The `__str__` and `__repr__` methods are called when Python needs to print a representation of a `FlexInt`. The `__int__` method is called by the `int` function to convert a `FlexInt` into a Python integer.

The `__bytes__` method responds to the `bytes` function by converting the list of bits first into a string and then an integer (`v`) before using the `to_bytes` integer method to format the value as a big-endian byte array. Use `bytes` to format the `FlexInt` for writing to disk or any other place that requires a binary representation. Note that the `__bytes__` method converts the entire `FlexInt`, bit by bit, not just the numeric value encoded within it. Finally, the `bin` method returns the `FlexInt`'s internal representation as a binary string.

Arithmetic methods come next:

```

1 def __add__(self, b):
2     v = self.__decode() + int(b)
3     return FlexInt(v)
4 def __sub__(self, b):
5     v = self.__decode() - int(b)
6     return FlexInt(v)
7 def __mul__(self, b):
8     v = self.__decode() * int(b)
9     return FlexInt(v)
10 def __truediv__(self, b):
11    v = self.__decode() // int(b)
12    return FlexInt(v)
13 def __pow__(self, b):
14    v = self.__decode() ** int(b)
15    return FlexInt(v)
16 def __mod__(self, b):
17    v = self.__decode() % int(b)
18    return FlexInt(v)
19 def __neg__(self):
20    return FlexInt(-self.__decode())
21 def __abs__(self):
22    v = self.__decode()
23    return FlexInt(abs(v))

```

Every method decodes the object's integer and, if binary, uses `int` to get the integer version of the other operand. This approach works for Python numbers and any object capable of being transformed into an integer, meaning objects supporting the `__int__` method. The method performs the necessary operation, returning the result as a new `FlexInt`.

Python supports two division operators, `/` and `//`, accessed via the `__truediv__` and `__floordiv__` methods, respectively. Python integers use these methods to divide and return a floating-point result or an integer result. How a class uses them is up to the designer. I've selected to use `/` but perform integer division internally; notice the `//` operator.

You'll find additional methods in *FlexInt.py* that mirror the dunder methods but begin with an "r". These are the reverse methods, and Python uses them for expressions where the second operand is a `FlexInt` object. For example, $A - 1$ where A is a `FlexInt` calls `__sub__`, while $1 - A$ calls `__rsub__`. If the operation commutes, the reverse method simply calls the main method. Noncommuting methods must perform the calculation with the `FlexInt` as the second operand, not the first.

The `FlexInt` class is complete; let's put it to work.

12.3.2 Examples

The `FlexInt` class operates in expressions where the operands can be converted to Python integers via the `int` function. For example,

```
>>> from FlexInt import FlexInt
>>> A,B,C = FlexInt(1234), FlexInt(34), FlexInt(8675309)
>>> A+1, 1+A, A-1, 1-A
(1235, 1235, 1233, -1233)
>>> A*B, B*A, (C+2*A-3*B)/B
(41956, 41956, 255225)
```

imports the `FlexInt` class, defines three `FlexInt`s, and then performs simple arithmetic with them. Notice that the expression in the final example is of arbitrary structure: the `FlexInt` class performs appropriately regardless of the complexity of the expression.

`FlexInt`s convert to and from the internal representation GPT-4 designed as needed. To use the representation as intended requires dumping `FlexInt`s to disk. Consider,

```

>>> from FlexInt import FlexInt
>>> A = FlexInt(1234)
>>> B = A**10
>>> A
1234
>>> B
8187505353567209228244052427776
>>> f = open("tmp", "wb")
>>> f.write(bytes(A))
8
>>> f.write(bytes(B))
16
>>> f.close()

```

First, A and B are defined and then dumped to disk in binary format via `bytes`. Notice that A occupies 8 bytes, a single 64-bit integer, while B is large enough to require 16 bytes. We can confirm this by examining the file `tmp`:

```

> xxd tmp
00000000: 0000 0000 0000 04d2 8000 0067 5741 b192
00000010: b69b f64c 85d5 4400

```

The `xxd` command dumps a file in hexadecimal and ASCII (removed). The first 8 bytes are `00000000000004D216` indicating mode 0 (fixed-width) and positive (sign bit 0). The value itself is $4D2_{16} = 1234$, as it should be.

The following 16 bytes represent B . The first hex digit is $8 = 1000_2$, implying mode 1 and a sign bit of zero (positive). The 4-bit E field is `0000`, implying a number that occupies one additional 64-bit word. In hex, then we have

$$675741B192B69BF64C85D54400_{16} = 8187505353567209228244052427776$$

again as it should be.

Recovering the FlexInts from disk is straightforward:

```

>>> f = open("tmp", "rb")
>>> A = FlexInt(f.read(8))
>>> B = FlexInt(f.read(16))
>>> A
1234
>>> B
8187505353567209228244052427776

```

Notice, however, that we already knew the FlexInts were such that the first occupied 8 bytes while the second used 16. In general, reading FlexInts in binary requires parsing the first byte of the number to determine the mode, sign, and, if extended, the number of additional 64-bit words to read. GPT-4 was aware of such issues when discussing the design:

- **Arithmetic Operations:** Implementing arithmetic operations requires algorithms capable of efficiently handling both fixed-width and arbitrary-precision integers. This might involve a slight overhead for determining the operation mode but is a worthy trade-off for the flexibility and scalability provided.
- **Memory Management:** Careful management of memory, especially in arbitrary-precision mode is crucial to avoid excessive memory allocation and fragmentation. Efficient memory management strategies, such as memory pooling for frequently used integer sizes, can mitigate these concerns.

The FlexInt class works well with other Python tools, including NumPy, Python's library for fast array processing. Python is the goto language for much of artificial intelligence in no small part to libraries like NumPy (`np`). Consider,

```
>>> from FlexInt import FlexInt
>>> import numpy as np
>>> A = [FlexInt(i) for i in [2,3,5,7,11,13]]
>>> A
[2, 3, 5, 7, 11, 13]
>>> a = np.array(A)
>>> a
array([2, 3, 5, 7, 11, 13], dtype=object)
>>> a+11
array([13, 14, 16, 18, 22, 24], dtype=object)
```

First, `A` is defined as a list of FlexInt objects. Next, NumPy converts this list into a vector, `a`, of object data type. NumPy is fast because it recognizes data types. Finally, NumPy adds 11 to every vector element, each a FlexInt integer, with a single expression: `a+11`.

As a final example, consider using NumPy and FlexInt to implement matrix multiplication:

```
>>> W = np.array([FlexInt(i+1) for i in range(9)])
>>> W = W.reshape((3,3))
>>> W
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]], dtype=object)
>>> x = np.array([FlexInt(2), FlexInt(3), FlexInt(5)])
>>> x = x.reshape((3,1))
>>> x
array([[2],
       [3],
       [5]], dtype=object)
>>> W @ x
array([[23],
       [53],
       [83]], dtype=object)
```

The example defines W first as a vector of FlexInt objects, then, via `reshape`, as a 3×3 matrix. Next, x is defined and reshaped as a 3×1 column vector. Finally, Wx is computed using NumPy's matrix multiplication operator (`@`) to return the expected column vector answer. Every multiplication and addition required by the matrix multiplication involved a FlexInt object.

It's evident that the FlexInt format works and serves to store and retrieve integer values correctly. GPT-4 was requested to be creative, and it delivered. Let's proceed to the next AI-designed number format.

12.4 MPIF: Mixed Precision Integer Float

GPT-4's next reply to the conservative prompt led to the Mixed Precision Integer Float (MPIF), another hybrid format that mixes integer and floating-point representation within a single 64-bit unsigned integer. The code is in *MPIF.py*, and the full transcript of my interaction with GPT-4 resides in *MPIF.pdf*.

GPT-4 began its reply with:

Designing a novel computer number format, especially with the constraint that it should support at least integers (signed or unsigned), is a fascinating challenge. It requires balancing between precision, range, performance, and ease of use.

then followed up by describing several standard number formats:

1. **Integer Formats:** Fixed-size formats like `int8`, `int16`, `int32`, `int64`, and their unsigned counterparts (`uint8`, `uint16`, etc.). These provide straightforward representations of whole numbers but have fixed limits on range and precision.
2. **Floating-Point Formats:** IEEE 754 formats like single-precision (`float32`) and double-precision (`float64`) floating-point numbers allow a wide range of values by sacrificing precision for numbers very large or small in magnitude.
3. **Arbitrary-Precision Formats:** Formats like those used in Python's `int` or libraries like GMP, where the number's size is only limited by available memory, allowing for very large integers but at the cost of performance.
4. **Decimal Formats:** Decimal formats (e.g., IEEE 754-2008 decimal floating-point formats) aim to provide more intuitive precision for decimal numbers, important in financial calculations to avoid rounding errors inherent in binary floating-point representations.

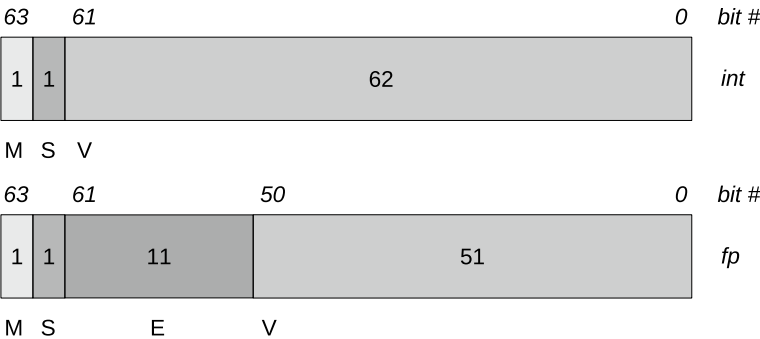
GPT-4's list contains many of the formats we've explored in this book. After this review, the model got down to business:

The Mixed Precision Integer-Float (MPIF) format is a hybrid numeric representation combining integer and floating-point representations, optimized for both range and precision.

Structure

- Sign bit (1 bit): Indicates whether the number is positive or negative.
- Mode bit (1 bit): Indicates whether the number is currently represented as an integer (0) or floating-point (1).
- Value bits (62 bits): The actual number, interpreted as either a large integer or a floating-point number based on the mode bit.
- In integer mode, these bits represent a 62-bit integer directly.
- In floating-point mode, these bits are split into a modified IEEE 754-like format with a smaller exponent and fraction part, optimized for precision around commonly used ranges.

Using GPT-4’s description as a guide, an MPIF number can be expressed in two ways depending on the mode bit: 0 for an integer and 1 for a floating-point number:



As with FlexInts, MPIF integers use a sign bit (1 = negative) and store the value as a positive 62-bit number. An MPIF float follows the structure of an IEEE *binary64* number but reduces the precision of the significand from 52 bits to 51 to accommodate the mode bit. GPT-4 didn’t state it explicitly, but its mention of IEEE implies that an MPIF float should assume a leading 1 for the significand so that a float is computed as

$$(-1)^S 1.b_{50}b_{49}b_{48} \dots b_0 \times 2^{E-1023}$$

where E is the excess-1023 11-bit exponent ala IEEE *binary64*, and $b_{50} \dots b_0$ are the 51 bits of the significand.

MPIF integers fit in $[-2^{62} + 1, 2^{62} - 1]$, which is less than the normal signed 64-bit integer range of $[-2^{63}, 2^{63} - 1]$ because of the bits lost to the mode and sign and the fact that not using two’s complement decreases the lower limit by one.

What about the floating-point range? MPIF floats cover much the same range as IEEE *binary64* floats, as only the least significant bit of the significand is missing. Therefore, MPIF floats fit in approximately $[-1.7977 \times 10^{308}, 1.7977 \times 10^{308}]$.

IEEE floats support infinity, not-a-number, and subnormal numbers. MPIF floats lack these features. Additionally, MPIF floats represent zero with entirely zero E and V fields corresponding to an IEEE float near $1.1125369292536007 \times 10^{-308}$. MPIF zero, like IEEE zero, may be signed or unsigned.

12.4.1 Implementation

Please review the file *MPIF.py* before continuing. As with *FlexInts* earlier in the chapter, the implementation is given in parts beginning with the constructor, followed by encoding, decoding, and select arithmetic and utility methods.

12.4.1.1 Definition and Constructor

As mentioned, all three GPT-4 designed number formats are similarly implemented as Python classes. The constructor accepts a Python byte array representing, bit for bit, an existing MPIF number, or an MPIF instance, an integer, or a float. All other data types are rejected. In code,

```

1 class MPIF:
2     MAX_INT = (1 << 62)
3     def __init__(self, n):
4         if isinstance(n, bytes):
5             v = int.from_bytes(n, byteorder='big')
6             b = len(n)*8
7             self.fx = list(f'{v:0{b}b}')
8         elif isinstance(n, MPIF):
9             self.fx = self.__encode(n())
10        elif (isinstance(n, int) or isinstance(n, float)):
11            self.fx = self.__encode(n)
12        else:
13            raise ValueError("Unsupported type")

```

The *MPIF* class, like the *FlexInt* class, stores the number as a list of bits. The goal of the constructor is to transform its argument into this format. If the argument (*n*) is a byte array, it is assumed already to be the binary representation of an MPIF number. All that remains is ensuring that the number is formatted correctly in binary, which lines 5 through 7 accomplish. If the argument is a Python integer or float, it is encoded (line 11). Finally, if the argument is already an MPIF instance, its current numeric value is encoded (line 9).

Notice that the argument to `__encode` is not merely *n* but *n()*. The *MPIF* class overloads the `__call__` dunder method to return an MPIF number's numeric value. The fact that a single MPIF number can be either an integer or a float means that using `int`, as was the case for the *FlexInt* format, is insufficient as that function must return an integer. Similarly, overloading `__float__` to return a value when using the `float` function is insufficient as it might lose precision when mapping an integer to a float. Therefore, there needs to be a way to get the current value of an MPIF number as integer *or* float according to the value of the mode bit. The selected solution in this case is to use `__call__` and treat returning the proper value of the number as a function call on the variable, hence using *n()* in line 9.

12.4.1.2 Encoding

The `__encode` method accepts an integer or float and represents it according to GPT-4's plan for an MPIF number. The method itself is straightforward:

```

1 def __encode(self, n):
2     if isinstance(n, int):
3         if (abs(n) > self.MAX_INT):
4             raise ValueError("Magnitude overflow")
5         fx = ['0']*64
6         fx[1] = '1' if (n<0) else '0'
7         b = 62
8         v = abs(n)
9         fx[2:] = list(f'{v:0{b}b}')
10    else:
11        fx = ['0']*64
12        fx[0] = '1'
13        if (n!=0):
14            fx[1] = '1' if (n<0) else '0'
15            s = abs(n).hex()[4:]
16            m,e = s.split('p')
17            e = int(e) + 1023
18            b=11
19            fx[2:13] = list(f'{e:0{b}b}')
20            b=13*4; v=int(m,16)
21            fx[13:] = list(f'{v:0{b}b}')[:-1]
22    return fx

```

If the argument (`n`) is an integer, and it fits (less than 10^{62} in magnitude), then create the required 64-element bit list (`fx`) and populate it with 0 for the mode, the proper sign, and a 62-bit magnitude with all necessary leading zeros (line 9).

Otherwise, assume the argument is a float, define `fx`, and mark it as mode 1. Line 13 asks if the argument is not zero. If the argument *is* zero, `fx` is already everything it needs to be as the *E* and *V* fields are all zeros. A nonzero argument is processed by setting the sign (line 14), then using Python's `hex` method on floats to represent the number as a hexadecimal string. For example, 3.141592×10^{11} becomes the string `'0x1.249563dc00000p+38'` which is a curious mixed-base representation using a base-16 significand and a decimal exponent on 2.

Encoding continues by skipping the first four characters (`'0x1.'`) and separating the significand from the exponent (line 16). The excess-1023 exponent is determined and then represented as an 11-bit binary string with leading zeros if necessary (line 19). This is the *E* field. The *V* field is set by interpreting the base-16 significand, without the leading 1, as a 52-bit binary string of which the first 51 bits are placed into `fx` (line 21).

Integers (mode 0) are decoded as sign (line 3) times magnitude (line 5). Floats (mode 1) are separated first into exponent (line 7) and significand (line 8). The former interprets the *E* field as a binary number and subtracts 1023. The latter creates a binary string from the significand. Notice that an extra zero is added to the end of the string, so it has 52 bits as required by an IEEE *binary64*.

Lines 9 through 11 first construct a Python float string using a mixed base-16 significand (line 10) with appended decimal exponent before using the `float.hex` class method to perform the floating-point conversion (line 11).

12.4.1.4 Methods

The MPIF class mimics the FlexInt class by converting the internal binary representation before performing the arithmetic operation. Again, this trick simplifies the implementation considerably, allowing us to focus on the number format. The remainder of *MPIF.py* is split between utility and arithmetic methods beginning with the utility methods:

```

1 def __call__(self):
2     return self.__decode()
3 def __str__(self):
4     return str(self.__decode())
5 def __repr__(self):
6     return self.__str__()
7 def __int__(self):
8     return int(self.__decode())
9 def __float__(self):
10    return float(self.__decode())
11 def __bytes__(self):
12    v = int("".join(self.fx), 2)
13    b = len(self.fx) // 8
14    return v.to_bytes(b, byteorder='big')
15 def bin(self):
16    return "".join(self.fx)
17 def mode(self):
18    return int(self.fx[0])

```

The previously mentioned `__call__` method is used to return the value of an MPIF number as an integer or float. To return an integer regardless of the storage format, use `int`. Similarly, use `float` to return a Python float. The call syntax is used by the arithmetic methods when necessary. The string representation and `bytes` methods act as they did for FlexInts. The `bin` method returns a big-endian binary string representing the bits of the MPIF number. The `mode` function returns 0 for integer and 1 for float.

Arithmetic methods follow with only the main ones listed here (i.e., no reverse methods):

```

1 def __add__(self, b):
2     v = self.__decode() + MPIF(b)()
3     return MPIF(v)
4 def __sub__(self, b):
5     v = self.__decode() - MPIF(b)()
6     return MPIF(v)
7 def __mul__(self, b):
8     v = self.__decode() * MPIF(b)()
9     return MPIF(v)
10 def __truediv__(self, b):
11     v = self.__decode() / MPIF(b)()
12     return MPIF(v)
13 def __floordiv__(self, b):
14     v = self.__decode() // MPIF(b)()
15     return MPIF(v)
16 def __pow__(self, b):
17     v = self.__decode() ** MPIF(b)()
18     return MPIF(v)
19 def __neg__(self):
20     return MPIF(-self.__decode())
21 def __abs__(self):
22     return MPIF(abs(self.__decode()))

```

The methods follow the form used by FlexInts, but notice that the call syntax is used to return MPIF objects as Python integers or floats when performing the actual operation. Also notice that both true division returning a float and floored division returning an integer are supported.

Let's explore the format with a few examples.

12.4.2 Examples

It's not necessary to show every form of the constructor; the pattern follows that of FlexInts. What concerns us most is how MPIF numbers are represented in binary, especially when reading from and writing to disk. Consider,

```

>>> from MPIF import MPIF
>>> A = MPIF(1234)
>>> B = MPIF(3.141592e11)
>>> f = open('tmp', 'wb')
>>> f.write(bytes(A))
8
>>> f.write(bytes(B))
8
>>> f.close()

```

The example creates two MPIF numbers, A (int) and B (float), then writes them to disk in binary format. Notice that each number occupies 8 bytes. FlexInts varied in size depending on their magnitude. MPIF numbers always occupy 8 bytes.

The `xxd` command demonstrates that the numbers were properly written in binary:

```
> xxd tmp
00000000: 0000 0000 0000 04d2 a129 24ab 1ee0 0000
```

and reading them from disk in binary validates the claim:

```
>>> f = open('tmp', 'rb')
>>> A = MPIF(f.read(8))
>>> B = MPIF(f.read(8))
>>> f.close()
>>> A
1234
>>> B
314159200000.0
```

NumPy works with MPIF numbers as it does with FlexInts; simply replace the FlexInts of the earlier example with MPIF numbers.

Let's proceed to the last of GPT-4's number formats.

12.5 MetaInt

GPT-4's *MetaInt* format combines a standard integer number with associated metadata at the binary level. My prompt merged conservatism with an exhortation to creativity:

Think deeply about all the computer number formats you know, especially those capable of supporting integers, signed or unsigned. Then, with this background in mind, create a novel computer number format for integers, one that can be implemented on existing computers. Be creative! Be wild! Don't be one of the herd!

The model's reply, found in *MetaInt.pdf*, began like this:

Creating a novel computer number format, especially for integers, requires stepping outside conventional boundaries while ensuring the format is practical for computation. Let's imagine a format that not only captures the value of an integer but also incorporates metadata that could enrich its computational context or utility. I'll call this format the "MetaInt" format.

The model continued:

Concept

The MetaInt format is designed to store integers along with metadata that could include information about the provenance of the data, its reliability, a timestamp for when it was last modified, and a unique identifier for the data source. This approach can be instrumental in data science, where understanding the origin and reliability of data is crucial.

Structure

A MetaInt value is composed of the following components:

1. **Value:** The actual integer value, stored in a conventional format (e.g., two's complement for signed integers).
2. **Metadata:** A structured collection of metadata, including
 - **Provenance:** Information about where the data came from.
 - **Reliability:** A score or flag indicating the reliability of the data.
 - **Timestamp:** When the data was last updated or modified.
 - **Source ID:** A unique identifier for the data source.

GPT-4's suggestions regarding what metadata might be associated with a value are reasonable.

I followed up on GPT-4's description by pointing out that the MetaInt format is readily extended to include floating-point values. The model agreed and expanded the design to "MetaNums", supporting integers and floats. I leave this extension to the exercises.

Ultimately, I adapted the metadata portion to include three fields: a flag indicating ORIGINAL or DERIVED, an owner ID number, and a 32-bit timestamp. The ORIGINAL and DERIVED concepts are borrowed from DICOM and indicate values understood as directly measured (e.g., from an MRI machine) or derived from original data, perhaps as the output of a filter operation.

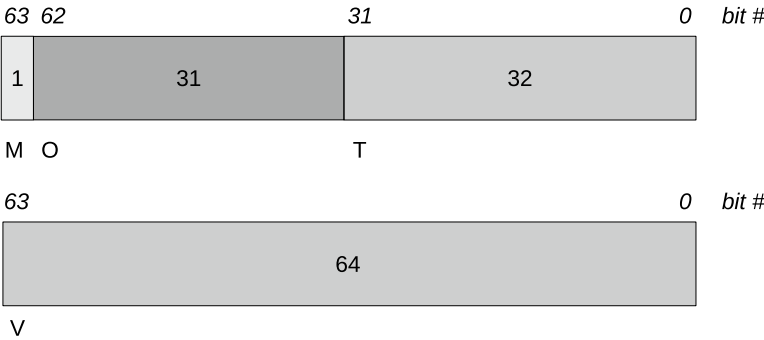
The owner ID provides a (minimal) way of declaring the source of the data values and enforcing operations such that the owner IDs must match before arithmetic operations proceed. An owner ID of zero implies public data compatible with all other owner IDs.

A 32-bit timestamp stored as an unsigned 32-bit number counts seconds from midnight January 1, 1970 UTC. This is known as the Unix epoch. Using an unsigned 32-bit integer means that timestamps will overflow on February 7, 2106, at 06:28:15 UTC, over 80 years beyond the date on which I’m writing. I think that an unsigned timestamp will be perfectly adequate.

It is worth noting that a signed 32-bit timestamp will overflow at 03:14:07 UTC on January 19, 2038. This date is likely to be a problem, though how bad of one remains to be seen. The Y2K bug scare amounted to little because significant effort was made ahead of time to catch and update most instances of 2-year dates. The amount of code in circulation in 2038 is likely orders of magnitude greater than that present on January 1, 2000. Will the code be patched on time?

The metadata described requires 1 bit for ORIGINAL or DERIVED and 32 bits for the timestamp, leaving 31 bits of a 64-bit word available for the owner ID. This configuration allows owner IDs in the range $[1, 2^{31} - 1]$, remembering that an ID of zero means public data.

The FlexInt and MPIF number formats store integers as signs and magnitudes. MetaInts use two’s complement to represent a single MetaInt in binary as two 64-bit words. The first is metadata, and the second is a standard 64-bit two’s complement integer for a range of $[-2^{63}, 2^{63} - 1]$. Visually, a MetaInt is



with M the ORIGINAL or DERIVED flag, O the owner ID, T the timestamp, and V the two’s complement value.

The previous GPT-4-designed number formats, FlexInt and MPIF, were immediately applied to arithmetic operations. The MetaInt format requires more care in two ways. First, in enforcing the rule that operations on A and B are only allowed if one of the following conditions are met:

- $\text{owner}(A) = \text{owner}(B)$
- $\text{owner}(A) = 0$ or $\text{owner}(B) = 0$ (or both)

with the understanding that an arbitrary integer has an owner ID of zero. In other words, the owner IDs must match, or at least one must be zero.

Second, care is required in updating the metadata of the result of an allowed operation. If $C = A + B$, and the owners of A and B are compatible, then C 's metadata is set to:

- $\text{owner}(C) = \text{owner}(A)$ if A the leftmost operand and $\text{owner}(A) \neq 0$
- $\text{owner}(C) = \text{owner}(B)$ if the reverse dunder method used and $\text{owner}(B) \neq 0$
- $\text{owner}(C) = 0$, otherwise

with the ORIGINAL or DERIVED flag always set to DERIVED (0). Creating a MetaInt object sets the flag to ORIGINAL by default. The timestamp is set whenever a new MetaInt object is created, regardless of the owner ID or the original flag.

12.5.1 Implementation

As mentioned, you'll find the code in the file *MetaInt.py*. Let's walk through the by-now-familiar stages.

12.5.1.1 Definition and Constructor

The constructor accepts a byte array for a bitwise copy of an existing MetaInt or an integer:

```

1 import time
2 from datetime import datetime
3 class MetaInt:
4     def __init__(self, n, owner=0, original=1):
5         if isinstance(n, bytes):
6             v = int.from_bytes(n, byteorder='big')
7             b = len(n)*8
8             self.fx = list(f'{v:0{b}b}')
9         else:
10            self.fx = self.__encode(int(n), owner, original)

```

By default, creating a MetaInt via the constructor implies that the value should be regarded as ORIGINAL.

12.5.1.2 Encoding

Encoding an integer involves ensuring that it is within range, converting the value to a list of bits and constructing the metadata in a similar fashion:

```

1 def __encode(self, n, owner, original):
2     if (n > 0):
3         if (n >= (1<<64)):
4             raise ValueError("Magnitude overflow")
5         v = n
6     else:
7         if (abs(n) > (1<<64)):
8             raise ValueError("Magnitude overflow")
9         v = n + (1<<64)
10    fx = ['0']*128
11    fx[64:] = list(f'{v:064b}')
12    fx[0] = "%1d" % original
13    if (owner < 0) or (owner >= (1<<31)):
14        raise ValueError("Owner magnitude overflow")
15    fx[1:32] = list(f'{owner:031b}')
16    ts = int(time.time()) & 0xFFFFFFFF
17    fx[32:64] = list(f'{ts:032b}')
18    return fx

```

Line 10 defines the 128-element list of bits, the first 64 of which are metadata, with the remaining 64 bits the two's complement integer value (line 11). Line 12 sets ORIGINAL or DERIVED, while lines 13 through 15 set the owner ID.

Python's `time` module is used to return the current epoch time, which is masked to 32 bits (line 16) and then placed into the lower 32 bits of the metadata word (line 17). As mentioned, using an unsigned 32-bit integer value for the timestamp supports dates into the early 22nd century, which should not be an issue going forward. If it becomes one, let me know.

12.5.1.3 Decoding

Decoding a `MetaInt` value is straightforward. Decoding the associated metadata requires Python library functions:

```

1 def __decode(self, meta=False):
2     orig = int(self.fx[0])
3     owner = int("".join(self.fx[1:32]), 2)
4     ts = int("".join(self.fx[32:64]), 2)
5     dt = datetime.utcfromtimestamp(ts)
6     tss = dt.strftime('%Y-%m-%d %H:%M:%S')
7     val = int("".join(self.fx[64:]), 2)
8     if (self.fx[64] == '1'):
9         val = val - (1<<64)
10    if (meta):
11        return orig, owner, tss, val
12    else:
13        return val

```

The ORIGINAL or DERIVED flag comes first (line 1), followed by the owner ID (line 3). Lines 4 through 6 decode the timestamp with help from Python's `datetime` library. Notice that line 5 uses UTC, not local time. A MetaInt shouldn't be tied to any particular location on Earth, implying UTC. The date string format of line 6 is adjustable as long as the requisite fields are present.

Line 7 converts the bits of the value into an integer. Here, care is required. Python will not interpret a leading "1" digit as a negative number, meaning `val` is initially a positive integer. Line 8 checks for a leading "1" digit. If present, line 9 subtracts 2^{64} to arrive at the corresponding negative value.

12.5.1.4 Methods

The methods section of *MetaInt.py* is more extensive than the other formats. The extra code checks for owner ID compatibility and manages metadata. Here, you'll find the utility functions, the complete code for addition as a stand-in for all binary operations, including the reverse methods, and negation, a unary operator. Let's begin with the utilities:

```

1 def __str__(self):
2     return "%d,%d,%s" %d" % self.__decode(meta=True)
3 def __repr__(self):
4     return self.__str__()
5 def __int__(self):
6     return self.__decode()
7 def __bytes__(self):
8     v = int("".join(self.fx), 2)
9     b = len(self.fx) // 8
10    return v.to_bytes(b, byteorder='big')
11 def bin(self):
12    return "".join(self.fx)
13 def metadata(self):
14    orig, owner, tss, _ = self.__decode(meta=True)
15    return orig, owner, tss

```

The set is standard and expected. MetaInt objects are only integers; therefore, the `int` function returns the value without metadata. Compare this to the `__call__` syntax used by MPIF earlier in the chapter. The `bytes` function returns bytes representing the complete set of 128 binary digits representing a MetaInt according to GPT-4's specification. The `bin` and `metadata` methods return what you expect. Notice that the string conversion methods present the value and metadata.

Addition and negation come next:

```

1 def __add__(self, b):
2     _, ownerA, _, A = self.__decode(meta=True)
3     if isinstance(b, MetaInt):
4         _, ownerB, _ = b.metadata()
5         v = A + int(b)
6         if (ownerA == 0) or (ownerB == 0):
7             if (ownerA == 0):
8                 ans = MetaInt(v, owner=ownerB, original=0)
9             else:
10                ans = MetaInt(v, owner=ownerA, original=0)
11        elif (ownerA == ownerB):
12            ans = MetaInt(v, owner=ownerA, original=0)
13        else:
14            raise ValueError("Owner mismatch")
15    else:
16        v = A + int(b)
17        ans = MetaInt(v, owner=ownerA, original=0)
18    return ans
19
20 def __neg__(self):
21     _, owner, _, v = self.__decode(meta=True)
22     return MetaInt(-v, owner=owner, original=0)

```

Let's begin with negation, `__neg__`. As a unary operator, there is no need to check the owner ID; therefore, line 21 merely acquires the current `owner` and value (`v`) before constructing a new DERIVED `MetaInt` with that owner ID and negated value.

Addition considers two cases: adding an existing `MetaInt` object (line 3 and following) or adding a value convertible to an integer via `int` (lines 16 and 17). The latter operates as expected and propagates the existing owner ID while ensuring the returned `MetaInt` is DERIVED.

Line 4 extracts the owner and value from the supplied `MetaInt` object. The sum is in line 5. The `if` in line 6 begins the process of confirming that the two values are compatible as to owner ID. If one or the other owner ID is zero, the sum proceeds using the owner ID of the other. Likewise, if the owner IDs match, the sum proceeds. Otherwise, a value error is raised because incompatible owner IDs were found.

12.5.2 Examples

A few `MetaInt` examples illustrate how to use the class. `MetaInt` operates like the `FlexInt` and `MPIF` classes discussed earlier in the chapter. First, create two `MetaInts`, perform arithmetic with them, and dump them to disk so we can examine their binary format:

```
>>> from MetaInt import MetaInt
>>> A = MetaInt(-1234, owner=42)
>>> B = MetaInt(34)
>>> C = A+2*B
>>> f = open("tmp", "wb")
>>> f.write(bytes(A))
16
>>> f.write(bytes(B))
16
>>> f.write(bytes(C))
16
>>> f.close()
```

The first *MetaInt*, *A*, has owner ID 42. The second, *B*, has no owner, therefore the expression defining *C* is valid and should result in *C* having the same owner ID as *A*. All three are dumped to disk, each occupying 16 bytes, as expected:

```
> xxd tmp
00000000: 8000 002a 661e 0224 ffff ffff ffff fb2e
00000010: 8000 0000 661e 022c 0000 0000 0000 0022
00000020: 0000 002a 661e 0236 ffff ffff ffff fb72
```

where each row is a complete *MetaInt* in binary. The leading 8 digit of the first byte is 1000_2 , implying the first two *MetaInt*s, *A* and *B*, are ORIGINAL. The third, *C*, is 0000_2 , as expected for a DERIVED value.

Reading the *MetaInt*s from disk produces

```
>>> from MetaInt import MetaInt
>>> f = open("tmp", "rb")
>>> A,B,C = [MetaInt(f.read(16)) for i in range(3)]
>>> A
(1,42,2024-04-23 18:07:56) -1234
>>> B
(1,0,2024-04-23 18:08:00) 34
>>> C
(0,42,2024-04-23 18:08:06) -1166
>>> f.close()
```

Notice that *A* and *C* share the same owner ID, 42, and that *C* is DERIVED while *A* and *B* are ORIGINAL. Finally, $-1234 + 2(34) = -1166$, implying proper application of *MetaInt*'s dunder methods.

12.6 Discussion

We asked GPT-4, a large language model, to develop multiple novel number formats for computers. We learned that *how* we asked the question significantly impacted the utility of the model's answer. Tailoring prompts to large language models is known as “prompt engineering”, and it has become a career of sorts, at least for now. Part of the issue lies in the fact that it is not presently known exactly how such models do what they do. The interactions within the, for GPT-4, 1.7 trillion or so weights are far beyond what the human mind can fathom. Therefore, some practice is required to learn how to interact with these new, partial minds. We shouldn't be too surprised. Those who remember a time before the Internet, or at least the web, remember that there was a learning curve to working with search engines effectively. LLMs present a similar, though perhaps more subtle, learning curve. As the old saying goes, be careful what you ask for.

In the end, GPT-4 was coaxed into inventing three new number formats that it named FlexInts, Mixed Precision Integer Floats, and MetaInts. This chapter described each format, with occasional interpretation on my part, and then implemented them for storage purposes as Python classes. As mentioned, developing hardware or software simulation of arithmetic acting on the bit patterns of each format is too complex a task for our purposes. Now, let's review each format with a somewhat critical eye.

FlexInt. FlexInts are capable of storing integers efficiently based on their magnitude. If small enough, a single 64-bit word suffices. Larger integers, up to a limit approaching 2^{1082} , are represented by appending the required number of additional 64-bit words to accommodate the number. This approach has a certain elegance and might be helpful for sequential storage, but the varying length of a number makes it impossible to index into an array of FlexInts stored sequentially in memory. Yes, in RAM, an array of pointers to FlexInt objects, however, accomplished, will suffice, but when written sequentially to a flat disk file, arbitrary access becomes impossible without reading the first FlexInt, followed by the second, and so on. Therefore, FlexInts become limited in utility to storing values that will not be accessed randomly. And this is to say nothing of the excessive effort (read probable nightmare) demanded of any hardware implementation. As mentioned earlier, FlexInts might find utility in highly precise fixed-point numbers.

Final assessment: A success overall, but with limited applicability.

Mixed Precision Integer Float. This number format uses a fixed-sized 64-bit representation. The number stored in each 64-bit word is an integer or a float with a range slightly reduced from that found in typical 64-bit integers (signed) and, by one bit in the significand, an IEEE *binary64* number—51 bits in place of 52.

At first blush, it might seem redundant to mix the integer and float representations, but by doing so, GPT-4's approach gains the ability to store larger integer values without loss of precision. For example, an MPIF number can store integers as large as 2^{62} , but the largest integer value that can be stored as a float is only 2^{52} . The

same situation applies to IEEE floating-point numbers with suitable exponents. This means an MPIF number, using the same number of bits for both integer or float, can store values with greater precision, assuming such values are integers. Consider,

```
>>> from MPIF import MPIF
>>> N = MPIF(2**52)
>>> N
4503599627370496
>>> F = MPIF(2.0**52)
>>> F
4503599627370496.0

>>> N = MPIF(2**52+1)
>>> F = MPIF(2.0**52+1)
>>> N
4503599627370497
>>> F
4503599627370496.0
```

The first example defines N as an integer and F as a float, showing that both are exact for 2^{52} . However, the second example illustrates the novel ability of MPIF numbers: $2^{52} + 1$ is exact as an integer but rounded as a float. The slight loss of precision, both integer and floating point, required of an MPIF number compared to standard formats is, I suspect, of no practical significance in most cases.

Implementing FlexInts in hardware would be difficult. Implementing MPIF numbers in hardware might be only slightly more complex than current implementations, as current implementations already require separate hardware paths for integers and floating point. Modifying the presented MPIF format to replace the sign-magnitude 62-bit integer representation with a two's complement 63-bit representation would contribute even more to simplifying the transition to existing hardware and increase the integer range to $[-2^{62}, 2^{62} - 1]$ from $[-2^{62} + 1, 2^{62} - 1]$.

Final assessment: A success with potential if suitable hardware implementations are constructed.

MetaInt. GPT-4's MetaInt format is the least useful of the three. It's undoubtedly effective to associate metadata with numerical data, and doing so is nothing new, but tagging each number with what would certainly be highly redundant metadata is wasteful. Why use 128 bits for every 64-bit integer when, for a collection representing some acquired data, say A/D samples, a single additional 64 bits of metadata is likely sufficient?

Final assessment: A failure, not for lack of imagination, but for unnecessary waste of memory.

Asking a large language model to generate a novel computer number format is itself something new in that even 2 years ago, such an experiment was not possible. I consider models like GPT-4 to be akin to Model T cars. They are the first of the next generation of AI models, not the last. A few years from this writing (April 2024), I suspect computer scientists will be able to repeat this simple exercise in earnest with models whose creativity will as far outrun GPT-4's as a Formula 1 race car can outrun a Model T.

Exercises

12.1 The `FlexInt` class implements basic arithmetic and a few utility methods. Extend the class by adding dunder methods for equality (`__eq__`), not equal (`__ne__`), less than (`__lt__`), less than or equal (`__le__`), greater than (`__gt__`), and greater than or equal (`__ge__`).

12.2 Repeat Problem 12.1 for the `MPIF` class.

12.3 Repeat Problem 12.1 for the `MetaInt` class.

12.4 Add metadata to the `MPIF` class to create a new number type (MetaMPIF?)

12.5 Create the algorithms, and optionally the code, to implement at least the addition of two MPIF numbers using the binary representations. When can the addition be performed between two integers? When must an integer be converted to a float? What about representing the final result? **

References

1. Arora S, Goyal A (2023) A theory for emergence of complex skills in language models. arXiv preprint [arXiv:2307.15936](https://arxiv.org/abs/2307.15936)
2. Kojima T, Gu SS, Reid M, Matsuo Y, Iwasawa Y (2022) Large language models are zero-shot reasoners. *Adv Neural Inf Process Syst* 35:22199–22213
3. Vaswani A et al (2017) Attention is all you need. *Adv Neural Inf Process Syst* 30



Abstract

In this chapter, we explore several somewhat unusual number systems. Most of these find use in hardware and are not encountered, typically, in software. Nonetheless, we will examine these number systems from a software perspective to understand how they operate and why and when they might be advantageous. Specifically, this chapter discusses the logarithmic number system, the double-base number system, the residue number system, and the redundant signed-digit number system.

13.1 Introduction

The book's other chapters examined many key number systems computers use. However, this list has not been exhaustive. Indeed, in this chapter, we will briefly consider four other number systems that are intellectually interesting in their own right, each with strengths and weaknesses. These number systems are typically implemented in hardware, but we will explore software versions here.

First, we consider the logarithmic number system (LNS), which stores numbers in log form. We follow with the double-base number system (DBNS), which, as the name implies, uses more than one base to store numbers. These numbers are representable as a two-dimensional array or image. Lastly, we consider the residue (RNS) and redundant signed-digit (RSD) number systems that seek to improve arithmetic by sacrificing specificity.

13.2 Logarithmic Number System

The IEEE 754 floating-point format, discussed in detail in Chap. 3, stores real numbers in memory as an approximation using a base-2 floating-point significand multiplying 2^i where i is an integer exponent. This directly preserves the magnitude of the number. While a completely natural and reasonable thing to do when faced with the question of “How do I store an arbitrary real number in finite computer memory?” as Chap. 3 shows, manipulating these numbers requires considerable effort for certain operations, namely, multiplication and division. So, the question could be asked, “Is there a way to store numbers so that multiplication and division become simpler?”. The answer is “yes”, and logarithmic numbers are the solution. The logarithmic number system (LNS) stores numbers as 2^x where x is now a floating-point exponent. The significand is always an implied one.

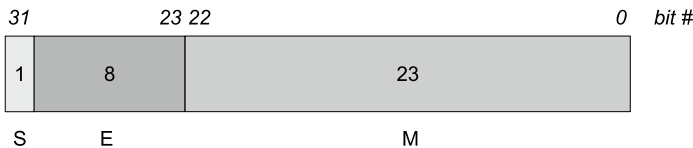
To understand why storing numbers this way helps with multiplication and division, we need only remember high school algebra. For two numbers, X and Y ,

$$\begin{aligned} A &= \log_2(X) \\ B &= \log_2(Y) \\ XY &= 2^A \times 2^B \\ &= 2^{A+B} \\ X/Y &= 2^A / 2^B \\ &= 2^{A-B} \end{aligned}$$

where we see that for numbers stored in log form (A and B), multiplication and division become addition and subtraction. This is quite convenient, but there is a price to pay. What about addition and subtraction of numbers stored in log form? Here is where the difficulty lies and is why the logarithmic number system isn’t widely used in practice.

It has been suggested that the advantage of casting multiplication and division as addition and subtraction outweighs the difficulty inherent in addition and subtraction of numbers stored this way. Indeed, LNS hardware projects exist. For example, the *European Logarithmic Microprocessor* [1] implements logarithmic numbers in hardware. We explore addition and subtraction of logarithmic numbers in time, but first, let’s discuss how to store the numbers in memory.

Representing and Storing LNS Numbers. A 32-bit IEEE 754 number is stored in memory as

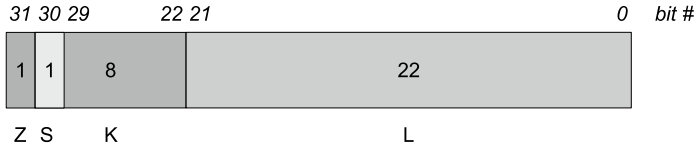


with a sign bit (S), 8 bits for the exponent (E), and 23 bits for the significand (M). The exponent is stored in excess-127 format with the special values of 0 and FF_{16} reserved. The implied base is 2, and the significand has an implied leading 1 bit for a normalized number. This results in a number stored as

$$\pm 1.b_{22}b_{21}b_{20}\dots b_0 \times 2^{E-127}$$

where b_n is the value of that bit of the significand. Note that this format scales the floating-point significand by 2 to an integer exponent ($2^i, i \in \mathbb{Z}$).

By contrast, a number stored in LNS format uses an implied integer significand of 1 and a floating-point exponent on the base of 2. This exponent is the log base-2 of the actual number, and it is the exponent that is stored along with the sign. There is currently no standard defining how LNS numbers are stored in memory. We will adopt the format used in [2] for a 32-bit representation of an LNS number with the modification that the 2 bits reserved for flags corresponding to the IEEE 754 values of *zero*, *+inf*, *-inf*, and *NaN* will be replaced with a single bit flag to indicate *zero*. Therefore, our numbers will be stored as



with 1 bit to indicate zero (Z), a sign bit (S), and floating-point exponent (E) stored in *fixed-point* format using 8 bits for the integer part (K) and 22 bits for the fractional part (L), i.e., as a *Q7.22* number. See Chap. 6 for a review of fixed-point numbers. We adopt this approach so that our LNS numbers can be represented as `uint32` numbers in C at the expense of some precision. This results in a number stored as

$$\pm 1 \times 2^E$$

where the significand is always an implied “1” and E is a fixed-point number stored in two’s complement format so that numbers < 1.0 have a negative exponent.

What is the range of numbers we can represent this way? The answer depends on the range of the exponent. A *Q7.22* number covers the range $[-2^7, 2^7 - 2^{-22}] = [-128, 127.9999998]$, which means that the smallest number, in magnitude, that can be represented is $2^{-128} \approx 2.9387359 \times 10^{-39}$ while the largest is $2^{27-2^{-22}} \approx 3.4028231 \times 10^{38}$ which compares favorably with the range of an IEEE 32-bit floating-point number: 1.17549×10^{-38} to 3.40282×10^{38} .

As with IEEE 754 floating-point numbers, the spacing between numbers in the LNS format is not constant. The minimum interval by which the exponent will change is $\Delta \equiv 2^{-22} \approx 2.3841858 \times 10^{-7} \approx 0.00000024$. Therefore, the difference between any number 2^N and the next largest number $2^{N+\Delta}$ is

$$2^{N+\Delta} - 2^N = 2^\Delta 2^N - 2^N = (2^\Delta - 1)(2^N)$$

Therefore, the difference between the smallest and the next smallest representable numbers is $2^{\Delta}2^{-128} \approx 4.9 \times 10^{-46}$. The difference between the largest and the next largest numbers is $2^{\Delta}2^{27-2^{-22}} \approx 5.6 \times 10^{31}$.

In IEEE 754, the smallest change to the significand for a 32-bit number is $2^{-23} \approx 1.19 \times 10^{-7}$ regardless of the exponent. Therefore, between any number and the next closest number, the difference is $2^{-23}2^{\beta}$ where β is the exponent of the number (ignoring steps between exponents for now). Therefore, between the smallest number and the next smallest, the difference is $2^{-23}2^{-126} \approx 1.4 \times 10^{-45}$ and between the largest and next largest we have $2^{-23}2^{127} \approx 2.0 \times 10^{31}$. This result indicates that LNS numbers are placed on the real number line like IEEE 754 floating points. However, unlike IEEE 754, which has a fixed spacing between numbers until the exponent changes by one, which then doubles the spacing, the spacing between LNS numbers is a smoothly varying function.

Let's define C functions to handle input, output, and storage of LNS numbers. Storage of LNS numbers is straightforward because we arranged matters so that a single LNS number uses the same number of bits as a 32-bit unsigned integer. Therefore, we define some preliminaries like so:

```

1 #include <stdio.h>
2 #include <math.h>
3 #include <inttypes.h>
4
5 #define K 8
6 #define L 22
7 #define LNS_ZERO 0x80000000
8 #define LNS_SIGN 0x40000000
9 #define LNS_MASK 0x3FFFFFFF
10
11 typedef uint32_t lns_t;
12
13 double lg2(double x) {
14     return log(x) / log(2.0);
15 }
```

We include necessary C libraries, define a local version of log base-2 (in case it is not in `math.h`), and declare our LNS number type to be an unsigned 32-bit int. We also define K and L , the integer and fractional parts of the fixed-point exponent. Next come three constants: `LNS_ZERO`, `LNS_SIGN`, and `LNS_MASK`. They are used to check and set the zero and sign bits, and to mask bits for the exponent.

Conversion to and from LNS numbers will cheat and use existing C functionality. The process of doing the conversion in hardware, like in an FPGA, is more involved. Similarly, we will cheat when going from LNS to floating point. Specifically,

```

1 | lns_t to_lns(double x) {
2 |     double e = lg2(fabs(x));
3 |     uint8_t s = (x >= 0.0) ? 0 : 1;
4 |     lns_t ans;
5 |
6 |     if (x == 0.0) return LNS_ZERO;
7 |     ans = (lns_t)(LNS_MASK & (uint32_t)floor(pow(2.0,L)*e + 0.5));
8 |     if (s) { ans |= LNS_SIGN; }
9 |     return ans;
10| }
11|
12| double to_double(lns_t x) {
13|     uint32_t z = (uint32_t)(LNS_ZERO & x);
14|     uint32_t s = (uint32_t)(LNS_SIGN & x);
15|     int32_t p = (uint32_t)(LNS_MASK & x);
16|     double m;
17|
18|     p = (0x20000000 & p) ? 0xC0000000+p : p;
19|     if (z != 0) { return 0.0; }
20|     m = pow(2.0, (double)p * pow(2.0,-L));
21|     return (double)(s == 0) ? m : -m;
22| }

```

Conversion to LNS (`to_lns`) first calculates the log base-2 of the input and stores it in `e` (line 2). It then captures the overall sign of the number in `s` (line 3). A quick special case check for zero is in line 6. If zero, return the constant for zero, which is the highest bit set, `0x80000000`.

Line 7 scales the log of the input by the number of bits in the fractional part of the exponent ($L = 22$), adding 0.5 to round to the nearest integer value. Casting to `uint32_t` makes an integer of the scaled float. Line 7 also AND's the output with `0x3FFFFFFF` to clear the highest 2 bits. Recall that the highest bit (bit 31) is the zero flag, and the next highest (bit 30) is the sign. If the magnitude of the input is less than 1.0, the log is negative, and the scaled integer output from the `pow` function will also be negative for all 32 bits. Therefore, we must mask the top 2 bits to ensure they are not set improperly. Line 8 ORs in the sign bit, one if negative. The LNS version is then returned in line 9.

Conversion from LNS (`to_double`) strips off the zero flag (`z`) and sign (`s`) in lines 13 and 14. Line 15 is where the fixed-point exponent is extracted. Note carefully that the extracted bits are stored in a *signed* integer, `p`. We need this integer to be signed so that the conversion back to double will properly account for the sign. Remember, `p` represents the exponent, so it is negative only when the magnitude of the number is less than one. Line 18 adjusts `p` for the case when it is less than one. If bit 29 of `p` is set, the exponent is negative because bits 0 through 29 are reserved for the exponent. If it is set, we sign-extend by adding `0xC0000000` to set all the high bits to make `p` a negative number. Line 19 checks for the zero flag and returns zero if set. Line 20 first “unscales” `p` by raising it to the negative `L` power, and then uses the unscaled floating-point value, which is the actual exponent of the LNS number, to calculate the final value by raising 2 to that power. Line 21 returns the number with the sign set according to `s`.

Multiplication and Division. Now that we have a way to represent LNS numbers, we can implement basic operations. The book's other number systems started with addition and subtraction and then developed multiplication and division. Here, we switch the order to begin with multiplication and division.

We know that $2^x 2^y = 2^{x+y}$, so we multiply LNS numbers by adding the exponents. We intentionally chose to store the exponents in a two's complement fixed-point format. Because of this, we can use normal integer addition to add the exponents and thereby multiply the numbers. For example, $123.45 \times 10.0 = 1234.5$ becomes

$$\begin{array}{rcl} 123.45 & \rightarrow & 1BCA87A \\ 10.00 & \rightarrow & 0D49A78 \\ \hline & & 29142F2 \rightarrow 1234.50 \end{array}$$

This multiplication works because the numbers are positive, and we need not consider the sign bit (bit 30), nor did we check for zero (bit 31). The exponent has an implied radix point between bit 21 and bit 22 ($L = 22$). Adding two numbers stored in this way works regardless of where the radix point is located, so we need to do nothing more than add bit by bit.

Some helper functions will simplify matters later,

```

1 unsigned char lns_isneg(lns_t x) {
2     return (LNS_SIGN & x) != 0;
3 }
4
5 unsigned char lns_iszero(lns_t x) {
6     return (LNS_ZERO & x) != 0;
7 }
8
9 lns_t lns_abs(lns_t x) {
10     if (lns_iszero(x))
11         return x;
12     return LNS_MASK & x;
13 }
14
15 lns_t lns_negate(lns_t x) {
16     if (lns_iszero(x))
17         return LNS_ZERO;
18     if (lns_isneg(x))
19         return LNS_MASK & x;
20     return LNS_SIGN | x;
21 }
```

Line 1 defines `lns_isneg` to check if its argument is negative by examining the sign bit. Line 5 performs a similar check for zero. Line 9 defines `lns_abs` to return a positive version of the input by masking off the zero and sign bits. Finally,

line 15 defines `lns_negate`, which returns its argument with the sign bit toggled. Multiplication now becomes

```

1 | lns_t lns_multiply(lns_t a, lns_t b) {
2 |     if (lns_iszero(a)) return LNS_ZERO;
3 |     if (lns_iszero(b)) return LNS_ZERO;
4 |
5 |     if (lns_isneg(a) == lns_isneg(b))
6 |         return LNS_MASK & (lns_abs(a) + lns_abs(b));
7 |     else
8 |         return lns_negate(LNS_MASK & (lns_abs(a) + lns_abs(b)));
9 | }
```

where the arguments are checked for zero in lines 2 and 3. If either is zero, return zero as the product. Line 5 checks if the signs of the arguments are the same. If so, the product is positive, and line 6 adds the positive versions of the inputs. The AND with `LNS_MASK` keeps the highest 2 bits clear, which is correct for a nonzero positive LNS number. If the signs differ, the product is negative, and line 8 negates the positive product to produce a negative result.

Division of LNS numbers, by analogy to multiplication, involves the subtraction of two exponents since $2^x/2^y = 2^{x-y}$. Again, the fixed-point exponent representation saves us from trouble by enabling simple subtraction with checks for the signs of the arguments to give us the proper result. Therefore, the division of two LNS numbers is simply multiplication, replacing addition with subtraction,

```

1 | lns_t lns_divide(lns_t a, lns_t b) {
2 |     if (lns_iszero(a)) return LNS_ZERO;
3 |     if (lns_iszero(b)) return LNS_ZERO;
4 |
5 |     if (lns_isneg(a) == lns_isneg(b))
6 |         return LNS_MASK & (lns_abs(a) - lns_abs(b));
7 |     else
8 |         return lns_negate(LNS_MASK & (lns_abs(a) - lns_abs(b)));
9 | }
```

where lines 6 and 8 now subtract the arguments instead of adding them. However, line 3 shows a weakness of our chosen LNS representation in that instead of simply returning zero when the second argument is zero, we should signal a division by zero error. We did not reserve any bits for infinity or NaN to maximize precision as we would in a more rigorous library.

Comparison Operators. Let's add logical comparisons to our library. Doing so is straightforward because $x > y \implies 2^x > 2^y$, meaning to decide *equality*, *greater than*, or *less than* requires comparing the exponents and signs. Additionally, since

we are using a two's complement fixed-point representation for the exponents, we can compare magnitudes with standard C comparison operators for signed integers. All the same, the implementation is a bit tedious because there are multiple cases to consider. Therefore, without further ado,

```

1 int32_t lns_exponent(lns_t x) {
2     int32_t e = LNS_MASK & x;
3     return (0x20000000 & e) ? 0xC0000000+e : e;
4 }
5
6 int8_t lns_compare(lns_t a, lns_t b) {
7     int32_t ea = lns_exponent(a);
8     int32_t eb = lns_exponent(b);
9
10    if (lns_iszero(a) && lns_iszero(b))
11        return 0;
12    if (lns_iszero(a))
13        return (lns_isneg(b)) ? 1 : -1;
14    if (lns_iszero(b))
15        return (lns_isneg(a)) ? -1 : 1;
16
17    if ((lns_isneg(a)==1) && (lns_isneg(b)==0))
18        return -1;
19    if ((lns_isneg(a)==0) && (lns_isneg(b)==1))
20        return 1;
21
22    if (lns_isneg(a) && lns_isneg(b))
23        return (ea == eb) ? 0
24                : (eb < ea) ? -1
25                : 1;
26    else
27        return (ea == eb) ? 0
28                : (ea < eb) ? -1
29                : 1;
30 }
31
32 unsigned char lns_equal(lns_t a, lns_t b) {
33     return (lns_compare(a,b) == 0);
34 }
35
36 unsigned char lns_less(lns_t a, lns_t b) {
37     return (lns_compare(a,b) == -1);
38 }
39
40 unsigned char lns_greater(lns_t a, lns_t b) {
41     return (lns_compare(a,b) == 1);
42 }

```


Wrapper functions are used for `lns_equal` (line 32), `lns_less` (line 36), and `lns_greater` (line 40) all of which call `lns_compare` (line 6). This function returns -1 if $a < b$, 0 if $a = b$, and $+1$ if $a > b$.

The `lns_compare` function uses several existing LNS library functions along with a new helper function, `lns_exponent`, which returns the exponent as a *signed* integer. The function is defined in line 1. Line 2 returns the fixed-point exponent by masking the zero and sign bits. Line 3 returns this exponent after setting the two highest bits if the exponent is negative (0×20000000 & e is nonzero). The comparison asks if the exponent's highest bit is set. If it is, the exponent is negative since it is a two's complement number, and we set the top 2 bits by AND-ing with $0 \times c0000000$ to extend the sign before returning. The properly sign-extended exponent allows normal integer comparisons to check the magnitude.

The `lns_compare` function extracts the exponents (lines 7, 8) and then begins a series of checks to cover all possible conditions. Lines 10 through 15 deal with numbers that are zero. If both are zero (line 10), return 0 to indicate that the numbers are equal. In line 12, we know that the second argument cannot be zero, so we check to see if the first argument is zero. If it is, we return the proper value depending upon the sign of the second argument, b (line 13). If b is negative, we know to return 1 because a is zero and $0 > b$ when b is negative. If this isn't the case, return -1 to indicate that $0 < b$ when b is positive. Lines 14 and 15 make the same check for the case where the second argument is zero. These explicit checks against zero are necessary because we elected to use a single bit to indicate zero.

Lines 17 through 20 handle cases where the signs of the numbers differ. If they differ, we already know the proper values to return regardless of the exponents. If a is negative and b is positive (line 17), we know $a < b$ and return -1 . In line 19, we perform the opposite check and, if true, return 1 because we know $a > b$.

It is only when the signs are the same that we need to look at the magnitude of the numbers. Line 27 handles the case where both arguments are positive. Here, we simply compare the exponents in e_a and e_b and, if equal, return 0 . We examine the outcome of $e_a < e_b$ if they are not equal. If this is the case, $a < b$, so return -1 (line 28). Otherwise, we know $a > b$ and return 1 (line 29). If both arguments are negative (line 22), similar comparisons are made for $e_b < e_a$. If so, we know that the exponent of argument a is less negative than the exponent of b , which means $a < b$ since both a and b are negative and $-2^{|e_a|} > -2^{|e_b|}$ for e_a and e_b the exponents of arguments. Return -1 in this case (line 24). Otherwise, the remaining case, $a > b$, is true and we return 1 (line 25).

Addition and Subtraction. Simple formulas exist for the multiplication and division of two LNS numbers, but, unfortunately, there are no simple formulas for addition and subtraction. However, all is not lost.

Following the notation in [2], we represent an LNS number mathematically as $A = -1^{S_A} \cdot 2^{E_A}$ where S_A is the sign bit and E_A is the floating-point exponent. Similarly, we write $B = -1^{S_B} \cdot 2^{E_B}$. To calculate $C = A \pm B$, where we assume

$|A| \geq |B|$, we write

$$\begin{aligned}
 S_C &= S_A \\
 E_C &= \log_2 |A \pm B| \\
 &= \log_2 |A(1 \pm \frac{B}{A})| \\
 &= \log_2 |A| + \log_2 |1 \pm \frac{B}{A}| \\
 &= E_A + f(E_B - E_A)
 \end{aligned}$$

with

$$f(E_B - E_A) = \log_2 |1 \pm \frac{B}{A}| = \log_2 |1 \pm 2^{E_B - E_A}|$$

where we have reduced the problem of addition and subtraction to function approximation since once we know the value of $f(E_B - E_A)$, we know the answer to our problem.

What does f look like? Consider Fig. 13.1 where we have limited the domain to $[-10, 0]$. Our LNS implementation uses a $Q7.22$ fixed-point number to represent exponents in the range $[-128, 127.9999998]$ so that the difference, $E_B - E_A$, $E_B < E_A$, is closer to $[-128, 0]$, but as can be seen, for differences below about -10 , the

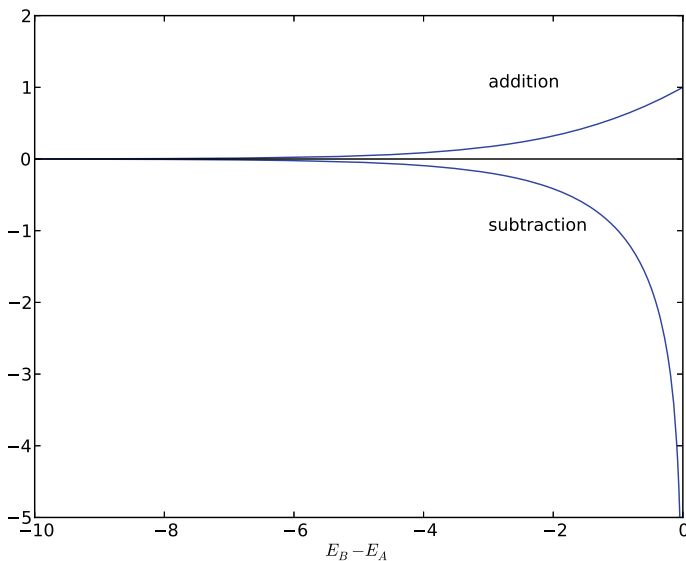


Fig. 13.1 The function f that must be approximated to implement LNS addition and subtraction. The argument to f is the difference between the two exponents where we require $|A| \geq |B|$, implying that the exponent for B , E_B , is less than the exponent for A , E_A . The figure is a recreation of Fig. 1 in [2]

value of f , for addition or subtraction, is virtually zero. Some hardware implementations approximate f mathematically, while others build large lookup tables. All use some form of interpolation to arrive at as accurate a value for f as possible, making $A \pm B$ as accurate as possible. We will take advantage of the closed functional form for f and in our simple library use the C library functions to calculate the value directly. Yes, it is a cheat. One aspect of Fig. 13.1 is worth a few words. If the two numbers, especially for subtraction, are close in value, the difference between their exponents is small, meaning the argument to f is close to zero. This is precisely the region where the function is changing most rapidly. This implies that accurate interpolation in this region is crucial for returning accurate sums and differences.

Suitably warned, let's implement addition and subtraction in stages. The first stage implements f for addition and subtraction of the magnitudes of two LNS numbers, A and B , where we already know that $|A| \geq |B|$. Consider,

```

1 double fminus(double ea, double eb) {
2     return pow(2.0, ea + lg2(1.0 - pow(2.0, eb-ea)));
3 }
4
5 double fplus(double ea, double eb) {
6     return pow(2.0, ea + lg2(1.0 + pow(2.0, eb-ea)));
7 }
8
9 lns_t lns_add_mag(lns_t a, lns_t b) {
10     double ea = (double)(LNS_MASK & a) * pow(2.0, -L);
11     double eb = (double)(LNS_MASK & b) * pow(2.0, -L);
12     return to_lns(fplus(ea, eb));
13 }
14
15 lns_t lns_sub_mag(lns_t a, lns_t b) {
16     double ea = (double)(LNS_MASK & a) * pow(2.0, -L);
17     double eb = (double)(LNS_MASK & b) * pow(2.0, -L);
18     return to_lns(fminus(ea, eb));
19 }
```

with the functions `lns_add_mag` and `lns_sub_mag` called appropriately by `lns_add` and `lns_sub`, yet to be defined, to handle all possible sign combinations of the arguments, a and b .

The function f is in `fminus` (line 1) and `fplus` (line 5) along with the remainder of the calculation to return the sum or difference except for the sign. Specifically, these functions calculate

$$E_C = E_A + \log_2(1.0 \pm 2^{E_B - E_A}), \quad C = 2^{E_C}$$

with E_A and E_B the exponents of the inputs A and B . Note that these functions accept and return standard C double-precision floats, not fixed-point floats. The helper functions are called by `lns_add_mag` (line 9) and `lns_sub_mag` (line 15). These functions perform the addition or subtraction, ignoring the overall sign of the inputs and under the assumption that $|A| \geq |B|$. The functions extract the fixed-

point exponents by AND'ing with `LNS_MASK` before scaling by 2^{-L} to create standard floating-point representations in `ea` and `eb`. The functions then call the proper helpers to determine the sum or difference and return it as an LNS number.

We must consider not only the signs of the arguments but also their relative magnitudes to know how to call `lns_add_mag` and `lns_sub_mag` appropriately. Addition and subtraction imply a total of eight cases to consider: four for the possible signs of the arguments and, for each of these, two cases for the relative magnitudes. We're now able to define the top-level addition function like so:

```

1  lns_t lns_add(lns_t a, lns_t b) {
2      lns_t A = lns_abs(a);
3      lns_t B = lns_abs(b);
4
5      if (lns_iszero(a)) return b;
6      if (lns_iszero(b)) return a;
7      if (lns_equal(a,b))
8          return lns_multiply(a, to_lns(2.0));
9
10     if ((lns_isneg(a)==0) && (lns_isneg(b)==0)) {
11         if (lns_greater(A,B))
12             return lns_add_mag(A,B);
13         else
14             return lns_add_mag(B,A);
15     }
16     if ((lns_isneg(a)==1) && (lns_isneg(b)==1)) {
17         if (lns_greater(A,B))
18             return lns_negate(lns_add_mag(A,B));
19         else
20             return lns_negate(lns_add_mag(B,A));
21     }
22     if ((lns_isneg(a)==0) && (lns_isneg(b)==1)) {
23         if (lns_greater(A,B))
24             return lns_sub_mag(A,B);
25         else
26             return lns_negate(lns_sub_mag(B,A));
27     }
28     if ((lns_isneg(a)==1) && (lns_isneg(b)==0)) {
29         if (lns_greater(A,B))
30             return lns_negate(lns_sub_mag(A,B));
31         else
32             return lns_sub_mag(B,A);
33     }
34 }
```

Subtraction becomes

```

1  lns_t lns_sub(lns_t a, lns_t b) {
2      lns_t A = lns_abs(a);
3      lns_t B = lns_abs(b);
4
5      if (lns_iszero(a)) return lns_negate(b);
6      if (lns_iszero(b)) return a;
7      if (lns_equal(a,b)) return LNS_ZERO;
8
9      if ((lns_isneg(a)==0) && (lns_isneg(b)==0)) {
10         if (lns_greater(A,B))
11             return lns_sub_mag(A,B);
12         else
13             return lns_negate(lns_sub_mag(B,A));
14     }
15     if ((lns_isneg(a)==1) && (lns_isneg(b)==1)) {
16         if (lns_greater(A,B))
17             return lns_negate(lns_sub_mag(A,B));
18         else
19             return lns_sub_mag(B,A);
20     }
21     if ((lns_isneg(a)==0) && (lns_isneg(b)==1)) {
22         if (lns_greater(A,B))
23             return lns_add_mag(A,B);
24         else
25             return lns_add_mag(B,A);
26     }
27     if ((lns_isneg(a)==1) && (lns_isneg(b)==0)) {
28         if (lns_greater(A,B))
29             return lns_negate(lns_add_mag(A,B));
30         else
31             return lns_negate(lns_add_mag(B,A));
32     }
33 }

```

where each possible case is handled so that the proper final sign will be used and the relative magnitudes are respected.

In particular, both functions store positive versions of their arguments in *A* and *B* (lines 2,3) and then check if any are zero. For addition, if either argument is zero, return the other one (lines 5,6). For subtraction, if the first argument is zero, negate the second one and return it (line 5); otherwise, return the first (line 6). If the arguments are equal, using the earlier comparison functions, return zero for subtraction (line 7) or multiply the argument by two if adding (line 8). Each function then checks the signs of the inputs to find which of the four possible cases matches. For each case, the magnitudes of the arguments are reviewed so that *lns_add_mag* or *lns_sub_mag* are called appropriately using the larger magnitude argument first. Some cases require negating the return value to get the proper sign.

The LNS library is available in the files *lns.c* and *lns64.c*. The former is the 32-bit library described here, while the latter replaces 32-bit unsigned integers with 64-bit unsigned integers. The fractional part is expanded from 22 to 54 bits for improved precision. Compile the files with

```
> gcc lns.c -o lns -lm
> gcc lns64.c -o lns64 -lm
```

Run the code with `test` as the only command line argument to display tests of the library functions. Run with no command line arguments to enter a simple calculator mode. Here's an example session using the 64-bit version:

```
Calculator: enter num op num for + - * /
(enter 0 for the first operand to exit)
] 3.141592 * 1001
    3144.733592000000
] 2.718281828459045 / 1
    2.718281828459
] 0.000033 * 33.0033
    0.001089108900
] 1/1.414
    0.707213578501
] 123 + 33.33
    156.330000000000
] 3.001 - 3.0001
    0.000900000000
] 123456 * 1000000
    123455999999.999969482422
] 0-0
```

Discussion. In this section, we implemented a basic library of arithmetic and comparison routines for LNS numbers. We saw how easy it is to implement multiplication and division, operations that are difficult to implement in other representations. We also saw that comparison operations are similarly straightforward because we selected a two's complement fixed-point representation. Of course, little in life is free. We learned that the power of LNS numbers comes at a price in regards to complicating addition and subtraction. A reasonable argument could be made that LNS numbers if efficiently implemented in hardware might become a viable alternative to the IEEE 754 standard.

13.3 Double-Base Number System

The Double-Base Number System (DBNS) was developed by Dimitrov and Jullien [3] and offers a particularly compact way to represent large unsigned integers. As the name suggests, instead of the single base used by every number system we have so far examined, be it binary, decimal, octal, or hexadecimal, the DBNS uses two bases, in

this case, 2 and 3. Specifically, for a given base, B , and digits, $d \in \{0, 1, \dots, B-1\}$, we can represent a number, x , as

$$x = \sum_i d_i B^i$$

In DBNS, we replace the single base B with two bases, 2 and 3. We also limit the choice of digits to $d \in \{0, 1\}$ so that we represent x as a sum of products of powers of 2 and 3, e.g., $2^i 3^j$,

$$x = \sum_{i,j} 2^i 3^j$$

where i, j are exponents ≥ 0 . Terms of the form $2^i 3^j$ are called *2-integers*. Note that we need not explicitly represent zero digits as is the case in standard place notation, meaning DBNS is not a positional number system. An example will clarify. The number 127 can be represented in DBNS as

$$\begin{aligned} 127 &= 2^2 3^3 + 2^1 3^2 + 2^0 3^0 \\ &= 2^2 3^3 + 2^4 3^0 + 2^0 3^1 \\ &= 2^5 3^1 + 2^0 3^3 + 2^2 3^0 \end{aligned}$$

which can be written as a set of tuples,

$$\begin{aligned} 127 &= \{(2, 3), (1, 2), (0, 0)\} \\ &= \{(2, 3), (4, 0), (0, 1)\} \\ &= \{(5, 1), (0, 3), (2, 0)\} \end{aligned}$$

listing only the powers of the terms $2^i 3^j$. We will use the tuple representation in our implementation.

The example reveals an essential aspect of the DBNS: representations are not unique. A number may be represented in many different ways. However, there are “smallest” representations in which the set of tuples is as small as possible. These are called *canonical* representations. The three representations of 127 can be shown by exhaustive search to be the three canonical representations of 127.

How does one find the canonical representation of a number? Locating the canonical representation requires an exhaustive search, implying that it is NP-complete. However, not all is lost. In [4], a simple greedy algorithm is given which locates a “near” canonical DBNS representation for any unsigned integer. The algorithm is easily implemented in Python,

```
1 while (x > 0):
2     w, i, j = Largest2Integer(x)
3     print((i, j))
4     x -= w
```

where `Largest2Integer` returns the largest 2-integer $\leq x$ along with the exponents, i.e., it returns $w = 2^i 3^j \leq x$. Print or otherwise store the tuple (i, j) and

subtract w from x . Repeat until $x < 0$. The algorithm will not always produce the canonical representation of x , but it will produce a useful one.

Graphical Representation of DBNS Numbers. DBNS numbers can be considered a matrix where the rows are the powers of 2, and the columns are the powers of 3. From there, it is a small step to visualize the matrix as an image. For example, a canonical representation of 127 is $\{(1, 2), (0, 0), (2, 3)\}$ which in matrix form becomes

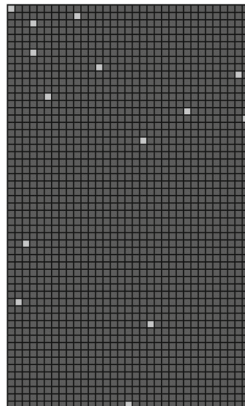
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

where a 1 indicates that the corresponding 2-integer is present in the representation. In the matrix above, the element at row 3, column 4 is present, meaning $2^23^3 = 108$ is present in the DBNS representation of 127. Notice that we index the rows and columns from one, as is typically the case in mathematics, but the exponents indicated are numbered from zero.

Similarly, there are elements in row 1, column 1, and row 2, column 3, indicating terms $2^03^0 = 1$ and $2^13^2 = 18$. The sum of the three terms is $108 + 18 + 1 = 127$, as expected. The matrix representation is useful for understanding the sparsity of a DBNS representation, but it is still more impressive when cast as an image. For example, 127 might be visualized as



where light squares indicate terms present in the DBNS representation. A number like 785,745,187,346,896,756,987,891 becomes



with only a handful of nonzero entries.

A Python Class for DBNS Numbers. Let's implement DBNS in Python. Python is useful in this case for two reasons. First, DBNS numbers are a good choice for representing huge integers. Python supports big integers, see Chap. 5, so it makes sense to use it here. Second, it is natural to represent a DBNS number as a set of tuples, (i, j) , which Python also supports.

The first thing our library must do is to accept an input number, x , and convert it to a set of DBNS tuples using the greedy algorithm. The code for this part is

```

1  from math import log, ceil
2
3  class DBNS:
4      def Largest2Integer(self, x):
5          m = [1,0,0]
6
7          a = int(ceil(log(x)/log(2))) + 1
8          b = int(ceil(log(x)/log(3))) + 1
9
10         for i in range(a):
11             for j in range(b):
12                 n = [(2**i)*(3**j), i, j]
13                 if (n[0] > m[0]) and (n[0] <= x):
14                     m = n
15         return m
16
17     def Greedy(self, x):
18         while (x > 0):
19             w,i,j = self.Largest2Integer(x)
20             self.n.add((i,j))
21             x -= w
22
23     def Limits(self):
24         self.a = 0
25         self.b = 0
26
27         for i,j in self.n:
28             if (i > self.a):
29                 self.a = i
30             if (j > self.b):
31                 self.b = j
32
33     def __init__(self, x=0):
34         self.n = set()
35
36         if (x > 0):
37             self.Greedy(x)
38             self.Ready()
39             self.Limits()

```

Ignore the `Ready()` method of line 38 for the moment. The `__init__` method accepts a number, x , to convert to DBNS stored as a set of tuples (line 34). The observant reader will have realized that zero is not directly representable as the sum

of $2^i 3^j$ terms. Therefore, we represent zero with an empty set, a fitting choice. If x is not zero, apply the greedy algorithm (line 37) and then call `Limits()` (line 39).

The greedy algorithm is the `while` loop starting on line 18. As long as x is greater than zero, subtract from it the largest possible 2-integer, $2^i 3^j$, less than or equal to x and keep the exponents, (i, j) , as a tuple. `Largest2Integer` starts on line 4.

The 2-integer is located by an exhaustive search of possible exponents using `limits` set from x 's current value: $a = \lceil \log_2(x) \rceil + 1$ and $b = \lceil \log_3(x) \rceil + 1$. The double loop searched all 2-integer pairs, keeping the largest. Alternative methods exist to find the largest 2-integer; see [5]. The last step uses `Limits` to track the largest powers of the bases. We'll use these for the 2D representation.

We can transform an integer into a DBNS number. The `Value` method goes the other way,

```

1 def Value(self):
2     m = 0
3     for i,j in self.n:
4         m += (2**i)*(3**j)
5     return m

```

Printing the tuple form of the DBNS number is straightforward,

```

>>> from DBNS import *
>>> x = DBNS(127)
>>> print x.n
      set([(1, 2), (0, 0), (2, 3)])

```

which tells us that the greedy algorithm worked since $127 = 2^1 3^2 + 2^0 3^0 + 2^2 3^3$. Note that there is no implied ordering in a set, and none is necessary as summation is commutative.

The 2D matrix representation is straightforward as well. Let's take advantage of Python's rich set of overloadable operators to produce the 2D representation when printing:

```

1 def __str__(self):
2     m = ""
3     for i in range(self.a+1):
4         n = "|"
5         for j in range(self.b+1):
6             if ((i,j) in self.n):
7                 n += "1 "
8             else:
9                 n += "0 "
10        m += n[0:-1] + "|\n"
11    return m[0:-1]
12
13 def __repr__(self):
14    return self.__str__()

```

Here, `__str__` returns the matrix representation as a string (lines 1-11). The `__repr__` method is overloaded as well (line 14).

We know the largest exponent used by any power of 2 (`self.a`) and 3 (`self.b`); therefore, we know the size of the output matrix: `self.a` rows by `self.b` columns. The loops at lines 3 and 5 cover all combinations. If the element, i.e., that power of 2 and 3, is present in the set of tuples (line 6), output a “1”; otherwise, output “0” to indicate that element is not in the set. When done, return the string (line 11), dropping the final newline character. Testing at the command line gives

```
>>> from DBNS import *
>>> x = DBNS(127)
>>> print(x)
|1 0 0 0|
|0 0 1 0|
|0 0 0 1|
```

matching the matrix and image representations of 127.

We are now ready to implement operations on DBNS numbers, all two of them: addition and multiplication. Others are left for the exercises. Note that unsigned DBNS numbers represented this way do not support subtraction or division.

Addition of DBNS Numbers. The addition of two DBNS numbers represented as sets of tuples is straightforward: take the union of the two sets accounting for any collisions, i.e., the same term, $2^i 3^j$, appearing in both sets. For example, consider $23 + 11 = 34$. Using matrix notation, we have

$$\begin{array}{ccc} 23 & + & 11 & = & 34 \\ \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} & + & \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} & = & \begin{pmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \end{array}$$

The sum overlays the two matrices matching the upper left corners. This example produces no collisions, so we have the final answer, 34. Note, however, that this answer isn’t the final form of the answer.

The sum using tuples notation is

$$\begin{array}{ccc} 23 & + & 11 & = & 34 \\ \{(0, 0), (2, 0), (1, 2)\} \cup \{(1, 0), (0, 2)\} & = & \{(0, 0), (1, 0), (2, 0), (0, 2), (1, 2)\} \end{array}$$

where this addition, because it lacks collisions, is merely set union. So far, so good, but what if the two DBNS numbers share one or more terms? For example, consider $10 + 11 = 21$, which is, using matrix form and adding element by element,

$$\begin{array}{ccc} 10 & + & 11 & = & 21 \\ (1 & 0 & 1) & + & \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} & = & \begin{pmatrix} 1 & 0 & 2 \\ 1 & 0 & 0 \end{pmatrix} \end{array}$$

We now have a problem, the bold 2, which is not allowed in the DBNS format. This is a collision, and we must deal with as a carry. The solution is to realize that a carry is

$$2(2^i 3^j) = 2^{i+1} 3^j$$

implying we can replace the 2 at (i, j) with a zero and increment $(i + 1, j)$ instead. If necessary, this process can be repeated if incrementing $(i + 1, j)$ results in a new carry. Then, we can rewrite the carry-free answer as

$$\begin{pmatrix} 1 & 0 & \mathbf{2} \\ 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \end{pmatrix} = 21$$

which is a valid DBNS number. Set notation gives

$$\begin{array}{ccccc} 10 & + & 11 & = & 21 \\ \{(0, 0), (0, 2)\} \cup \{(1, 0), (0, 2)\} & \rightarrow & \{(0, 0), (1, 0), (0, 2)\} \end{array}$$

We have a problem because there are two $(0, 2)$ elements, one from each operand. Conveniently, since we are using Python sets, to know if there are collisions when adding, we need only check for a non-empty intersection between the numbers. If there are elements in the intersection, as there are here, the element is a collision, and we apply carry reduction. If carry reduction introduces a new carry, we put that carry in the set of elements in the intersection and repeatedly pull elements until the intersection set is empty.

For example, since $\{(0, 0), (0, 2)\} \cap \{(1, 0), (0, 2)\} = \{(0, 2)\}$ we know we have at least one carry. Carry reduction replaces $(i, j) = (0, 2)$ in the answer with $(i + 1, j) = (1, 2)$. The term $(1, 2)$ is not in the intersection; therefore, there is no new carry, and the final output set is $\{(0, 0), (1, 0), (1, 2)\}$, with $2^0 3^0 + 2^1 3^0 + 2^1 3^2 = 1 + 2 + 18 = 21$, as expected.

We get addition by overloading the $+$ operator,

```

1 def __add__(self, y):
2     z = DBNS()
3
4     z.n = self.n.union(y.n)
5     b = self.n.intersection(y.n)
6
7     while (b != set()):
8         i, j = b.pop()
9         z.n.remove((i, j))
10        if ((i+1, j) in z.n):
11            b.add((i+1, j))
12        else:
13            z.n.add((i+1, j))
14
15    z.Ready()
16    z.Limits()
17
18    return z

```

In line 2, we create a new output DBNS number to hold the answer. Nominally, the answer is the union of the two numbers, the object instance and the given object instance in y . The union is assigned to $z.n$.

We check for collisions via the intersection assigned to b (line 5). Lines 7 through 13 loop over the elements of the intersection, if any, performing carry reduction when necessary. While there are still elements in the intersection (line 7), pull an element out of the set (line 8) and remove it from the answer (line 9).

Line 10 asks whether the carry reduction element, $(i, j) \rightarrow (i+1, j)$, is already in the answer. If so, we don't add it to the answer but do add it to the intersection (line 11). If the element is not already in the answer, add it (line 13). Continue this process until the intersection is empty. Line 15 calls the mysterious `Ready()` method we ignored earlier. Let's continue to ignore it for the moment. Line 16 limits the exponents of the answer for display purposes. Finally, line 18 returns the carry-free sum.

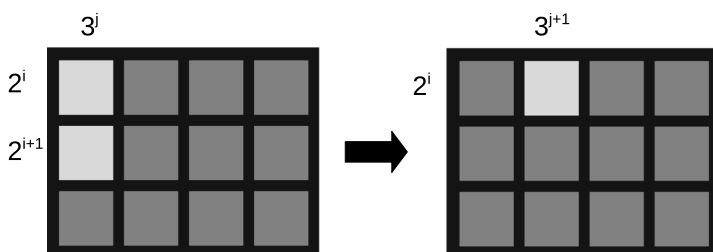
A Problem with Addition and Its Solution. Our first addition example, $23 + 11 = 34$, produced a solution of the form

$$\begin{pmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

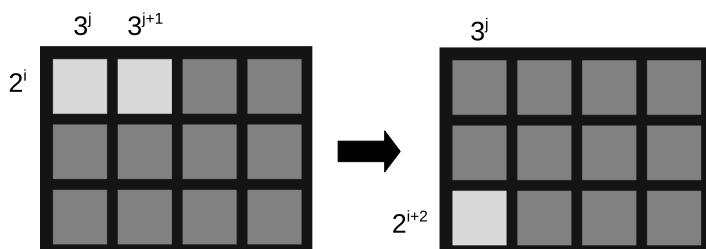
which has quite a few nonzero elements. In other words, the representation is no longer sparse. We can typically improve on this and change the number into a near-canonical, sparse representation by repeatedly applying two reduction rules based on observations of sums of adjacent terms. Namely, we note that

$$\begin{aligned} 2^i 3^j + 2^{i+1} 3^j &= 2^i 3^{j+1} \\ 2^i 3^j + 2^i 3^{j+1} &= 2^{i+2} 3^j \end{aligned}$$

which we can visualize as



and



Additionally, the reduction rules may introduce new carries. The `Ready` method applies the reduction rules and any needed carry reductions until the number is in a sparse, near-canonical form. The `Ready` method itself is straightforward

```

1 def Ready(self):
2     rows = cols = True
3
4     while (rows or cols):
5         cols = self.AdjacentColumn()
6         rows = self.AdjacentRow()

```

where it repeatedly calls `AdjacentColumn` and `AdjacentRow` until both of these return `False` indicating that all instances of adjacent elements have been processed.

The `AdjacentColumn` method is a bit dense

```

1 def AdjacentColumn(self):
2     modified = True
3     ever = False
4
5     while (modified):
6         modified = False
7         b = self.n.copy()
8         while (b != set()):
9             i, j = b.pop()
10            if ((i+1, j) in self.n):
11                modified = True
12                ever = True
13                self.n.remove((i, j))
14                self.n.remove((i+1, j))
15                elem = (i, j+1)
16                if (elem not in self.n):
17                    self.n.add(elem)
18                    b = self.n.copy()
19            else:
20                done = False
21                while (not done):
22                    self.n.remove(elem)
23                    elem = (elem[0]+1, elem[1])
24                    if (elem not in self.n):
25                        self.n.add(elem)
26                        b = self.n.copy()
27                    done = True
28     return ever

```

The main loop (line 5) runs as long as we are making changes to the representation. Each iteration of the loop starts by assuming no changes will be made (line 6) and by duplicating the current representation (line 7). We duplicate so we can examine each element ($2^i 3^j$) in the loop starting on line 8. As we are looking for cases where, for the current element at (i, j) , the adjacent element in the same column, $(i + 1, j)$, is

also present, we ask if that element is in the representation (line 10). If it isn't, move to the next element. If it is, we know that we will make a change, so we indicate that fact (line 11) and that we changed at least one element (line 12).

The reduction rule is $2^i 3^j + 2^{i+1} 3^j \rightarrow 2^i 3^{j+1}$, so if both (i, j) and $(i + 1, j)$ are in the representation, we remove them (lines 13 and 14) and try to add $(i, j + 1)$ (line 15). There are two possibilities. If $(i, j + 1)$ is not already in the representation, we add it (line 17), make a new copy of the representation (line 18), and continue looking at elements until b is empty (line 8). However, if $(i, j + 1)$ is already in the representation, we have a new carry. Line 21 loops through the representation, removing the element that caused the carry and replacing it with a new element: $(i, j) \rightarrow (i + 1, j)$. The loop covers the case in which a chain of carries occurs.

Fortunately, the `AdjacentRow` method is structurally identical to `AdjacentColumn`, only the modified elements change. For example, we make `AdjacentRow` from `AdjacentColumn` by changing the name and modifying a few lines (in bold)

```

1 def AdjacentRow(self):
2     modified = True
3     ever = False
4
5     while (modified):
6         modified = False
7         b = self.n.copy()
8         while (b != set()):
9             i,j = b.pop()
10            if ((i,j+1) in self.n):
11                modified = True
12                ever = True
13                self.n.remove((i,j))
14                self.n.remove((i,j+1))
15                elem = (i+2,j)
16                if (elem not in self.n):
17                    self.n.add(elem)
18                    b = self.n.copy()
19            else:
20                done = False
21                while (not done):
22                    self.n.remove(elem)
23                    elem = (elem[0]+1,elem[1])
24                    if (elem not in self.n):
25                        self.n.add(elem)
26                        b = self.n.copy()
27                    done = True
28     return ever

```

`AdjacentRow` acts like `AdjacentColumn` but implements the reduction $2^i 3^j + 2^{i+2} 3^j \rightarrow 2^{i+1} 3^{j+1}$. Calling these methods repeatedly until neither changes the representation implies that we have arrived at a suitable form to minimize carries on future operations.

Multiplication of DBNS Numbers. Let's add multiplication to our little DBNS implementation. It's particularly straightforward. To calculate xy , multiply every term in y by every term in x . Multiplying two terms is as easy as adding,

$$2^i 3^j \times 2^a 3^b = 2^i 2^a 3^j 3^b = 2^{i+a} 3^{j+b}$$

implying that the answer contains the element $(i + a, j + b)$. If the element already exists, we have a carry and must do our usual carry reduction, $(i + a, j + b) \rightarrow (i + a + 1, j + b)$. The code becomes

```

1 def __mul__(self, y):
2     z = DBNS()
3
4     for i,j in self.n:
5         for a,b in y.n:
6             elem = (i+a,j+b)
7             if (elem not in z.n):
8                 z.n.add(elem)
9             else:
10                done = False
11                while (not done):
12                    z.n.remove(elem)
13                    elem = (elem[0]+1,elem[1])
14                    if (elem not in z.n):
15                        z.n.add(elem)
16                    done = True
17
18     z.Ready()
19     z.Limits()
20     return z

```

The answer is in z . Loop over every element of the first number (line 4); for that element, loop over every element of the second number (line 5). The product of these two elements (line 6) is sought in the answer (line 7). If not present, it is added, and the loops continue (line 8). Otherwise, we have a carry, so we remove the element in the answer (line 12) and add the next element $((i + 1, j))$ until carries are done (line 13). To finish, we put the answer in proper form by calling `Ready` (line 18) and set the exponent limits (line 19) before returning the new DBNS number as the answer (line 20).

Let's test the code by multiplying $1234567890 \times 9876543210$. The representations are

$$\begin{aligned}
 1234567890 &= \{(8, 14), (14, 1), (9, 9), (2, 5), (1, 1)\} \\
 9876543210 &= \{(0, 0), (7, 0), (5, 5), (17, 1), (11, 14), (12, 9), (0, 2)\}
 \end{aligned}$$

implying multiplication requires $5 \times 7 = 35$ pairs of additions and occasional carry reductions.

Representing Real Numbers. We can extend DBNS to represent signed floating-point numbers like the logarithmic number system described earlier in the chapter, by allowing the exponents in $2^i 3^j$ to take on positive and negative integer values. Specifically, we can approximate any number, x , by finding exponents b and t satisfying

$$|x - s_x 2^b 3^t| < \varepsilon, \quad b, t \in \mathbb{Z}$$

for ε a given tolerance and s_x the sign of x .

In practice, the exponents are restricted to a specified precision (number of bits), limiting the set of possible values that can be well represented. Like the LNS system, this system, which we will call the double-base logarithmic number system (DBLNS), or “2DLNS” in [6], easily supports multiplication and division but requires interpolation of a function for addition and subtraction. We will implement only multiplication and division here and leave addition and subtraction as projects for the enthusiastic reader.

Let’s fix the exponent size so DBLNS numbers can be represented as Python floats. To convert x to DBLNS, then, means locating good integer values for b and t so that $|x - s_x 2^b 3^t|$ is as small as possible for the allowed range of exponents. Let’s create a sorted list of all representable values for $b \in [-600, 1023]$ and $t \in [-600, 646]$. We need only to generate this list once, so we do it when the module is imported

```

1 from bisect import bisect_left
2
3 p = []
4 for i in range(-600,1023+1):
5     for j in range(-600,646+1):
6         p.append((2.0**i)*(3.0**j),i,j))
7 p.sort()
8
9 gValues = []
10 gBexp = []
11 gTexp = []
12
13 for i in p:
14     v,b,t = i
15     gValues.append(v)
16     gBexp.append(b)
17     gTexp.append(t)
18 del p,v,b,t,i
19
20 class DBLNS:
21     def __init__(self, x):
22         idx = self.Closest(abs(x))
23         self.b = gBexp[idx]
24         self.t = gTexp[idx]
25         self.s = 0 if (x >= 0.0) else 1

```

Line 1 imports from the standard Python `bisect` module. This module allows for rapid searching of sorted lists. We'll use it to locate the best approximation to any input to the class; see the `Closest` method below. For now, understand that `Closest` returns the index into the list of representable values in global `gValues` closest to `x`. The corresponding exponents are in `gBexp` and `gTexp`. The last step sets the sign (line 25).

The table is generated on import (lines 3 through 7) and computes all possible values and associated exponents. The list is sorted (line 7). N.B., Python sorts on the first value of the tuple. The list is then split into the global values `gValues`, `gBexp`, and `gTexp` (lines 9 through 18).

We now add the `Closest` method and a method to return the floating-point value,

```

1 def Closest(self, x):
2     pos = bisect_left(gValues, x)
3
4     if (pos == 0) or (pos == len(gValues)):
5         return pos
6
7     if (gValues[pos]-x) < (x-gValues[pos-1]):
8         return pos
9     else:
10        return pos-1
11
12 def Value(self):
13     try:
14         v = ((-1.0)**self.s)*(2.0**self.b)*(3.0**self.t)
15     except:
16         v = float('inf') if (self.s==0) else float('-inf')
17     return v
18
19 def __str__(self):
20     return "%0.6f" % self.Value()
21
22 def __repr__(self):
23     return self.__str__()

```

Recall that `gValues` contains a sorted list of all representable floating-point values. Therefore, `bisect_left` returns the position in that list where `x` would best fit (line 2). If we are outside the range, return whatever `pos` is (line 5). Lines 7 through 10 decide if `x` is closer to the returned position or the one before it. We use the returned position as an index into the `b` and `t` exponent lists (`gBexp` and `gTexp`) to define the DBLNS representation. Line 12 defines a function to return the floating-point value of the number. The `try` is necessary to capture the case where the exponents are beyond what can be represented as a Python float. In that case, return positive or negative infinity. Lastly, overload Python methods to print the floating-point value when requested.

Let's complete the DBLNS class by adding multiplication and division,

$$xy = s_x 2^a 3^b \times s_y 2^c 3^d = ((s_x + s_y) \bmod 2) 2^{a+c} 3^{b+d}$$

$$x/y = s_x 2^a 3^b / s_y 2^c 3^d = ((s_x + s_y) \bmod 2) 2^{a-c} 3^{b-d}$$

which translates to

```

1 def __mul__(self, y):
2     z = DBLNS(0)
3     z.s = (self.s + y.s) % 2
4     z.b = self.b + y.b
5     z.t = self.t + y.t
6     return z
7
8 def __div__(self, y):
9     z = DBLNS(0)
10    z.s = (self.s + y.s) % 2
11    z.b = self.b - y.b
12    z.t = self.t - y.t
13    return z

```

where for both multiplication and division we create an output object (lines 2 and 9) and update its sign and exponents appropriately with the proper sum or difference. Lastly, we return the new DBLNS number (lines 6 and 13).

A few examples illustrate how to use the DBLNS class. Note that there will be a second or two pause when importing the class as the lookup table is generated. A hardware implementation would have this table in permanent memory. For example,

```

>>> from DBLNS import *
>>> x = DBLNS(3.141592)
>>> x
3.140757
>>> x.b
260
>>> x.t
-163
>>> y = DBLNS(13)
>>> y
13.002569
>>> p,q = x*y, x/y
>>> p, 3.141592*13
(40.837911, 40.840696)
>>> q, 3.141592 / 13
(0.241549, 0.24166092307692308)

```

The example asks for $x = 3.141592$ and gets 3.140757 the closest value calculated from the exponents $x.b$ and $x.t$, $3.140757 = 2^{260}3^{-163}$. Similarly, $y = 13$ becomes $y = 13.002569$. The product (p) and quotient (q) are calculated and compared to the

corresponding floating-point results. We'll return to these results in the Discussion below.

As the exponent range allowed by our DBLNS numbers is fixed, we might quickly generate exponents that cannot be represented. A solution to this problem involves approximations of unity. For example, all of the following are within ± 0.005 of 1:

value	<i>b</i>	<i>t</i>
0.9958324437176391	168	-106
0.9968509430967009	653	-412
0.9968944606878651	-401	253
0.9979140462573112	84	-53
0.9989346746199259	569	-359
0.998978283176522	-485	306
1.0	0	0
1.0010227617964118	485	-306
1.001066461508586	-569	359
1.0020903140410862	-84	53
1.0031152137308417	401	-253
1.004141161648845	886	-559
1.0041849974949628	-168	106

We might pre-multiply a value by one of these approximations to reduce the exponents to ones that fit the number of exponent bits. Consider,

```
>>> from DBLNS import *
>>> x = DBLNS(3.141592)
>>> x
    3.140757
>>> A = x*x*x*x
>>> A.b
    1040
>>> A.t
   -652
>>> A
    inf
>>> u = DBLNS(0)
>>> u.b = -401
>>> u.t = 253
>>> u
    0.996894
>>> z = x*u
>>> B = z*z*z*z
>>> B
    96.102380
>>> B.b
   -564
>>> B.t
    360
```

where we seek x^4 .

First, we multiply x by itself four times to get A and again after scaling x by an approximation of unity before multiplying $z = xu \rightarrow B = z^4$. Without scaling, A is declared infinite because the exponents exceed the representation table. After scaling, however, we arrive at a reasonable approximation of the true answer.

Scaling by u , a manually constructed DBLNS number using exponents from the table above, changes the exponents on x from $260 \rightarrow -141$ and $-163 \rightarrow 90$ so that repeated multiplication remains confined to the representation table. A complete DBLNS library would use this transformation to keep exponents in range, much as a rational number library uses the GCD to keep fractions in lowest terms.

Discussion. This section investigated two methods to represent numbers using a double-base system. The first represented unsigned integers as sets of tuples, exponents of $2^i 3^j$, that sum to the number. We learned that an efficient mechanism exists to generate this representation. We also learned how to implement addition and multiplication of numbers in this form and tricks for keeping the representation sparse. As interesting as double-base numbers are, the lack of subtraction and division operations for the unsigned integer form significantly limits their practical use. As with all the number systems in this chapter, DBNS numbers are intended for implementation in hardware for specific calculations requiring high speed. For example, [6] reports an algorithm for computing $y = A^E \pmod{p}$ using unsigned DBNS numbers. This is a frequently encountered operation for cryptographic systems, so when the numbers involved are large, unsigned DBNS can be seen as an advantage.

The second method used double-base logarithmic numbers (DBLNS). DBLNS numbers are of the form $x = s_x 2^b 3^t$, where b and t are signed integer exponents and s_x is the sign of x . For a fixed range of exponents, we learned how to implement multiplication and division operations and to take advantage of unity approximations to keep the exponents in range.

Multiplication and division of DBLNS numbers are efficiently implemented, as they are in LNS. However, it is not difficult to appreciate that unless the range of integer exponents is large enough, there are significant losses in precision when converting floating-point values to DBLNS. Errors are further compounded when performing operations on these already approximate values and then again when unity approximations are used. These effects limit the utility of DBLNS numbers, but [6] offers examples of DBLNS numbers for filter calculations along with block diagrams for hardware implementations.

13.4 Residue Number System

The residue number system (RNS) is described by Garner [7]. It is handy for rapid multiplication and addition and, unlike the double-base number system, also supports subtraction. However, division is difficult, as are other operations like magnitude comparison. Conversions to and from standard number formats are also less than straightforward. Our implementation will focus on representation, conversion, and

the operations of unsigned addition, multiplication, and subtraction. Like the other number systems in this chapter, the advantages of RNS are best realized in hardware.

Representing Numbers in RNS. The word “residue” in “residue number system” refers to the fact that numbers are represented as collections of remainders for a chosen set of bases.

For the representations to be unique, the selected bases must be relatively prime, i.e., have no factors in common. Prime numbers naturally satisfy this requirement. Therefore, we will use prime bases for our implementation. How many bases and their values are up to the implementation. In Garner’s original paper, the bases for the examples are 2, 3, 5, and 7. The number of possible remainders for a number divided by 2 is, of course, two: 0 or 1. Similarly, for 3, it is three: 0, 1, 2. The pattern is clear: if the base is n then there are n possible remainders, 0, 1, $\dots$, $n - 1$. So, if we select 2, 3, 5, and 7 as bases, we can uniquely represent $2 \times 3 \times 5 \times 7 = 210$ possible numbers. Let’s make matters concrete using Garner’s set of bases. Representing a number means using the remainders (residues) found by dividing the number by each base. For example, the numbers 0 through 10 become

	2	3	5	7
0	0	0	0	0
1	1	1	1	1
2	0	2	2	2
3	1	0	3	3
4	0	1	4	4
5	1	2	0	5
6	0	0	1	6
7	1	1	2	0
8	0	2	3	1
9	1	0	4	2
10	0	1	0	3

where 3 becomes 1 0 3 3 because $3 \bmod 2 = 1$, $3 \bmod 3 = 0$, $3 \bmod 5 = 3$, and $3 \bmod 7 = 3$.

The largest representable number is 209, which becomes 1 2 4 6. What happens if we try to represent a larger number? Following the conversion rule tells us that 210 becomes 0 0 0 0 and 211 is 1 1 1 1, identical to zero and one, respectively. Therefore, after 209, the pattern of residues repeats. A moment’s thought informs us why: 210 is the product of the bases and is, therefore, evenly divisible by each.

Conversion To RNS. Let’s begin our implementation by deciding on the set of bases and how we will store them in a 32-bit unsigned C integer. Four bases implies allocating 8 bits to each residue. To maximize the range, we should use bases that are close to $2^8 - 1 = 255$ to best use the 8 bits available to each residue. A table of prime numbers reveals that the four largest primes less than 255 are 233, 239, 241, and 251. Therefore, we will use these as the bases and allot 8 bits of the 32-bit

unsigned integer to each of them. What sort of range will this give us compared to a standard 32-bit unsigned integer? Consider,

$$2^{32} - 1 = 4,294,967,295$$

$$233 \times 239 \times 241 \times 251 - 1 = 3,368,562,317$$

meaning we will retain 78.4 percent of the range of a standard 32-bit unsigned integer.

Storing residues implies dividing the 32 bits into four blocks of 8 bits. It will be handy to define bit masks and shift values to set and retrieve the value for any base. Defining conversion to RNS comes next. Putting it all together begins of our library,

```

1  #include <stdint.h>
2
3  typedef uint32_t rns_t;
4
5  #define B0 233
6  #define B1 239
7  #define B2 241
8  #define B3 251
9
10 #define M0 0xFF
11 #define M1 0xFF00
12 #define M2 0xFF0000
13 #define M3 0xFF000000
14
15 #define S0 0
16 #define S1 8
17 #define S2 16
18 #define S3 24
19
20 void pp(rns_t n) {
21     uint16_t n0,n1,n2,n3;
22     n0 = (n & M0) >> S0;
23     n1 = (n & M1) >> S1;
24     n2 = (n & M2) >> S2;
25     n3 = (n & M3) >> S3;
26     printf("%02x %02x %02x %02x", n0,n1,n2,n3);
27 }
28
29 rns_t to_rns(uint32_t n) {
30     rns_t ans = 0;
31     ans |= (n % B0) << S0;
32     ans |= (n % B1) << S1;
33     ans |= (n % B2) << S2;
34     ans |= (n % B3) << S3;
35     return ans;
36 }
```

Including `stdint.h` gives us the `uint32_t` data type (line 1). We define the RNS data type in line 3. Lines 5 through 8 define the prime number bases. Lines 10 through 13 define bit masks on the 32-bit integer that, along with the shift values defined in lines 15 through 18, let us select the residue for any base. We store the bases such

that the bits assigned to B_0 cover bits 0 through 7, B_1 bits 8 through 15, B_2 bits 16 through 23, and B_3 bits 24 through 31.

Lines 20 through 27 define a helper “pretty print” function to display the residues. It also serves as an example of extracting the residues using the idiom of $(n \& M1) \gg S1$ to select the desired bits using the mask and then shift right to move the bits into the lowest order byte.

Lastly, lines 30 through 36 define `to_rns` to calculate the residue for each base and shift it into place. The bits are then logically OR’ed to set them without modifying other bits.

Let’s use these functions to examine the conversion of 1234567890 to RNS. Call `to_rns(1234567890)` to get the RNS representation followed by `pp` to print the residues. The result is

$$\begin{aligned} 1234567890 &= 94_{16}, 06_{16}, 52_{16}, 2B_{16} \\ &= 148, 6, 82, 43 \end{aligned}$$

which is what we expect to see because

$$\begin{aligned} 1234567890 \% 233 &= 148 \\ 1234567890 \% 239 &= 6 \\ 1234567890 \% 241 &= 82 \\ 1234567890 \% 251 &= 43 \end{aligned}$$

Now that we can represent RNS numbers let’s implement addition and multiplication.

Unsigned Addition and Multiplication. The point behind the residue number system is that addition and multiplication are fast, highly parallel operations. This is especially true when RNS is implemented in hardware.

Addition is performed base by base without carries between the bases. For each base, we add the corresponding residues, modulo the base, and that is all. It is the lack of a carry that makes RNS potentially useful. It is not difficult to imagine a hardware implementation that performs addition for each base simultaneously. For example, using the bases 2, 3, 5, and 7,

$$\begin{array}{r} 104 \quad 0 \ 2 \ 4 \ 6 \\ + \ 95 \quad + \ 1 \ 2 \ 0 \ 4 \\ \hline \quad \quad \quad \mathbf{1 \ 4 \ 4 \ 10} \\ 199 \quad 1 \ 1 \ 4 \ 3 \end{array}$$

Each residue is added for each base. The bold entries indicate residues that exceed the base value. Here, bold 4 and 10 exceed their bases, 3 and 7, respectively, and so must be adjusted by subtracting the base value. This produces the last line, where it is claimed that 1 1 4 3 is an RNS representation of 199, which it is since: $199 \bmod 2 = 1$, $199 \bmod 3 = 1$, $199 \bmod 5 = 4$, and $199 \bmod 7 = 3$.

Addition requires two helper functions, `rns_set` and `rns_get`, to set or get the value of a particular residue,


```

1 void rns_set(rns_t *num, uint16_t val, uint8_t base) {
2     switch (base) {
3         case 0: *num |= (uint32_t)val << S0; break;
4         case 1: *num |= (uint32_t)val << S1; break;
5         case 2: *num |= (uint32_t)val << S2; break;
6         case 3: *num |= (uint32_t)val << S3; break;
7         default: break;
8     }
9 }
10
11 uint16_t rns_get(rns_t num, uint8_t base) {
12     uint16_t ans = 0;
13     switch (base) {
14         case 0: ans = (uint16_t)((num & M0) >> S0); break;
15         case 1: ans = (uint16_t)((num & M1) >> S1); break;
16         case 2: ans = (uint16_t)((num & M2) >> S2); break;
17         case 3: ans = (uint16_t)((num & M3) >> S3); break;
18         default: break;
19     }
20     return ans;
21 }

```

To set a residue, supply the residue number to update, the residue value, and which base it belongs in (line 1). To get a residue, supply the RNS number and base (line 11) and, after masking the number appropriately, shift the residue down and return it.

The helper functions make addition particularly terse,

```

1 rns_t rns_add(rns_t x, rns_t y) {
2     rns_t ans = 0;
3     rns_set(&ans, (rns_get(x,0)+rns_get(y,0)) % B0, 0);
4     rns_set(&ans, (rns_get(x,1)+rns_get(y,1)) % B1, 1);
5     rns_set(&ans, (rns_get(x,2)+rns_get(y,2)) % B2, 2);
6     rns_set(&ans, (rns_get(x,3)+rns_get(y,3)) % B3, 3);
7     return ans;
8 }

```

where we set each residue to the sum of the corresponding residues of each input, modulo the base. For example, line 3 sets the residue for base 0. Once all residues are set, the new RNS number, with the proper sum value, is returned (line 7).

Multiplication is similarly performed in parallel without carries. However, instead of adding each residue, we multiply them (modulo the base). Consider,

$$\begin{array}{r}
 \begin{array}{r}
 14 \quad 0 \ 2 \ 4 \ 0 \\
 \times 13 \quad \times 1 \ 1 \ 3 \ 6 \\
 \hline
 \end{array} \\
 \begin{array}{r}
 0 \ 2 \ \mathbf{12} \ 0 \\
 182 \quad 0 \ 2 \ 2 \ 0
 \end{array}
 \end{array}$$

where the product of each residue is given, followed by the residue product modulo at the base. The bold 12 implies that, unlike addition, simple subtraction of the

base value to find the residue is insufficient for multiplication. The final answer is claimed to be 0 2 2 0, which is correct because $182 \bmod 2 = 0$, $182 \bmod 3 = 2$, $182 \bmod 5 = 2$, and $182 \bmod 7 = 0$. The code is nearly identical to addition,

```

1 rns_t rns_mul(rns_t x, rns_t y) {
2     rns_t ans = 0;
3     rns_set(&ans, (rns_get(x,0)*rns_get(y,0)) % B0, 0);
4     rns_set(&ans, (rns_get(x,1)*rns_get(y,1)) % B1, 1);
5     rns_set(&ans, (rns_get(x,2)*rns_get(y,2)) % B2, 2);
6     rns_set(&ans, (rns_get(x,3)*rns_get(y,3)) % B3, 3);
7     return ans;
8 }
```

where the only difference is to replace “+” with “*” for each residue.

Unsigned Subtraction. Recalling that subtraction is defined as the addition of the additive inverse points us toward how to implement unsigned subtraction in RNS. The residues are modulo their base, which means that each residue is a group and, by definition, every element of a group has an inverse such that the group operation performed on element a and its inverse, a^{-1} , will result in the identity element, e .

Here, the identity element is 0. Therefore, for any of the residues, the inverse to a value, a , is what must be added to a to get to 0. Further, this value is always the base minus a . In other words, if the bases are 2, 3, 5, and 7, and we have a set of residues, 1 2 0 4 (95), then we know the inverse is 1 1 5 3 because $1\ 2\ 0\ 4 + 1\ 1\ 5\ 3 = 0\ 0\ 0\ 0$. Therefore, subtraction becomes the addition of the inverse. For example, if we want to subtract 95 (1 2 0 4) from 134 (0 2 4 1) we should get $134 - 95 = 39$ or 1 0 4 4 because

$$\begin{aligned}
 0\ 2\ 4\ 1 - 1\ 2\ 0\ 4 &= 0\ 2\ 4\ 1 + 1\ 1\ 5\ 3 \\
 &= \mathbf{1\ 3\ 9\ 4} \\
 &= 1\ 0\ 4\ 4
 \end{aligned}$$

which is indeed 39, as expected. As before, the bold digits exceed the base value, requiring reduction by the base.

Adding the inverse, which is itself easy to find, makes $A - B$, assuming $A > B$, a straightforward addition to our RNS library,

```

1 rns_t rns_inverse(rns_t y) {
2     rns_t ans = 0;
3     rns_set(&ans, B0 - rns_get(y,0), 0);
4     rns_set(&ans, B1 - rns_get(y,1), 1);
5     rns_set(&ans, B2 - rns_get(y,2), 2);
6     rns_set(&ans, B3 - rns_get(y,3), 3);
7     return ans;
8 }
9
10 rns_t rns_sub(rns_t x, rns_t y) {
11     rns_t ans = 0, iy = rns_inverse(y);
12     rns_set(&ans, (rns_get(x,0)+rns_get(iy,0)) % B0, 0);
13     rns_set(&ans, (rns_get(x,1)+rns_get(iy,1)) % B1, 1);
14     rns_set(&ans, (rns_get(x,2)+rns_get(iy,2)) % B2, 2);
15     rns_set(&ans, (rns_get(x,3)+rns_get(iy,3)) % B3, 3);
16     return ans;
17 }
```

The `rns_sub` function first calculates the inverse of y , then adds as before. The inverse is determined in `rns_inverse` where each residue, r , of the argument is replaced by $B - r$, where B is the residue base.

Conversion From RNS. Conversion from RNS is less obvious than conversion to RNS. We must locate values mapping to special, yet-to-be-defined RNS representations. Once we have said values, we multiply the residues by them and sum to arrive at the answer (modulo the product of the bases).

The special RNS representations are those where one base has a residue of one, and all the other residues are zero. For example, our library uses four bases, implying we must find four numbers whose RNS representations are 1 0 0 0, 0 1 0 0, 0 0 1 0, and 0 0 0 1.

Knowledge of the four special numbers helps in the following way. Assume we know the number with residue representation 1 0 0 0, let's call it b_0 . Then any RNS number with a residue of, say, 3, in the first base contributes $3b_0$ to the value of the full RNS number. Repeating this operation for all bases and summing modulo the bases' product gives us the equivalent decimal value. An example will clarify.

Assume the bases are 2, 3, 5, and 7. We have four bases, so we need numbers with RNS representations as indicated above: 1 0 0 0, 0 1 0 0, 0 0 1 0, and 0 0 0 1. Let's find the first such number, the one with RNS representation 1 0 0 0. Call it A_0 .

We know that whatever A_0 is, the following statements are true:

$$A_0 \bmod 2 = 1$$

$$A_0 \bmod 3 = 0$$

$$A_0 \bmod 5 = 0$$

$$A_0 \bmod 7 = 0$$

If $a \bmod n = 0$, then $n|a$. Therefore, A_0 must be a multiple of the least common multiple of 3, 5, and 7 so that each base divides A_0 . The bases are all primes, implying their least common multiple is their product: $3 \times 5 \times 7 = 105$. In other words, we know that A_0 must be a multiple of 105. Additionally, we want A_0 to be odd, so $A_0 \bmod 2 = 1$. As 105 is already odd, we have $105 \bmod 2 = 1$, telling us that the smallest A_0 is 105. Similar reasoning reveals the remaining three special values:

$$105 \rightarrow 1\ 0\ 0\ 0$$

$$70 \rightarrow 0\ 1\ 0\ 0$$

$$126 \rightarrow 0\ 0\ 1\ 0$$

$$120 \rightarrow 0\ 0\ 0\ 1$$

Let's use these values to show that $0\ 2\ 2\ 0 \rightarrow 182$,

$$[105(0) + 70(2) + 126(2) + 120(0)] \bmod 210 = 182$$

remembering that $2 \times 3 \times 5 \times 7 = 210$.

To add such a conversion to our library, we need to find the four integers that map to the special RNS representations for the bases 233, 239, 241, and 251. Brute force searching, with time to get a cup of coffee, locates the required integers:

$3021585941 \rightarrow 1\ 0\ 0\ 0$
 $1099363434 \rightarrow 0\ 1\ 0\ 0$
 $1663315003 \rightarrow 0\ 0\ 1\ 0$
 $952860257 \rightarrow 0\ 0\ 0\ 1$

which define constants:

```

1 #define A0 3021585941u
2 #define A1 1099363434u
3 #define A2 1663315003u
4 #define A3 952860257u
5
6 uint32_t to_int(rns_t n) {
7     uint64_t ans;
8     ans = (uint64_t)A0*rns_get(n,0) +
9           (uint64_t)A1*rns_get(n,1) +
10          (uint64_t)A2*rns_get(n,2) +
11          (uint64_t)A3*rns_get(n,3);
12     return (uint32_t)(ans % (rns_maxint()+1));
13 }
```

The function `to_int` uses `A0` through `A3` to transform an RNS number to the corresponding unsigned integer by multiplying and summing before modulo the product of the bases. Notice the use of a 64-bit intermediate value to avoid overflow. The modulo operation ensures that the result will fit in a 32-bit unsigned value.

Discussion. This section examined the essence of the residue number system. In particular, how to perform basic arithmetic operations (sans division), and how to convert numbers to and from RNS. RNS is a plausible option for integer operations, especially in hardware. Division remains difficult, however. In [7], Garner speculates that division could be implemented as a trial multiplication and subtraction process, similar to long division by hand. Implementing such an algorithm in hardware might be difficult and, worse yet, slow enough to undo the performance gains for addition, subtraction, and multiplication. Remember, however, that hardware implementations of RNS take full advantage of its carry-free nature to implement operations in parallel, something our library does not do.

The residue number system is still an active research area because it promises fast, carry-free, parallel operations. For example, consider [8–10]. To dive deep into RNS, try [11].

13.5 Redundant Signed-Digit Number System

The chapter's final number system is the redundant signed-digit number system (RSD). Like the residue and double-base number systems above, the redundant system allows a number to be represented in multiple ways to perform certain operations

quickly and efficiently. And, like the other number systems, this flexibility comes at a price when attempting to implement other operations.

The phrase “redundant signed-digit number system” refers to a positional number system where digits are allowed to take on more values than are necessary. For example, a standard positional number, x , using radix (base) r requires digits $0, 1, \dots, r-1$ to represent the number as a string of digits, $d_n d_{n-1} d_{n-2} \dots d_0$, with the value,

$$x = \sum_{i=0}^{n-1} d_i r^i$$

The RSD system loosens the digit requirement, allowing digits to take on values from $-\alpha, \dots, -1, 0, 1, \dots, \beta$ where α and β are $< r$. If $\alpha = \beta$, the system is symmetric. If $\alpha = \beta = r-1$ the system is symmetric and maximally redundant. We will consider only symmetric, maximally redundant systems here. With these constraints, then, a radix-10 RSD number system has digits that range from $-9, -8, \dots, 8, 9$ allowing numbers to be represented in multiple ways depending on the number of digits used. For example, this table shows select values, $0, \dots, 99$, as they might be represented with two or three digits,

$0 = 00,$	$0 = 000$
$1 = 01, 1\bar{9}$	$1 = 001, 01\bar{9}, 1\bar{9}\bar{9}$
$2 = 02, 1\bar{8}$	$2 = 002, 01\bar{8}, 1\bar{9}\bar{8}$
$3 = 03, 1\bar{7}$	$3 = 003, 01\bar{7}, 1\bar{9}\bar{7}$
$4 = 04, 1\bar{6}$	$4 = 004, 01\bar{6}, 1\bar{9}\bar{6}$
$17 = 17, 2\bar{3}$	$17 = 017, 02\bar{3}, 1\bar{9}\bar{7}, 1\bar{8}\bar{3}$
$18 = 18, 2\bar{2}$	$18 = 018, 02\bar{2}, 1\bar{9}\bar{8}, 1\bar{8}\bar{2}$
$19 = 19, 2\bar{1}$	$19 = 019, 02\bar{1}, 1\bar{9}\bar{9}, 1\bar{8}\bar{1}$
$20 = 20,$	$20 = 020, 1\bar{8}0,$
$85 = 85, 9\bar{5}$	$85 = 085, 09\bar{5}, 1\bar{2}5, 1\bar{1}\bar{5}$
$86 = 86, 9\bar{4}$	$86 = 086, 09\bar{4}, 1\bar{2}6, 1\bar{1}\bar{4}$
$87 = 87, 9\bar{3}$	$87 = 087, 09\bar{3}, 1\bar{2}7, 1\bar{1}\bar{3}$
$88 = 88, 9\bar{2}$	$88 = 088, 09\bar{2}, 1\bar{2}8, 1\bar{1}\bar{2}$
$89 = 89, 9\bar{1}$	$89 = 089, 09\bar{1}, 1\bar{2}9, 1\bar{1}\bar{1}$
$90 = 90,$	$90 = 090, 1\bar{1}0,$
$99 = 99,$	$99 = 099, 1\bar{1}\bar{9}, 10\bar{1}$

where $\bar{9} = -9$, and so on.

The number of possible representations increases as the number of digits in the representation increases. Note that the range of an n -digit RSD number is twice that of an n -digit number for any radix if symmetric. For example, a three-digit decimal number can represent values from $[0, 999]$, but a symmetric radix-10 RSD system can represent values from $[-999, 999]$.

Redundant Binary Representation (RBR) RSD supports any radix, but the majority of RSD implementations of focus on radix-2 or binary. This is the “redundant

binary representation” (RBR) and what we’ll use to create a small library of RBR routines.

RBR digits are drawn from the set $\{-1, 0, 1\}$, implying we must represent three distinct values. In binary, this requires 2 bits per digit using this mapping:

Value	r	s
-1	0	0
0	0	1
0	1	0
+1	1	1

where r and s are the 2 bits stored for each digit value. Notice that 0 has two representations, those where the 2 bits differ in value.

For example, the RBR number $01\bar{1}011\bar{1}0_2$ is stored as

$$01\bar{1}011\bar{1}0_2 = 01\ 11\ 00\ 01\ 11\ 11\ 00\ 01_2$$

It’s immediately evident that RBR uses twice as much memory as standard binary. Conversion from RBR to a numerical value is straightforward using the table above. If there are n RBR digits, then memory is accessed in pairs of bits and converted according to

$$x = \sum_{k=0}^{n-1} d_k 2^k = \sum_{k=0}^{n-1} (r_k + s_k - 1) 2^k$$

where d_k is the RBR digit, $\{-1, 0, 1\}$, and r_k and s_k are the actual bits stored in memory using.

Let’s verify that this is indeed true. Consider,

$$\begin{aligned} 01\bar{1}011\bar{1}0_2 &= 0 \times 2^7 + 1 \times 2^6 - 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 - 1 \times 2^1 + 0 \times 2^0 \\ &= 2^6 - 2^5 + 2^3 + 2^2 - 2^1 \\ &= 64 - 32 + 8 + 4 - 2 \\ &= 42 \end{aligned}$$

and

$$\begin{aligned} 01\ 11\ 00\ 01\ 11\ 11\ 00\ 01_2 &= (0 + 1 - 1)2^7 + (1 + 1 - 1)2^6 + (0 + 0 - 1)2^5 + (0 + 1 - 1)2^4 + \\ &\quad (1 + 1 - 1)2^3 + (1 + 1 - 1)2^2 + (0 + 0 - 1)2^1 + (0 + 1 - 1)2^0 \\ &= 2^6 - 2^5 + 2^3 + 2^2 - 2^1 \\ &= 64 - 32 + 8 + 4 - 2 \\ &= 42 \end{aligned}$$

Conversion to RBR from standard binary is equally straightforward: simply take the binary value and encode it. There is no need to search for any representation (as was done for DBNS above). Therefore, we start our simple library with conversion routines to map a signed 32-bit integer to an encoded RBR integer stored in an unsigned 64-bit integer,

```
1 #include <stdint.h>
2 typedef uint64_t rbr_t;
3
4 rbr_t to_rbr(int32_t n) {
5     rbr_t ans = 0;
6     uint8_t i, s=0;
7
8     if (n < 0) {
9         s = 1;
10        n = abs(n);
11    }
12
13    for(i=0; i < 32; i++) {
14        ans += ((n & 1) ? 3 : 1) * (1llu << (2*i));
15        n >>= 1;
16    }
17    return (s) ? ~ans : ans;
18 }
19
20
21 int8_t rbr_digit(uint8_t b) {
22     switch (b) {
23         case 0: return -1;
24         case 1: return 0;
25         case 2: return 0;
26         case 3: return 1;
27     }
28 }
29
30 int32_t to_int(rbr_t n) {
31     int64_t ans = 0;
32     uint8_t i;
33
34     for(i=0; i < 32; i++) {
35         ans += rbr_digit(n & 3) * (1 << i);
36         n >>= 2;
37     }
38     return (int32_t)ans;
39 }
```

where we use `stdint.h` (line 1) to access the `uint32_t` data type (and similar). Line 2 defines the RBR type as an unsigned 64-bit integer. Conversion from binary to RBR starts in line 4. We track the sign of the argument, keeping a flag if it is negative and work with the absolute value (lines 8–11). We then loop over every bit of the argument (line 13) and encode that bit as 11_2 if it is set and 01_2 if it is not set (line 14) where `n & 1` returns the value of the lowest order bit of the argument. Once we have the encoded value, we need to move it to the proper place in the 64-bit

output and add it. The phrase `*(111u << (2 * i))` shifts the encoded value to the proper place in the output, `ans`, where it is added to the existing output bits. Recall that each encoded bit takes two output bits, hence shifting by $2*i$. Line 15 shifts the argument down 1 bit so that the next highest bit is in the lowest position. The loop then continues until all the bits of the argument have been encoded and stored. Line 17 returns the encoded RBR representation where if the argument is negative, we flip all the bits. Why? Because the encoding table tells us that flipping the bits of the pairs changes $01_2 \leftrightarrow 10_2$ and $00_2 \leftrightarrow 11_2$, which maps zero to zero and negative one to positive one, and vice versa. This is equivalent to making the number negative because every positive power added (every 1 bit in the argument) is now added with a negative one.

Conversion from RBR to a standard binary integer begins on line 30. The result will fit in a signed 32-bit integer, but a signed 64-bit integer is used as an accumulator. Loop over all 32 output bits (line 34) and work with the encoded bits as pairs. The phrase `n & 3` extracts the two lowest bits, an encoded digit, and passes them to the `rbr_digit` helper function. The helper function implements the conversion table to return the proper bit value, $\{-1, 0, 1\}$. Once we know the digit value for the current bit, we multiply it by $1 << i$ to determine the value to add to the result. Line 30 shifts down by 2 bits to ready the next encoded digit. Line 32 returns the final answer cast to a 32-bit signed integer.

Let's now consider the RBR encoding and decoding process,

$$\begin{aligned} 123 &\rightarrow 5555555555557FDF_{16} \\ -123 &\rightarrow \text{AAAAAAAAAAAA}8020_{16} \end{aligned}$$

where the right value is the RBR encoded version of 123 or -123 . The number 123 fits in 8 bits to become 16 encoded bits. All other encoded bits become zero using either 01_2 or 10_2 . In other words, each pair of zero digits becomes either $0101_2 = 5_{16}$ or $1010_2 = A_{16}$, thereby explaining all the leading 5's and A's in the encoded versions.

Let's examine $7FDF_{16}$ in binary,

$$01\ 11\ 11\ 11\ 11\ 01\ 11\ 11_{RBR} = 01111011_2 = 123$$

and for -123 , which is 8020_{16} ,

$$10\ 00\ 00\ 00\ 00\ 10\ 00\ 00_{RBR} = 0\bar{1}\bar{1}\bar{1}\bar{1}0\bar{1}\bar{1}_2 = -123$$

showing that the RBR representation does indeed map back to 123 or -123 , as we expect.

RBR Addition and Subtraction. Let's implement addition and subtraction now that we can convert to and from RBR. In hardware, the addition process is highly parallelizable, but we will use standard C here. The key to addition in RBR, which is slightly different from RSD addition in bases greater than 2, is a translation table that converts the sum of 2 bits in $\{-1, 0, 1\}$ to a partial sum and transfer bit added to the partial sum of the next highest bit. In place of bit-by-bit addition with carry, we generate the partial sum and transfer bits simultaneously for each digit in the operands and then shift the transfer bits and add. Selecting the partial sum and transfer bit values ensures that the sum will never require a carry. An example will clarify matters.

Let's add $x = 123$ and $y = 35$ to see if we get 158. The RBR representations for 123 and 35 are, respectively, 01111011_{RBR} and 00100011_{RBR} . We add these values using a lookup table to convert the bits of x and y to a new representation, the partial sum, and the transfer bits. The table is

x_i	y_i	$\rightarrow$	t_i	s_i
-1	-1		-1	0
-1	0		-1	1
0	-1		0	-1
-1	1		0	0
1	-1		0	0
0	0		0	0
0	1		0	1
1	0		1	-1
1	1		1	0

where the i -th bit of x and y are used to locate the i -th bits of s and t . Once we have s and t , we must shift t up 1 bit and then add to get the final result,

$$\begin{array}{r}
 0\ 1\ 1\ 1\ 1\ 0\ 1\ 1 \leftarrow x \\
 +\ 0\ 0\ 1\ 0\ 0\ 0\ 1\ 1 \leftarrow y \\
 \hline
 0\ \bar{1}\ 0\ \bar{1}\ \bar{1}\ 0\ 0\ 0 \leftarrow s \\
 +\ 0\ 1\ 1\ 1\ 1\ 0\ 1\ 1 \leftarrow t(\text{shifted left}) \\
 \hline
 0\ 1\ 0\ 1\ 0\ \bar{1}\ 1\ 1\ 0 \rightarrow 158
 \end{array}$$

Subtraction is implemented by adding the inverse, $x - y \rightarrow x + \bar{y}$, which is the same as adding the negation of y . As we saw above for our conversion routines, flipping the bits of an RBR number changes the sign, so subtraction is simply adding the flipped bits of y .

Let's add addition and subtraction to our set of RBR routines,

```

1 int8_t rbr_enc(int8_t s) {
2     switch (s) {
3         case -1: return 0;
4         case 0: return 1;
5         case 1: return 3;
6     }
7 }
8
9 int8_t rbr_add_partial(rbr_t s, rbr_t t) {
10     int8_t z = rbr_digit(s & 3) + rbr_digit(t & 3);
11     return rbr_enc(z);
12 }
13
14 int8_t rbr_add_st(rbr_t x, rbr_t y, int8_t *s, int8_t *t) {
15     int8_t dx = rbr_digit(x & 3);
16     int8_t dy = rbr_digit(y & 3);
17     if ((dx == -1) && (dy == -1)) { *t = -1; *s = 0; }
18     if ((dx == -1) && (dy == 0)) { *t = -1; *s = 1; }
19     if ((dx == 0) && (dy == -1)) { *t = 0; *s = -1; }
20     if ((dx == -1) && (dy == 1)) { *t = 0; *s = 0; }
21     if ((dx == 1) && (dy == -1)) { *t = 0; *s = 0; }
22     if ((dx == 0) && (dy == 0)) { *t = 0; *s = 0; }
23     if ((dx == 0) && (dy == 1)) { *t = 0; *s = 1; }
24     if ((dx == 1) && (dy == 0)) { *t = 1; *s = -1; }
25     if ((dx == 1) && (dy == 1)) { *t = 1; *s = 0; }
26 }
27
28 rbr_t rbr_add(rbr_t x, rbr_t y) {
29     rbr_t ans=0, s=0, t=0;
30     int8_t ds,dt;
31     uint8_t i;
32
33     for(i=0; i < 32; i++) {
34         rbr_add_st(x, y, &ds, &dt);
35         s += ds * (1llu << (i*2));
36         t += dt * (1llu << (i*2));
37         x >>= 2;
38         y >>= 2;
39     }
40     t <<= 2;
41
42     for(i=0; i < 32; i++) {
43         ans += rbr_add_partial(s,t) * (1llu << (2*i));
44         s >>= 2;
45         t >>= 2;
46     }
47     return ans;
48 }
49
50 rbr_t rbr_sub(rbr_t x, rbr_t y) {
51     return rbr_add(x, ~y);
52 }

```

The code appears a little daunting at first, but it isn't so bad. Let's begin with subtraction (line 50). As stated above, subtraction adds the inverse of the second argument, so we return the sum of x and $\neg y$ where $\neg$ means logical NOT. The addition itself begins on line 28. We process each bit of the operands (x and y) in the loop at line 33. The current lowest order 2 bits of each (an encoded RBR digit) are used to decide the values put in the corresponding bit positions of s and t . We use the scaling trick described above for our conversion routines to put encoded bit pairs in the proper place (lines 35 and 36). The call to `rbr_add_st` accomplishes this by converting the lowest digit of x and y to their actual digit values (lines 15 and 16) before searching the lookup table, here implemented via a set of `if` statements (lines 17 through 25). The statements set s and t properly. The current lowest order digit processed, x and y , are shifted to the next digit (lines 37 and 38). When all digits are processed, line 40 shifts the transfer digits (t) up one position before adding. The addition happens in the loop at line 42. The lookup table's design ensures the additions never produce a carry. The sum is in `ans` (line 47).

The RBR representation is cumbersome in code, but in practice, hardware implementations, particularly on FPGAs, are parallelized, making addition a rapid operation, even with the transfer digits.

Discussion. As with all the systems in this chapter, the RSD system (including RBR) is a special-purpose system used in custom hardware situations for specific algorithms, where their advantages are helpful. RSD is intellectually entertaining, and RBR remains an area of active research. For example, consider [12–15].

13.6 Summary

In this chapter, we reviewed four special-purpose number systems: logarithmic numbers, double-base numbers, residue numbers, and redundant numbers, each offering advantages in certain situations most often realized best in hardware. However, all, except possibly logarithmic numbers, are unlikely alternatives to standard representations. Logarithmic numbers offer a viable alternative to IEEE 754 floating-point numbers. Multiplication and division are highly efficient, while addition and subtraction are less precise, requiring function interpolation. Modern hardware can store detailed information on the interpolation function, making even addition and subtraction effective at the expense of some memory use. This makes LNS an attractive possible alternative for floating-point operations [16].

Double-base numbers are interesting because of their ability to be represented in two dimensions. Addition and multiplication are rapid, but subtraction and division are likely not possible in the unsigned form. The double-base logarithmic form offers the same possible advantages as LNS, but using integer exponents introduces a severe loss of precision.

Residue numbers allow for speedy implementation of addition, subtraction, and multiplication. The set of routines developed in this chapter was limited to unsigned integers for space considerations. Still, there is no reason why signed numbers cannot be represented as residues. Fixed-point floating-point operations are also possible. Again, special-purpose hardware can take advantage of the representation for specific algorithms.

Lastly, we considered redundant signed-digit numbers, specifically radix-2 redundant binary numbers. We learned how this system allows for parallel implementation of addition and subtraction. The RSD representation is curious in that since each digit is signed, there is no need for an overall sign for the number. This also makes comparison between numbers more difficult. The constant-time addition speed of RBR, in hardware, offers a distinct performance advantage over two's complement addition, especially for larger bit widths (≥ 32) as the runtime for two's complement addition increases as the log of the bit width [17].

Exercises

13.1 Add new LNS library functions: `lns_lg2(x)` to compute $\log_2(x)$, `lns_pow(x, n)` to compute x^n , $n \in \mathbb{Z}$, and `lns_sqrt(x)` to compute $\sqrt{x}$.

13.2 Add a new method to the Python DBNS class to compute x^y where x and y are both DBNS instances.

13.3 Add a new method to the Python DBNS class to compute $x!$, the factorial of x .

13.4 Addition and subtraction of DBLNS numbers involve interpolation of a 2D function like the 1D interpolation of the logarithmic number system. Specifically, addition (subtraction) is $2^a 3^b \pm 2^c 3^d = 2^a 3^b (1 \pm 2^{c-a} 3^{d-b})$, where the trick is to interpolate $1 \pm 2^{c-a} 3^{d-b}$ in a reasonable way. For this exercise, skip interpolation and implement the function exactly using Python.

13.5 Add a new function to the RNS library, `rns_pow`, that takes an RNS number and an unsigned integer (`uint16_t`) as input and returns the RNS number raised to that power.

13.6 Add a new function to the RNS library, `rns_fact`, that takes an RNS number, x , and returns $x!$, the factorial of x .

13.7 Add multiplication to the RBR library. A naïve approach to $x \times y$ would add x to itself y times or add y to itself x times. Looping over the smaller of x and y helps a little. Implement this less-than-ideal approach.

13.8 Now, add proper multiplication to the RBR library. The multiplication of two integers in binary is accomplished by adding shifted versions of the multiplicand depending on the value of the multiplier bits. RBR multiplication works similarly with the added condition that a bit value might be negative, in which case the negation of the multiplicand is added. See Sect. 2.3.5 for an explanation of binary multiplication. Implement the corresponding RBR version. **

References

1. Coleman JN et al (2008) The European logarithmic microprocessor. *IEEE Trans Comput* 57(4):532–546
2. Haselman M et al (2005) A comparison of floating point and logarithmic number systems for FPGAs. In: 13th annual IEEE symposium on field-programmable custom computing machines, 2005. FCCM 2005. IEEE

3. Dimitrov Vassil S, Jullien Graham A, Miller William C (1999) Theory and applications of the double-base number system. *IEEE Trans Comput* 48(10):1098–1106
4. Dimitrov Vassil S, Jullien Graham A, Miller William C (1998) An algorithm for modular exponentiation. *Inf Process Lett* 66(3):155–159
5. Berthe V, Imbert L (2004) On converting numbers to the double-base number system. In: Optical science and technology, the SPIE 49th annual meeting. International society for optics and photonics
6. Dimitrov Vassil S, Jullien Graham A (2003) Loading the bases: a new number representation with applications. *IEEE Circuits Syst Mag* 3(2):6–23
7. Garner HL (1959) The residue number system. *IRE Trans Electron Comput* 2:140–147
8. Molahosseini AS, Sorouri S, Zarandi AAE (2012) Research challenges in next-generation residue number system architectures. 2012 7th international conference on computer science and education (ICCSE). IEEE
9. Chen, J, Hu J (2013) Energy-efficient digital signal processing via voltage-overscaling-based residue number system. *IEEE Trans Very Large Scale Integrat (VLSI) Syst* 21(7):1322–1332
10. Zalekian A, Esmaeildoust M, Kaabi A (2015) Efficient implementation of NTRU cryptography using residue number system. *Int J Comput Appl* 124(7)
11. Mohan PVA (2012) Residue number systems: algorithms and architectures, vol 677. Springer Science and Business Media
12. Ganesh KV et al (2012) Constructing a low power multiplier using modified booth encoding algorithm in redundant binary number system. *Int J Eng Res Appl* 2:2734–2740
13. Shukla V et al (2015) A novel approach to design a redundant binary signed digit adder cell using reversible logic gates. In: 2015 IEEE UP section conference on electrical computer and electronics (UPCON). IEEE
14. El-Slehdar AA, Fouad AH, Radwan AG (2015) Memristor based N-bits redundant binary adder. *Microelectron J* 46(3):207–213
15. Cui X et al (2016) A modified partial product generator for redundant binary multipliers. *IEEE Trans Comput* 65(4):1165–1171
16. Coleman JN, Che Ismail R (2016) LNS with co-transformation competes with floating-point. *IEEE Trans Comput* 65(1):136–146
17. Pai Y-T, Chen Y-K (2004) The fastest carry lookahead adder. *DELTA*

Index

A

- AI number formats, 306
 - bfloat16, 306
 - Effect on inference, 315, 316
 - float16, 306
 - FP4, 308, 311
 - FP8, 308
 - INT4, 313–315
 - INT8, 313, 314
 - Mixed-precision training, 320, 321
 - NormalFloat (NF4), 312
- AND, 23
- Arbitrary-precision floating point
 - Arithmetic, 256
 - Comparing, 258
 - Definition, 251
 - Representing, 251
 - Software libraries, 264
 - Trigonometric and transcendental functions, 259
- Ariane 5 rocket explosion, 117

B

- Big integers, 135
 - Addition and subtraction, 141
 - Comba multiplication, 153
 - Comparing, 139
 - Divide-and-conquer, 160
 - Division, 150
 - in Python, 165
 - Input and output, 137
 - Karatsuba multiplication, 155
 - Knuth division, 159
 - Libraries, 163

- Representation, 136
- Schönhage and Strassen multiplication, 156
- School method multiplication, 147
- Bits, 17
 - Clearing, 30
 - Masking, 29
 - Rotations, 33
 - Setting, 29
 - Shifting, 31
 - Testing, 29
 - Toggling, 30
- Boole, George, 23
- Boolean algebra, 23
- Bytes, 18

C

- Checksum, 27
- Compare instructions, 35
- Converting numbers
 - Binary to binary-coded decimal, 69
 - Binary to decimal, 11
 - Binary-coded decimal to binary, 71
 - Decimal to binary, 9
 - Fixed-point to floating-point, 183
 - Floating-point to fixed-point, 183
 - Hexadecimal and binary, 8
 - Octal and binary, 9
 - Others to decimal, 12
- Cryptography, 172

D

- Decimal floating point, 203
 - Biased exponent continuation field, 204
 - Combination field, 204

- Continuation field, 204
- Declet, 206
- Densely packed decimal (DPD), 206
- in software (C), 213
- in software (Python), 218
- Infinity, 208
- Not-a-number (NaN), 208
- Rounding modes, 208
- Storage formats, 204
- Storage order, 209
- Diffie-Hellman key exchange, 173
- Digital comparator, 35
- Distribution of floating-point numbers, 81
- Double-base number system
 - Addition, 373
 - Definition, 368
 - Discussion, 383
 - Graphical representation, 370
 - Improving addition, 375
 - Multiplication, 378
 - Python implementation, 371
 - Real number multiplication and division, 380
 - Real number unity approximations, 382
 - Representing integers, 370
 - Representing real numbers, 379

E

- Encryption, 26
- Endianness
 - Big-endian, 22
 - Little-endian, 22
- Experiments
 - Comparison of fixed-point trigonometric functions, 194
 - Decimal floating point to ASCII string, 209
 - Decimal floating-point logistic map, 213
 - Floating-point uncertainty in comparing numbers, 123
 - Floating-point uncertainty in repeated subtractions, 120
 - Floating-point uncertainty in summing an array of numbers, 122
 - Floating-point uncertainty in the Logistic Map, 118
 - Illustrating uncertainty in floating-point calculations, 117
 - Using rational numbers, 171

F

- Fashion MNIST, 294

Fixed-point numbers

- Addition, 184
- Cosine (polynomial), 193
- Cosine (table), 191
- Cosine (Taylor series), 192
- Division, 188
- DOOM (case study), 199
- Exponential (Taylor series), 196
- Multiplication, 185
- Natural logarithm (Newton's method), 198
- Q notation, 179
- Sine (polynomial), 193
- Sine (table), 189
- Sine (Taylor series), 191
- Square root (Newton's method), 195
- Subtraction, 184
- Trigonometric functions, 189
- When to use, 199

Floating-point

- Addition/subtraction algorithm, 101
- Avoiding the pitfalls, 128
- Binary-coded decimal floating-point, 108
- Comparison, 97
- Floating-point number, 79
- IEEE 754 rounding modes (binary), 93
- IEEE addition and subtraction, 100
- IEEE exceptions, 103
- IEEE floating-point in hardware, 106
- IEEE infinity (binary), 88
- IEEE multiplication, 101
- IEEE NaN (binary), 89
- Mantissa, 80
- Multiplication algorithm, 102
- Pitfalls, 115
- Real number, 79
- Rules of thumb for floating-point numbers, 128
- Significand, 80
- Subnormal numbers, 92
- Trapping exceptions, 104
- Using a tool to improve floating-point calculations, 129

G

- GPT-4, 323

H

- Herbie, 129

I

- IBM S/360 floating-point, 84

IEEE 754 number formats, 87

IEEE 754-2008, 79

Integers

Binary addition, 40

Binary subtraction, 43

Binary-code decimal subtraction, 68

Binary-coded decimal, 66

Binary-coded decimal addition, 68

Densely Packed Decimal, 72

One's complement numbers, 55

Power of two test, 46

Sign extension, 62

Sign-magnitude numbers, 54

Signed addition, 58

Signed comparison, 56

Signed division, 63

Signed multiplication, 60

Signed subtraction, 58

Two's complement numbers, 55

Unsigned, 19

Unsigned addition, 41

Unsigned division, 48

Unsigned multiplication, 46

Unsigned square root, 52

Unsigned subtraction, 44

Zoned Decimal, 73

Interval arithmetic, 224

Absolute value, 233

Addition and subtraction, 226

Comparisons, 234

Dependency problem, 243

in Python, 235

Monotonic functions, 240

MPFI library, 244

Multiplication, 228

Negation, 232

Powers, 230

Properties of intervals, 239

Reciprocal and division, 229

Sine and cosine, 241

L

Large language model (LLM), 323

FlexInt, 326, 328

Examples, 334

Implementation, 330

MetaInt, 326, 344

Examples, 350

Implementation, 347

Mixed Precision Integer Float (MPIF), 326, 337

Examples, 343

Implementation, 339

Prompting, 326

Transformer layer, 324

Least-squares curve fitting, 193, 290

LLaMa 3, 323

Logarithmic number system, 356

Addition and subtraction, 363

Comparing numbers, 361

Discussion, 368

Multiplication and division, 360

Representation, 356

Logical operators, 23

M

Memory

Addresses, 20

Bit order, 21

Byte order, 21

Microscaling Formats (MX) Specification, 313

MNIST, 294

N

Neural network

Training and inference, 304

Nibbles, 18

NOT, 24

Numbers

Babylonian numbers, 4

Binary numbers, 6

Decimal numbers, 6

Egyptian numbers, 2

Hexadecimal numbers, 7

Mayan numbers, 5

Octal numbers, 7

Roman numerals, 1

NumPy, 295

O

OR, 24

P

Parity, 27

Patriot missile failure, 116

Place notation, 3

Posits, 277

Interpretation, 280, 283

Neural networks, 294

Python libraries, 285

Representation, 278

Standard formats, [281](#)
Propagation of errors, [224](#)

R

Radix point, [3](#)
Rational arithmetic, [166](#)
 Addition, [169](#)
 Division, [170](#)
 GCD, [168](#)
 in Python, [167](#)
 Multiplication, [170](#)
 Subtraction, [169](#)
Redundant signed-digit number system, [390](#)
 Addition and subtraction, [395](#)
 Discussion, [397](#)
 Redundant binary representation, [391](#)
Residue number system, [383](#)
 Addition and multiplication, [386](#)
 Conversion from RNS, [389](#)
 Conversion to RNS, [384](#)
 Discussion, [390](#)
 Representing, [384](#)
 Subtraction, [388](#)

RSA encryption, [173](#)

S

Schleswig-Holstein parliament election, [117](#)
Scikit-Learn, [295](#)
Shannon, Claude, [17](#)
Swift, Jonathan, [22](#)

T

Torres y Quevedo, Leonardo, [82](#)
Truth table, [23](#)

V

Vancouver stock exchange, [117](#)

W

Words, [18](#)

X

XOR, [24](#)

Z

Zuse, Konrad, [83](#)